Firmware Reference Manual

for Ezairo® 7100

M-20785-018
November 2016

© SCILLC, 2015
Previous Edition © 2014
"All Rights Reserved"

ON Semiconductor®

ON Semiconductor and  are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is either a trademark or registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

Table of Contents

Page

1. Introduction 22
   1.1 Purpose 22
   1.2 Intended Audience 22
   1.3 Conventions 22
   1.4 Manual Organization 22
   1.5 Further Reading 23
2. Firmware Overview 24
   2.1 Description 24
   2.2 Components 24
      2.2.1 Firmware Files 26
      2.2.2 Compliance Exceptions 29
   2.3 Device Variants 29
   2.4 Using the Firmware 29
      2.4.1 ROM-Based Libraries 30
   2.5 Memory Map 30
      2.5.1 Memory Allocation 31
         2.5.1.1 Stack Initialization 32
      2.5.2 System Variables. 33
   2.6 Firmware Naming Conventions 33
      2.6.1 Macro and ARM Cortex-M3 Processor Inline Function Naming 34
      2.6.2 Library Function Naming 34
   2.7 Firmware Resource Usage 34
      2.7.1 Register Usage 34
      2.7.2 Stack Usage 35
   2.8 Versions 35
   2.9 Operating Frequencies 36
      2.9.1 Current Operating Frequency 37
      2.9.2 Changing Operating Frequencies 37
   2.10 Booting 37
      2.10.1 Boot Sequence 38
      2.10.2 Boot Errors 38
3. CFX System Include Files and Macros 40
   3.1 Description 40
   3.2 Hardware Definitions 40
      3.2.1 Register Naming Conventions 40
         3.2.1.1 Register Definitions Usage 41
      3.2.2 Memory Map and Vector Table Definitions 41
   3.3 Macros 41
      3.3.1 Macro Naming Conventions 42
      3.3.2 Macro Usage 43
         3.3.2.1 Set, Get and SetReg Macros 43
   3.4 System Configuration 43
   3.5 Memory Management 44

3.6 Application Switching . . . 46
3.7 Firmware Utilities . . . 46
3.8 CFX External Interfaces . . . 48
3.9 CFX Peripherals . . . 50
3.10 ARM Cortex-M3 Processor External Interfaces . . . 52
3.11 ARM Cortex-M3 Processor Peripherals . . . 53
3.12 Analog Block Configuration . . . 54
3.13 HEAR Configuration . . . 58
3.14 ARM Cortex-M3 Processor Control and Configuration . . . 58
3.15 Filter Engine Configuration . . . 59
4. CFX System Library . . . 61
4.1 Description . . . 61
4.2 System Library Usage . . . 61
4.2.1 Naming Convention . . . 61
4.3 System Library Functions . . . 61
4.4 System Library Macros . . . 61
4.5 Version . . . 62
5. ARM Cortex-M3 Processor Header Files and System Library . . . 63
5.1 Description . . . 63
5.2 Hardware Definitions . . . 63
5.2.1 Register and Register Bit-field Definition . . . 63
5.2.2 Memory Map Definition . . . 64
5.2.3 Interrupt Vector Definition . . . 64
5.3 CFX Control . . . 64
5.4 ARM Cortex-M3 Core Support Macros and Functions . . . 64
5.5 External Interfaces on the ARM Cortex-M3 Processor . . . 65
5.6 Peripherals on the ARM Cortex-M3 Processor . . . 66
6. ARM Cortex-M3 Processor Startup Implementation . . . 70
6.1 Description . . . 70
7. CFX Math Library . . . 72
7.1 Description . . . 72
7.2 Content of the Library . . . 72
7.3 Math Library Usage . . . 76
7.3.1 Naming Conventions . . . 77
7.4 Numeric Types . . . 78
7.4.1 24-bit Fractional (frac24) . . . 78
7.4.2 48-bit Fractional (frac48) . . . 78
7.4.3 24-bit Mixed (m8p16) . . . 78
7.4.4 48-bit Mixed (m24p24) . . . 79
7.4.5 48-bit Mixed (m2p46) . . . 79
7.4.6 24-bit Integer (int24). . . . 79
7.4.7 24-bit Floating-Point (fl8n16) . . . 79
7.5 Math Library Macros . . . 79
7.5.1 Square Root . . . 80
7.5.2 Base-2 Logarithm . . . 81
7.5.3 Base-2 Exponent . . . 82

7.5.4 Reciprocal . . . 83
7.5.5 Division . . . 85
7.5.6 Linear Interpolation . . . 86
7.5.7 Exponential Average Filter . . . 86
7.5.8 Attack-Release Filter . . . 87
7.5.9 RMS-to-Decibel Conversion . . . 88
7.5.10 Decibel-to-RMS Conversion . . . 89
7.5.11 Power-to-Decibel Conversion . . . 89
7.5.12 Decibel-to-Power Conversion . . . 90
7.5.13 RMS-to-Decibel (Base 2) Conversion . . . 91
7.5.14 dB (Base 2)-to-RMS Conversion . . . 91
7.5.15 Power-to-Decibel (Base 2) Conversion . . . 92
7.5.16 dB (Base 2)-to-Power Conversion . . . 92
7.5.17 Squared Magnitude . . . 93
7.5.18 Magnitude . . . 94
7.5.19 Sine Function . . . 94
7.5.20 Cosine Function . . . 95
7.5.21 Fixed-Point-to-Floating-Point Conversion . . . 95
7.5.22 Floating-Point-to-Fixed-Point Conversion . . . 97
7.6 Examples . . . 98
7.6.1 Squared Magnitude . . . 98
7.6.2 Square Root . . . 98
7.6.3 Cosine . . . 99
7.6.4 Sine . . . 100
7.6.5 Format Conversion . . . 100
7.6.6 Logarithm . . . 101
7.6.7 Exponential . . . 101
7.6.8 Attack Release . . . 101
7.6.9 Reciprocal . . . 102
7.6.10 RMS to db10 . . . 103
7.6.11 Division . . . 103
7.6.12 Linear Interpolation . . . 104
8. ARM Cortex-M3 Processor Math Library . . . 105
8.1 Overview . . . 105
8.2 Data Types . . . 105
8.2.1 Fixed-point Data Saturation . . . 105
8.3 Content of the Library . . . 105
8.4 Math Library Usage . . . 106
8.4.1 Naming Conventions . . . 106
8.5 Math Library Functions . . . 107
8.5.1 32-Bit Fixed-Point Basic Arithmetic Operations . . . 107
8.5.2 Exponential Averaging Filter . . . 107
8.5.3 Dual-Time Constant Attack-Release Averaging Filter . . . 107

8.5.4 Linear Interpolation . . . 107
8.5.5 Single-Variable Linear Regression . . . 108
9. EEPROM Library . . . 109
9.1 Description . . . 109
9.2 EEPROM Library Usage . . . 109
9.2.1 SPI Interface Configuration . . . 110
9.2.2 EEPROM Library Addressing Mode. . . . 110
9.2.3 EEPROM Library Naming Conventions . . . 110
9.3 Write Protection . . . 111
9.4 Word Sizes . . . 111
9.5 Page Sizes . . . 111
9.6 Cyclic Redundancy Check (CRC) . . . 111
9.7 EEPROM Library Functions . . . 112
9.8 EEPROM Library Operation . . . 114
10. EEPROM File System . . . 117
10.1 Description . . . 117
10.2 Version . . . 117
10.3 Word Size and Addressing . . . 117
10.4 File System Integrity . . . 118
10.5 Security . . . 119
10.5.1 Level 1 Security. . . . 119
10.5.2 Level 2 Security. . . . 119
10.6 File System Components . . . 119
10.7 System Area . . . 121
10.7.1 Boot Variables . . . 122
10.7.2 File System Version . . . 123
10.7.3 Application Variables . . . 123
10.7.4 System Area Extension Vector . . . 123
10.8 Manufacturing Area . . . 123
10.8.1 Clock Calibration . . . 124
10.8.2 ADC Offset Calibration . . . 125
10.8.3 VDDC and VDDM Calibration . . . 125
10.8.4 VREG, VDDA and VDBL Calibration. . . . 126
10.8.5 Manufacturing Data Area . . . 126
10.8.6 Manufacturing Area Extension Vector . . . 126
10.9 Key Area . . . 126
10.10 Bootloader . . . 127
10.11 Applications . . . 127
10.11.1 Application Files . . . 128
10.11.2 Application Execution . . . 129
10.11.3 Sections . . . 129
10.11.4 Volatile Sections . . . 129
10.11.5 Encrypted Sections . . . 129
11. Calibration Library . . . 131
11.1 Introduction . . . 131

11.2 Power Supply Calibration . . . 131
11.3 Clock Calibration . . . 132
11.4 ADC Calibration . . . 133
11.5 Storing Calibration Settings . . . 134
11.6 Calibration Library Errors . . . 136
12. Program ROM . . . 138
12.1 Description . . . 138
12.1.1 Versions . . . 138
12.2 Boot ROM . . . 138
12.2.1 Boot Initialization . . . 138
12.2.2 Stabilization Delay . . . 139
12.2.3 Loading the Bootloader . . . 139
12.2.4 Boot Failures. . . . 139
12.3 Function Tables . . . 139
12.4 File System Decryption Support . . . 142
12.4.1 Advanced Encryption Standard (AES) Functions . . . 142
12.4.1.1 Memory, Register and Other Resource Usage by AES Routines. . . . 142
12.4.2 Encryption Blocks . . . 143
13. Bootloader . . . 145
13.1 Description . . . 145
13.2 Installation . . . 145
13.3 Bootloader Version . . . 145
13.4 Memory Usage . . . 145
13.5 Operation . . . 145
13.6 Bootloader Errors . . . 146
14. CFX System Macro Reference . . . 147
14.1 Sys_ACB_Clear_State . . . 147
14.2 Sys_ACB_DisableAll . . . 148
14.3 Sys_ACB_Int_Ack . . . 149
14.4 Sys_ACB_Int_Enable . . . 150
14.5 Sys_AudioMux_Config . . . 151
14.6 Sys_BootApp . . . 152
14.7 Sys_C_I2C_ACK . . . 153
14.8 Sys_C_I2C_Config . . . 154
14.9 Sys_C_I2C_NACK . . . 155
14.10 Sys_C_I2C_NACK_And_Stop . . . 156
14.11 Sys_C_I2C_Reset . . . 157
14.12 Sys_C_I2C_Slave_Disable . . . 158
14.13 Sys_C_I2C_Slave_Enable . . . 159
14.14 Sys_C_I2C_Start_Read . . . 160
14.15 Sys_C_I2C_Start_Write . . . 161
14.16 Sys_C_PCM_Config . . . 162
14.17 Sys_C_PCM_Disable . . . 163
14.18 Sys_C_PCM_Enable . . . 164
14.19 Sys_C_SPI_CS_High . . . 165
14.20 Sys_C_SPI_CS_Low . . . 166
14.21 Sys_C_SPI_Disable . . . 167

14.22 Sys_C_SPI_Enable . . . 168
14.23 Sys_C_SPI_Read . . . 169
14.24 Sys_C_SPI_ReadWrite . . . 170
14.25 Sys_C_SPI_WaitDone . . . 171
14.26 Sys_C_SPI_Write . . . 172
14.27 Sys_Calc_Gain_Factor_Val . . . 173
14.28 Sys_Calc_Sample_Freq_Val . . . 174
14.29 Sys_CFX_BiasDisable . . . 175
14.30 Sys_CFX_BiasEnable . . . 176
14.31 Sys_CFX_CircAddrConfig . . . 177
14.32 Sys_CFX_FlagUpdateDisable . . . 178
14.33 Sys_CFX_FlagUpdateEnable . . . 179
14.34 Sys_CFX_InitLoopStack . . . 180
14.35 Sys_CFX_InitStack . . . 181
14.36 Sys_CFX_IntDisable . . . 182
14.37 Sys_CFX_IntEnable . . . 183
14.38 Sys_CFX_RoundDisable . . . 184
14.39 Sys_CFX_RoundEnable . . . 185
14.40 Sys_CFX_SaturateDisable . . . 186
14.41 Sys_CFX_SaturateEnable . . . 187
14.42 Sys_CFX_SRConfig . . . 188
14.43 Sys_CFX_StepConfig . . . 189
14.44 Sys_Clk_Config . . . 190
14.45 Sys_Clk_Config0 . . . 191
14.46 Sys_Clk_Config1 . . . 192
14.47 Sys_Clk_MultThrottleConfig . . . 193
14.48 Sys_Clk_Throttle_Config . . . 194
14.49 Sys_Clk_Throttle_MultConfig . . . 195
14.50 Sys_CM3_Command . . . 196
14.51 Sys_CM3_Pause . . . 197
14.52 Sys_CM3_Reset . . . 198
14.53 Sys_CM3_Start . . . 199
14.54 Sys_CM3AndSubsystem_Reset . . . 200
14.55 Sys_CVSD_Decoder_Clear_State . . . 201
14.56 Sys_CVSD_Decoder_Config . . . 202
14.57 Sys_CVSD_Encoder_Clear_State . . . 203
14.58 Sys_CVSD_Encoder_Config . . . 204
14.59 Sys_Data_FENG_Coefficients . . . 205
14.60 Sys_Data_fl8n16 . . . 206
14.61 Sys_Data_frac24 . . . 207
14.62 Sys_Data_frac32 . . . 208
14.63 Sys_Data_frac48 . . . 209
14.64 Sys_Data_m24p24 . . . 210
14.65 Sys_Data_m2p22 . . . 211
14.66 Sys_Data_m2p46 . . . 212
14.67 Sys_Data_m8p16 . . . 213
14.68 Sys_DebugPort_Get_AudioMode . . . 214
14.69 Sys_DebugPort_Set_AccessMode . . . 215
14.70 Sys_DebugPort_Set_Address . . . 216

14.71 Sys_DebugPort_Set_CM3Address . . . 217
14.72 Sys_DebugPort_Set_ExternalAddress . . . 218
14.73 Sys_DIO_Config . . . 219
14.74 Sys_DIO_Config_C_I2C . . . 220
14.75 Sys_DIO_Config_C_PCM . . . 221
14.76 Sys_DIO_Config_C_SPI . . . 222
14.77 Sys_DIO_Config_DAC_SCALE . . . 223
14.78 Sys_DIO_Config_DMICInput . . . 224
14.79 Sys_DIO_Config_PCM . . . 225
14.80 Sys_DIO_Config_PLL . . . 226
14.81 Sys_DIO_Config_SPI . . . 227
14.82 Sys_DIO_Config_SYNC_SLAVE . . . 228
14.83 Sys_DIO_Config_UART . . . 229
14.84 Sys_DIO_Get_Mode . . . 230
14.85 Sys_ECB_Int_Ack . . . 231
14.86 Sys_ECB_Int_Enable . . . 232
14.87 Sys_FENG_Config . . . 233
14.88 Sys_FENG_DelayedPause . . . 234
14.89 Sys_FENG_Execute . . . 235
14.90 Sys_FENG_IntAck . . . 236
14.91 Sys_FENG_Pause . . . 237
14.92 Sys_FENG_Reset . . . 238
14.93 Sys_FENG_SingleStep . . . 239
14.94 Sys_FENG_Start . . . 240
14.95 Sys_FIFO_AutoMute . . . 241
14.96 Sys_FIFO_CFXIntConfig . . . 242
14.97 Sys_FIFO_CM3IntConfig . . . 243
14.98 Sys_FIFO_Configure . . . 244
14.99 Sys_FIFO_HEARIntConfig . . . 245
14.100 Sys_fl8n16 . . . 246
14.101 Sys_frac24 . . . 247
14.102 Sys_frac32 . . . 248
14.103 Sys_frac48 . . . 249
14.104 Sys_G722_Decoder_Clear_State . . . 250
14.105 Sys_G722_Decoder_Conceal . . . 251
14.106 Sys_G722_Decoder_Config . . . 252
14.107 Sys_G722_Encoder_Clear_State . . . 253
14.108 Sys_G722_Encoder_Config . . . 254
14.109 Sys_Get_ADC_CFG . . . 255
14.110 Sys_Get_ADC_CTRL . . . 256
14.111 Sys_Get_ADC_DEC_CTRL . . . 257
14.112 Sys_Get_ADC_SAMPLE_FREQ_CFG . . . 258
14.113 Sys_Get_AppNum . . . 259
14.114 Sys_Get_ChipDebugVersion . . . 260
14.115 Sys_Get_ChipFamily . . . 261
14.116 Sys_Get_ChipRevision . . . 262
14.117 Sys_Get_ChipVersion . . . 263
14.118 Sys_Get_CM3ChipDebugVersion . . . 264
14.119 Sys_Get_CP_VDBL_CTRL . . . 265

14.120 Sys_Get_CP_VDDA_CTRL . . . 266
14.121 Sys_Get_DMIC_OUTPUT_CTRL . . . 267
14.122 Sys_Get_FENG_CoeffOffset . . . 268
14.123 Sys_Get_FENG_CoeffOffsetEnable . . . 269
14.124 Sys_Get_FENG_IntStatus . . . 270
14.125 Sys_Get_FENG_State . . . 271
14.126 Sys_Get_FENG_StateWriteEnable . . . 272
14.127 Sys_Get_HEAR_BFP_BLOCK_EXP . . . 273
14.128 Sys_Get_IDRX_CFG . . . 274
14.129 Sys_Get_LSAD_CTRL . . . 275
14.130 Sys_Get_O_DAC_AO_CTRL . . . 276
14.131 Sys_Get_O_DAC_GF_CTRL . . . 277
14.132 Sys_Get_O_SDM_CTRL . . . 278
14.133 Sys_Get_O_SDMCLK . . . 279
14.134 Sys_Get_OSC_CTRL_0 . . . 280
14.135 Sys_Get_OSC_CTRL_1 . . . 281
14.136 Sys_Get_OSCCompOut . . . 282
14.137 Sys_Get_OUTPUT_CTRL . . . 283
14.138 Sys_Get_PLL_CalibrationOutput . . . 284
14.139 Sys_Get_PLL_ClockSync . . . 285
14.140 Sys_Get_PLL_DigitalCal . . . 286
14.141 Sys_Get_PLL_LoopFilterAndVCOGain . . . 287
14.142 Sys_Get_PLL_RefClkDetectConfig . . . 288
14.143 Sys_Get_PLL_TempCompensation . . . 289
14.144 Sys_Get_SDU_CFG . . . 290
14.145 Sys_Get_SUP_COUNT_DATA . . . 291
14.146 Sys_Get_SUP_THRSH_DATA . . . 292
14.147 Sys_GPIO_CM3IntConfig . . . 293
14.148 Sys_GPIO_Direction . . . 294
14.149 Sys_GPIO_Get_CM3IntStatus . . . 295
14.150 Sys_GPIO_Get_Data . . . 296
14.151 Sys_GPIO_Get_Data_1 . . . 297
14.152 Sys_GPIO_Get_Data_2 . . . 298
14.153 Sys_GPIO_Get_Data_3 . . . 299
14.154 Sys_GPIO_Get_IntStatus . . . 300
14.155 Sys_GPIO_IntConfig . . . 301
14.156 Sys_GPIO_Set_Data . . . 302
14.157 Sys_GPIO_Set_Data_1 . . . 303
14.158 Sys_GPIO_Set_Data_2 . . . 304
14.159 Sys_GPIO_Set_Data_3 . . . 305
14.160 Sys_GPIO_Set_High . . . 306
14.161 Sys_GPIO_Set_Low . . . 307
14.162 Sys_GPIO_Toggle . . . 308
14.163 Sys_GPIO_Toggle_Bits . . . 309
14.164 Sys_GPIO_Toggle_Bits_1 . . . 310
14.165 Sys_GPIO_Toggle_Bits_2 . . . 311
14.166 Sys_GPIO_Toggle_Bits_3 . . . 312
14.167 Sys_HEAR_Command . . . 313
14.168 Sys_HEAR_FunctionChain_Enable . . . 314

14.169 Sys_HEAR_FunctionChain_Priority . . . 315
14.170 Sys_HEAR_Pause . . . 316
14.171 Sys_HEAR_Reset . . . 317
14.172 Sys_HEAR_RunTime_Addr . . . 318
14.173 Sys_HEAR_Start . . . 319
14.174 Sys_HMI_Disable . . . 320
14.175 Sys_HMI_Encoder_Config . . . 321
14.176 Sys_I2C_ACK . . . 322
14.177 Sys_I2C_Config . . . 323
14.178 Sys_I2C_NACK . . . 324
14.179 Sys_I2C_NACK_And_Stop . . . 325
14.180 Sys_I2C_Pin_Config . . . 326
14.181 Sys_I2C_Reset . . . 327
14.182 Sys_I2C_Slave_Disable . . . 328
14.183 Sys_I2C_Slave_Enable . . . 329
14.184 Sys_I2C_Start_Read . . . 330
14.185 Sys_I2C_Start_Write . . . 331
14.186 Sys_I2C_Watchdog_Config . . . 332
14.187 Sys_IHM_Decoder_Clear_Status . . . 333
14.188 Sys_IHM_Decoder_Config . . . 334
14.189 Sys_IMM_n . . . 335
14.190 Sys_Int_Ack . . . 336
14.191 Sys_Int_Enable . . . 337
14.192 Sys_Int_Master_Disable . . . 338
14.193 Sys_Int_Master_Enable . . . 339
14.194 Sys_Int_MuxConfig . . . 340
14.195 Sys_Int_Priority . . . 341
14.196 Sys_IOC_ADCInputConfig . . . 342
14.197 Sys_IOC_FIFOAccessConfig . . . 343
14.198 Sys_IOC_FIFOShiftConfig . . . 344
14.199 Sys_IOC_OutputConfig . . . 345
14.200 Sys_IOC_PCMInputConfig . . . 346
14.201 Sys_Log2 . . . 347
14.202 Sys_Log2_Ceil . . . 348
14.203 Sys_Log2_Floor . . . 349
14.204 Sys_m24p24 . . . 350
14.205 Sys_m2p22 . . . 351
14.206 Sys_m2p46 . . . 352
14.207 Sys_m8p16 . . . 353
14.208 Sys_Mem_Config . . . 354
14.209 Sys_Mem_CopyPP . . . 355
14.210 Sys_Mem_CopyPX . . . 356
14.211 Sys_Mem_CopyPY . . . 357
14.212 Sys_Mem_CopyXP . . . 358
14.213 Sys_Mem_CopyXX . . . 359
14.214 Sys_Mem_CopyXY . . . 360
14.215 Sys_Mem_CopyXY_Quick . . . 361
14.216 Sys_Mem_CopyYP . . . 362
14.217 Sys_Mem_CopyYX . . . 363

14.218 Sys_Mem_CopyYX_Quick . . . 364
14.219 Sys_Mem_CopyYY . . . 365
14.220 Sys_Mem_FillP . . . 366
14.221 Sys_Mem_FillX . . . 367
14.222 Sys_Mem_FillY . . . 368
14.223 Sys_Mem_Priority . . . 369
14.224 Sys_Mem_Priority2 . . . 370
14.225 Sys_Mem_Reserve . . . 371
14.226 Sys_Mem_Reserve_Init . . . 372
14.227 Sys_Mem_Reserve_Init_Default . . . 373
14.228 Sys_Mem_Start_Absolute_Segment . . . 374
14.229 Sys_Mem_Start_Reloc_Segment . . . 375
14.230 Sys_Mem_Start_Segment . . . 376
14.231 Sys_PCM_Config . . . 377
14.232 Sys_PCM_Disable . . . 378
14.233 Sys_PCM_Enable . . . 379
14.234 Sys_PLL_ClearStatus . . . 380
14.235 Sys_PLL_ConfigLock . . . 381
14.236 Sys_PLL_GetStatus . . . 382
14.237 Sys_PLL_WaitLock . . . 383
14.238 Sys_Pop_Reg_1 . . . 384
14.239 Sys_Pop_Reg_10 . . . 385
14.240 Sys_Pop_Reg_2 . . . 386
14.241 Sys_Pop_Reg_3 . . . 387
14.242 Sys_Pop_Reg_4 . . . 388
14.243 Sys_Pop_Reg_5 . . . 389
14.244 Sys_Pop_Reg_6 . . . 390
14.245 Sys_Pop_Reg_7 . . . 391
14.246 Sys_Pop_Reg_8 . . . 392
14.247 Sys_Pop_Reg_9 . . . 393
14.248 Sys_Pop_Reg_Long_1 . . . 394
14.249 Sys_Pop_Reg_Long_10 . . . 395
14.250 Sys_Pop_Reg_Long_2 . . . 396
14.251 Sys_Pop_Reg_Long_3 . . . 397
14.252 Sys_Pop_Reg_Long_4 . . . 398
14.253 Sys_Pop_Reg_Long_5 . . . 399
14.254 Sys_Pop_Reg_Long_6 . . . 400
14.255 Sys_Pop_Reg_Long_7 . . . 401
14.256 Sys_Pop_Reg_Long_8 . . . 402
14.257 Sys_Pop_Reg_Long_9 . . . 403
14.258 Sys_PopLoopsFromStack . . . 404
14.259 Sys_Power2 . . . 405
14.260 Sys_Push_Reg_1 . . . 406
14.261 Sys_Push_Reg_10 . . . 407
14.262 Sys_Push_Reg_2 . . . 408
14.263 Sys_Push_Reg_3 . . . 409
14.264 Sys_Push_Reg_4 . . . 410
14.265 Sys_Push_Reg_5 . . . 411
14.266 Sys_Push_Reg_6 . . . 412

14.267 Sys_Push_Reg_7 . . . 413
14.268 Sys_Push_Reg_8 . . . 414
14.269 Sys_Push_Reg_9 . . . 415
14.270 Sys_Push_Reg_Long_1 . . . 416
14.271 Sys_Push_Reg_Long_10 . . . 417
14.272 Sys_Push_Reg_Long_2 . . . 418
14.273 Sys_Push_Reg_Long_3 . . . 419
14.274 Sys_Push_Reg_Long_4 . . . 420
14.275 Sys_Push_Reg_Long_5 . . . 421
14.276 Sys_Push_Reg_Long_6 . . . 422
14.277 Sys_Push_Reg_Long_7 . . . 423
14.278 Sys_Push_Reg_Long_8 . . . 424
14.279 Sys_Push_Reg_Long_9 . . . 425
14.280 Sys_PushLoopsToStack . . . 426
14.281 Sys_Reg_fl8n16 . . . 427
14.282 Sys_Reg_frac24 . . . 428
14.283 Sys_Reg_frac32 . . . 429
14.284 Sys_Reg_frac48 . . . 430
14.285 Sys_Reg_m24p24 . . . 431
14.286 Sys_Reg_m2p22 . . . 432
14.287 Sys_Reg_m2p46 . . . 433
14.288 Sys_Reg_m8p16 . . . 434
14.289 Sys_Restore_HEAR_BFP . . . 435
14.290 Sys_RS_Clear_State . . . 436
14.291 Sys_RS_Decoder_Config . . . 437
14.292 Sys_RS_Encoder_Config . . . 438
14.293 Sys_Save_HEAR_BFP . . . 439
14.294 Sys_Set_ADC_CFG . . . 440
14.295 Sys_Set_ADC_CTRL . . . 441
14.296 Sys_Set_ADC_CTRL_NoCalibration . . . 442
14.297 Sys_Set_ADC_DEC_CTRL . . . 443
14.298 Sys_Set_ADC_SAMPLE_FREQ_CFG . . . 444
14.299 Sys_Set_Clk_ThrottleConfigMode . . . 445
14.300 Sys_Set_CP_VDBL_CTRL . . . 446
14.301 Sys_Set_CP_VDDA_CTRL . . . 447
14.302 Sys_Set_DMIC_OUTPUT_CTRL . . . 448
14.303 Sys_Set_FENG_Coefficients . . . 449
14.304 Sys_Set_FENG_CoeffOffset . . . 450
14.305 Sys_Set_FENG_CoeffOffsetEnable_Active . . . 451
14.306 Sys_Set_FENG_CoeffOffsetEnable_Shadow . . . 452
14.307 Sys_Set_FENG_IntEnable . . . 453
14.308 Sys_Set_FENG_MRCounterConfig . . . 454
14.309 Sys_Set_FENG_State . . . 455
14.310 Sys_Set_FENG_State_Shadow . . . 456
14.311 Sys_Set_FENG_StateWriteEnable . . . 457
14.312 Sys_Set_HEAR_BFP . . . 458
14.313 Sys_Set_HEAR_FunctionChain_Vector . . . 459
14.314 Sys_Set_IDRX_CFG . . . 460
14.315 Sys_Set_LSAD_CTRL . . . 461

14.316 Sys_Set_MultiSyncConfig . . . 462
14.317 Sys_Set_O_DAC_AO_CTRL . . . 463
14.318 Sys_Set_O_DAC_GF_CTRL . . . 464
14.319 Sys_Set_O_SDM_CTRL . . . 465
14.320 Sys_Set_O_SDMCLK . . . 466
14.321 Sys_Set_OSC_CTRL . . . 467
14.322 Sys_Set_OSC_CTRL_0 . . . 468
14.323 Sys_Set_OSC_CTRL_1 . . . 469
14.324 Sys_Set_OSCComp . . . 470
14.325 Sys_Set_OUTPUT_CTRL . . . 471
14.326 Sys_Set_PLL_ClockSync . . . 472
14.327 Sys_Set_PLL_DigitalCal . . . 473
14.328 Sys_Set_PLL_Frequency . . . 474
14.329 Sys_Set_PLL_LockInterrupt . . . 475
14.330 Sys_Set_PLL_LoopFilterAndVCOGain . . . 476
14.331 Sys_Set_PLL_ModeControl . . . 477
14.332 Sys_Set_PLL_RefClkDetectConfig . . . 478
14.333 Sys_Set_PLL_TempCompensation . . . 479
14.334 Sys_Set_SDU_CFG . . . 480
14.335 Sys_Set_SUP_THRSH_DATA . . . 481
14.336 Sys_Set_VDBL_CPClk . . . 482
14.337 Sys_Set_VDDA_CPClk . . . 483
14.338 Sys_Set_VDDC_CPClk . . . 484
14.339 Sys_Set_VDDM_CPClk . . . 485
14.340 Sys_Set_VMIC_SW_CTRL . . . 486
14.341 Sys_SetReg_ADC_CFG . . . 487
14.342 Sys_SetReg_ADC_CTRL . . . 488
14.343 Sys_SetReg_ADC_CTRL_NoCalibration . . . 489
14.344 Sys_SetReg_ADC_DEC_CTRL . . . 490
14.345 Sys_SetReg_ADC_SAMPLE_FREQ_CFG . . . 491
14.346 Sys_SetReg_CP_VDBL_CTRL . . . 492
14.347 Sys_SetReg_CP_VDDA_CTRL . . . 493
14.348 Sys_SetReg_DMIC_OUTPUT_CTRL . . . 494
14.349 Sys_SetReg_FENG_CoeffOffset . . . 495
14.350 Sys_SetReg_FENG_CoeffOffsetEnable_Active . . . 496
14.351 Sys_SetReg_FENG_CoeffOffsetEnable_Shadow . . . 497
14.352 Sys_SetReg_FENG_IntEnable . . . 498
14.353 Sys_SetReg_FENG_State . . . 499
14.354 Sys_SetReg_FENG_State_Shadow . . . 500
14.355 Sys_SetReg_FENG_StateWriteEnable . . . 501
14.356 Sys_SetReg_HEAR_BFP_BLOCK_EXP . . . 502
14.357 Sys_SetReg_IDRX_CFG . . . 503
14.358 Sys_SetReg_LSAD_CTRL . . . 504
14.359 Sys_SetReg_O_DAC_AO_CTRL . . . 505
14.360 Sys_SetReg_O_DAC_GF_CTRL . . . 506
14.361 Sys_SetReg_O_SDM_CTRL . . . 507
14.362 Sys_SetReg_O_SDMCLK . . . 508
14.363 Sys_SetReg_OSC_CTRL_0 . . . 509
14.364 Sys_SetReg_OSC_CTRL_1 . . . 510

14.365 Sys_SetReg_OUTPUT_CTRL . . . 511
14.366 Sys_SetReg_PLL_DigitalCal . . . 512
14.367 Sys_SetReg_PLL_LoopFilterAndVCOGain . . . 513
14.368 Sys_SetReg_PLL_RefClkDetectConfig . . . 514
14.369 Sys_SetReg_PLL_TempCompensation . . . 515
14.370 Sys_SetReg_SDU_CFG . . . 516
14.371 Sys_SetReg_SUP_THRSH_DATA . . . 517
14.372 Sys_SPI_CS_High . . . 518
14.373 Sys_SPI_CS_Low . . . 519
14.374 Sys_SPI_Disable . . . 520
14.375 Sys_SPI_Enable . . . 521
14.376 Sys_SPI_Read . . . 522
14.377 Sys_SPI_WaitDone . . . 523
14.378 Sys_SPI_Write . . . 524
14.379 Sys_Stack_Alloc . . . 525
14.380 Sys_Stack_Free . . . 526
14.381 Sys_Timer1_Config . . . 527
14.382 Sys_Timer1_Start . . . 528
14.383 Sys_Timer1_Stop . . . 529
14.384 Sys_Timer2_Config . . . 530
14.385 Sys_Timer2_Start . . . 531
14.386 Sys_Timer2_Stop . . . 532
14.387 Sys_Timers_Start . . . 533
14.388 Sys_Timers_Stop . . . 534
14.389 Sys_UART_Disable . . . 535
14.390 Sys_UART_Enable . . . 536
14.391 Sys_Value_XY . . . 537
14.392 Sys_Watchdog_Refresh . . . 538
14.393 Sys_Watchdog_TimeoutConfig . . . 539

15. ARM Cortex-M3 Header Files and System Library Reference . . . 540
15.1 Sys_ACB_Clear_State . . . 540
15.2 Sys_ACB_DisableAll . . . 541
15.3 Sys_ACB_Int_Ack . . . 542
15.4 Sys_ACB_Int_Enable . . . 543
15.5 SYS_CALC_BITBAND . . . 544
15.6 Sys_CFX_Command . . . 545
15.7 Sys_CM3_Delay . . . 546
15.8 Sys_CVSD_Decoder_Clear_State . . . 547
15.9 Sys_CVSD_Decoder_Config . . . 548
15.10 Sys_CVSD_Encoder_Clear_State . . . 549
15.11 Sys_CVSD_Encoder_Config . . . 550
15.12 Sys_DIO_Get_IntStatus . . . 551
15.13 Sys_DIO_Get_Mode . . . 552
15.14 Sys_DIO_IntConfig . . . 553
15.15 Sys_DIO_Set_Direction . . . 554
15.16 Sys_DMA_Channel0_Config . . . 555
15.17 Sys_DMA_Channel0_Disable . . . 557
15.18 Sys_DMA_Channel0_Enable . . . 558
15.19 Sys_DMA_Channel1_Config . . . 559

15.20 Sys_DMA_Channel1_Disable . . . 561
15.21 Sys_DMA_Channel1_Enable . . . 562
15.22 Sys_DMA_Channel2_Config . . . 563
15.23 Sys_DMA_Channel2_Disable . . . 565
15.24 Sys_DMA_Channel2_Enable . . . 566
15.25 Sys_DMA_Channel3_Config . . . 567
15.26 Sys_DMA_Channel3_Disable . . . 569
15.27 Sys_DMA_Channel3_Enable . . . 570
15.28 Sys_DMA_Clear_Channel0_Status . . . 571
15.29 Sys_DMA_Clear_Channel1_Status . . . 572
15.30 Sys_DMA_Clear_Channel2_Status . . . 573
15.31 Sys_DMA_Clear_Channel3_Status . . . 574
15.32 Sys_DMA_Clear_Status . . . 575
15.33 Sys_DMA_Get_Channel0_Status . . . 576
15.34 Sys_DMA_Get_Channel1_Status . . . 577
15.35 Sys_DMA_Get_Channel2_Status . . . 578
15.36 Sys_DMA_Get_Channel3_Status . . . 579
15.37 Sys_DMA_Set_Channel0_DestAddress . . . 580
15.38 Sys_DMA_Set_Channel0_SourceAddress . . . 581
15.39 Sys_DMA_Set_Channel1_DestAddress . . . 582
15.40 Sys_DMA_Set_Channel1_SourceAddress . . . 583
15.41 Sys_DMA_Set_Channel2_DestAddress . . . 584
15.42 Sys_DMA_Set_Channel2_SourceAddress . . . 585
15.43 Sys_DMA_Set_Channel3_DestAddress . . . 586
15.44 Sys_DMA_Set_Channel3_SourceAddress . . . 587
15.45 Sys_ECB_Int_Ack . . . 588
15.46 Sys_ECB_Int_Enable . . . 589
15.47 Sys_G722_Decoder_Clear_State . . . 590
15.48 Sys_G722_Decoder_Conceal . . . 591
15.49 Sys_G722_Decoder_Config . . . 592
15.50 Sys_G722_Encoder_Clear_State . . . 593
15.51 Sys_G722_Encoder_Config . . . 594
15.52 Sys_G722_ExtendFrom48 . . . 595
15.53 Sys_G722_ExtendFrom56 . . . 596
15.54 Sys_G722_TruncateTo48 . . . 597
15.55 Sys_G722_TruncateTo56 . . . 598
15.56 Sys_Get_CRC_Config . . . 599
15.57 Sys_GPIO_Set_High . . . 600
15.58 Sys_GPIO_Set_Low . . . 601
15.59 Sys_GPIO_Toggle . . . 602
15.60 Sys_HMI_Disable . . . 603
15.61 Sys_HMI_Encoder_Config . . . 604
15.62 Sys_I2C_ACK . . . 605
15.63 Sys_I2C_Config . . . 606
15.64 Sys_I2C_Get_Status . . . 607
15.65 Sys_I2C_NACK . . . 608
15.66 Sys_I2C_NACKAndStop . . . 609
15.67 Sys_I2C_Reset . . . 610
15.68 Sys_I2C_StartRead . . . 611

15.69 Sys_I2C_StartWrite . . . 612
15.70 Sys_IHM_Decoder_Clear_Status . . . 613
15.71 Sys_IHM_Decoder_Config . . . 614
15.72 Sys_NVIC_ClearAllPendingInt . . . 615
15.73 Sys_NVIC_DisableAllInt . . . 616
15.74 Sys_PCM_Config . . . 617
15.75 Sys_PCM_Disable . . . 618
15.76 Sys_PCM_Enable . . . 619
15.77 Sys_RS_Decoder_Clear_State . . . 620
15.78 Sys_RS_Decoder_Config . . . 621
15.79 Sys_RS_Decoder_GetOutput . . . 622
15.80 Sys_RS_Decoder_SetInput . . . 623
15.81 Sys_RS_Encoder_Clear_State . . . 624
15.82 Sys_RS_Encoder_Config . . . 625
15.83 Sys_RS_Encoder_GetOutput . . . 626
15.84 Sys_RS_Encoder_SetInput . . . 627
15.85 Sys_Set_CRC_Config . . . 628
15.86 Sys_SPI_Config . . . 629
15.87 Sys_SPI_MasterInit . . . 630
15.88 Sys_SPI_Read . . . 631
15.89 Sys_SPI_ReadWrite . . . 632
15.90 Sys_SPI_TransferConfig . . . 633
15.91 Sys_SPI_Write . . . 634
15.92 SYS_WAIT_FOR_EVENT . . . 635
15.93 SYS_WAIT_FOR_INTERRUPT . . . 636
16. CFX System Library Reference . . . 637
16.1 Sys_CalibrateADC . . . 637
16.2 Sys_CalibrateADCs . . . 638
16.3 Sys_Delay . . . 639
16.4 Sys_DelayReg . . . 640
16.5 Sys_Reset . . . 641
16.6 Sys_SetOperatingFrequency . . . 642
16.7 Sys_SetOperatingFrequency_Throttle . . . 643
16.8 Sys_SoftPowerdownDisable . . . 644
16.9 Sys_SoftPowerdownEnable . . . 645
16.10 Sys_WaitForSPIDone . . . 646
16.11 SYSLIB_CalibrateADC . . . 647
16.12 SYSLIB_Delay . . . 648
16.13 SYSLIB_Reset . . . 649
16.14 SYSLIB_SetOperatingFrequency . . . 650
16.15 SYSLIB_SetOperatingFrequency_Throttle . . . 651
16.16 SYSLIB_SoftPowerdownDisable . . . 652
16.17 SYSLIB_SoftPowerdownEnable . . . 653
16.18 SYSLIB_WaitForSPIDone . . . 654
17. CFX Math Library Macro Reference . . . 655
17.1 MATHLIB_ATTACK_RELEASE . . . 655
17.2 MATHLIB_ATTACK_RELEASE_VEC . . . 656
17.3 MATHLIB_CONV_fl8n16_frac48 . . . 657
17.4 MATHLIB_CONV_fl8n16_m24p24 . . . 658

17.5 MATHLIB_CONV_fl8n16_m8p16 . . . 659
17.6 MATHLIB_CONV_frac24_fl8n16 . . . 660
17.7 MATHLIB_CONV_frac48_fl8n16 . . . 661
17.8 MATHLIB_CONV_int24_fl8n16 . . . 662
17.9 MATHLIB_CONV_m24p24_fl8n16 . . . 663
17.10 MATHLIB_CONV_m8p16_fl8n16 . . . 664
17.11 MATHLIB_CONV_VEC_fl8n16_frac48 . . . 665
17.12 MATHLIB_CONV_VEC_fl8n16_m24p24 . . . 666
17.13 MATHLIB_CONV_VEC_fl8n16_m8p16 . . . 667
17.14 MATHLIB_CONV_VEC_frac24_fl8n16 . . . 668
17.15 MATHLIB_CONV_VEC_frac48_fl8n16 . . . 669
17.16 MATHLIB_CONV_VEC_int24_fl8n16 . . . 670
17.17 MATHLIB_CONV_VEC_m24p24_fl8n16 . . . 671
17.18 MATHLIB_CONV_VEC_m8p16_fl8n16 . . . 672
17.19 MATHLIB_COS_frac24_frac24 . . . 673
17.20 MATHLIB_COS_VEC_frac24_frac24 . . . 674
17.21 MATHLIB_DB10_PWR_m8p16_frac48 . . . 675
17.22 MATHLIB_DB10_PWR_VEC_m8p16_frac48 . . . 676
17.23 MATHLIB_DB10_RMS_m8p16_frac48 . . . 677
17.24 MATHLIB_DB10_RMS_VEC_m8p16_frac48 . . . 678
17.25 MATHLIB_DB2_PWR_m8p16_frac48 . . . 679
17.26 MATHLIB_DB2_PWR_VEC_m8p16_frac48 . . . 680
17.27 MATHLIB_DB2_RMS_m8p16_frac48 . . . 681
17.28 MATHLIB_DB2_RMS_VEC_m8p16_frac48 . . . 682
17.29 MATHLIB_DIVIDE_frac24 . . . 683
17.30 MATHLIB_DIVIDE_int24 . . . 684
17.31 MATHLIB_DIVIDE_m24p24 . . . 685
17.32 MATHLIB_EXP2_m8p16_frac48 . . . 686
17.33 MATHLIB_EXP2_m8p16_m8p16 . . . 687
17.34 MATHLIB_EXP2_VEC_m8p16_frac48 . . . 688
17.35 MATHLIB_EXP2_VEC_m8p16_m8p16 . . . 689
17.36 MATHLIB_EXP_AVG . . . 690
17.37 MATHLIB_EXP_AVG_VEC . . . 691
17.38 MATHLIB_LIN_INTERP . . . 692
17.39 MATHLIB_LOG2_fl8n16_m8p16 . . . 693
17.40 MATHLIB_LOG2_frac24_m8p16 . . . 694
17.41 MATHLIB_LOG2_frac48_m8p16 . . . 695
17.42 MATHLIB_LOG2_m8p16_m8p16 . . . 696
17.43 MATHLIB_LOG2_VEC_fl8n16_m8p16 . . . 697
17.44 MATHLIB_LOG2_VEC_frac24_m8p16 . . . 698
17.45 MATHLIB_LOG2_VEC_frac48_m8p16 . . . 699
17.46 MATHLIB_LOG2_VEC_m8p16_m8p16 . . . 700
17.47 MATHLIB_MAG_frac24_m2p46 . . . 701
17.48 MATHLIB_MAG_VEC_frac24_m2p46 . . . 702
17.49 MATHLIB_PWR_DB10_frac48_m8p16 . . . 703
17.50 MATHLIB_PWR_DB10_VEC_frac48_m8p16 . . . 704
17.51 MATHLIB_PWR_DB2_frac48_m8p16 . . . 705
17.52 MATHLIB_PWR_DB2_VEC_frac48_m8p16 . . . 706
17.53 MATHLIB_RECIP_fl8n16_fl8n16 . . . 707

17.54 MATHLIB_RECIP_frac48_fl8n16 . . . 708
17.55 MATHLIB_RECIP_int24_fl8n16 . . . 709
17.56 MATHLIB_RECIP_m8p16_fl8n16 . . . 710
17.57 MATHLIB_RECIP_POS_fl8n16_fl8n16 . . . 711
17.58 MATHLIB_RECIP_POS_frac48_fl8n16 . . . 712
17.59 MATHLIB_RECIP_POS_int24_fl8n16 . . . 713
17.60 MATHLIB_RECIP_POS_m8p16_fl8n16 . . . 714
17.61 MATHLIB_RECIP_POS_VEC_fl8n16_fl8n16 . . . 715
17.62 MATHLIB_RECIP_POS_VEC_frac48_fl8n16 . . . 716
17.63 MATHLIB_RECIP_POS_VEC_int24_fl8n16 . . . 717
17.64 MATHLIB_RECIP_POS_VEC_m8p16_fl8n16 . . . 718
17.65 MATHLIB_RECIP_VEC_fl8n16_fl8n16 . . . 719
17.66 MATHLIB_RECIP_VEC_frac48_fl8n16 . . . 720
17.67 MATHLIB_RECIP_VEC_int24_fl8n16 . . . 721
17.68 MATHLIB_RECIP_VEC_m8p16_fl8n16 . . . 722
17.69 MATHLIB_RMS_DB10_frac48_m8p16 . . . 723
17.70 MATHLIB_RMS_DB10_VEC_frac48_m8p16 . . . 724
17.71 MATHLIB_RMS_DB2_frac48_m8p16 . . . 725
17.72 MATHLIB_RMS_DB2_VEC_frac48_m8p16 . . . 726
17.73 MATHLIB_SIN_frac24_frac24 . . . 727
17.74 MATHLIB_SIN_VEC_frac24_frac24 . . . 728
17.75 MATHLIB_SQMAG_frac24_m2p46 . . . 729
17.76 MATHLIB_SQMAG_VEC_frac24_m2p46 . . . 730
17.77 MATHLIB_SQRT_frac24_frac48 . . . 731
17.78 MATHLIB_SQRT_frac48_frac48 . . . 732
17.79 MATHLIB_SQRT_VEC_frac24_frac48 . . . 733
17.80 MATHLIB_SQRT_VEC_frac48_frac48 . . . 734

18. CFX Math Library Function Reference . . . 735
18.1 _MATHLIB_DIVIDE . . . 735
18.2 _MATHLIB_EXP2_m8p16_frac48 . . . 736
18.3 _MATHLIB_LOG2_frac48_m8p16 . . . 737
18.4 _MATHLIB_RECIP_GEN_frac48_fl8n16 . . . 738
18.5 _MATHLIB_RECIP_POS_frac48_fl8n16 . . . 739
18.6 _MATHLIB_SQRT_frac48_frac48 . . . 740

19. ARM Cortex-M3 Math Library Reference . . . 741
19.1 Math_Add_frac32 . . . 741
19.2 Math_AttackRelease . . . 742
19.3 Math_AttackRelease_frac32 . . . 743
19.4 Math_ExpAvg . . . 744
19.5 Math_ExpAvg_frac32 . . . 745
19.6 Math_LinearInterp . . . 746
19.7 Math_LinearInterp_frac32 . . . 747
19.8 Math_Mult_frac32 . . . 748
19.9 Math_SingleVar_Reg . . . 749
19.10 Math_Sub_frac32 . . . 750

20. EEPROM Library Reference . . . 751
20.1 EELIB_AutoSet_AddressMode . . . 751
20.2 EELIB_Close . . . 752

20.3 EELIB_Commit . . . 753
20.4 EELIB_Initialize . . . 754
20.5 EELIB_InitWriteWord . . . 755
20.6 EELIB_LibWriteDisable . . . 756
20.7 EELIB_LibWriteEnable . . . 757
20.8 EELIB_PowerDown . . . 758
20.9 EELIB_PowerUp . . . 759
20.10 EELIB_ReadBlock_P . . . 760
20.11 EELIB_ReadBlock_P16 . . . 761
20.12 EELIB_ReadBlock_X . . . 762
20.13 EELIB_ReadBlock_X16 . . . 763
20.14 EELIB_ReadBlock_Y . . . 764
20.15 EELIB_ReadNextBlock_P . . . 765
20.16 EELIB_ReadNextBlock_P16 . . . 766
20.17 EELIB_ReadNextBlock_X . . . 767
20.18 EELIB_ReadNextBlock_X16 . . . 768
20.19 EELIB_ReadNextBlock_Y . . . 769
20.20 EELIB_ReadNextWord16 . . . 770
20.21 EELIB_ReadNextWord24 . . . 771
20.22 EELIB_ReadNextWord32 . . . 772
20.23 EELIB_ReadNextWord8 . . . 773
20.24 EELIB_ReadStatusRegister . . . 774
20.25 EELIB_ReadWord16 . . . 775
20.26 EELIB_ReadWord24 . . . 776
20.27 EELIB_ReadWord32 . . . 777
20.28 EELIB_ReadWord8 . . . 778
20.29 EELIB_SelectSegment . . . 779
20.30 EELIB_Set_AddressMode . . . 780
20.31 EELIB_Set_PageSize . . . 781
20.32 EELIB_VerifyEEPROMConnection . . . 782
20.33 EELIB_WriteBlock_Const32 . . . 783
20.34 EELIB_WriteBlock_P . . . 784
20.35 EELIB_WriteBlock_X . . . 785
20.36 EELIB_WriteBlock_Y . . . 786
20.37 EELIB_WriteDisable . . . 787
20.38 EELIB_WriteEnable . . . 788
20.39 EELIB_WriteNextWord16 . . . 789
20.40 EELIB_WriteNextWord24 . . . 790
20.41 EELIB_WriteNextWord32 . . . 791
20.42 EELIB_WriteNextWord8 . . . 792
20.43 EELIB_WriteStatusRegister . . . 793
20.44 EELIB_WriteWord16 . . . 794
20.45 EELIB_WriteWord24 . . . 795
20.46 EELIB_WriteWord32 . . . 796
20.47 EELIB_WriteWord8 . . . 797
21. Calibration Library Reference . . . 798
21.1 CALIBRATE_ADC_CalibrateCompleteSystem . . . 798
21.2 CALIBRATE_ADC_CalibrateCurrentConfiguration . . . 799
21.3 CALIBRATE_ADC_Initialize . . . 800

21.4 Calibrate_ADCCopyToRAM 801
21.5 Calibrate_AddADCCalibration 802
21.6 Calibrate_AddClockCalibration 803
21.7 Calibrate_AddVDBLCalibration 804
21.8 Calibrate_AddVDDACalibration 805
21.9 Calibrate_AddVDDCAltCalibration 806
21.10 Calibrate_AddVDDCCalibration 807
21.11 Calibrate_AddVDDMCalibration 808
21.12 Calibrate_AddVREGCalibration 809
21.13 CALIBRATE_Clock 810
21.14 Calibrate_Clock 811
21.15 CALIBRATE_Clock_Initialize 812
21.16 Calibrate_InitializeMA 813
21.17 Calibrate_LoadMA 814
21.18 CALIBRATE_Power_Initialize 815
21.19 CALIBRATE_Power_VDBLAltCalibrate 816
21.20 CALIBRATE_Power_VDBLCalibrate 817
21.21 CALIBRATE_Power_VDDACalibrate 818
21.22 CALIBRATE_Power_VDDCAltCalibrate 819
21.23 CALIBRATE_Power_VDDCCalibrate 820
21.24 CALIBRATE_Power_VDDMAltCalibrate 821
21.25 CALIBRATE_Power_VDDMCalibrate 822
21.26 CALIBRATE_Power_VREGAltCalibrate 823
21.27 CALIBRATE_Power_VREGCalibrate 824
21.28 CALIBRATE_StoreMA 825
21.29 Calibrate_StoreMA 826
21.30 Calibrate_UpdateMA24 827
21.31 Calibrate_UpdateMA32 828
21.32 Calibrate_UpdateMA8 829
A. Glossary 830

CHAPTER 1
Introduction

1.1 Purpose

This manual describes the Ezairo® 7100 firmware. The firmware provides developers with a convenient software layer on which to build their applications. It is also responsible for system-level tasks such as booting the system and implementing portions of the security layer. It consists of include files, macros, libraries, ROM code and executables. This manual includes descriptions, function listings and usage examples to help you understand the firmware and its parts.

1.2 Intended Audience

This manual is intended for use by developers who are designing and implementing applications for the Ezairo 7100 series. Both novice and experienced developers can benefit. The descriptions and code examples will help novice users to learn the system, while experienced developers may go straight to the reference chapters that describe the available functions, macros, libraries and executables.

The manual assumes the reader has a basic understanding of:

- CFX assembly language
- C and the fundamentals of the ARM® Thumb-2 assembly language
- The integrated development environment and toolchains that form the development tools for the Ezairo 7100 series
- The architecture of the Ezairo 7100 chip

1.3 Conventions

`monospace font`
: Assembly code, macros, functions, defines and addresses.

italics
: File and path names, or any portion of them.

`<angle brackets>`
: Optional parameters and placeholders for specific information. To use an optional parameter or replace a placeholder, specify the information within the brackets; do not include the brackets themselves.

1.4 Manual Organization

The *Firmware Reference Manual for Ezairo 7100* contains the following chapters:

- **Chapter 1: Introduction** describes the purpose of this manual, the intended audience, the typographical conventions used, and gives an overview of the manual.
- **Chapter 2: Firmware Overview** provides a top level look at the firmware described in this manual, including a memory map showing what memory is reserved by the firmware.
- **Chapter 3: CFX System Include Files and Macros** describes the hardware definitions and macros that are contained in the CFX system include files.
- **Chapter 4: CFX System Library** describes the functions that applications can use to control and configure the CFX system hardware.
- **Chapter 5: ARM Cortex-M3 Processor Header Files and System Library** describes the hardware definitions and macros that are contained in the ARM Cortex™-M3 system include files.

- **Chapter 6: ARM Cortex-M3 Processor Startup Implementation** describes the CMSIS-compatible startup implementation provided for ARM Cortex-M3 processor development.
- **Chapter 7: CFX Math Library** describes the CFX math macros and functions that applications can use to perform common mathematical operations.
- **Chapter 8: ARM Cortex-M3 Processor Math Library** describes the ARM Cortex-M3 processor math macros and functions that applications can use to perform common mathematical operations.
- **Chapter 9: EEPROM Library** describes the functions that provide access to an EEPROM connected to the Ezairo 7100 device on the CFX SPI port.
- **Chapter 10: EEPROM File System** describes the EEPROM file system, which is designed for storing applications and data.
- **Chapter 11: Calibration Library** describes the calibration macros and functions that are provided to calibrate the power supply, clock and ADC components of the Ezairo 7100 system.
- **Chapter 12: Program ROM** describes the functions of the Program ROM, including the boot sequence and JumpROM functionality.
- **Chapter 13: Bootloader** describes the operation of the CFX-based bootloader.
- **Chapter 14: CFX System Macro Reference** provides a description of the CFX system macros.
- **Chapter 15: ARM Cortex-M3 Header Files and System Library Reference** provides a description of the ARM Cortex-M3 core system macros, inline functions and library functions.
- **Chapter 16: CFX System Library Reference** provides a description of the functions and macros in the CFX system library.
- **Chapter 17: CFX Math Library Macro Reference** provides a description of the macros in the CFX math library.
- **Chapter 18: CFX Math Library Function Reference** provides a description of the functions in the CFX math library.
- **Chapter 19: ARM Cortex-M3 Math Library Reference** provides a description of the functions and macros in the ARM Cortex-M3 core math library.
- **Chapter 20: EEPROM Library Reference** provides a description of the functions in the EEPROM library.
- **Chapter 21: Calibration Library Reference** provides a description of the calibration macros and functions in the calibration library.
- **Appendix A: Glossary** provides definitions of many of the terms defined and used throughout this document

1.5 Further Reading

For more information about the CFX DSP, refer to the following documents:

- *CFX DSP Architecture Manual*
- *SK4 ELF Toolchain Reference for Ezairo 7100*

For more information about the ARM® Cortex™-M3 processor, refer to the following documents:

- *ARMv7M Architecture Reference Manual*
- *ARM and Thumb-2 Instruction Set Quick Reference Card*
- *ARM Core Cortex-M3/Cortex-M3 with ETM (AT420/AT425) Errata Notice*
- *ARM Cortex-M3 Processor Toolchain Reference for Ezairo 7100*

For more information on other related topics, refer to the following documents:

- *Hardware Reference Manual for Ezairo 7100*
- *HEAR Configurable Accelerator Reference Manual*
- *Filter Engine Reference Manual*
- *Communication Protocols Manual for Ezairo 7100*

CHAPTER 2
Firmware Overview

2.1 DESCRIPTION

The firmware provides a layer of support between the application developer and the underlying hardware. It is a collection of files, macros, functions and libraries designed to make application development simpler, quicker and more reliable.

The firmware serves many functions, including performing system-level tasks (e.g., booting the system), implementing part of the security layer, storing and retrieving EEPROM data, and improving application readability.

By providing a layer of support between the hardware and the developer, the system firmware allows the developer to focus on application development and avoid having to unnecessarily learn details about the underlying Ezairo 7100 hardware. The firmware provides a clear, easy to understand interface to common operations—such as controlling and configuring the underlying hardware—while avoiding the use of absolute addresses and limiting the need for traditional assembly code. All of these help developers to quickly produce error-free code.

When multiple programmers are involved in development, using the firmware leads to increased consistency, which in turn leads to increased overall robustness and correctness of code. Development teams also need not worry about memory conflicts as there is no need to define specific memory maps.

In some cases, depending on the particulars of the application, the firmware implementation may not be optimal; however, even in these situations, the firmware will serve as an example and advanced starting point for custom-developed functions and macros.

The firmware layer also encapsulates the details of the hardware such that many changes due to hardware revisions will be transparent at the application level. The encapsulation also provides a basic level of error checking of the hardware usage.

This chapter explores the system design details of the firmware, including components, memory maps, global variables, versions, operating frequencies, and the boot sequence.

2.2 COMPONENTS

The firmware consists of two sets of components, those supporting the CFX DSP and those supporting the ARM Cortex-M3 core microcontroller. The firmware includes the following components supporting the CFX DSP:

Hardware Definitions Applies a layer of names and labels to the underlying hardware for ease of use.

CFX System Macros A set of macros which are inserted inline in the application to configure and control the underlying hardware.

CFX System Library A set of macros and callable functions that configure and control the underlying hardware.

EEPROM Library A set of callable functions that perform operations on an attached EEPROM, including initialize, read and write.

CFX Math Library A set of macros and callable functions that perform useful mathematical operations.

EEPROM File System A structure designed to store system data, application code and data tables in EEPROM.

Calibration Library A set of macros and callable functions that can be used to calibrate and store calibration settings for various analog components of the Ezairo 7100 system.

Program ROM Firmware code stored in ROM that controls system start-up, loads the bootloader to RAM, implements security functions, and reduces RAM usage by mirroring the implementation of some CFX library code.

Bootloader A system program that loads a full application (code and data) from EEPROM to RAM and executes it.

The firmware includes the following components supporting the ARM Cortex-M3 core microcontroller, all of which are CMSIS-compatible:

Hardware Definitions
Applies a layer of names and labels to the underlying hardware for ease of use.

Startup Implementation
Provides a startup vector table with a set of weakly defined interrupt handlers that link to dummy interrupt and exception handling functions.

ARM Cortex-M3 Core System Macros, Inline Functions and Callable Functions
A set of macros and inline functions that implement the CMSIS peripheral access layer and are used in the application to configure and control the underlying hardware. This is supplemented by a set of callable functions where the use of inline functions is impractical.

ARM Cortex-M3 Core Math Library
A set of callable functions that provide support to perform useful mathematical operations using non-standard data types.

The programs and libraries included with the EDK are listed in Table 1.

Table 1. Installed Locations of Programs and Libraries

Programs and Libraries	Location
CFX System Library	• The files required to link the CFX system library into projects can be found under the **<EDK_HOME>***lib**cfx* folder. • The CFX system library source code is in **<EDK_HOME>***source**cfx**syslib*.
CFX Math Library	• The files required to link the CFX math library into projects can be found under the **<EDK_HOME>***lib**cfx* folder. • The CFX math library source code is in **<EDK_HOME>***source**cfx**mathlib*.
EEPROM Library	• The files required to link the EEPROM library into projects can be found under the **<EDK_HOME>***lib**cfx* folder. • The EEPROM library source code is in **<EDK_HOME>***source**cfx**eeprom*.
Calibration Library	• The files required to link the calibration library into projects can be found under the **<EDK_HOME>***lib**cfx* folder. • The calibration library source code is in **<EDK_HOME>***source**cfx**calibrate*
Bootloader	The Bootloader source code is in **<EDK_HOME>***source**cfx**bootloader*.
CMSIS-Compatible Startup Implementations	Source code for the CMSIS-compatible startup implementation can be found under **<EDK_HOME>***source**cm3**isr_defaults*.

Table 1. Installed Locations of Programs and Libraries (Continued)

Programs and Libraries	Location
ARM Cortex-M3 Core System Library	• The files required to link the ARM Cortex-M3 core system library into projects can be found under **the <EDK_HOME>***lib**cm3* folder. • The ARM Cortex-M3 core system library source code is in **<EDK_HOME>***source**cm3**syslib*.
ARM Cortex-M3 Core Math Library	• The files required to link the ARM Cortex-M3 core library into projects can be found under the **<EDK_HOME>***lib**cm3* folder. • The ARM Cortex-M3 core math library source code is in **<EDK_HOME>***source**cm3**mathlib*.

2.2.1 Firmware Files

The firmware files consist of include files (denoted with *.cfi* extensions for CFX include files and *.h* extensions for ARM Cortex-M3 processor include files), precompiled library binaries (denoted with *.a* extensions) and precompiled program executables (denoted with *.cfe* extensions). Some precompiled libraries and programs are also included in source code format.

The CFX firmware include files are listed in Table 2. With the exception of the library include files (*sk4_syslib.cfi*, *sk4_eeprom.cfi*, *sk4_mathlib.cfi* and *sk4_calibrate.cfi*), the top-level firmware include file, *sk4_sys.cfi*, includes all other CFX include files. Thus, an application only needs to include *sk4_sys.cfi* to be able to access all of the provided hardware definitions and macros. The macros are divided into separate files based on their components. This organization is intended to assist developers in finding the implementation for specific macros and improve version control.

Table 2 also lists the CFX firmware libraries. The firmware libraries are required only if functions from the libraries are used in application code. The application source code must include the associated firmware include file as well as link against the desired library.

Table 2. CFX Firmware Include Files and Libraries

Firmware Component	Include Files	Library or Executable
Hardware Definitions	*sk4_cfx.cfi* *sk4_hw.cfi* *sk4_hw_cid102.cfi* *sk4_map.cfi*	-
System Macros and Components	*sk4_sys.cfi* *sk4_sys_acb.cfi* *sk4_sys_analog_clk.cfi* *sk4_sys_analog_in.cfi* *sk4_sys_analog_lsad.cfi* *sk4_sys_analog_out.cfi* *sk4_sys_analog_pwr.cfi* *sk4_sys_cfx.cfi* *sk4_sys_clk.cfi* *sk4_sys_cm3.cfi* *sk4_sys_debugport.cfi* *sk4_sys_dio.cfi* *sk4_sys_ecb.cfi* *sk4_sys_feng.cfi* *sk4_sys_fifo.cfi* *sk4_sys_gpio.cfi* *sk4_sys_hear.cfi* *sk4_sys_i2c.cfi* *sk4_sys_int.cfi* *sk4_sys_ioc.cfi* *sk4_sys_mem.cfi* *sk4_sys_pcm.cfi* *sk4_sys_spi.cfi* *sk4_sys_system.cfi* *sk4_sys_timer.cfi* *sk4_sys_uart.cfi* *sk4_sys_utils.cfi* *sk4_sys_watchdog.cfi*	-
System Library	*sk4_syslib.cfi*	*sk4_sys.a* *sk4_sys_cid102.a*
Math Library	*sk4_mathlib.cfi* *sk4_math.cfi*	*sk4_math.a* *sk4_math_cid102.a*
EEPROM Library	*sk4_eeprom.cfi*	*sk4_eeprom.a* *sk4_eeprom_cid102.a*
EEPROM File System	*sk4_map_nvm.cfi*	-

Table 2. CFX Firmware Include Files and Libraries (Continued)

Firmware Component	Include Files	Library or Executable
Calibration Library	*sk4_calibrate.cfi*	*sk4_calibrate.a*
Program ROM Vectors	*sk4_romvect.cfi*	-
Bootloader	-	*sk4_bootload_cid102.cfe*

The ARM Cortex-M3 processor firmware include files are listed in Table 3. With the exception of the ARM Cortex-M3 math library include file (*sk4_math.h*), the top-level firmware include file, *sk4.h*, includes all other ARM Cortex-M3 processor include files. Thus, an application only needs to include *sk4.h* to be able to access all of the provided hardware definitions, macros, and inline functions. The macros are divided into separate files based on their components. This organization is similarly intended to assist developers in finding the implementation for specific macros or functions and improve version control.

Table 3 also lists the ARM Cortex-M3 processor firmware libraries. The firmware libraries are required only if functions from the libraries are used in application code. The application source code must include the associated firmware include file as well as link against the desired library.

Table 3. ARM Cortex-M3 Processor Include Files and Libraries

Firmware Component	Include Files	Library or Executable
Hardware Definition and CMSIS Source Files	*core_cm3.h* *core_cmFunc.h* *core_cmInstr.h* *sk4.h* *sk4_hw.h* *sk4_hw_cid102.h* *sk4_map.h* *sk4_reg.h* *sk4_vectors.h* *system_sk4.h*	-
System Macros and Library	*sk4.h* *sk4_sys_acb.h* *sk4_sys_cfx.h* *sk4_sys_cm3.h* *sk4_sys_crc.h* *sk4_sys_dio.h* *sk4_sys_dma.h* *sk4_sys_ecb.h* *sk4_sys_i2c.h* *sk4_sys_pcm.h* *sk4_sys_spi.h*	*sk4_cm3_sys.a*

Table 3. ARM Cortex-M3 Processor Include Files and Libraries

Firmware Component	Include Files	Library or Executable
Math Library	*sk4_math.h*	*sk4_cm3_math.a*
CMSIS-Compatible Startup Implementation	*sk4_isr_defaults.h*	*sk4_isr_defaults.a* *sk4-cm3-isr-names.inc*

2.2.2 Compliance Exceptions

The firmware provided for the ARM Cortex-M3 processor is generally compliant with the MISRA-C:2004 rules, as required by the CMSIS standard. The exceptions in compliance to this standard that can be found in the Ezairo 7100 firmware are the same compliance exceptions that are part of the CMSIS Core standard.

The Ezairo 7100 firmware and CMSIS-CORE violate the following MISRA-C:2004 rules:

- Required Rule 8.5, object/function definition in header file. Violated since function definitions in header files are used to allow “inlining” of functions.
- Required Rule 18.4, declaration of union type or object of union type: { . . . }. Violated since unions are used for effective representation of core registers.
- Advisory Rule 19.7, Function-like macro defined. Violated since function-like macros are used to allow for more efficient code.

2.3 Device Variants

Each chip version might contain different hardware elements, as was the case with previous devices. Thus, when developing an application, the target chip within this technology family must be specified in the project build settings using a chip identifier (CID). Also, CID-targeted versions of the register definitions, firmware libraries and the bootloader exist, as listed in Table 2 on page 27, Table 3 on page 28, and explained in Section 2.4.1, “ROM-Based Libraries” on page 30.

The CID for a device is derived from combining the value of the `D_CHIP_VERSION` control register with the major revision component of the `D_CHIP_REVISION` register. See the *Hardware Reference Manual for Ezairo 7100* for more information.

When including *sk4_sys.cfi* or *sk4.h*, specify the CID by using the symbol `SK4_CID`. Set this symbol to the target platform either in the source files or through the project build settings. The following example instructs the system libraries to provide the CID 102 variant of the constants:

```
#define SK4_CID 102              // Target Ezairo 7100 version 1.02.xx
#include <sk4_sys.cfi>
#include <sk4_syslib.cfi>
```

By default, new projects include the correct settings to target CID 102 for the Ezairo 7100 chip. To modify the target CID, update the build settings for your project. You can change the chip ID using build properties or preprocessor settings. See the *SK4 ELF Toolchain Reference for Ezairo 7100* for more information about build settings.

2.4 Using the Firmware

Applications must include the appropriate include files and link against the appropriate libraries to use the provided firmware. The top-level system include file, *sk4_sys.cfi*, should be included using the preprocessor `#include` directive at the beginning of each CFX source file. The top-level CMSIS system include file, *sk4.h*, should be included using the

preprocessor `#include` directive at the beginning of each ARM Cortex-M3 processor source file. Subsequently, if the application makes use of the firmware libraries, the associated include files for those libraries must be included wherever the library's functions or macros are called.

For example, a CFX application source file that uses the firmware macros and hardware definitions, system library, EEPROM library, and CFX math library would contain the following code:

```
#include <sk4_sys.cfi>
#include <sk4_syslib.cfi>
#include <sk4_eeprom.cfi>
#include <sk4_math.cfi>
```

The application's setting within the IDE would specify that it must link against the *sk4_sys.a*, *sk4_eeprom.a* and *sk4_math.a* libraries (or their ROM-based equivalents) for the CFX portion of the application.

Similarly, an ARM Cortex-M3 processor application source file that uses the firmware macros, inline functions and hardware definitions along with the ARM Cortex-M3 processor math library (but not ARM Cortex-M3 system library functions) would contain the following code:

```
#include <sk4.h>
#include <sk4_math.h>
```

The application's setting within the IDE would specify that it must link against the *sk4_math.a* library for the Cortex-M3 portion of the application.

2.4.1 ROM-Based Libraries

The Program ROM contains implementations for some CFX library functions. The ROM-based implementations are intended to reduce memory usage in the EEPROM and RAM. To use the ROM-based implementations, the application must link against the appropriate library. The ROM-based libraries contain a *_cid***xxx** suffix in their filenames, where **xxx** is the chip ID (see Table 2 on page 27 for specifics).

During development it is generally recommended that the standard RAM-based libraries be used. This allows full use of the IDE's debugging capabilities when using the libraries. For example, if the EEPROM library is being used the application would initially link against *sk4_eeprom.a*. After the EEPROM library usage has been fully integrated and tested, the user can modify the application settings to link against *sk4_eeprom_cid***xxx***.a*. This switches from using the RAM-based implementations to using ROM-based implementations for the specified chip ID, which will reduce RAM and EEPROM usage with respect to the application footprint.

The Program ROM contains a function vector table so that the ROM-based libraries do not need to know the specific addresses of the ROM functions. The advantage of this design is that if a ROM function is relocated to a different memory location, application code that uses that function does not need to be changed or recompiled. The disadvantage is that, since the function table uses the `goto.0d` instruction to vector to the ROM functions, an application using the ROM-based libraries needs two more processor cycles per function call to execute than an otherwise identical application using the RAM-based libraries. Links to these ROM vectors are defined by *sk4_romvect.cfi*.

2.5 Memory Map

The firmware reserves space in program memory, X memory and Y memory to ensure efficient operation. The reserved space is used for code, variables, error codes and other general firmware storage needs; it must not be used by user applications. To ensure this, applications must exclude the reserved areas in the linker command options. This is the

default in the IDE-generated projects. Table 4 lists the firmware-reserved memory.

Table 4. Firmware-Reserved Memory

Memory Space	Reserved Range	Usage
Program memory	0xDC00 - 0xDFFF	Bootloader, other code and encryption keys
X memory	0x0000 - 0x001F	System variables
Y memory	0x0000 - 0x001F	System variables

If using the ROM-based AES function implementation, user applications should also avoid memories temporarily reserved by the AES library, as described in Section 12.4.1.1, “Memory, Register and Other Resource Usage by AES Routines” on page 142.

User applications may use any RAM outside of the reserved memory ranges.

2.5.1 Memory Allocation

The memory allocation model is designed for ease of use. It uses dynamic link-time allocation of code and data memories. When using this dynamic memory allocation model, you will not need to specify absolute memory addresses. It also frees you from manually tracking the memory allocation of your code, as it automatically avoids overlapping memory regions amongst multiple components. Furthermore, it abstracts the details of the memory map from you, so that you can write code and debug your application symbolically.

Several memory allocation macros are provided to support the memory allocation model on the Ezairo 7100 series. They are located in the *sk4_sys_mem.cfi* include file and listed in Table 5.

Table 5. Memory Allocation Macros

Macro	Description	Reference Section
`Sys_Mem_Start_Segment`	Starts a new memory segment	14.230
`Sys_Mem_Start_Absolute_Segment`	Starts a new memory segment at a specific address	14.228
`Sys_Mem_Start_Reloc_Segment`	Starts a new memory segment after a specific address	14.229
`Sys_Mem_Reserve`	Reserved uninitialized memory in the current segment	14.225
`Sys_Mem_Reserve_Init_Default`	Reserve memory in the current segment and initialize with 0	14.227
`Sys_Mem_Reserve_Init`	Reserve memory in the current segment and initialize with specified value	14.226

Applications specify their memory requirements at compile time using these memory allocation macros. Segments are declared symbolically using the `Sys_Mem_Start_*` macros. The macros set up segments that are stored in the compiled binary object file. At link time, the linker gathers the segment information for all components, and the application and its data are placed appropriately based on the available memory.

The IDE supports this memory allocation model by allowing symbolic memory addresses to be used. An application can therefore be debugged using symbols, so you will not need to know the actual program memory

addresses at which your application, variables and functions are located. Within the IDE, the application's build settings indicate to the linker the available memory for the application.

Note that segments need to be defined for both program code and data. Variables should be initialized within the appropriate data segment using the `Sys_Mem_Reserve_*` macros.

An application normally begins by declaring a data memory segment and then declaring variables and buffers that will reside in that segment. The application must also allocate memory for the stack. The start of an application should then declare the main program memory segment and indicate the entry point of the application. The main entry point is specified to the linker with the `--entry` option.

The following example demonstrates a possible start to an application that uses the memory allocation macros:

```
Sys_Mem_Start_Segment( variables_segment, SYS_MEMSPACE_X )
Sys_Mem_Reserve_Init( x_buffer_ptr, 1, 0 )
Sys_Mem_Reserve_Init( x_buffer, 100, 0 )
Sys_Mem_Reserve_Init( x_count, 1, 0 )

Sys_Mem_Start_Segment( main_segment, SYS_MEMSPACE_P )
.global main
main:
    // Code starts here
```

This example allocates two variables (`x_buffer_ptr` and `x_count`) as well as a 100-word buffer (`x_buffer`). These variables and the buffer are created sequentially in the `variables_segment` segment in X memory. Next, the `main_segment` segment is created in program memory and the code starts after the `main:` label.

IMPORTANT: The `Sys_Mem_Reserve_Init*` macros initialize memory only when the application is loaded by the debugger or bootloader. After a soft reset, the memory remains allocated. If your application requires the values in the memory to be cleared out, it will have to re-initialize them.

2.5.1.1 Stack Initialization

The CFX portion of the application is responsible for initializing and allocating space for the system stack. The same memory allocation macros used to allocate space for variables are used to allocate space for the stack. The stack can be created at a static address or it can be allocated dynamically. The following example shows how to allocate a 100-word stack and initialize it at the start of the application:

```
#define STACK_SIZE                100
Sys_Mem_Start_Absolute_Segment(X_System_Stack_Memory_Reserved, \
                               SYS_MEMSPACE_X,                 \
                               (D_XMEM_TOP - STACK_SIZE + 1))
Sys_Mem_Reserve(X_STACK_BOTTOM, STACK_SIZE)

Sys_Mem_Start_Segment( main, SYS_MEMSPACE_P )
.global main
main:
    Sys_CFX_InitStack( xstack )       // Initialize the stack pointer
```

The ARM Cortex-M3 processor portion of the application is responsible for initializing and allocating space for the ARM Cortex-M3 processor's stack. This memory is allocated as a static memory area placed by the linker. The

following example shows how to allocate a 100-word stack that can be referenced by the ARM Cortex-M3 processor application's vector table:

```
#define INITIAL_STACK_SIZE        100
static unsigned int romStack[INITIAL_STACK_SIZE] @ ".stack";
```

2.5.2 System Variables

The firmware contains a number of global variables located at fixed addresses. These variables are allocated in the firmware-reserved space at the base of X memory and Y memory. The system variables are used for system purposes and generally should not be directly modified by user applications (exceptions to this are noted in Table 6). You may sometimes find it useful to read the value of some of the system variables in your application code. Table 6 lists these variables.

Table 6. Firmware System Variables

Variable	Address	Description
SYSVAR_BOOTLOADER_VER	X:0x0000	Bootloader version
SYSVAR_PROGROM_VER	X:0x0001	Program ROM version
SYSVAR_BOOTLOAD_VECT	X:0x0002	Bootloader address
SYSVAR_OPERATING_FREQ	X:0x0003	Current operating frequency
SYSVAR_APPNUM	X:0x0004	Application number
SYSVAR_EE_KEYCHECK	X:0x0005	EEPROM library key check
SYSVAR_KROM_VERIFY	X:0x0006	AES-key-related error code
SYSVAR_KEY	X:0x0007	Selected AES key
SYSVAR_SYS_ERRNO	X:0x0008	System error code
SYSVAR_EE_ERRNO	X:0x0009	EEPROM library error code
SYSVAR_EE_COMMIT_TIMEOUT	X:0x000A	EEPROM library commit timeout status
SYSVAR_EE_WREN	X:0x000B	Write-enable opcode used by the EEPROM library; default is AT_WRDI
SYSVAR_EE_MEM_SELECT	X:0x000C	EEPROM library memory selection
SYSVAR_APP_ERRNO	X:0x000D	Application error code (user accessible)
SYSVAR_CLKCAL_ENTRY*	X:0x000E to 0x0019	Clock calibration entries
SYSVAR_STANDBY_CLKCFG	X:0x001A	Temporary storage for oscillator configuration while in standby mode
SYSVAR_BOOT_ERRNO	X:0x001B	Boot ROM error code
SYSVAR_EE_ADDR_MODE	X:0x001C	EEPROM library addressing mode
SYSVAR_EE_PAGE_SIZE	X:0x001D	EEPROM library page size
SYSVAR_CAL_ERRNO	X:0x001E	Calibration library error code
SYSVAR_Y_ADCCAL_START	Y:0x0000 to 0x0013	ADC calibration entries

2.6 Firmware Naming Conventions

For clarity and ease of use, the CFX portion of the firmware follows several naming conventions for library functions and macros. The Cortex-M3 portion of the firmware follows similar naming conventions where these conventions are compatible with the CMSIS requirements.

2.6.1 Macro and ARM Cortex-M3 Processor Inline Function Naming

The firmware macros for the CFX are all prefixed with `Sys_` and generally continue with the name of component they operate on. Similarly, the inline firmware functions for the ARM Cortex-M3 processor are all prefixed with `Sys_` and generally continue with the name of component they operate on. ARM Cortex-M3 macros that are implemented using a `#define` statement are prefixed with `SYS_`.

The components supported by these macros and inline functions include the CFX, HEAR, Filter Engine and ARM Cortex-M3 processor control registers and peripherals. For instance, macros related to the CFX are prefixed with `Sys_CFX_`. If there is no specific target component, the component is omitted. The full name of the macro is intended to represent its underlying functionality.

2.6.2 Library Function Naming

The firmware libraries contain functions that are directly callable or callable through associated macros. The library functions are all prefixed with the name of the library, followed by the function name. For instance, the CFX system library functions are prefixed with `SYSLIB_`. Table 7 lists the prefixes for each of the firmware libraries. Further details about the naming conventions for each library are available in their respective chapters.

Table 7. Library Function Naming Convention

Library	Filename	Function Prefix
ARM Cortex-M3 Core System Library	*sk4_cm3_sys.a*	Sys_
CFX System Library	*sk4_sys.a* *sk4_sys_cid102.a*	`SYSLIB_`
CFX Math Library	*sk4_math.a* *sk4_math_cid102.a*	`_MATHLIB_`
ARM Cortex-M3 Core Math Library	*sk4_cm3_math.a*	`Math_`
EEPROM Library	*sk4_eeprom.a* *sk4_eeprom_cid102.a*	`EELIB_`
Calibration Library	*sk4_calibrate.a*	`CALIBRATE_`

ROM-based libraries have a *_cid***xxx** suffix on their file names (e.g., *sk4_sys_cid102.a*). For more information on ROM-based libraries, see Section 2.4.1, “ROM-Based Libraries” on page 30.

2.7 Firmware Resource Usage

2.7.1 Register Usage

The firmware attempts to minimize the number of register modifications where possible. In its function header, every firmware macro and function lists the registers it modifies, including any scratch registers. If the calling function needs to preserve the content of any of the registers, the caller must take care of saving and restoring those registers.

The CFX firmware generally specifies registers `y1`, `b1`, `xp7` and `yp3` as scratch registers. As an exception, the CFX math library uses registers `a0`, `x0` and `xp0` as scratch registers. Additionally, the status register (SR) is treated as a scratch register with specific bit fields possibly being modified by a macro or function. These registers are not being saved and restored from the stack by the firmware. This ensures efficient implementation of the firmware functions and macros because using dedicated scratch registers avoids excessive use of the stack to save and restore register values.

The register modifications list for each macro and function indicate which scratch registers are used. In general though, it is good practice to avoid using these registers in application code that uses the firmware as this avoids the overhead associated with saving and restoring those registers.

The ARM Cortex-M3 processor based firmware does not use any scratch registers or register modifications in its implementation.

2.7.2 Stack Usage

The firmware makes use of the CFX system stack. It expects the CFX stack pointer to point to a valid stack that grows upward (i.e., increasing memory addresses). Furthermore, it assumes the memory location at `x[sp]` is not yet occupied by stack data, so it allocates the space it requires starting at the address `sp`.

Similarly, the firmware makes use of the ARM Cortex-M3 processor system stack. It expects that the ARM Cortex-M3 stack pointer points to a valid stack that grows downward (i.e., decreasing memory addresses).

For more information on allocating stacks, please refer to Section 2.5.1.1, “Stack Initialization” on page 32.

2.8 VERSIONS

Version symbols are provided for each major firmware component. The version symbols can be used directly or indirectly to verify the version of the components used to build an application. There are three types of version symbols available:

Define A preprocessor define containing the version information

Symbolic A compiled symbol contained within a binary library

Global Variable A global variable in memory containing the symbol

The available version information for each firmware component is listed in Table 8.

Table 8. Firmware Version Symbols

Component	Type	Version Symbol
CFX Firmware includes	Define	`SYS_FW_VER`
CFX System Library	Symbolic	`SYSLIB_Version`
CFX Math Library	Symbolic	`MATHLIB_Version`
ARM Cortex-M3 Core Math Library	Symbolic	`SK4_MathLib_Version`
EEPROM Library	Symbolic	`EELIB_Version`
Calibration Library	Symbolic	`CALIBRATELIB_Version`
Program ROM	Global Variable	`SYSVAR_PROGROM_VER`
Bootloader	Global Variable	`SYSVAR_BOOTLOADER_VER`

The version information contains a major version, minor version and revision. The major version is updated to indicate significant changes to the component. Significant changes may involve a total redesign of the component including its interfaces and functionality. The minor version is updated to indicate minor changes or additions that are generally backwards compatible. This would generally indicate a change to the interfaces or underlying functionality, including different register modifications or leaving a different system state. The revision is updated to indicate a fairly

insignificant change to the component. For example, the revision is updated when the interfaces and functionality of the component remains the same and some other change has occurred. This may include non-functional changes to the source code or optimizations that do not affect the calling application.

The major version, minor version and revision are contained within a 16-bit value. The version is described as Major.Minor.Revision. For example, Bootloader v1.0.0 indicates major version 1, minor version 0, and revision 0. Table 9 shows the bit fields in the version symbols.

Table 9. Firmware Version Bit Fields

Bits 15-12	Bits 11-8	Bits 7-0
Major Version	Minor Version	Revision

The version information for the Program ROM does not include a revision number, as all changes to the ROM image are considered to be at least minor version improvements. Table 10 shows the bit fields in the Program ROM version symbol.

Table 10. Program ROM Version Bit Fields

Bits 15-8	Bits 7-0
Major Version	Minor Version

2.9 Operating Frequencies

The firmware is designed with support for standard operating frequencies. A standard operating frequency is a valid system clock frequency that is recognized by all the firmware components requiring timing-specific code, including the bootloader, EEPROM library and system functions. The operating frequency is used to determine the appropriate division factors where such factors are required. It may also be used by the user application as desired. Table 11 lists the standard Ezairo 7100 operating frequencies and their associated indices and defines. For more information on the storage of clock calibration values in EEPROM, see Section 7.8.1, “Clock Calibration” on page 80.

Table 11. Operating Frequencies

Frequency Index	Symbol	Frequency
0x00	SYS_FREQ_UNKNOWN	Unknown operating frequency
0x01	SYS_FREQ_1M28	1.28 MHz
0x02	SYS_FREQ_1M92	1.92 MHz
0x03	SYS_FREQ_2M56	2.56 MHz
0x04	SYS_FREQ_3M20	3.20 MHz
0x05	SYS_FREQ_3M84	3.84 MHz
0x06	SYS_FREQ_4M48	4.48 MHz
0x07	SYS_FREQ_5M12	5.12 MHz
0x08	SYS_FREQ_5M76	5.76 MHz
0x09	SYS_FREQ_6M40	6.40 MHz
0x0A	SYS_FREQ_7M04	7.04 MHz
0x0B	SYS_FREQ_7M68	7.68 MHz

Table 11. Operating Frequencies (Continued)

Frequency Index	Symbol	Frequency
0x0C	SYS_FREQ_8M32	8.32 MHz
0x0D	SYS_FREQ_8M96	8.96 MHz
0x0E	SYS_FREQ_9M60	9.60 MHz
0x0F	SYS_FREQ_10M24	10.24 MHz
0x10	SYS_FREQ_10M88	10.88 MHz
0x11	SYS_FREQ_11M52	11.52 MHz
0x12	SYS_FREQ_12M16	12.16 MHz
0x13	SYS_FREQ_12M80	12.80 MHz
0x14	SYS_FREQ_13M44	13.44 MHz
0x15	SYS_FREQ_14M08	14.08 MHz
0x16	SYS_FREQ_14M72	14.72 MHz
0x17	SYS_FREQ_15M36	15.36 MHz

2.9.1 Current Operating Frequency

The current operating frequency is stored in the system variable SYSVAR_OPERATING_FREQ. The value is initially set by the Program ROM and subsequently by the bootloader and through Sys_SetOperatingFrequency. The file system contains specific clock calibration information for each Ezairo 7100 chip. The bootloader uses this information to set the internal oscillator to the indicated operating frequency, and stores that frequency index value in the system variable.

2.9.2 Changing Operating Frequencies

The current operating frequency can be changed using the system library function SYSLIB_SetOperatingFrequency, or the corresponding macro Sys_SetOperatingFrequency. The function takes a parameter specifying the operating frequency index. An operating frequency symbol from the list Table 11 should be used. For example, to change the clock frequency to 2.56 MHz, execute the following code:

```
// Change operating frequency to 2.56 MHz
Sys_SetOperatingFrequency( SYS_FREQ_2M56 )
```

The Sys_SetOperatingFrequency macro and the corresponding SYSLIB_SetOperatingFrequency function indicate success or failure code in the status register.

IMPORTANT: The bootloader loads the clock calibration information from EEPROM to the firmware-reserved X memory system variables block during boot. The clock frequency can only be changed through the system library function if the calibration data present in the X memory system variables is valid.

2.10 BOOTING

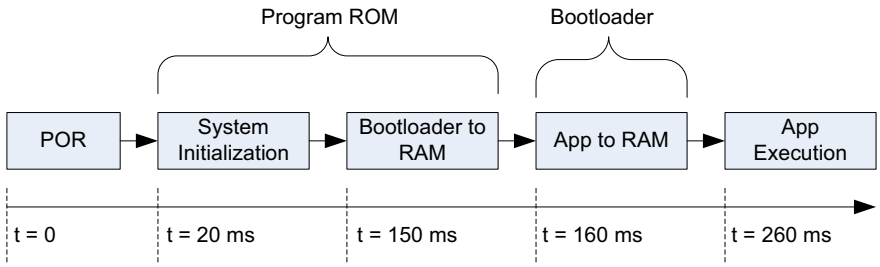
The firmware provides support for booting the system from an attached EEPROM. The user application is stored on the EEPROM. It is usually pre-loaded during manufacturing and stored along with manufacturing data such as clock calibration values. A variable in the EEPROM file system indicates which application to boot if multiple applications are available on the EEPROM. For more information on the EEPROM file system see Chapter 10, “EEPROM File System” on page 117.

The Program ROM and bootloader work sequentially to load the application from EEPROM to RAM. The integrity of the application is verified while being loaded. If an error occurs during the boot process, an error code is stored in memory and the system reboots after a delay.

The firmware also provides support for switching between multiple applications. This is provided through a firmware macro as well as a JumpROM function. The firmware macro `Sys_BootApp` can be used to allow the existing application to boot into another application. The JumpROM function can be used to allow an external system connected to the Ezairo 7100 debug port to switch between different applications. See the *Communication Protocols Manual for Ezairo 7100* for more information.

2.10.1 Boot Sequence

The boot process is a multi-stage sequence involving the reset circuitry, Program ROM and bootloader. After a Power-On Reset (POR), the reset signal to the CFX is released after a short delay of approximately 20 ms, and program execution begins at address 0x0000. This corresponds to the base of program memory where the Program ROM resides. Figure 1 shows the boot sequence and typical times for each stage. The example below assumes the default operating frequency set as part of the bootload process is 5.12 MHz and an EEPROM read data rate of 2.56 Mbps is used to load the application to RAM.



NOTE: Times are typical.
Assumptions:
1) Fast boot option set
2) Bootloader size is 256 words
3) Application size is 250 kbit total

Figure 1. Boot Sequence

2.10.2 Boot Errors

During the boot process the file system and stored applications are verified for integrity. Several errors may be generated during the boot process. If an error is detected, the corresponding error code is written to the `SYSVAR_SYS_ERRNO` system variable. If this error occurs during the boot ROM portion of the program ROM, this error is also recorded in the `SYSVAR_BOOT_ERRNO` system variable. This value may be retrieved by examining memory (if possible) or through the JumpROM Get Error function. Table 12 lists the boot error codes. The Program ROM and bootloader will both detect errors. The bootloader error handler uses a logical `OR` to combine the error code corresponding to the boot failure with the bootloader error constant, 0x100. The bootloader error constant indicates that the error occurred in the bootloader.

Table 12. Boot Error Codes

Error code	Symbol	Description
0x000	BOOT_ERR_NOERR	No error during boot
0x001	BOOT_ERR_BUSY	Boot currently in progress
0x002	BOOT_ERR_HEADER	Bootloader header invalid
0x003	BOOT_ERR_ALT	Alternate boot failed
0x004	BOOT_ERR_CRC	Bootloader CRC invalid
0x005	BOOT_ERR_DECRYPT	Bootloader decryption failed
0x006	BOOT_ERR_ADDR	Bootloader load address out of range
0x101	BL_ERR_SYSAREA_INVALID_CRC	System area CRC invalid
0x102	BL_ERR_MANUAREA_INVALID_CRC	Manufacturing area CRC invalid
0x103	BL_ERR_INVALID_KEY_INFO	KEY_INFO invalid
0x104	BL_ERR_INVALID_APPNUM	APPNUM invalid
0x105	BL_ERR_INVALID_APPSIG	Invalid application signature
0x106	BL_ERR_APP_INVALID_SECTCOUNT	Invalid section count
0x107	BL_ERR_APP_INVALID_CRC	Application CRC invalid
0x108	BL_ERR_APP_INVALID_SECTTYPE	Application section type invalid
0x109	BL_ERR_APP_INVALID_KEY	Application decryption key invalid
0x10A	BL_ERR_APP_INVALID_SECTCRC	Application section CRC invalid
0x10B	BL_ERR_APP_INVALID_SECTSIZE	Application section size invalid
0x10C	BL_ERR_APP_INVALID_START	Application start section invalid
0x10D	BL_ERR_CORRUPT_VOLATILE	Volatile section CRC invalid
0x10E	BL_ERR_INCORRECT_FSVERSION	Incorrect file system version
0x10F	BL_ERR_INVALID_EESIG	The system area does not start with a valid EEPROM signature

CHAPTER 3
CFX System Include Files and Macros

3.1 DESCRIPTION

The CFX system include files contain hardware definitions and macros. Hardware definitions are used in place of absolute addresses and constants. The macros perform specific system functions to support CFX development; they are expanded inline by the preprocessor.

3.2 HARDWARE DEFINITIONS

The CFX processor is supported by a set of header files defining the Ezairo 7100 device. This hardware definition applies a layer of names and labels to the underlying hardware. Every control register and field in the system is assigned a name and value. These are defined using preprocessor defines, so they can be used in your application code.

Using the hardware definitions file allows you to refer to system components by preprocessor symbols instead of by addresses and bit fields. This greatly improves the readability and reliability of your application code. The use of hardware definitions in an application also means that some hardware changes, such as changes to addresses or field values, are transparent to your application code.

The Ezairo 7100 CFX hardware definition includes:

- CFX core register bit descriptions for the status register and circular addressing registers
- Register and bit descriptions for all control and configuration registers accessible by the CFX
- Memory maps for memory that the CFX can access using any of the P memory, X memory, or Y memory buses
- Interrupt vector table descriptions

3.2.1 Register Naming Conventions

The hardware definitions file divides the control and configuration registers into three major categories:

- Digital registers under the CFX processor
- Digital register under the ARM Cortex-M3 processor
- Analog registers

CFX digital control and configuration registers are prefixed with `D_`. These registers control and configure the digital system components such as cores, peripherals and digital interfaces. ARM Cortex-M3 digital control and configuration registers are prefixed with `C_`. These registers control and configure the digital system components that are part of the ARM Cortex-M3 processor's subsystem. All registers with this prefix are also accessible using from the ARM Cortex-M3 processor peripheral bus. Analog registers are prefixed with `A_`. These registers control and configure the predominantly analog components, such as audio inputs, outputs and amplifiers.

The full register name follows the `D_`, `C_` or `A_` prefix (i.e., D_**<REGISTERNAME>**, C_**<REGISTERNAME>** and A_**<REGISTERNAME>**). The register name is chosen to clearly represent the underlying functionality of the register.

Each register name is defined as a preprocessor symbol with the register's address as the symbol value. Following each register definition, the fields contained within that register are defined. Each field is defined by a bit position definition and additionally by a mask definition for a multi-bit field. Bit position definitions are indicated by a suffix of `_POS` (i.e., **<REGISTERNAME>_<FIELD>_POS)**, while bit mask definitions are indicated by a suffix of `_MASK` (i.e., **<REGISTERNAME>_<FIELD>_MASK**). A bit position definition indicates the starting bit of the

corresponding field in the register. When a bit mask is applied to a register, all the bits except for those corresponding to the specified multi-bit field are zeroed. In both bit position and bit mask definitions, bit numbers start at the least significant bit (right-most bit, Bit 0).

The bit position and mask together allow for easy extraction of any field in the register. The preprocessor expression

```
(<D|C|A>_<REGISTERNAME> & <REGISTER_FIELD_MASK>) >> <REGISTER_FIELD_POS>
```

extracts the field value.

After all the fields for the register are defined, preprocessor symbols for all valid values for each of those fields are defined. As part of the definition, the settings are already aligned to the proper bit position of the field. This allows the preprocessor symbol to be used directly to write the setting to the register. Further, you may OR symbols together to form a full control word.

3.2.1.1 Register Definitions Usage

The hardware register definitions are provided by the *sk4_hw.cfi* and *sk4_hw_cid*.cfi* CFX include files. The include files may be included directly in the application source code or as part of another include file. The files are automatically included in the top-level system firmware include file (*sk4_sys.cfi).*

Once the file has been included in the application, you may use the preprocessor symbols in place of absolute addresses and constants. For most control and configuration registers all the available settings are listed in the include file. To create a control word, OR together all the desired settings and write the resulting word to the control register (either by writing directly to its defined address, or by using the appropriate macro).

The system macros provide an additional layer of abstraction by eliminating the need to use the register symbols directly. The Set or SetReg macros can be used to store the settings in the associated register, while the Get macros can be used to retrieve the value of a register.

3.2.2 Memory Map and Vector Table Definitions

Memory map and vector table definitions for the CFX core are provided by *sk4_map.cfi*. This file provides:

- Defines pointing to the base and top of each memory available on the P-, X- and Y-memory buses
- Definitions providing the size of each memory available on the P-, X- and Y-memory buses
- Defines for each of the CFX interrupt vectors that are mapped into the P memory space

3.3 MACROS

Macros provide a convenient layer of abstraction between the user application and the underlying system. They are generally targeted towards a specific system component. Instead of manually writing the code to directly modify any control register, you can simply use the macro. Other macros are provided for easy data conversion. Macros are also available to handle any required bit field manipulation and masking. For instance, if only certain fields are relevant within a control register, the other bits are left unmodified. Where possible, the macros provide a level of error checking.

The following sections describe the macro usage and syntax and include lists of all the available macros, organized by their operation.

3.3.1 Macro Naming Conventions

A macro name describes its core functionality. Macros that target a specific control register use the format:

<macro identifier>_<type>_<register> [(arguments)]

while those used to control a particular system component use the format

<macro identifier>_[**<component>**_]**<operation>** [(arguments)]

The macro identifier distinguishes the system firmware macros from user-defined macros. The <macro identifier> for all system firmware macros is Sys_.

The macro type or component follows the macro identifier. When targeting a specific control register, the type is one of Set, Get or SetReg. When the macro is used to control a particular system component, the component is specified after the macro identifier; for instance, when the macro controls a specific peripheral. The system component is one of:

- ACB: Audio codec block
- CFX: CFX DSP core
- Clk: Clock distribution
- CM3: ARM Cortex-M3 core
- Data: Immediate data initialization
- DebugPort: Debug Port
- DIO: Digital I/O
- ECB: Error coding blocks
- FENG: Filter Engine core
- FIFO: All FIFO buffers
- GPIO: General Purpose I/Os
- HEAR: HEAR Configurable Accelerator
- I2C: I^2C interface
- Input: Analog input stage blocks
- Int: Interrupt Controller
- IOC: Input/Output Controller
- Mem: Memory configuration and management
- Output: Analog output stage blocks
- PCM: PCM interface
- PLL: Phase lock loop controller
- SPI: SPI interface
- System: Top-level system
- Timer: Timers 1 and 2
- UART: UART interface
- Watchdog: Watchdog

The Set, Get and SetReg macros set new values and get existing values from the control registers. The SetReg macro transfers a value from a CFX register. The system component macros are specific to the particular operation of that component.

Macros that do not require any arguments end without any parentheses following the <operation> or <register> field. Macros that take arguments have the arguments specified within parentheses after the name of the macro. For a complete list of the macro parameters, refer to Chapter 14, “CFX System Macro Reference” on page 147.

3.3.2 Macro Usage

The macros are contained in several CFX include files. The top-level system include file, *sk4_sys.cfi*, includes all the subsequent include files and thus provides access to all the macros. Since all system firmware macros are implemented using the preprocessor directive #macro, you can simply use the macro names directly in your application source code. The preprocessor will expand (or “inline”) each macro name to the appropriate assembly code at the point in your code where the macro name appears. See the *SK4 ELF Toolchain Reference for Ezairo 7100* for more information about the preprocessor.

3.3.2.1 Set, Get and SetReg Macros

The `Set`, `Get` and `SetReg` macros are designed for simplified access to control registers. Each control register contains a bit field mapping and associated field values in the hardware definitions include file. To execute the desired `Set`, `Get` or `SetReg` operation on the CFX, choose the macro corresponding to the control register to be accessed. If it is a Set or SetReg operation, you will also need to specify the desired field settings. You can `OR` together the field values to form the control word that will be written to the control register. For `Set` macros, the control word is passed to the macro directly as a macro argument. For `SetReg` macros, the control word is first stored in a CFX register and the name of the CFX register is passed to the macro as an argument. For `Get` macros, the field values of the control register will be placed in the CFX register specified as the argument to the macro.

3.4 System Configuration

The system configuration macros relate to top-level system configuration. This includes:

- The CFX DSP core (Table 13)
- Interrupts (Table 14)
- Clocking (Table 15)
- Debug port (Table 16)
- System versions (Table 17)

Table 13. CFX Configuration Macros

Function	Description	Reference
Sys_CFX_BiasDisable	Set the SR_BIAS bit to disable bias	14.29 on p. 175
Sys_CFX_BiasEnable	Clear the SR_BIAS bit to enable bias	14.30 on p. 176
Sys_CFX_CircAddrConfig	Configure one of the CFX's circular registers	14.31 on p. 177
Sys_CFX_FlagUpdateDisable	Clear the SR_FLAG_UPD bit to disable updates to SR flags	14.32 on p. 178
Sys_CFX_FlagUpdateEnable	Set the SR_FLAG_UPD bit to enable updates to SR flags	14.33 on p. 179
Sys_CFX_InitLoopStack	Initialize the loop stack	14.34 on p. 180
Sys_CFX_InitStack	Initialize the stack pointer	14.35 on p. 181
Sys_CFX_IntDisable	Clear the SR_IE bit to disable interrupts	14.36 on p. 182
Sys_CFX_IntEnable	Set the SR_IE bit to enable interrupts	14.37 on p. 183
Sys_CFX_RoundDisable	Clear the SR_ROUND bit to select truncation	14.38 on p. 184
Sys_CFX_RoundEnable	Set the SR_ROUND bit to select rounding	14.39 on p. 185
Sys_CFX_SaturateDisable	Clear the SR_SATURATE bit to disable saturation	14.40 on p. 186
Sys_CFX_SaturateEnable	Set the SR_SATURATE bit to enable saturation	14.41 on p. 187
Sys_CFX_SRConfig	Configure the CFX status register	14.42 on p. 188
Sys_CFX_StepConfig	Configure one of the CFX's step registers	14.43 on p. 189

Table 14. Interrupt Configuration Macros

Function	Description	Reference
Sys_Int_Ack	Acknowledge a set of interrupts	14.190 on p. 336
Sys_Int_Enable	Enable a set of interrupts	14.191 on p. 337
Sys_Int_Master_Disable	Disable all interrupts at the interrupt controller	14.192 on p. 338
Sys_Int_Master_Enable	Enable all interrupts at the interrupt controller	14.193 on p. 339
Sys_Int_MuxConfig	Configure the interrupt multiplexing	14.194 on p. 340
Sys_Int_Priority	Configure the interrupt priority	14.195 on p. 341

Table 15. Clock Configuration Macros

Function	Description	Reference
Sys_Clk_Config	Configure the main clocks used by the system	14.44 on p. 190
Sys_Clk_Config0	Configure the HEARCLK, EXTCLK, SYSCLK and CM3CLK clocks	14.45 on p. 191
Sys_Clk_Config1	Configure the SLOWCLK and UCLK clocks, and double access mode	14.46 on p. 192
Sys_Clk_MultThrottleConfig	Configure whether UCLK, SLOWCLK and SDMCLK are prescaled by the VCO multiplier value	14.47 on p. 193
Sys_Clk_Throttle_Config	Configure SYSCLK, SLOWCLK, and the SDMCLK using the clock throttling mechanism	14.48 on p. 194
Sys_Clk_Throttle_MultConfig	Configure SYSCLK using the VCO multiplier clock throttling mechanism	14.49 on p. 195

Table 16. Debug Port Macros

Function	Description	Reference
Sys_DebugPort_Get_AudioMode	Return the debug port audio mode setting	14.68 on p. 214
Sys_DebugPort_Set_AccessMode	Set the debug port access mode	14.69 on p. 215
Sys_DebugPort_Set_Address	Set the internally-programmable CFX debug port address	14.70 on p. 216
Sys_DebugPort_Set_CM3Address	Set the debug port address for the ARM Cortex-M3 debug port	14.71 on p. 217
Sys_DebugPort_Set_ExternalAddress	Set the externally-programmable CFX debug port address	14.72 on p. 218

Table 17. System Version Macros

Function	Description	Reference
Sys_Get_ChipDebugVersion	Return the CFX (primary) debug port version	14.114 on p. 260
Sys_Get_ChipFamily	Return chip family	14.115 on p. 261
Sys_Get_ChipRevision	Return chip revision	14.116 on p. 262
Sys_Get_ChipVersion	Return chip version	14.117 on p. 263

3.5 Memory Management

The memory management macros facilitate the memory use paradigm. Macros are included for configuring memory segments and stack manipulation, including pushing and popping of core registers and the hardware loop stack registers. The memory management macros are listed in Table 18.

Table 18. Memory Management Macros

Function	Description	Reference
Sys_Push_Reg_1	Store the specified register onto stack	14.260 on p. 406
Sys_Push_Reg_10	Store the ten specified registers onto stack	14.261 on p. 407
Sys_Push_Reg_2	Store the two specified registers onto stack	14.262 on p. 408
Sys_Push_Reg_3	Store the three specified registers onto stack	14.263 on p. 409
Sys_Push_Reg_4	Store the four specified register onto stack	14.264 on p. 410
Sys_Push_Reg_5	Store the five specified registers onto stack	14.265 on p. 411
Sys_Push_Reg_6	Store the six specified register onto stack	14.266 on p. 412
Sys_Push_Reg_7	Store the seven specified registers onto stack	14.267 on p. 413
Sys_Push_Reg_8	Store the eight specified registers onto stack	14.268 on p. 414
Sys_Push_Reg_9	Store the nine specified register onto stack	14.269 on p. 415
Sys_Push_Reg_Long_1	Store the specified register onto the stack	14.270 on p. 416
Sys_Push_Reg_Long_10	Store the ten specified registers onto the stack	14.271 on p. 417
Sys_Push_Reg_Long_2	Store the two specified registers onto the stack	14.272 on p. 418
Sys_Push_Reg_Long_3	Store the three specified registers onto the stack	14.273 on p. 419
Sys_Push_Reg_Long_4	Store the four specified registers onto the stack	14.274 on p. 420
Sys_Push_Reg_Long_5	Store the five specified registers onto the stack	14.275 on p. 421
Sys_Push_Reg_Long_6	Store the six specified registers onto the stack	14.276 on p. 422
Sys_Push_Reg_Long_7	Store the seven specified registers onto the stack	14.277 on p. 423
Sys_Push_Reg_Long_8	Store the eight specified registers onto the stack	14.278 on p. 424
Sys_Push_Reg_Long_9	Store the nine specified registers onto the stack	14.279 on p. 425
Sys_PushLoopsToStack	Push all four sets of the hardware loop registers onto the stack; uses ROMVECT_PushLoopsToStack	14.280 on p. 426
Sys_Pop_Reg_1	Restore the specified register from stack	14.238 on p. 384
Sys_Pop_Reg_10	Restore the ten specified registers from stack	14.239 on p. 385
Sys_Pop_Reg_2	Restore the two specified registers from stack	14.240 on p. 386
Sys_Pop_Reg_3	Restore the three specified registers from stack	14.241 on p. 387
Sys_Pop_Reg_4	Restore the four specified registers from stack	14.242 on p. 388
Sys_Pop_Reg_5	Restore the five specified registers from stack	14.243 on p. 389
Sys_Pop_Reg_6	Restore the six specified registers from stack	14.244 on p. 390
Sys_Pop_Reg_7	Restore the seven specified registers from stack	14.245 on p. 391
Sys_Pop_Reg_8	Restore the eight specified registers from stack	14.246 on p. 392
Sys_Pop_Reg_9	Restore the nine specified registers from stack	14.247 on p. 393
Sys_Pop_Reg_Long_1	Restore the specified register from the stack	14.248 on p. 394
Sys_Pop_Reg_Long_10	Restore the ten specified registers from the stack	14.249 on p. 395
Sys_Pop_Reg_Long_2	Restore the two specified registers from the stack	14.250 on p. 396
Sys_Pop_Reg_Long_3	Restore the three specified registers from the stack	14.251 on p. 397
Sys_Pop_Reg_Long_4	Restore the four specified registers from the stack	14.252 on p. 398
Sys_Pop_Reg_Long_5	Restore the five specified registers from the stack	14.253 on p. 399
Sys_Pop_Reg_Long_6	Restore the six specified registers from the stack	14.254 on p. 400
Sys_Pop_Reg_Long_7	Restore the seven specified registers from the stack	14.255 on p. 401

Table 18. Memory Management Macros (Continued)

Function	Description	Reference
Sys_Pop_Reg_Long_8	Restore the eight specified registers from the stack	14.256 on p. 402
Sys_Pop_Reg_Long_9	Restore the nine specified registers from the stack	14.257 on p. 403
Sys_PopLoopsFromStack	Pop all four sets of the hardware loop registers from the stack; uses ROMVECT_PopLoopsFromStack	14.258 on p. 404
Sys_Stack_Alloc	Allocate space on the stack	14.379 on p. 525
Sys_Stack_Free	Free up previously allocated space on the stack	14.380 on p. 526
Sys_Mem_Config	Set either the CFX or the ARM Cortex-M3 as the PRAM0 memory instance controller	14.208 on p. 354
Sys_Mem_Priority	Set either the CFX or the HEAR as having higher priority when accessing shared memories	14.223 on p. 369
Sys_Mem_Priority2	Set either the CFX or the ARM Cortex-M3 as having higher priority when accessing shared memories	14.224 on p. 370
Sys_Mem_Reserve	Reserve uninitialized memory in the current memory segment	14.225 on p. 371
Sys_Mem_Reserve_Init	Reserve memory in the current memory segment and initialize with the specified value	14.226 on p. 372
Sys_Mem_Reserve_Init_Default	Reserve memory in the current memory segment and initialize with 0	14.227 on p. 373
Sys_Mem_Start_Absolute_Segment	Start an absolute memory segment	14.228 on p. 374
Sys_Mem_Start_Reloc_Segment	Start a relocatable memory segment	14.229 on p. 375
Sys_Mem_Start_Segment	Start a relocatable memory segment	14.230 on p. 376

3.6 Application Switching

The firmware provides a method to switch the active application, by booting an application from EEPROM. The application-switching macro and the macro to query the current application are listed in Table 19.

Table 19. Application Switching Macros

Function	Description	Reference
Sys_BootApp	Boot the specified application	14.6 on p. 152
Sys_Get_AppNum	Return the number of the currently running application (APPNUM)	14.113 on p. 259

3.7 Firmware Utilities

The firmware utilities include convenient and efficient macros for common repetitive operations and useful data conversions. Memory copy macros are provided for copying data between the various memory spaces. Memory fill macros are provided for filling blocks of memory with data. The memory copy and fill macros are listed in Table 20. The data conversion macros are listed in Table 21.

Table 20. Memory Copy and Fill Macros

Function	Description	Reference
Sys_Mem_CopyPP	Copy data from program memory to program memory	14.209 on p. 355
Sys_Mem_CopyPX	Copy data from program memory to X memory	14.210 on p. 356
Sys_Mem_CopyPY	Copy data from program memory to Y memory	14.211 on p. 357

Table 20. Memory Copy and Fill Macros (Continued)

Function	Description	Reference
Sys_Mem_CopyXP	Copy data from X memory to program memory	14.212 on p. 358
Sys_Mem_CopyXX	Copy data from X memory to X memory	14.213 on p. 359
Sys_Mem_CopyXY	Copy data from X memory to Y memory	14.214 on p. 360
Sys_Mem_CopyXY_Quick	Copy data from X memory to Y memory	14.215 on p. 361
Sys_Mem_CopyYP	Copy data from Y memory to program memory	14.216 on p. 362
Sys_Mem_CopyYX	Copy data from Y memory to X memory	14.217 on p. 363
Sys_Mem_CopyYX_Quick	Copy data from Y memory to X memory	14.218 on p. 364
Sys_Mem_CopyYY	Copy data from Y memory to Y memory	14.219 on p. 365
Sys_Mem_FillP	Fill a range of program memory with a value	14.220 on p. 366
Sys_Mem_FillX	Fill a range of X memory with a value	14.221 on p. 367
Sys_Mem_FillY	Fill a range of Y memory with a value	14.222 on p. 368

Table 21. Numeric Type Conversion Macros

Function	Description	Reference
Sys_Data_fl8n16	Create a .data directive using the Sys_fl8n16 macro	14.60 on p. 206
Sys_Data_frac24	Create a .data directive using the Sys_frac24 macro	14.61 on p. 207
Sys_Data_frac32	Create a .data directive using the Sys_frac32 macro	14.62 on p. 208
Sys_Data_frac48	Create a .data directive using the Sys_frac48 macro	14.63 on p. 209
Sys_Data_m24p24	Create a .data directive using the Sys_m24p24 macro	14.64 on p. 210
Sys_Data_m2p22	Create a .data directive using the Sys_m2p22 macro	14.65 on p. 211
Sys_Data_m2p46	Create a .data directive using the Sys_m2p46 macro	14.66 on p. 212
Sys_Data_m8p16	Create a .data directive using the Sys_m8p16 macro	14.67 on p. 213
Sys_fl8n16	Convert a decimal number with up to seven digits of precision to floating-point format with an 8-bit signed exponent and a 16-bit signed mantissa	14.100 on p. 246
Sys_frac24	Convert a decimal number with up to seven digits of precision to <1,23> fractional format	14.101 on p. 247
Sys_frac32	Convert a decimal number with up to seven digits of precision to <9,23> fixed-point format	14.102 on p. 248
Sys_frac48	Convert a decimal number with up to seven digits of precision to <1,47> fractional format	14.103 on p. 249
Sys_IMM_n	Sign-extend a constant expression from its n LSBs, for n = 1, 2, ..., 32	14.189 on p. 335
Sys_Value_XY	Define an XY value list that can be used with the .data directive and similar use cases	14.391 on p. 537
Sys_Log2	Calculate the integer log base-2 (ceiling, always positive) for a supplied constant	14.201 on p. 347
Sys_Log2_Ceil	Calculate the integer log base-2 (ceiling) for a supplied constant	14.202 on p. 348
Sys_Log2_Floor	Calculate the integer log base-2 (floor) for a supplied constant	14.203 on p. 349
Sys_m24p24	Convert a decimal number with up to seven digits of precision to <24,24> fixed-point format	14.204 on p. 350

Table 21. Numeric Type Conversion Macros (Continued)

Function	Description	Reference
Sys_m2p22	Convert a decimal number with up to seven digits of precision to <2,22> fixed-point format	14.205 on p. 351
Sys_m2p46	Convert a decimal number with up to seven digits of precision to <2,46> fixed-point format	14.206 on p. 352
Sys_m8p16	Convert a decimal number with up to seven digits of precision to <8,16> fixed-point format	14.207 on p. 353
Sys_Power2	Calculate the power of 2 for a supplied constant	14.259 on p. 405
Sys_Reg_fl8n16	Load a data register or high part of an accumulator with a floating-point number with an 8-bit signed exponent and a 16-bit signed mantissa using the Sys_fl8n16 macro	14.281 on p. 427
Sys_Reg_frac24	Load a data register or high part of an accumulator with a 24-bit signed immediate using the Sys_frac24 macro	14.282 on p. 428
Sys_Reg_frac32	Load the high and extension part of an accumulator with a 32-bit signed immediate using the Sys_frac32 macro	14.283 on p. 429
Sys_Reg_frac48	Load an accumulator with a 48-bit signed immediate using the Sys_frac48 macro	14.284 on p. 430
Sys_Reg_m24p24	Load an accumulator with a 48-bit signed immediate using the Sys_m24p24 macro	14.285 on p. 431
Sys_Reg_m2p22	Load a data register or high part of an accumulator with a 24-bit signed immediate using the Sys_m2p22 macro	14.286 on p. 432
Sys_Reg_m2p46	Load an accumulator with a 48-bit signed immediate using the Sys_m2p46 macro	14.287 on p. 433
Sys_Reg_m8p16	Load a data register or high part of an accumulator with an 8-bit signed integer and a 16-bit unsigned fractional part using the Sys_m8p16 macro	14.288 on p. 434

3.8 CFX External Interfaces

Macros are provided to control most of the external interfaces, including the following:

- GPIO interface (Table 22)
- I^2C interface (Table 23)
- PCM interface (Table 24)
- SPI (Table 25)
- UART interface (Table 26)

Table 22. GPIO Macros

Function	Description	Reference
Sys_GPIO_Direction	Set the GPIO directions	14.148 on p. 294
Sys_GPIO_Get_Data	Get the GPIO input and store it in the specified accumulator register	14.150 on p. 296
Sys_GPIO_Get_Data_1	Get the GPIO input for the VDDO 1 supported DIOs and store it in the specified register	14.151 on p. 297
Sys_GPIO_Get_Data_2	Get the GPIO input for the VDDO 2 supported DIOs and store it in the specified register	14.152 on p. 298
Sys_GPIO_Get_Data_3	Get the GPIO input for the VDDO 3 supported DIOs and store it in the specified register	14.153 on p. 299

Table 22. GPIO Macros (Continued)

Function	Description	Reference
Sys_GPIO_Get_IntStatus	Get the current status for the DIO interrupts; clears the current status	14.154 on p. 300
Sys_GPIO_Get_CM3IntStatus	Get the current status for the ARM Cortex-M3 subsystem's GPIO (DIO) interrupt; this clears the current status	14.149 on p. 295
Sys_GPIO_IntConfig	Configure a CFX GPIO interrupt source	14.155 on p. 301
Sys_GPIO_CM3IntConfig	Configure an ARM Cortex-M3 subsystem GPIO interrupt source	14.147 on p. 293
Sys_GPIO_Set_Data	Set the GPIO output to the specified value	14.156 on p. 302
Sys_GPIO_Set_Data_1	Set the GPIO output for the VDDO 1 supported DIOs to the specified value	14.157 on p. 303
Sys_GPIO_Set_Data_2	Set the GPIO output for the VDDO 2 supported DIOs to the specified value	14.158 on p. 304
Sys_GPIO_Set_Data_3	Set the GPIO output for the VDDO 3 supported DIOs to the specified value	14.159 on p. 305
Sys_GPIO_Set_High	Set the specified CFX GPIO output value to high	14.160 on p. 306
Sys_GPIO_Set_Low	Set the specified CFX GPIO output value to low	14.161 on p. 307
Sys_GPIO_Toggle	Toggle the specified CFX GPIO output pin	14.162 on p. 308
Sys_GPIO_Toggle_Bits	Toggle the specified CFX GPIO output pins	14.163 on p. 309
Sys_GPIO_Toggle_Bits_1	Toggle the specified CFX GPIO output pins supported by VDDO1	14.164 on p. 310
Sys_GPIO_Toggle_Bits_2	Toggle the specified CFX GPIO output pins supported by VDDO2	14.165 on p. 311
Sys_GPIO_Toggle_Bits_3	Toggle the specified CFX GPIO output pins supported by VDDO3	14.166 on p. 312

Table 23. CFX I^2C Macros

Function	Description	Reference
Sys_I2C_ACK	Manually ACK the latest I^2C byte	14.176 on p. 322
Sys_I2C_Config	Configure the I^2C interface	14.177 on p. 323
Sys_I2C_NACK	Manually NACK the latest I^2C byte	14.178 on p. 324
Sys_I2C_NACK_And_Stop	Manually NACK the last I^2C byte and issue a stop condition (applies to I^2C master mode transfers only)	14.179 on p. 325
Sys_I2C_Pin_Config	Configure the I^2C interface bus	14.180 on p. 326
Sys_I2C_Reset	Reset the general purpose I^2C interface	14.181 on p. 327
Sys_I2C_Slave_Disable	Disable the I^2C interface by setting the I^2C_SLAVE_ENABLE bit	14.182 on p. 328
Sys_I2C_Slave_Enable	Enable the I^2C interface by setting the I^2C_SLAVE_ENABLE bit	14.183 on p. 329
Sys_I2C_Start_Read	Start an I^2C read transfer (applies to I^2C master mode transfers only)	14.184 on p. 330
Sys_I2C_Start_Write	Start an I^2C write transfer (applies to I^2C master mode transfers only)	14.185 on p. 331
Sys_I2C_Watchdog_Config	Configure the I^2C interface bus watchdog timeout	14.186 on p. 332

Table 24. CFX PCM Macros

Function	Description	Reference
Sys_PCM_Config	Configure the PCM interface	14.231 on p. 377

Table 24. CFX PCM Macros (Continued)

Function	Description	Reference
Sys_PCM_Disable	Disable the PCM interface by clearing the PCM_CFG_ENABLE bit	14.232 on p. 378
Sys_PCM_Enable	Enable the PCM interface by setting the PCM_CFG_ENABLE bit	14.233 on p. 379

Table 25. CFX SPI Macros

Function	Description	Reference
Sys_SPI_CS_High	Set the SPI_CS line high	14.372 on p. 518
Sys_SPI_CS_Low	Set the SPI_CS line low	14.373 on p. 519
Sys_SPI_Disable	Disable the SPI interface (clears the SPI configuration)	14.374 on p. 520
Sys_SPI_Enable	Configure the SPI interface	14.375 on p. 521
Sys_SPI_Read	Read the specified number of bits over the SPI interface	14.376 on p. 522
Sys_SPI_WaitDone	Wait for the SPI operation to complete; exit when the SPI interface is no longer busy	14.377 on p. 523
Sys_SPI_Write	Write the specified number of bits over the SPI interface	14.378 on p. 524

Table 26. UART Macros

Function	Description	Reference
Sys_UART_Disable	Disable the UART	14.389 on p. 535
Sys_UART_Enable	Enable the UART at the specified baud rate	14.390 on p. 536

3.9 CFX PERIPHERALS

Macros are provided to control most of the system peripherals, including:

- DIO (Table 27)
- FIFO (Table 28)
- IOC (Table 29)
- Timers (Table 30)
- Watchdog timer (Table 31)

No macros are provided to support the following peripherals:

- Cyclic redundancy code (CRC) generator
- Access mode security blocks

Table 27. DIO Macros

Function	Description	Reference
Sys_DIO_Config	Configure the specified digital I/O	14.73 on p. 219
Sys_DIO_Config_C_I2C	Configure two DIOs as the ARM Cortex-M3 I^2C interface	14.74 on p. 220
Sys_DIO_Config_C_PCM	Configure four DIOs as the ARM Cortex-M3 PCM interface	14.75 on p. 221
Sys_DIO_Config_C_SPI	Configure four DIOs as the ARM Cortex-M3 SPI interface	14.76 on p. 222

Table 27. DIO Macros (Continued)

Function	Description	Reference
Sys_DIO_Config_DAC_SCALE	Configure the DIO to be used as the DAC scaling control pin	14.77 on p. 223
Sys_DIO_Config_DMICInput	Configure DIOs as the DMIC inputs	14.78 on p. 224
Sys_DIO_Config_PCM	Configure four DIOs as the CFX PCM interface	14.79 on p. 225
Sys_DIO_Config_PLL	Configure the DIO to be used as the PLL reference clock input	14.80 on p. 226
Sys_DIO_Config_SPI	Configure four DIOs as the CFX SPI interface	14.81 on p. 227
Sys_DIO_Config_SYNC_SLAVE	Configure the DIO to be used as the synchronization pin for synchronizing with another device	14.82 on p. 228
Sys_DIO_Config_UART	Configure two DIOs as the CFX UART interface	14.83 on p. 229
Sys_DIO_Get_Mode	Get the DIO modes (DIO or GPIO) of all of the DIOs and store them to the specified accumulator register	14.84 on p. 230

Table 28. FIFO Macros

Function	Description	Reference
Sys_FIFO_AutoMute	Configure the auto-mute feature of one of the system FIFOs	14.95 on p. 241
Sys_FIFO_CFXIntConfig	Configure source for a CFX FIFO interrupt	14.96 on p. 242
Sys_FIFO_CM3IntConfig	Configure source for an ARM Cortex-M3 FIFO interrupt	14.97 on p. 243
Sys_FIFO_Configure	Configure one of the system FIFOs for use within an application	14.98 on p. 244
Sys_FIFO_HEARIntConfig	Configure source for a HEAR FIFO interrupt	14.99 on p. 245

Table 29. IOC Macros

Function	Description	Reference
Sys_IOC_ADCInputConfig	Set up the IOC ADC input configuration	14.196 on p. 342
Sys_IOC_FIFOAccessConfig	Set up the IOC FIFO access configuration for each FIFO; access is either one or two data values at a time	14.197 on p. 343
Sys_IOC_FIFOShiftConfig	Set up the IOC FIFO data alignment for ADCs and DACs	14.198 on p. 344
Sys_IOC_OutputConfig	Set up the IOC output configuration	14.199 on p. 345
Sys_IOC_PCMInputConfig	Set up the IOC PCM input configuration	14.200 on p. 346

Table 30. Timer Macros

Function	Description	Reference
Sys_Timer1_Config	Configure general-purpose timer 1	14.381 on p. 527
Sys_Timer1_Start	Start general-purpose timer 1	14.382 on p. 528
Sys_Timer1_Stop	Stop general-purpose timer 1	14.383 on p. 529
Sys_Timer2_Config	Configure general-purpose timer 2	14.384 on p. 530
Sys_Timer2_Start	Start general-purpose timer 2	14.385 on p. 531
Sys_Timer2_Stop	Stop general-purpose timer 2	14.386 on p. 532
Sys_Timers_Start	Start both general-purpose timers	14.387 on p. 533
Sys_Timers_Stop	Stop both general-purpose timers	14.388 on p. 534

Table 31. Watchdog Macros

Function	Description	Reference
Sys_Watchdog_Refresh	Refresh the watchdog timer count	14.392 on p. 538
Sys_Watchdog_TimeoutConfig	Configure the watchdog timeout	14.393 on p. 539

3.10 ARM CORTEX-M3 PROCESSOR EXTERNAL INTERFACES

Macros are provided to control most of the external interfaces on the ARM Cortex-M3 processor, including:

- I2C (Table 32)
- PCM interface (Table 33)
- SPI (Table 34)

Table 32. ARM Cortex-M3 Processor I^2C Macros

Function	Description	Reference
Sys_C_I2C_ACK	Manually ACK the latest ARM Cortex-M3 subsystem's I^2C interface byte	14.7 on p. 153
Sys_C_I2C_Config	Configure the ARM Cortex-M3 subsystem's I^2C interface	14.8 on p. 154
Sys_C_I2C_NACK	Manually NACK the latest ARM Cortex-M3 subsystem's I^2C interface byte	14.9 on p. 155
Sys_C_I2C_NACK_And_Stop	Manually NACK the last ARM Cortex-M3 subsystem's I^2C interface byte and issue a stop condition (applies to I^2C master mode transfers only)	14.10 on p. 156
Sys_C_I2C_Reset	Reset the ARM Cortex-M3 subsystem's I^2C interface	14.11 on p. 157
Sys_C_I2C_Slave_Disable	Disable the ARM Cortex-M3 subsystem's I^2C interface by clearing the C_I^2C_SLAVE_ENABLE bit	14.12 on p. 158
Sys_C_I2C_Slave_Enable	Enable the ARM Cortex-M3 subsystem's I^2C interface by setting the C_I^2C_SLAVE_ENABLE bit	14.13 on p. 159
Sys_C_I2C_Start_Read	Start an I^2C read transfer (applies to I^2C master mode transfers only) using the ARM Cortex-M3 subsystem's I^2C interface	14.14 on p. 160
Sys_C_I2C_Start_Write	Start an I^2C write transfer (applies to I^2C master mode transfers only) using the ARM Cortex-M3 subsystem's I^2C interface	14.15 on p. 161

Table 33. ARM Cortex-M3 Processor PCM Macros

Function	Description	Reference
Sys_C_PCM_Config	Configure the ARM Cortex-M3 subsystem's PCM interface	14.16 on p. 162
Sys_C_PCM_Disable	Disable the ARM Cortex-M3 subsystem's PCM interface by clearing the PCM_CTRL_ENABLE bit	14.17 on p. 163
Sys_C_PCM_Enable	Enable the ARM Cortex-M3 subsystem's PCM interface by setting the PCM_CTRL_ENABLE bit	14.18 on p. 164

Table 34. ARM Cortex-M3 Processor SPI Macros

Function	Description	Reference
Sys_C_SPI_CS_High	Set the ARM Cortex-M3 subsystem's SPI_CS line high	14.19 on p. 165
Sys_C_SPI_CS_Low	Set the ARM Cortex-M3 subsystem's SPI_CS line low	14.20 on p. 166
Sys_C_SPI_Disable	Disable the ARM Cortex-M3 subsystem's SPI interface (clears the SPI configuration)	14.21 on p. 167
Sys_C_SPI_Enable	Configure the ARM Cortex-M3 subsystem's SPI interface	14.22 on p. 168
Sys_C_SPI_Read	Read the specified number of bits over the ARM Cortex-M3 subsystem's SPI interface	14.23 on p. 169
Sys_C_SPI_ReadWrite	Read and write the specified number of bits over the ARM Cortex-M3 subsystem's SPI interface	14.24 on p. 170
Sys_C_SPI_WaitDone	Wait for the ARM Cortex-M3 subsystem's SPI operation to complete; exit when the SPI interface is no longer busy	14.25 on p. 171
Sys_C_SPI_Write	Write the specified number of bits over the ARM Cortex-M3 subsystem's SPI interface	14.26 on p. 172

3.11 ARM CORTEX-M3 PROCESSOR PERIPHERALS

Macros are provided to control key system peripherals on the ARM Cortex-M3 processor, including:

- Audio coding blocks, including CVSD and G.722 (Table 35)
- Error correction coding blocks, including hamming and interleaving and Reed-Solomon (Table 36)

No CFX macros are provided to support other ARM Cortex-M3 processor peripherals, including:

- Nested vectored interrupt controller (NVIC)
- System tick (SYSTICK) timer
- Direct memory access (DMA) controller
- Cyclic redundancy code (CRC) generator

Table 35. ACB Block Macros

Function	Description	Reference
Sys_ACB_Clear_State	Clear the internal state of all of the audio codec blocks	14.1 on p. 147
Sys_ACB_DisableAll	Disable all of the audio codec blocks	14.2 on p. 148
Sys_ACB_Int_Ack	Acknowledge all audio codec block interrupt sources	14.3 on p. 149
Sys_ACB_Int_Enable	Enable/disable a set of audio codec block interrupts	14.4 on p. 150
Sys_CVSD_Decoder_Clear_State	Clear the internal state of the CVSD decoder	14.55 on p. 201
Sys_CVSD_Decoder_Config	Configure the CVSD decoder	14.56 on p. 202
Sys_CVSD_Encoder_Clear_State	Clear the internal state of the CVSD encoder	14.57 on p. 203
Sys_CVSD_Encoder_Config	Configure the CVSD encoder	14.58 on p. 204
Sys_G722_Decoder_Clear_State	Clear the internal state of the G.722 decoder	14.104 on p. 250
Sys_G722_Decoder_Conceal	Set the necessary registers for concealment of a missing G.722 sample	14.105 on p. 251
Sys_G722_Decoder_Config	Configure the G.722 decoder	14.106 on p. 252

Table 35. ACB Block Macros (Continued)

Function	Description	Reference
Sys_G722_Encoder_Clear_State	Clear the internal state of the G.722 encoder	14.107 on p. 253
Sys_G722_Encoder_Config	Configure the G.722 encoder	14.108 on p. 254

Table 36. ECB Block Macros

Function	Description	Reference
Sys_ECB_Int_Ack	Acknowledge all ECB interrupt sources	14.85 on p. 231
Sys_ECB_Int_Enable	Enable/disable a set of error correction coding block interrupts	14.86 on p. 232
Sys_HMI_Disable	Disable all of the Hamming and interleaving components (disables both the encoder and decoder)	14.174 on p. 320
Sys_HMI_Encoder_Config	Configure the Hamming and interleaving encoder	14.175 on p. 321
Sys_RS_Clear_State	Clear the internal state and status outputs from the Reed-Solomon encoder and decoder, including the input/output pointers	14.290 on p. 436
Sys_RS_Decoder_Config	Configure the Reed-Solomon decoder	14.291 on p. 437
Sys_RS_Encoder_Config	Configure the Reed-Solomon encoder	14.292 on p. 438

3.12 Analog Block Configuration

Macros related to control and configuration of the analog blocks are divided into the following groups:

- Clocking (Table 37)
- Phase lock loop (Table 38)
- Power supply (Table 39)
- LSAD (Table 41)
- Inputs (Table 42)
- Signal detection unit (Table 41)
- Outputs (Table 43)
- Analog audio settings (Table 44)
- Digital microphone output (Table 45)

Table 37. Clocking Macros

Function	Description	Reference
Sys_Get_OSC_CTRL_0	Get the oscillator frequency	14.134 on p. 280
Sys_Get_OSC_CTRL_1	Get the oscillator multiplier and enable settings	14.135 on p. 281
Sys_Get_OSCCompOut	Get the outputs from the oscillator compensation circuitry	14.136 on p. 282
Sys_Set_OSC_CTRL	Set the internal oscillator frequency, enable and multiplier	14.321 on p. 467
Sys_Set_OSC_CTRL_0	Set the oscillator frequency	14.322 on p. 468
Sys_Set_OSC_CTRL_1	Set the oscillator multiplier and enable	14.323 on p. 469
Sys_SetReg_OSC_CTRL_0	Set the oscillator frequency	14.363 on p. 509
Sys_SetReg_OSC_CTRL_1	Set the oscillator multiplier and enable	14.364 on p. 510
Sys_Set_OSCComp	Configure the internal oscillator's temperature compensation hardware	14.324 on p. 470
Sys_Set_Clk_ThrottleConfigMode	Configure the clock throttling mode	14.299 on p. 445

Table 38. Phase Lock Loop (PLL) Macros

Function	Description	Reference
Sys_Get_PLL_CalibrationOutput	Get the VCO frequency setting output from the digital calibration feature of the PLL	14.138 on p. 284
Sys_Get_PLL_ClockSync	Get the PLL and SDMCLK clock synchronization status	14.139 on p. 285
Sys_Get_PLL_DigitalCal	Get the PLL digital calibration configuration	14.140 on p. 286
Sys_Get_PLL_LoopFilterAndVCOGain	Get the PLL loop filter and VCO gain configuration	14.141 on p. 287
Sys_Get_PLL_RefClkDetectConfig	Get the PLL reference clock detection configuration and status	14.142 on p. 288
Sys_Get_PLL_TempCompensation	Get the PLL temperature compensation configuration	14.143 on p. 289
Sys_PLL_ClearStatus	Clear the PLL status in A_PLL_CTRL_7	14.234 on p. 380
Sys_PLL_ConfigLock	Ensure the PLL and VCO are enabled, configure the variables that are used to set the PLL frequency, then lock the PLL	14.235 on p. 381
Sys_PLL_GetStatus	Get the PLL status (and the PLL lock interrupt configuration settings) from A_PLL_CTRL_7	14.236 on p. 382
Sys_PLL_WaitLock	Wait for the PLL to lock; exit when the PLL indicates that the PLL is locked or if a timeout occurs	14.237 on p. 383
Sys_Set_PLL_DigitalCal	Set the PLL digital calibration configuration	14.327 on p. 473
Sys_Set_PLL_Frequency	Configure the variables that are used to set the PLL frequency; if the VCO oscillator was not previously enabled, this macro enables it	14.328 on p. 474
Sys_Set_PLL_LockInterrupt	Set the PLL lock interrupt configuration	14.329 on p. 475
Sys_Set_PLL_LoopFilterAndVCOGain	Set the PLL loop filter and VCO gain configuration	14.330 on p. 476
Sys_Set_PLL_ModeControl	Configure the PLL mode	14.331 on p. 477
Sys_Set_PLL_RefClkDetectConfig	Set the PLL reference clock detection configuration	14.332 on p. 478
Sys_Set_PLL_TempCompensation	Set the PLL temperature compensation configuration	14.333 on p. 479
Sys_SetReg_PLL_DigitalCal	Set the PLL digital calibration configuration from a register	14.366 on p. 512
Sys_SetReg_PLL_LoopFilterAndVCOGain	Set the PLL loop filter and VCO gain configuration from a register	14.367 on p. 513
Sys_SetReg_PLL_RefClkDetectConfig	Set the PLL reference clock detection configuration from a register	14.368 on p. 514
Sys_SetReg_PLL_TempCompensation	Set the PLL temperature compensation configuration from a register	14.369 on p. 515

Table 39. Power Supply Macros

Function	Description	Reference
Sys_Get_CP_VDBL_CTRL	Get the digital interface supply voltage (VDBL) charge pump configuration	14.119 on p. 265
Sys_Get_CP_VDDA_CTRL	Get the analog supply voltage (VDDA) charge pump configuration	14.120 on p. 266
Sys_Get_SUP_COUNT_DATA	Get the value of control register A_SUP_COUNT_DATA	14.145 on p. 291
Sys_Get_SUP_THRSH_DATA	Get the value of control register A_SUP_THRSH_DATA	14.146 on p. 292
Sys_Set_CP_VDBL_CTRL	Set the digital interface supply voltage (VDBL) charge pump configuration	14.300 on p. 446
Sys_Set_CP_VDDA_CTRL	Set the analog supply voltage (VDDA) charge pump configuration	14.301 on p. 447

Table 39. Power Supply Macros (Continued)

Function	Description	Reference
Sys_Set_SUP_THRSH_DATA	Set the voltage threshold when the supply threshold counter (SUP_COUNT) will be updated	14.335 on p. 481
Sys_Set_VDBL_CPClk	Set the clock divisor for the VDBL charge pump	14.336 on p. 482
Sys_Set_VDDA_CPClk	Set the clock divisor for the VDDA charge pump	14.337 on p. 483
Sys_Set_VDDC_CPClk	Set the clock divisor for the VDDC charge pump	14.338 on p. 484
Sys_Set_VDDM_CPClk	Set the clock divisor for the VDDM charge pump	14.339 on p. 485
Sys_Set_VMIC_SW_CTRL	Configure the source for VMIC	14.340 on p. 486
Sys_SetReg_CP_VDBL_CTRL	Set the digital interface supply voltage (VDBL) charge pump configuration from a register	14.346 on p. 492
Sys_SetReg_CP_VDDA_CTRL	Set the analog supply voltage (VDDA) charge pump configuration from a register	14.347 on p. 493
Sys_SetReg_SUP_THRSH_DATA	Set the voltage threshold when the supply threshold counter (SUP_COUNT) will be updated using a register	14.371 on p. 517

Table 40. LSAD Macros

Function	Description	Reference
Sys_Get_LSAD_CTRL	Get the value of control register A_LSAD_CTRL	14.129 on p. 275
Sys_Set_LSAD_CTRL	Set the control register A_LSAD_CTRL	14.315 on p. 461
Sys_SetReg_LSAD_CTRL	Set the control register A_LSAD_CTRL from a specified register	14.358 on p. 504

Table 41. SDU Macros

Function	Description	Reference
Sys_Get_SDU_CFG	Load the signal detection unit (SDU) configuration	14.144 on p. 290
Sys_Set_SDU_CFG	Configure the signal detection unit (SDU)	14.334 on p. 480
Sys_SetReg_SDU_CFG	Configure the signal detection unit (SDU)	14.370 on p. 516

Table 42. Input Macros

Function	Description	Reference
Sys_Get_ADC_CFG	Load the ADC configuration from one of the input channels	14.109 on p. 255
Sys_Get_ADC_CTRL	Load the ADC control configuration from one of the input channels	14.110 on p. 256
Sys_Get_ADC_DEC_CTRL	Load the decimation filter configuration from one of the input channels	14.111 on p. 257
Sys_Get_ADC_SAMPLE_FREQ_CFG	Load the sampling frequency configuration	14.112 on p. 258
Sys_Get_IDRX_CFG	Load the input dynamic range extended (IDRX) input channel configuration	14.128 on p. 274
Sys_Set_ADC_CFG	Configure the ADC for one of the input channels	14.294 on p. 440
Sys_Set_ADC_CTRL	Configure the ADC control settings for one of the input channels and calibrate the ADC DC gain offset	14.295 on p. 441
Sys_Set_ADC_CTRL_NoCalibration	Configure the ADC control settings for one of the input channels	14.296 on p. 442

Table 42. Input Macros (Continued)

Function	Description	Reference
Sys_Set_ADC_DEC_CTRL	Configure the decimation filter for one of the input channels	14.297 on p. 443
Sys_Set_ADC_SAMPLE_FREQ_CFG	Configure the sampling frequency	14.298 on p. 444
Sys_Set_IDRX_CFG	Configure an input dynamic range extended (IDRX) input channel	14.314 on p. 460
Sys_SetReg_ADC_CFG	Configure the ADC for one of the input channels	14.341 on p. 487
Sys_SetReg_ADC_CTRL	Configure the ADC control settings for one of the input channels and calibrate the ADC DC gain offset	14.342 on p. 488
Sys_SetReg_ADC_CTRL_NoCalibration	Configure the ADC control settings for one of the input channels	14.343 on p. 489
Sys_SetReg_ADC_DEC_CTRL	Configure the decimation filter for one of the input channels	14.344 on p. 490
Sys_SetReg_ADC_SAMPLE_FREQ_CFG	Configure the sampling frequency	14.345 on p. 491
Sys_SetReg_IDRX_CFG	Configure an input dynamic range extended (IDRX) input channel	14.357 on p. 503

Table 43. Output Macros

Function	Description	Reference
Sys_Get_O_DAC_AO_CTRL	Get the analog output configuration for both output channels	14.130 on p. 276
Sys_Get_O_DAC_GF_CTRL	Get the output stage gain factor control settings	14.131 on p. 277
Sys_Get_O_SDM_CTRL	Get the sigma-delta modulator configuration	14.132 on p. 278
Sys_Get_O_SDMCLK	Load the output modulator prescaler	14.133 on p. 279
Sys_Get_OUTPUT_CTRL	Load the output driver configuration settings	14.137 on p. 283
Sys_Set_O_DAC_AO_CTRL	Configure the analog outputs for both output channels	14.317 on p. 463
Sys_Set_O_DAC_GF_CTRL	Configure the output stage gain factor control settings	14.318 on p. 464
Sys_Set_O_SDM_CTRL	Configure the sigma-delta modulator	14.319 on p. 465
Sys_Set_O_SDMCLK	Configure the sigma-delta modulator prescaler	14.320 on p. 466
Sys_Set_OUTPUT_CTRL	Configure the output driver configuration settings	14.325 on p. 471
Sys_SetReg_O_DAC_AO_CTRL	Configure the analog outputs for both output channels	14.359 on p. 505
Sys_SetReg_O_DAC_GF_CTRL	Configure the output stage gain factor control settings	14.360 on p. 506
Sys_SetReg_O_SDM_CTRL	Configure the sigma-delta modulator	14.361 on p. 507
Sys_SetReg_O_SDMCLK	Configure the sigma-delta modulator prescaler	14.362 on p. 508
Sys_SetReg_OUTPUT_CTRL	Configure the output driver configuration settings	14.365 on p. 511

Table 44. Analog Audio Settings Macros

Function	Description	Reference
Sys_AudioMux_Config	Configure the audio multiplexers and transfer formats	14.5 on p. 151
Sys_Calc_Gain_Factor_Val	Calculate the decimation gain factor for a desired gain given an adc_cfg_sample_freq field	14.27 on p. 173
Sys_Calc_Sample_Freq_Val	Calculate the sampling frequency field for A_ADC_SAMPLE_FREQ_CFG	14.28 on p. 174

Table 45. Digital Microphone Macros

Function	Description	Reference
Sys_Get_DMIC_OUTPUT_CTRL	Get the DMIC output configuration for both output channels	14.121 on p. 267
Sys_Set_DMIC_OUTPUT_CTRL	Configure the DMIC outputs for both output channels	14.302 on p. 448
Sys_SetReg_DMIC_OUTPUT_CTRL	Configure the DMIC outputs for both output channels	14.348 on p. 494

3.13 HEAR Configuration

Macros related to configuring the HEAR Configurable Accelerator are listed in Table 46.

Table 46. HEAR Macros

Function	Description	Reference
Sys_HEAR_Command	Issue a command to execute a HEAR function chain	14.167 on p. 313
Sys_HEAR_FunctionChain_Enable	Enable specified HEAR function chains	14.168 on p. 314
Sys_HEAR_FunctionChain_Priority	Set the priority of HEAR function chains	14.169 on p. 315
Sys_HEAR_Pause	Pause the HEAR Configurable Accelerator	14.170 on p. 316
Sys_HEAR_Reset	Reset the HEAR Configurable Accelerator and the FIFO controllers	14.171 on p. 317
Sys_HEAR_Start	Start (or resume) the HEAR Configurable Accelerator	14.173 on p. 319
Sys_HEAR_RunTime_Addr	Calculate the equivalent HEAR address to a CFX address	14.172 on p. 318
Sys_Set_HEAR_BFP	Configure the HEAR block floating point guard bits	14.312 on p. 458
Sys_Set_HEAR_FunctionChain_Vector	Set a function chain vector to the specified function chain entry point	14.313 on p. 459
Sys_Restore_HEAR_BFP	Configure the HEAR block floating point using a stored value	14.289 on p. 435
Sys_Save_HEAR_BFP	Store the current HEAR block floating point configuration	14.293 on p. 439
Sys_SetReg_HEAR_BFP_BLOCK_EXP	Set the value of the block floating point (BFP) block exponent from a register	14.356 on p. 502
Sys_Get_HEAR_BFP_BLOCK_EXP	Store the value of the block floating point (BFP) block exponent into a register	14.127 on p. 273

3.14 ARM Cortex-M3 Processor Control and Configuration

Macros related to configuring the ARM Cortex-M3 processor are listed in Table 47.

IMPORTANT: Use of the CFX macro Sys_CM3AndSubsystem_Reset resets the CM3 debug port, which causes the CM3 to no longer respond to any breakpoints or debug commands from the EDK. Take care to start the GDB server to be able to continue to debug ARM Cortex-M3 processor application components alongside your CFX application after running this macro.

This macro will cause failures if using coordinated debug features.

Table 47. ARM Cortex-M3 Processor Macros

Function	Description	Reference
Sys_CM3_Command	Issue a command to trigger an ARM Cortex-M3 interrupt	14.50 on p. 196
Sys_CM3_Pause	Pause the ARM Cortex-M3 processor	14.51 on p. 197
Sys_CM3_Reset	Reset the ARM Cortex-M3 processor	14.52 on p. 198
Sys_CM3_Start	Start (or resume) the ARM Cortex-M3 processor	14.53 on p. 199
Sys_CM3AndSubsystem_Reset	Reset the ARM Cortex-M3 processor along with its interfaces and peripherals	14.54 on p. 200
Sys_Get_CM3ChipDebugVersion	Return the ARM Cortex-M3 debug port version	14.118 on p. 264

3.15 FILTER ENGINE CONFIGURATION

Macros related to configuring the Filter Engine are listed in Table 48.

Table 48. Filter Engine Macros

Function	Description	Reference
Sys_Data_FENG_Coefficients	Create a data record for one set of Filter Engine coefficients for use with the Filter Engine BIQ and FIR instructions	14.59 on p. 205
Sys_FENG_Config	Configure the inputs to, outputs from, output shadow register enable, and coefficient offset for the Filter Engine	14.87 on p. 233
Sys_FENG_DelayedPause	Pause the Filter Engine the next time it would start execution	14.88 on p. 234
Sys_FENG_Execute	Execute one phase of the current Filter Engine program	14.89 on p. 235
Sys_FENG_IntAck	Acknowledge a set of Filter Engine interrupt sources	14.90 on p. 236
Sys_FENG_Pause	Pause the Filter Engine immediately	14.91 on p. 237
Sys_FENG_Reset	Reset the Filter Engine program decoder registers and audio path registers	14.92 on p. 238
Sys_FENG_SingleStep	Single-step the Filter Engine	14.93 on p. 239
Sys_FENG_Start	Start (or resume) the Filter Engine	14.94 on p. 240
Sys_Get_FENG_CoeffOffset	Get the current Filter Engine coefficient offset	14.122 on p. 268
Sys_Get_FENG_CoeffOffsetEnable	Get the current Filter Engine coefficient offset enable settings	14.123 on p. 269
Sys_Get_FENG_IntStatus	Get the status of various possible sources of the Filter Engine interrupt	14.124 on p. 270
Sys_Get_FENG_State	Get the current Filter Engine state codes	14.125 on p. 271
Sys_Get_FENG_StateWriteEnable	Get the current Filter Engine state code write-enable settings	14.126 on p. 272
Sys_Set_FENG_Coefficients	Configure one set of Filter Engine coefficients for use with the Filter Engine BIQ and FIR instructions	14.303 on p. 449
Sys_Set_FENG_CoeffOffset	Set the Filter Engine coefficient offset	14.304 on p. 450
Sys_Set_FENG_CoeffOffsetEnable_Active	Select which Filter Engine coefficient sets use the offset by writing the enable value to the active register	14.305 on p. 451
Sys_Set_FENG_CoeffOffsetEnable_Shadow	Select which Filter Engine coefficient sets use the offset by writing the enable value to the shadow register	14.306 on p. 452

Table 48. Filter Engine Macros (Continued)

Function	Description	Reference
Sys_Set_FENG_IntEnable	Configure the possible sources for the Filter Engine interrupt	14.307 on p. 453
Sys_Set_FENG_MRCounterConfig	Configure the multi-rate counter divisors and reset the multi-rate counters	14.308 on p. 454
Sys_Set_FENG_State	Configure the Filter Engine state codes by writing to the active register	14.309 on p. 455
Sys_Set_FENG_State_Shadow	Configure the Filter Engine state codes by writing to the shadow register	14.310 on p. 456
Sys_Set_FENG_StateWriteEnable	Configure the Filter Engine state code write-enable settings	14.311 on p. 457
Sys_SetReg_FENG_CoeffOffset	Set the Filter Engine coefficient offset	14.349 on p. 495
Sys_SetReg_FENG_CoeffOffsetEnable_Active	Select which Filter Engine coefficient sets use the offset by writing the enable value to the active register	14.350 on p. 496
Sys_SetReg_FENG_CoeffOffsetEnable_Shadow	Select which Filter Engine coefficient sets use the offset by writing the enable value to the shadow register	14.351 on p. 497
Sys_SetReg_FENG_IntEnable	Configure the possible sources for the Filter Engine interrupt	14.352 on p. 498
Sys_SetReg_FENG_State	Configure the Filter Engine state codes by writing to the active register	14.353 on p. 499
Sys_SetReg_FENG_State_Shadow	Configure the Filter Engine state codes by writing to the shadow register	14.354 on p. 500
Sys_SetReg_FENG_StateWriteEnable	Configure the Filter Engine state code write-enable settings using a register	14.355 on p. 501

CHAPTER 4
CFX System Library

4.1 Description

The CFX system library contains a set of useful functions that may be called from the user application to easily control and configure the system hardware. The functions perform common tasks such as resetting the system and changing the operating frequency. These functions are linked into the application.

4.2 System Library Usage

System library functions are contained within a compiled library. Both RAM-based and ROM-based versions are available. For more information, see Section 2.4.1, “ROM-Based Libraries” on page 30.

You must include the associated system library include file, *sk4_syslib.cfi*, in the source code. This include file contains the function declarations for the system library as well as system library macros. You must then configure your application settings to link in the appropriate compiled library.

Function parameters and return values are passed through registers. The specific registers required for each function are listed in the function reference.

4.2.1 Naming Convention

System library functions are named according to their operation, in mixed case with a prefix of `SYSLIB_`.

4.3 System Library Functions

Table 49 lists the functions available in the system library.

Table 49. System Library Functions

Function	Description	Reference
SYSLIB_CalibrateADC	Load the calibrated DC offset for the specified ADC	16.11 on p. 647
SYSLIB_Delay	Delay the CFX core for the specified period of time	16.12 on p. 648
SYSLIB_Reset	Reset the system with a watchdog timeout	16.13 on p. 649
SYSLIB_SetOperatingFrequency	Set the internal oscillator base operating frequency; the oscillator multiplication setting is not a component of the base operating frequency configuration	16.14 on p. 650
SYSLIB_SetOperatingFrequency_Throttle	Set the internal oscillator base operating frequency, using clock throttling to also configure SLOWCLK and SDMCLK	16.15 on p. 651
SYSLIB_SoftPowerdownDisable	Exit from soft power-down mode	16.16 on p. 652
SYSLIB_SoftPowerdownEnable	Put the system in soft power-down mode	16.17 on p. 653
SYSLIB_WaitForSPIDone	Wait for the SPI interface to become idle	16.18 on p. 654

4.4 System Library Macros

The system library macros are mostly wrappers around the system library functions. The macros provide a convenient method for using the system library functions. The macros take parameters that are subsequently set in the appropriate registers for the library function. The macro implementation can also be used as a reference on how to use the corresponding function directly. The system library macros are listed in Table 50.

Table 50. System Library Macros

Macro	Description	Reference
Sys_CalibrateADC	Load the calibrated DC offset for the specified ADC	16.1 on p. 637
Sys_CalibrateADCs	Load the calibrated DC offsets for all ADCs	16.2 on p. 638
Sys_Delay	Delay the CFX for the specified period of time	16.3 on p. 639
Sys_DelayReg	Delay the CFX for an amount of time specified in a register	16.4 on p. 640
Sys_Reset	Reset the system through a watchdog timeout	16.5 on p. 641
Sys_SetOperatingFrequency	Set the base operating frequency	16.6 on p. 642
Sys_SetOperatingFrequency_Throttle	Set the operating frequency throttle using the clock throttling mechanism	16.7 on p. 643
Sys_SoftPowerdownDisable	Exit from soft power-down mode	16.8 on p. 644
Sys_SoftPowerdownEnable	Put the system in soft power-down mode	16.9 on p. 645
Sys_WaitForSPIDone	Wait for the SPI interface to become idle	16.10 on p. 646

4.5 VERSION

The system library version is contained within the library as a 16-bit symbol. The symbol `SYSLIB_Version` may be used directly in the application code to identify the version of the library linked with the application. See Section 2.8, “Versions” on page 35 for more information about version symbols.

CHAPTER 5
ARM Cortex-M3 Processor Header Files and System Library

5.1 DESCRIPTION

The ARM Cortex-M3 processor on the Ezairo 7100 chip is supported by a set of header files and system library functions. These provide a description that defines the ARM Cortex-M3 subsystem of the Ezairo 7100 device. This includes:

- Register and bit descriptions for control and configuration registers in the ARM Cortex-M3 core memory map
- A memory map for the ARM Cortex-M3 core
- Interrupt vector table descriptions
- Macros that support basic ARM Cortex-M3 core functionality
- Inline functions and library function calls that support more complex firmware requirements of the ARM Cortex-M3 subsystem

The format and configuration of all of these firmware components conform to Cortex Microcontroller Software Interface Standard (CMSIS) compatibility requirements. Therefore, most of the system library consists of inline functions that are defined within the library header files.

The top-level include file for the system library, *sk4.h*, combines all of the system hardware definition, system support macro and system library firmware components provided for the ARM Cortex-M3 processor subsystem of the Ezairo 7100 chip. If an application includes this file and defines `SK4_CID` to match the chip identifier of Ezairo 7100, then all of the support macros and system library functions that are available to support ARM Cortex-M3 development on the Ezairo 7100 chip are made accessible to that application.

IMPORTANT: The system initialization and clock configuration for Ezairo 7100 are both controlled by the CFX DSP. As a result, the `SystemInit(void)` and `SystemCoreClockUpdate()` functions that are required for CMSIS compatibility are only provided as part of *system_sk4.c* for completeness. These functions do not provide any functional value and are not described further here.

5.2 HARDWARE DEFINITIONS

Hardware definition files are integral to the system firmware. The hardware definitions apply a layer of data structures and address mappings to the underlying hardware, so that every control register and bit field in the system is easily accessible from C code.

5.2.1 Register and Register Bit-field Definition

Using the hardware definition files allows you to refer to system components by C structures and preprocessor symbols instead of by addresses and bit fields. This greatly improves the readability, reliability and maintainability of your application code. The use of hardware definitions in an application also means that some hardware changes, such as changes to addresses or bit field values, are transparent to your application code.

Hardware register descriptions for components that are linked to the Ezairo 7100 ARM Cortex-M3 peripheral bus are defined in the file *sk4_hw.h* and *sk4_hw_cid*.h*. Register descriptions for standard ARM Cortex-M3 peripherals, such as NVIC and SysTick, are defined in the core CMSIS header file, core_cm3.h.

5.2.2 Memory Map Definition

Memory map definitions from the perspective of the ARM Cortex-M3 processor are provided in *sk4_map.h*. Specifically, this file defines the locations of the following structures:

- Instruction and data bus memory structures
- System bus memory structures
- Peripheral bus memory-mapped control registers (including the base of control register groups for each system component)
- Private peripheral bus internal memory-mapped control registers
- System variables

For more information on the ARM Cortex-M3 memory map see the *Hardware Reference Manual for Ezairo 7100*.

5.2.3 Interrupt Vector Definition

Interrupt vector definitions are defined in *sk4_vectors.h* for both internal and external interrupts. These definitions have the form **<interrupt_name>**_IRQn. You can use these definitions with the NVIC-related functions included with the core CMSIS (and defined in *core_cm3.h*).

5.3 CFX Control

This ARM Cortex-M3 inline function allows you to control the CFX interrupt state for ARM Cortex-M3 related events. See Table 51.

Table 51. CFX Inline Functions

Function	Description	Reference
Sys_CFX_Command	Issue a command to trigger a set of CFX interrupts	15.6 on p. 545

5.4 ARM Cortex-M3 Core Support Macros and Functions

Macros are provided to control the ARM Cortex-M3 core while it waits for an event or interrupt (Table 52).

Table 52. Core Support Macros

Function	Description	Reference
SYS_WAIT_FOR_EVENT	Hold the ARM Cortex-M3 core waiting for an event, interrupt request, abort or debug entry request (ARM Thumb-2 WFE instruction)	15.92 on p. 635
SYS_WAIT_FOR_INTERRUPT	Hold the ARM Cortex-M3 core waiting for an interrupt request, abort or debug entry request (ARM Thumb-2 WFI instruction)	15.93 on p. 636

A macro is also provided to calculate the access addresses for memory, or registers that are mapped into memory areas, that can be accessed using the bit-band (atomic single bit) access provided by the ARM Cortex-M3 processor (Table 53).

Table 53. Bit-Band Support Macros

Function	Description	Reference
SYS_CALC_BITBAND	Calculate the bit-band location for a given address and bit position	15.5 on p. 544

Standard ARM Cortex-M3 processor support macros that are defined as part of CMSIS are provided by the CMSIS Cortex-M function and instruction header files, *core_cmFunc.h* and *core_cmInstr.h*.

The system library includes a system delay function for the ARM Cortex-M3 processor as a core support function (Table 54).

Table 54. Core Support Functions

Function	Description	Reference
Sys_CM3_Delay	Delay by at least the specified number of clock cycles	15.7 on p. 546

5.5 External Interfaces on the ARM Cortex-M3 Processor

Inline functions are provided to control most of the external interfaces on the ARM Cortex-M3 processor, including the following:

- GPIO (Table 55)
- I2C (Table 56)
- PCM (Table 57)
- SPI (Table 58)

Table 55. GPIO Inline Functions

Function	Description	Reference
Sys_GPIO_Set_High	Set the specified GPIO output value to high	15.57 on p. 600
Sys_GPIO_Set_Low	Set the specified GPIO output value to low	15.58 on p. 601
Sys_GPIO_Toggle	Toggle the current value of the specified GPIO output	15.59 on p. 602

Table 56. I^2C Inline Functions

Function	Description	Reference
Sys_I2C_ACK	Manually acknowledge the latest transfer	15.62 on p. 605
Sys_I2C_Config	Configure the I^2C interface for operation	15.63 on p. 606
Sys_I2C_Get_Status	Get the current I^2C interface status	15.64 on p. 607
Sys_I2C_NACK	Manually not-acknowledge the latest transfer; releases the bus to continue with a transfer	15.65 on p. 608
Sys_I2C_NACKAndStop	Manually not-acknowledge the latest transfer and send a stop condition (Master mode only)	15.66 on p. 609
Sys_I2C_Reset	Reset the I^2C interface	15.67 on p. 610
Sys_I2C_StartRead	Initialize an I^2C master read transfer on the I^2C interface	15.68 on p. 611
Sys_I2C_StartWrite	Initialize an I^2C master write transfer on the I^2C interface	15.69 on p. 612

Table 57. PCM Inline Functions

Function	Description	Reference
Sys_PCM_Config	Configure the PCM scheme and data transfer configuration	15.74 on p. 617
Sys_PCM_Disable	Disable the PCM interface without updating the other PCM configuration settings	15.75 on p. 618
Sys_PCM_Enable	Enable the PCM interface without updating the other PCM configuration settings	15.76 on p. 619

Table 58. SPI Inline Functions

Function	Description	Reference
Sys_SPI_Config	Configure the ARM Cortex-M3's SPI operation and controller information	15.86 on p. 629
Sys_SPI_MasterInit	Initialize an SPI operation on the ARM Cortex-M3's SPI when running this interface in master mode	15.87 on p. 630
Sys_SPI_Read	Read the specified number of bits over the SPI port	15.88 on p. 631
Sys_SPI_ReadWrite	Read and write the specified number of bits over the SPI port (full-duplex)	15.89 on p. 632
Sys_SPI_TransferConfig	Configure the ARM Cortex-M3's SPI interface transfer information	15.90 on p. 633
Sys_SPI_Write	Write the specified number of bits over the SPI port	15.91 on p. 634

5.6 Peripherals on the ARM Cortex-M3 Processor

Inline functions are provided to control most of the peripherals on the ARM Cortex-M3 processor, including the following:

- Audio coding blocks, including CVSD and G.722 (Table 59)
- Error correction coding blocks, including hamming and interleaving and Reed-Solomon (Table 60)
- Cyclic Redundancy Check (Table 61)
- DIO (Table 62)
- DMA (Table 63)
- NVIC (Table 64)

Additional inline functions supporting the standard ARM Cortex-M3 peripherals, including the NVIC and SysTick, are defined in the core CMSIS header file, *core_cm3.h*.

Table 59. Audio Codec Block Inline Functions

Function	Description	Reference
Sys_ACB_Clear_State	Clear the internal state of all of the audio codec blocks	15.1 on p. 540
Sys_ACB_DisableAll	Disable all of the audio codec blocks	15.2 on p. 541
Sys_ACB_Int_Ack	Acknowledge all of the audio codec block interrupts	15.3 on p. 542
Sys_ACB_Int_Enable	Enable/disable a set of audio codec block interrupts	15.4 on p. 543
Sys_CVSD_Decoder_Clear_State	Clear the internal state of the CVSD decoder	15.8 on p. 547
Sys_CVSD_Decoder_Config	Configure the CVSD decoder	15.9 on p. 548
Sys_CVSD_Encoder_Clear_State	Clear the internal state of the CVSD encoder	15.10 on p. 549

Table 59. Audio Codec Block Inline Functions (Continued)

Function	Description	Reference
Sys_CVSD_Encoder_Config	Configure the CVSD encoder	15.11 on p. 550
Sys_G722_Decoder_Clear_State	Clear the internal state of the G.722 decoder	15.47 on p. 590
Sys_G722_Decoder_Conceal	Set the necessary registers for concealment of a missing G.722 sample	15.48 on p. 591
Sys_G722_Decoder_Config	Configure the G.722 decoder	15.49 on p. 592
Sys_G722_Encoder_Clear_State	Clear the internal state of the G.722 encoder	15.50 on p. 593
Sys_G722_Encoder_Config	Configure the G.722 encoder	15.51 on p. 594
Sys_G722_ExtendFrom48	Convert 6-bit samples to 8-bit samples for a 64 kbps rate	15.52 on p. 595
Sys_G722_ExtendFrom56	Convert 7-bit samples to 8-bit samples for a 64 kbps rate	15.53 on p. 596
Sys_G722_TruncateTo48	Convert 8-bit samples to 6-bit samples for a 48 kbps rate	15.54 on p. 597
Sys_G722_TruncateTo56	Convert 8-bit samples to 7-bit samples for a 56 kbps rate	15.55 on p. 598

Table 60. Error Correction Coding Block Interrupt Inline Functions

Function	Description	Reference
Sys_ECB_Int_Ack	Acknowledge all of the error correction coding block interrupts	15.45 on p. 588
Sys_ECB_Int_Enable	Enable/disable a set of error correction coding block interrupts	15.46 on p. 589
Sys_HMI_Disable	Disable all of the Hamming and interleaving components (disables both the encoder and decoder)	15.60 on p. 603
Sys_HMI_Encoder_Config	Configure the Hamming encoder and interleaver	15.61 on p. 604
Sys_IHM_Decoder_Clear_Status	Clear the data, and hence the status, for the Hamming decoder and de-interleaver	15.70 on p. 613
Sys_IHM_Decoder_Config	Configure the Hamming decoder and de-interleaver	15.71 on p. 614
Sys_RS_Decoder_Clear_State	Clear the internal state and status outputs from the Reed-Solomon decoder, including the input/output pointers	15.77 on p. 620
Sys_RS_Decoder_Config	Configure the Reed-Solomon decoder	15.78 on p. 621
Sys_RS_Decoder_GetOutput	Get the 11 Reed-Solomon decoder output data samples	15.79 on p. 622
Sys_RS_Decoder_SetInput	Set the 15 Reed-Solomon decoder data and parity input samples	15.80 on p. 623
Sys_RS_Encoder_Clear_State	Clear the internal state of the Reed-Solomon encoder, including the input/output pointers	15.81 on p. 624
Sys_RS_Encoder_Config	Configure the Reed-Solomon encoder	15.82 on p. 625
Sys_RS_Encoder_GetOutput	Get the 15 Reed-Solomon encoder data and parity output samples	15.83 on p. 626
Sys_RS_Encoder_SetInput	Set the 11 Reed-Solomon encoder data input samples	15.84 on p. 627

Table 61. Cyclic Redundancy Check Inline Functions

Function	Description	Reference
Sys_Get_CRC_Config	Get the CRC generator configuration	15.56 on p. 599
Sys_Set_CRC_Config	Configure the CRC generator type, endianness of the input data, and standard vs non-standard CRC behavior	15.85 on p. 628

Table 62. DIO Inline Functions

Function	Description	Reference
Sys_DIO_Get_Mode	Get the DIO mode (DIO or ARM Cortex-M3 controlled GPIO) for all of the DIOs	15.13 on p. 552
Sys_DIO_Get_IntStatus	Get the current status for the DIO interrupts; this clears the current status	15.12 on p. 551
Sys_DIO_IntConfig	Configure an ARM Cortex-M3 DIO interrupt source	15.14 on p. 553
Sys_DIO_Set_Direction	Set the input/output configuration for those DIOs configured as ARM Cortex-M3 controlled GPIOs	15.15 on p. 554

Table 63. DMA Inline Functions

Function	Description	Reference
Sys_DMA_Channel0_Config	Configure DMA channel 0 for a data transfer	15.16 on p. 555
Sys_DMA_Channel0_Disable	Disable DMA channel 0	15.17 on p. 557
Sys_DMA_Channel0_Enable	Enable DMA channel 0	15.18 on p. 558
Sys_DMA_Channel1_Config	Configure DMA channel 1 for a data transfer	15.19 on p. 559
Sys_DMA_Channel1_Disable	Disable DMA channel 1	15.20 on p. 561
Sys_DMA_Channel1_Enable	Enable DMA channel 1	15.21 on p. 562
Sys_DMA_Channel2_Config	Configure DMA channel 2 for a data transfer	15.22 on p. 563
Sys_DMA_Channel2_Disable	Disable DMA channel 2	15.23 on p. 565
Sys_DMA_Channel2_Enable	Enable DMA channel 2	15.24 on p. 566
Sys_DMA_Channel3_Config	Configure DMA channel 3 for a data transfer	15.25 on p. 567
Sys_DMA_Channel3_Disable	Disable DMA channel 3	15.26 on p. 569
Sys_DMA_Channel3_Enable	Enable DMA channel 3	15.27 on p. 570
Sys_DMA_Clear_Channel0_Status	Clear the current status for DMA channel 0	15.28 on p. 571
Sys_DMA_Clear_Channel1_Status	Clear the current status for DMA channel 1	15.29 on p. 572
Sys_DMA_Clear_Channel2_Status	Clear the current status for DMA channel 2	15.30 on p. 573
Sys_DMA_Clear_Channel3_Status	Clear the current status for DMA channel 3	15.31 on p. 574
Sys_DMA_Clear_Status	Clear the current status for all of the DMA channels	15.32 on p. 575
Sys_DMA_Get_Channel0_Status	Get the current status of DMA channel 0	15.33 on p. 576
Sys_DMA_Get_Channel1_Status	Get the current status of DMA channel 1	15.34 on p. 577
Sys_DMA_Get_Channel2_Status	Get the current status of DMA channel 2	15.35 on p. 578
Sys_DMA_Get_Channel3_Status	Get the current status of DMA channel 3	15.36 on p. 579
Sys_DMA_Set_Channel0_DestAddress	Set the base destination address for DMA channel 0	15.37 on p. 580
Sys_DMA_Set_Channel0_SourceAddress	Set the base source address for DMA channel 0	15.38 on p. 581
Sys_DMA_Set_Channel1_DestAddress	Set the base destination address for DMA channel 1	15.39 on p. 582
Sys_DMA_Set_Channel1_SourceAddress	Set the base source address for DMA channel 1	15.40 on p. 583
Sys_DMA_Set_Channel2_DestAddress	Set the base destination address for DMA channel 2	15.41 on p. 584
Sys_DMA_Set_Channel2_SourceAddress	Set the base source address for DMA channel 2	15.42 on p. 585
Sys_DMA_Set_Channel3_DestAddress	Set the base destination address for DMA channel 3	15.43 on p. 586
Sys_DMA_Set_Channel3_SourceAddress	Set the base source address for DMA channel 3	15.44 on p. 587

Table 64. NVIC Inline Functions

Function	Description	Reference
Sys_NVIC_ClearAllPendingInt	Clear the pending status for all of the external interrupts	15.72 on p. 615
Sys_NVIC_DisableAllInt	Clear the enable for all of the external interrupts	15.73 on p. 616

CHAPTER 6
ARM Cortex-M3 Processor Startup Implementation

6.1 DESCRIPTION

The Ezairo 7100 firmware contains a CMSIS-compatible startup implementation that provides:

- A weakly-defined standard ARM Cortex-M3 processor vector table implementation
- Weakly-defined default interrupt request implementations for all ARM Cortex-M3 processor interrupts

This implementation can be linked into an application as a library, *sk4_isr_defaults.a*. If linking against this standard startup implementation, a user application can redefine any of the blocks that form this startup implementation to override the default implementations. Here is a list of components which can be overridden by defining a corresponding component with the same name in your own application:

- `sk4_vector_table_entry SK4_ISR_Vector_Table[]`
- `unsigned char SK4_Initial_SP[]`
- `void NMI_Handler(void)`
- `void HardFault_Handler(void)`
- `void MemManage_Handler(void)`
- `void BusFault_Handler(void)`
- `void UsageFault_Handler(void)`
- `void SVC_Handler(void)`
- `void DebugMon_Handler(void)`
- `void PendSV_Handler(void)`
- `void SysTick_Handler(void)`
- `void FIFO_0_IRQHandler(void)`
- `void FIFO_1_IRQHandler(void)`
- `void FIFO_2_IRQHandler(void)`
- `void FIFO_7_IRQHandler(void)`
- `void CFX_0_IRQHandler(void)`
- `void CFX_1_IRQHandler(void)`
- `void CFX_2_IRQHandler(void)`
- `void CFX_3_IRQHandler(void)`
- `void CFX_4_IRQHandler(void)`
- `void CFX_5_IRQHandler(void)`
- `void CFX_6_IRQHandler(void)`
- `void CFX_7_IRQHandler(void)`
- `void DMA0_IRQHandler(void)`
- `void DMA1_IRQHandler(void)`
- `void DMA2_IRQHandler(void)`
- `void DMA3_IRQHandler(void)`
- `void GPIO_IRQHandler(void)`
- `void SPI_RX_IRQHandler(void)`
- `void SPI_TX_IRQHandler(void)`
- `void SPI_ERROR_IRQHandler(void)`
- `void PCM_RX_IRQHandler(void)`
- `void PCM_TX_IRQHandler(void)`
- `void PCM_ERROR_IRQHandler(void)`
- `void I2C_IRQHandler(void)`
- `void CVSD_ENC_IN_RDY_IRQHandler(void)`

- void CVSD_ENC_OUT_RDY_IRQHandler(void)
- void CVSD_DEC_IN_RDY_IRQHandler(void)
- void CVSD_DEC_OUT_RDY_IRQHandler(void)
- void G722_ENC_IN_RDY_IRQHandler(void)
- void G722_ENC_OUT_RDY_IRQHandler(void)
- void G722_DEC_IN_RDY_IRQHandler(void)
- void G722_DEC_OUT_RDY_IRQHandler(void)
- void RS_ENC_IN_RDY_IRQHandler(void)
- void RS_ENC_OUT_RDY_IRQHandler(void)
- void RS_DEC_IN_RDY_IRQHandler(void)
- void RS_DEC_OUT_RDY_IRQHandler(void)
- void ECB_STATUS_IRQHandler(void)
- void ACB_STATUS_IRQHandler(void)

CHAPTER 7
CFX Math Library

7.1 DESCRIPTION

This chapter describes the mathematical library deployed for the CFX processor.

The purpose of this library is to provide developers with commonly used mathematical functions that support the development of digital signal processing algorithms. This library offers optimized mathematical operations in macros and functions. Additionally, since the source code is provided, these macros can be examples for implementing other desired mathematical functions. All provided routines are optimized for speed of execution and small memory usage, within a self-contained code module. Their use is recommended, as they simplify programming by eliminating the need to write basic functions.

In the following sections, you will find:

- A list of the math library functions
- Context and usage of the functions, including a description of the naming convention used
- Descriptions of the data formats used
- Details of the functions including accuracy and precision of results, especially for functions using lookup table-based approximations
- Assembly code examples that use the math library

7.2 CONTENT OF THE LIBRARY

The library provides the mathematical and digital signal processing operations listed in Table 65.

Table 65. Operations Included in the CFX Math Library

Function	Operation
$y = \sqrt{x}$	Square root of *x*
$y = \log_2 x$	Base 2 logarithm of *x*
$y = 2^x$	Base 2 exponent of *x*
$y = \frac{1}{x}$	Reciprocal of *x*
$y = \frac{a}{b}$	Division *a*/*b*
$y = t0 + x*(t1 - t0)$	Linear interpolation (typically for table lookup operations)
$y(n) = a*x(n) + (1 - a)*y(n - 1)$	Exponential averaging filter (first order)
$y(n) = a*x(n) + (1 - a)*y(n - 1)$ if $x(n) \geq y(n - 1)$ $y(n) = b*x(n) + (1 - b)*y(n - 1)$ if $x(n) < y(n - 1)$	Attack-release smoothing filter (first order) with *a* controlling the attack time and *b* the release time
$y = 20*\log_{10} x$	RMS to dB conversion

Table 65. Operations Included in the CFX Math Library

Function	Operation
$y = 10^{\frac{x}{20}}$	dB to RMS conversion
$y = 10*\log_{10}x$	Power to dB conversion
$y = 10^{\frac{x}{10}}$	dB to power conversion
$y = 20*\log_{2}x$	RMS to dB (base2) conversion
$y = 2^{\frac{x}{20}}$	dB (base 2) to RMS conversion
$y = 10*\log_{2}x$	Power to dB (base 2) conversion
$y = 2^{\frac{x}{10}}$	dB (base 2) to power conversion
$y = real(x)^2 + imag(x)^2$	Squared magnitude of a complex number
$y = \sqrt{real(x)^2 + imag(x)^2}$	Magnitude of a complex number
$y = sin(x)$	Sine function
$y = cos(x)$	Cosine function
Fixed-point-to-floating-point conversion	Convert data represented in one of the fixed-point formats to floating-point format
Floating-point-to-fixed-point conversion	Convert data represented in floating-point format to one of the fixed-point formats

Where required, several macros are provided for the same mathematical operation in order to support different numerical data formats (in either input or output; see Section 7.4, “Numeric Types” on page 78 for more details).

For some of the mathematical operations, different versions of the macros are provided to process either scalar or vector inputs optimally. The vector-based macros are particularly useful to process data in the frequency domain, when the same math operation is required on every band. In such a case, data corresponding to the successive bands must be stored in a vector, where the elements of the vector occupy consecutive locations in X memory (for 24-bit data) or XY memory (for 48-bit data).

All the macros and functions in the library are listed in Table 66 on page 73 and Table 67 on page 76. Each macro and function is cross-referenced to the page with detailed information, including supported formats, calling parameters, modified registers, returned values and execution time.

Table 66. CFX Math Library Macro Reference Functions

Function	Description	Reference
MATHLIB_ATTACK_RELEASE	Calculate a first-order attack-release	17.1 on p. 655
MATHLIB_ATTACK_RELEASE_VEC	Vector-wise first-order attack-release	17.2 on p. 656
MATHLIB_CONV_fl8n16_frac48	Convert fl8n16 format to frac48 format	17.3 on p. 657

Table 66. CFX Math Library Macro Reference Functions (Continued)

Function	Description	Reference
MATHLIB_CONV_fl8n16_m24p24	Convert fl8n16 format to m24p24 format	17.4 on p. 658
MATHLIB_CONV_fl8n16_m8p16	Convert fl8n16 format to m8p16 format	17.5 on p. 659
MATHLIB_CONV_frac24_fl8n16	Convert frac24 format to fl8n16 format	17.6 on p. 660
MATHLIB_CONV_frac48_fl8n16	Convert frac48 format to fl8n16 format	17.7 on p. 661
MATHLIB_CONV_int24_fl8n16	Convert int24 format to fl8n16 format	17.8 on p. 662
MATHLIB_CONV_m24p24_fl8n16	Convert m24p24 format to fl8n16 format	17.9 on p. 663
MATHLIB_CONV_m8p16_fl8n16	Convert m8p16 format to fl8n16 format	17.10 on p. 664
MATHLIB_CONV_VEC_fl8n16_frac48	Convert vector from fl8n16 format to frac48 format	17.11 on p. 665
MATHLIB_CONV_VEC_fl8n16_m24p24	Convert vector from fl8n16 format to m24p24 format	17.12 on p. 666
MATHLIB_CONV_VEC_fl8n16_m8p16	Convert vector from fl8n16 format to m8p16 format	17.13 on p. 667
MATHLIB_CONV_VEC_frac24_fl8n16	Convert vector from frac24 format to fl8n16 format	17.14 on p. 668
MATHLIB_CONV_VEC_frac48_fl8n16	Convert vector from frac48 format to fl8n16 format	17.15 on p. 669
MATHLIB_CONV_VEC_int24_fl8n16	Convert vector from int24 format to fl8n16 format	17.16 on p. 670
MATHLIB_CONV_VEC_m24p24_fl8n16	Convert vector from m24p24 format to fl8n16 format	17.17 on p. 671
MATHLIB_CONV_VEC_m8p16_fl8n16	Convert vector from m8p16 format to fl8n16 format	17.18 on p. 672
MATHLIB_COS_frac24_frac24	Approximate the cosine of a frac24 input	17.19 on p. 673
MATHLIB_COS_VEC_frac24_frac24	Approximate the cosine of each frac24 element in a vector	17.20 on p. 674
MATHLIB_DB10_PWR_m8p16_frac48	Convert from dB10 unit to power value	17.21 on p. 675
MATHLIB_DB10_PWR_VEC_m8p16_frac48	Convert vector of values from dB10 units to power values	17.22 on p. 676
MATHLIB_DB10_RMS_m8p16_frac48	Convert from dB10 unit to RMS value	17.23 on p. 677
MATHLIB_DB10_RMS_VEC_m8p16_frac48	Convert vector of values from dB10 units to RMS values	17.24 on p. 678
MATHLIB_DB2_PWR_m8p16_frac48	Convert from dB2 unit to power value	17.25 on p. 679
MATHLIB_DB2_PWR_VEC_m8p16_frac48	Convert vector of values from dB2 units to power values	17.26 on p. 680
MATHLIB_DB2_RMS_m8p16_frac48	Convert from dB2 unit to RMS value	17.27 on p. 681
MATHLIB_DB2_RMS_VEC_m8p16_frac48	Convert vector of values from dB2 units to RMS values	17.28 on p. 682
MATHLIB_DIVIDE_frac24	Divide positive values and produce a frac24 quotient	17.29 on p. 683
MATHLIB_DIVIDE_int24	Divide positive values and produce an int24 quotient	17.30 on p. 684
MATHLIB_DIVIDE_m24p24	Divide positive values and produce an m24p24 quotient	17.31 on p. 685
MATHLIB_EXP2_m8p16_frac48	Approximate 2^x for m8p16 input x, with frac48 output	17.32 on p. 686
MATHLIB_EXP2_m8p16_m8p16	Approximate 2^x for m8p16 input x, with m8p16 output	17.33 on p. 687
MATHLIB_EXP2_VEC_m8p16_frac48	Approximate 2^x for each m8p16 element in a vector, with frac48 output values	17.34 on p. 688
MATHLIB_EXP2_VEC_m8p16_m8p16	Approximate 2^x for each m8p16 element in a vector, with m8p16 output values	17.35 on p. 689
MATHLIB_EXP_AVG	Calculate a first-order exponential average	17.36 on p. 690
MATHLIB_EXP_AVG_VEC	Vector-wise first-order exponential average	17.37 on p. 691

Table 66. CFX Math Library Macro Reference Functions (Continued)

Function	Description	Reference
MATHLIB_LIN_INTERP	Linear interpolation	17.38 on p. 692
MATHLIB_LOG2_fl8n16_m8p16	Approximate the base-2 logarithm of a fl8n16 input	17.39 on p. 693
MATHLIB_LOG2_frac24_m8p16	Approximate the base-2 logarithm of a frac24 input	17.40 on p. 694
MATHLIB_LOG2_frac48_m8p16	Approximate the base-2 logarithm of a frac48 input	17.41 on p. 695
MATHLIB_LOG2_m8p16_m8p16	Approximate the base-2 logarithm of an m8p16 input	17.42 on p. 696
MATHLIB_LOG2_VEC_fl8n16_m8p16	Approximate log2 of each fl8n16 element in a vector	17.43 on p. 697
MATHLIB_LOG2_VEC_frac24_m8p16	Approximate log2 of each frac24 element in a vector	17.44 on p. 698
MATHLIB_LOG2_VEC_frac48_m8p16	Approximate log2 of each frac48 element in a vector	17.45 on p. 699
MATHLIB_LOG2_VEC_m8p16_m8p16	Approximate log2 of each m8p16 element in a vector	17.46 on p. 700
MATHLIB_MAG_frac24_m2p46	Calculate the magnitude of a complex number	17.47 on p. 701
MATHLIB_MAG_VEC_frac24_m2p46	Vector-wise magnitude of complex values	17.48 on p. 702
MATHLIB_PWR_DB10_frac48_m8p16	Convert from power value to dB10 unit	17.49 on p. 703
MATHLIB_PWR_DB10_VEC_frac48_m8p16	Convert vector of power values to dB10 units	17.50 on p. 704
MATHLIB_PWR_DB2_frac48_m8p16	Convert from power value to dB2 unit	17.51 on p. 705
MATHLIB_PWR_DB2_VEC_frac48_m8p16	Convert vector of power values to dB2 units	17.52 on p. 706
MATHLIB_RECIP_fl8n16_fl8n16	Approximate the reciprocal of a signed fl8n16 input	17.53 on p. 707
MATHLIB_RECIP_frac48_fl8n16	Approximate the reciprocal of a signed frac48 input	17.54 on p. 708
MATHLIB_RECIP_int24_fl8n16	Approximate the reciprocal of a signed int24 input	17.55 on p. 709
MATHLIB_RECIP_m8p16_fl8n16	Approximate the reciprocal of a signed m8p16 input	17.56 on p. 710
MATHLIB_RECIP_POS_fl8n16_fl8n16	Approximate the reciprocal of a positive fl8n16 input	17.57 on p. 711
MATHLIB_RECIP_POS_frac48_fl8n16	Approximate the reciprocal of a positive frac48 input	17.58 on p. 712
MATHLIB_RECIP_POS_int24_fl8n16	Approximate the reciprocal of a positive int24 input	17.59 on p. 713
MATHLIB_RECIP_POS_m8p16_fl8n16	Approximate the reciprocal of a positive m8p16 input	17.60 on p. 714
MATHLIB_RECIP_POS_VEC_fl8n16_fl8n16	Approximate the reciprocal of each positive fl8n16 element in a vector	17.61 on p. 715
MATHLIB_RECIP_POS_VEC_frac48_fl8n16	Approximate the reciprocal of each positive frac48 element in a vector	17.62 on p. 716
MATHLIB_RECIP_POS_VEC_int24_fl8n16	Approximate the reciprocal of each positive int24 element in a vector	17.63 on p. 717
MATHLIB_RECIP_POS_VEC_m8p16_fl8n16	Approximate the reciprocal of each positive m8p16 element in a vector	17.64 on p. 718
MATHLIB_RECIP_VEC_fl8n16_fl8n16	Approximate the reciprocal of each signed fl8n16 element in a vector	17.65 on p. 719
MATHLIB_RECIP_VEC_frac48_fl8n16	Approximate the reciprocal of each signed frac48 element in a vector	17.66 on p. 720
MATHLIB_RECIP_VEC_int24_fl8n16	Approximate the reciprocal of each signed int24 element in a vector	17.67 on p. 721
MATHLIB_RECIP_VEC_m8p16_fl8n16	Approximate the reciprocal of each signed m8p16 element in a vector	17.68 on p. 722
MATHLIB_RMS_DB10_frac48_m8p16	Convert from RMS value to dB10 unit	17.69 on p. 723

Table 66. CFX Math Library Macro Reference Functions (Continued)

Function	Description	Reference
MATHLIB_RMS_DB10_VEC_frac48_m8p16	Convert vector of RMS values to dB10 units	17.70 on p. 724
MATHLIB_RMS_DB2_frac48_m8p16	Convert from RMS value to dB2 unit	17.71 on p. 725
MATHLIB_RMS_DB2_VEC_frac48_m8p16	Convert vector of RMS values to dB2 units	17.72 on p. 726
MATHLIB_SIN_frac24_frac24	Approximate the sine of a frac24 input	17.73 on p. 727
MATHLIB_SIN_VEC_frac24_frac24	Approximate the sine of each frac24 element in a vector	17.74 on p. 728
MATHLIB_SQMAG_frac24_m2p46	Calculate the squared magnitude of complex number x	17.75 on p. 729
MATHLIB_SQMAG_VEC_frac24_m2p46	Vector-wise squared magnitude of complex values	17.76 on p. 730
MATHLIB_SQRT_frac24_frac48	Approximate the square root of a frac24 input	17.77 on p. 731
MATHLIB_SQRT_frac48_frac48	Approximate the square root of a frac48 input	17.78 on p. 732
MATHLIB_SQRT_VEC_frac24_frac48	Approximate the square root of each frac24 element in a vector	17.79 on p. 733
MATHLIB_SQRT_VEC_frac48_frac48	Approximate the square root of each frac48 element in a vector	17.80 on p. 734

Table 67. CFX Math Library Function Reference Functions

Function	Description	Reference
_MATHLIB_DIVIDE	Divide a non-negative dividend by a positive divisor	18.1 on p. 735
_MATHLIB_EXP2_m8p16_frac48	Approximate 2^x for m8p16 input x, with frac48 output	18.2 on p. 736
_MATHLIB_LOG2_frac48_m8p16	Approximate the base-2 logarithm of a frac48 input	18.3 on p. 737
_MATHLIB_RECIP_GEN_frac48_fl8n16	Approximate the reciprocal of a signed frac48 input	18.4 on p. 738
_MATHLIB_RECIP_POS_frac48_fl8n16	Approximate the reciprocal of a positive frac48 input	18.5 on p. 739
_MATHLIB_SQRT_frac48_frac48	Approximate the square root of a frac48 input	18.6 on p. 740

The library is organized into two parts: macros available from the include file *sk4_math.cfi* and functions stored in the source file *sk4_mathlib.cfs*. The functions—which are the core of the macros—are stored together with the associated tables in the ROM of the DSP. You can examine the source code, but direct calls to the core functions are usually unnecessary. Using the macros will suffice in most situations.

IMPORTANT: All of the math library macros and functions modify the status register because of the operations done within them. Also, for vector-based macros, ensure that at least one loop is available in the loop stack. For fixed-point-to-floating-point conversion macros, enable Flag Updates mode.

7.3 Math Library Usage

To access the math functions, the application source code must include the associated system library file, *sk4_math.cfi*. This include file contains the function declarations for the math library as well as the math library macros. The project must then link against the appropriate binary library. Both RAM- and ROM-based libraries are available. See Section 2.4.1, “ROM-Based Libraries” on page 30 for more information. Both versions of the library use the same lookup tables, located in X-memory. The lookup tables are in separate sections of memory from the code that implements the library functions.

This example calculates the square root of the number 0.25, previously stored in the accumulator a0 as a 48-bit fixed-point fractional number. After execution, the 48-bit result (0.5) becomes available in a0.

```
// Example of math library call using a math library macro

#include <sk4_math.cfi>          // Include math library file

    load a0, 0x200000            // Load input (0.25) in a0

    MATHLIB_SQRT_frac48_frac48   // Call SQRT function for 48-bit frac
                                 //   input and output

    // Result (approx. 0.5) can be read in a0
```

7.3.1 Naming Conventions

The name of a math library macro follows the convention as described below. Note that all math library macros are prefixed with MATHLIB_ to indicate that they are part of the math library.

MATHLIB_**<Math Op Name>_<Input data format>_<Output data format>**

In situations where the data format can be user-defined, it is omitted in the macro name. When only one data format is specified (as in the divide macros), it is the output format.

When the macro is intended for use with vector inputs, "VEC" is added after the name of the macro:

MATHLIB_**<Math Op Name>**_VEC_**<Input data format>_<Output data format>**

For instance, the SQRT operation is supplied in 4 library macros. One of the macros calculates the square root (SQRT) of a fractional 48-bit input value, and provides the result as a fractional 48-bit value. It would be launched as follows:

```
MATHLIB_SQRT_frac48_frac48
```

For all single-input scalar-based macros, such as the SQRT macro above, the input data is to be available in the accumulator a0 before launching the routine. The result of the calculation is returned in the accumulator a0.

Another SQRT macro (named MATHLIB_SQRT_frac24_frac48) expects a 24-bit fractional input. As in the previous case, it returns a 48-bit result. Here the input occupies only the high part of the accumulator (a0.h).

For both macros, equivalent vector-based versions are also available. They are named MATHLIB_SQRT_VEC_frac48_frac48 and MATHLIB_SQRT_VEC_frac24_frac48 respectively. The 48-bit input version of the macros would be launched as follows:

```
MATHLIB_SQRT_VEC_frac48_frac48(XP_IN, XP_OUT, VEC_LENGTH)
```

In both cases, an input vector of data is expected at the location pointed to by the pointer register XP_IN, and the vector length must be specified as the third macro parameter VEC_LENGTH. The results are returned, starting at location addressed by XP_OUT. All vector-based macros use this scheme. They assume input/output data in X memory (for 24-bit data) or XY memory (for 48-bit data).

7.4 Numeric Types

This section describes the different numeric types supported in the Math Library. Based on the dynamic range characteristics of the signals going through the math functions, seven different numerical formats are provided to represent data. The CFX native two's-complement fixed-point format is deployed in six variants using either 24 or 48 bits (frac24, frac48, m8p16, m24p24, m2p46, int24). The most significant bit (MSB) is always the sign bit, and only the position of the decimal point changes among those formats. In addition, a 24-bit floating-point format is introduced for operations requiring a wider dynamic range.

Table 68. Numerical Formats Used in the Math Library

Format	Description	Range
frac24	24-bit fractional fixed-point format	$[-1, \ldots -2^{-23}, 0, 2^{-23}, \ldots, 1-2^{-23}]$
frac48	48-bit fractional fixed-point format	$[-1, \ldots -2^{-47}, 0, 2^{-47}, \ldots, 1-2^{-47}]$
m8p16	24-bit mixed fixed-point format with 8-bit signed integer and 16-bit unsigned fractional	$[-128, \ldots -2^{-16}, 0, 2^{-16}, \ldots, 128-2^{-16}]$
m24p24	48-bit mixed fixed-point format with 24-bit signed integer and 24-bit unsigned fractional	$[-2^{23}, \ldots -2^{-24}, 0, 2^{-24}, \ldots, 2^{23}-2^{-24}]$
m2p46	48-bit mixed fixed-point format with 2-bit signed integer and 46-bit unsigned fractional	$[-2, \ldots -2^{-46}, 0, 2^{-46}, \ldots, 2-2^{-46}]$
int24	24-bit fixed-point integer format	$[-2^{23}, \ldots -1, 0, 1, \ldots, 2^{23} - 1]$
fl8n16	24-bit floating-point format with 8-bit signed exponent and 16-bit signed mantissa	$[-2^{127}, \ldots, -2^{-128}, \ldots, -2^{-128-15}, 0, 2^{-128-15}, \ldots, (2^{-128})*(1-2^{-15}), \ldots, (2^{127})*(1-2^{-15})]$

The following subsections describe each format in greater detail.

7.4.1 24-bit Fractional (frac24)

This fractional fixed-point format represents data in the range [-1, 1) using a 24-bit two's-complement value. The MSB is the sign bit, making the resolution equal to 2^{-23}. As for all fractional formats, the implicit decimal point is between the sign bit and the next lesser significant bit.

7.4.2 48-bit Fractional (frac48)

In this fractional fixed-point format, data in the range [-1, 1) is represented using a 48-bit two's-complement value. The MSB is the sign bit, making the resolution equal to 2^{-47}. As for all fractional formats, the implicit decimal point is between the sign bit and the next lesser significant bit.

This format extends the fractional representation to 48 bits, for use in operations that need very high precision. Both 24-bit words are loaded in parallel when accessing XY memory. However, if the 48-bit data is interleaved in X or Y memory, more processing cycles are required, since the access to the data would be done in two steps (loading the higher 24 bits first, and then the lower 24 bits).

7.4.3 24-bit Mixed (m8p16)

In this 8.16 mixed fixed-point format, data in range [-128, 128) is represented using a 24-bit two's-complement value. The eight MSBs represent the signed integer part, while the 16 least significant bits (LSBs) store the unsigned fractional part. The fractional resolution of numbers in this format is equal to 2^{-16}.

This format is a compromise between pure fractional and pure integer representation, in that it can represent numbers with both an integer and a fractional portion. This is useful for small numbers (128 as a maximum value) that need some fractional precision (up to 16-bit). This format is appropriate for representing sound levels in decibels.

7.4.4 48-bit Mixed (m24p24)

In this 24.24 mixed fixed-point format, data in range [-2^{23}, 2^{23}) is represented using a 48-bit two's-complement value. The 24 MSBs are dedicated to the signed integer part, while the 24 LSBs store the unsigned fractional part. The fractional resolution of numbers in this format is equal to 2^{-24}.

This format extends the 24-bit mixed format (m8p16) to 48-bit for situations in which a larger integer portion with good fractional resolution is needed. In fact, this format is the concatenation of the 24-bit integer format to the 24-bit unsigned fractional format.

7.4.5 48-bit Mixed (m2p46)

This format is used in routines calculating the magnitude and squared magnitude of complex numbers. In those operations, where the fractional values representing the real part and imaginary part of the input are frac48 numbers, the output can reach a value of 2. Hence, the frac48 format was modified to cope with this, leading to a 2.46 mixed format. Data in range [-2, 2) can be represented as a 48-bit two's-complement value. The two MSBs represent the signed integer part, while the 46 LSBs store the unsigned fractional part. The fractional resolution of numbers in this format is equal to 2^{-46}.

7.4.6 24-bit Integer (int24)

In this integer fixed-point format, data in the range [-2^{23}, 2^{23}) is represented using a 24-bit two's-complement value. All 24 bits are dedicated to the signed integer part with the decimal point to the right of the LSB bit. The fractional resolution of numbers in this format is 1.

This format is typically used in counting and indexing tasks, pattern matching, and error codes.

7.4.7 24-bit Floating-Point (fl8n16)

In this floating-point format, the 24 bits are split into an 8-bit signed exponent (an integer, located in the top eight bits), and a 16-bit signed mantissa. The exponent is expressed in base 2, and it represents the number of times the mantissa should be shifted to represent the original number in fixed-point. Data in range [-2^{127}, ..., -2^{-128},..., -2^{-143}, 0, 2^{-143}, ..., 2^{-128}, ..., 2^{127}) can be represented.

For macros that output a value in fl8n16 format, the mantissa of the output is always normalized. If the mantissa is zero, the exponent is also zero.

For macros that take an input value in fl8n16 format, the mantissa of the input is not required to be normalized. Zero is represented by a mantissa of zero with any value for the exponent.

This format is particularly adapted to the large dynamic range required in operations such as the reciprocal. Support for this format is provided through the conversion routines of the math library.

7.5 Math Library Macros

In the following sections, the mathematical operations provided by the library are described in more detail. The associated macros are listed, and their performance is addressed.

7.5.1 Square Root

The square root of a number is calculated by decomposing the input *x* into $x = m \cdot 2^E$, where *E* is the integer exponent and *m* is the mantissa in the range [0.5, 1.0). Therefore, the square root of *x* is calculated as:

$$\sqrt{x} = 2^{E/2} \cdot \sqrt{m}$$

for even *E*, or

$$\sqrt{x} = 2^{(E+1)/2} \cdot \sqrt{m/2}$$

for odd *E*.

In the code, *E* is calculated using the CFX log2abs instruction, and the square root of the mantissa is extracted from a lookup table using the five MSBs of the input value as the index number. The lookup table has 32 entries, where the first 16 correspond to $\sqrt{m/2}$ and the last half of the table includes values of $\sqrt{m}$. The lookup table is stored in ROM, starting at `D_LUT_SQRT_BASE`.

IMPORTANT: The input value x=0 is not allowed. An undefined result will be returned. If the value of x might be zero, the user must test the input value before launching the function.

Four macros are provided for calculating the square root, supporting scalar or vectors and 24- or 48-bit fractional fixed-point inputs. In all cases, the input must be a positive non-zero number. All output results are 48-bit fractional values.

```
// 48-bit input scalar
MATHLIB_SQRT_frac48_frac48
// 48-bit input vector
MATHLIB_SQRT_VEC_frac48_frac48(XP_IN, XP_OUT, VEC_LENGTH)
// 24-bit input scalar
MATHLIB_SQRT_frac24_frac48
// 24-bit input vector
MATHLIB_SQRT_VEC_frac24_frac48(XP_IN, XP_OUT, VEC_LENGTH)
```

All macros use the core function `_MATHLIB_SQRT_frac48_frac48`, after converting the input format if necessary.

The precision of the square root functions is the same for 24-bit and 48-bit inputs. The relative calculation error is 0.56% on average. It reaches the maximum of 1.55% for results very close to zero. The average absolute error is ±0.0035, while its maximum stands at ±0.011. This occurs at the middle of the input range, for $\sqrt{(x = 0.5)}$.

Table 69. Properties for SQRT Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_SQRT_frac48_frac48, MATHLIB_SQRT_VEC_frac48_frac48	(0, 1)	avg: 0.56% max: 1.55%	avg: ±0.0035 max: ±0.011	The input value x=0 is not valid, since an undefined result would be returned
MATHLIB_SQRT_frac24_frac48, MATHLIB_SQRT_VEC_frac24_frac48	(0, 1)	avg: 0.56% max: 1.55%	avg: ±0.0035 max: ±0.011	

7.5.2 Base-2 Logarithm

The log2(x) function is a key component of many audio algorithms because it is used as part of the conversion from signal power to signal level in decibels. For level measurements, the required precision is small, since:

$$\text{Level(dB)} = 10\log_{10}x = 3.0103\log_2 x \approx 3\log_2 x$$

The base-2 logarithm of a number is calculated based on decomposing the input *x* into $x = m \cdot 2^E$, where *E* is the integer exponent and *m* is the mantissa in the range [0.5, 1.0). Therefore, the base-2 logarithm of *x* can be calculated as:

$$\log_2 x = \log_2(m \cdot 2^E) = \log_2 m + E$$

In the code, the exponent E is determined by using the CFX log2abs instruction. The base-2 logarithm of *m* is extracted from a lookup table using the five MSBs of the input value as the index number. The lookup table has 32 entries, stored in ROM starting at `D_LUT_LOG2_BASE`.

Eight macros are provided for calculating the base-2 logarithm, supporting scalar or vector input and output data. The output format is always m8p16, while the input formats can be frac48, frac24, m8p16 or float (fl8n16). In all cases, the input must be a positive non-zero number. An undefined value is returned for zero input.

NOTE: The `vLinLog2` module in the HEAR microcode library performs a similar calculation. The `vLinLog2` module also returns an undefined value for a zero input, but the value returned is different from the value returned by the Math library base-2 logarithm for a zero input.

```
// 48-bit input scalar
MATHLIB_LOG2_frac48_m8p16
// 48-bit input vector
MATHLIB_LOG2_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
// 24-bit input scalar
MATHLIB_LOG2_frac24_m8p16
// 24-bit input vector
MATHLIB_LOG2_VEC_frac24_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
// Mixed input scalar
MATHLIB_LOG2_m8p16_m8p16
// Mixed input vector
MATHLIB_LOG2_VEC_m8p16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
// Float input scalar
MATHLIB_LOG2_fl8n16_m8p16
// Float input vector
MATHLIB_LOG2_VEC_fl8n16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
```

All macros use the core function `_MATHLIB_LOG2_frac48_m8p16`, after converting the input format to frac48 when necessary.

The average absolute error of the logarithm functions is ±0.008, except for the macro with floating-point input data which shows an average of ±0.010. The maximum reaches ±0.023 for all functions. This occurs when the input is as close to zero as possible for an input numerical format. The relative error of the logarithm functions is not meaningful, because the result of the log2 function is close to zero when the input is close to 1. In the case of float input, overflow of the result occurs when the input (which can be as small as 2^{-143} in this format) is less than 2^{-128}. All macros will provide indeterminate results in case of zero inputs.

Table 70. Properties for LOG2 Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_LOG2_frac48_m8p16, MATHLIB_LOG2_VEC_frac48_m8p16	(0, 1)	Not applicable	avg: ±0.008 max: ±0.023	Relative error is not meaningful because the log2 output can be very close to zero; the input value x=0 is not valid and will result in an undefined value being returned.
MATHLIB_LOG2_frac24_m8p16, MATHLIB_LOG2_VEC_frac24_m8p16	(0, 1)	Not applicable	avg: ±0.008 max: ±0.023	
MATHLIB_LOG2_m8p16_m8p16, MATHLIB_LOG2_VEC_m8p16_m8p16	(0, 128)	Not applicable	avg: ±0.008 max: ±0.023	
MATHLIB_LOG2_fl8n16_m8p16, MATHLIB_LOG2_VEC_fl8n16_m8p16	$[2^{-128}, 2^{127})$	Not applicable	avg: ±0.010 max: ±0.023	Overflow if input in range $[2^{-143}, 2^{-128})$; the input value x=0 is not valid and will result in an undefined value being returned.

7.5.3 Base-2 Exponent

The 2^x function is often used as part of the conversion from signal level in dB to signal power. For level measurements, the required precision is small, since:

$$\text{Power} = 10^{\text{Level}/10} = 2^{\text{Level} \cdot 0.33219}$$

The base-2 exponent of a number is calculated based on decomposing the input *x* into x = *E*+*r*, where *E* is the integer component and *r* is the fractional component. Therefore, the base-2 logarithm of *x* can be calculated as:

$$2^x = 2^E \cdot 2^r = 2^{E+1} \cdot 2^{r-1}$$

The second form is preferred, improving the precision of the calculation. The function first extracts the exponent *E*, which is the integer part of the input. Then, 2^{r-1} is approximated from a lookup table, using the five MSBs of (*r*-1) as the index number. Finally, the approximation from the table is shifted to the left by *E*+1 bits. The lookup table has 32 entries, and is stored in ROM, starting at location `D_LUT_EXP2_BASE`.

Four macros are provided for calculating the base-2 exponent, supporting scalar or vector input and output data. The input format is always m8p16, while the output formats can be frac48 or m8p16.

```
// 48-bit output scalar
MATHLIB_EXP2_m8p16_frac48
// 48-bit output vector
MATHLIB_EXP2_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)
```

```
// Mixed output scalar
MATHLIB_EXP2_m8p16_m8p16
// Mixed output vector
MATHLIB_EXP2_VEC_m8p16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
```

The first two macros must have negative inputs, as they cannot return values equal to or higher than 1 because of the fractional output format. The other macros can accept inputs as high as 7, thanks to the m8p16 format. For all macros, the output format limits the representation of very small values approaching zero. Using the 48-bit fractional format, the smallest output is 2^{-47}, which makes the minimum input equal to -47. To avoid underflows, smaller inputs are internally saturated to -47 before running the function. Similarly, for m8p16 output formats, the smallest output is 2^{-16}, which limits the input to a minimum value of -16. However, the macro `MATHLIB_EXP2_m8p16_m8p16` is actually a special case, because the output results are made available on the entire 48-bit accumulator, extending the precision of the fractional part from 16 to 16+24 = 40 bits. Hence, the actual minimum output value is not 2^{-16}, but 2^{-40}, just like in a m8p40 format. The user can use the extended output word and extend the input data range from [-16, 7) to [-40, 7). The inputs are saturated to -40 internally to avoid underflows.

In all cases, the core function `_MATHLIB_EXP2_m8p16_frac48` is used, converting its output to the appropriate format when necessary.

The precision of the exponent functions is different for fractional input format and for m8p16 format. In the first case, the absolute error is of the order $\pm 2^{-4}$ on average, but it can reach 0.011 for inputs close to zero (results close to 1). In the second case, this error is higher. While its average is about ±0.05, it can reach up to ± 1.2 for the largest outputs close to 2^7. The relative error is not meaningful, as the output value can be very close to zero for certain inputs.

Table 71. Properties for EXP2 Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_EXP2_m8p16_frac48, MATHLIB_EXP2_VEC_m8p16_frac48	[-47, 0)	Not applicable	avg: ±0.0002 max: ±0.011	Inputs < -47 will be saturated, resulting in an output of 0
MATHLIB_EXP2_m8p16_m8p16, MATHLIB_EXP2_VEC_m8p16_m8p16	[-40, 7)	Not applicable	avg: ±0.050 max: ±1.2	Inputs < -40 will be saturated, resulting in an output of 0

7.5.4 Reciprocal

The reciprocal function is approximated by expressing the input *x* as $x = m \cdot 2^E$, where *E* is the integer exponent and *m* is the mantissa in the range [0.5, 1.0). Therefore, the reciprocal of *x* can be calculated as:

$$\frac{1}{x} = \left(\frac{1}{m}\right) \cdot 2^{-E} = \frac{1}{2m} \cdot 2^{-E+1}$$

The second form is preferred in the code because $\frac{1}{2m}$ is a fractional number. *E* is calculated using the CFX instruction log2abs, and $\frac{1}{2m}$ is extracted from a lookup table using the five MSBs of the modified mantissa (*2m*) as the index number. Finally, the approximation from the table is shifted by -*E*+1 bits. The lookup table has 32 entries. It is stored in ROM, starting at `D_LUT_RECIP_BASE`.

There are 16 macros for calculating the reciprocal, supporting scalar and vector data for both input and output. Macros supporting scalar data use accumulator a0 for input and output, while macros supporting vector data require pointers to the input and output locations.

To take advantage of CFX capabilities, two sets of macros are provided: one for positive numbers only and the other for signed numbers. The macro for positive-only inputs is more computationally efficient. For both kinds of macros, several input formats (frac48, m8p16, fl8n16 and int24) are acceptable in different versions of the macros, while the output is always represented in floating-point format.

General Input

```
// 48-bit input scalar
MATHLIB_RECIP_frac48_fl8n16
// 48-bit input vector
MATHLIB_RECIP_VEC_frac48_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// Mixed input scalar
MATHLIB_RECIP_m8p16_fl8n16
// Mixed input vector
MATHLIB_RECIP_VEC_m8p16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// Float input scalar
MATHLIB_RECIP_fl8n16_fl8n16
// Float input vector
MATHLIB_RECIP_VEC_fl8n16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// Integer input scalar
MATHLIB_RECIP_int24_fl8n16
// Integer input vector
MATHLIB_RECIP_VEC_int24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
```

Positive Input

```
// 48-bit positive input scalar
MATHLIB_RECIP_POS_frac48_fl8n16
// 48-bit positive input vector
MATHLIB_RECIP_POS_VEC_frac48_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// Mixed positive input scalar
MATHLIB_RECIP_POS_m8p16_fl8n16
// Mixed positive input vector
MATHLIB_RECIP_POS_VEC_m8p16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// Float positive input scalar
MATHLIB_RECIP_POS_fl8n16_fl8n16
// Float positive input vector
MATHLIB_RECIP_POS_VEC_fl8n16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// Integer positive input scalar
MATHLIB_RECIP_POS_int24_fl8n16
// Integer positive input vector
MATHLIB_RECIP_POS_VEC_int24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_RECIP_GEN_frac48_fl8n16` is called by the general-input macros, while `_MATHLIB_RECIP_POS_frac48_fl8n16` is called by the positive-input macros. Conversion of the input to the appropriate format is performed when necessary.

The relative error of the reciprocal functions is 0.56% on average, except for the function with floating-point input data which has an average around 0.7%. The maximum relative error is smaller than 1.55% for all functions. Absolute

errors can become huge with very small inputs (large results). Overflow occurs with floating-point input formats, when the input is in the range (-2^{-127}, 2^{-127}). All macros will provide indeterminate results in case of zero inputs.

Table 72. Properties for RECIP Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_RECIP_frac48_fl8n16, MATHLIB_RECIP_VEC_frac48_fl8n16	[-1, 1)	avg: 0.56% max: 1.55%	Not applicable	Results and absolute errors are huge for inputs close to 0
MATHLIB_RECIP_m8p16_fl8n16, MATHLIB_RECIP_VEC_m8p16_fl8n16	[-128, 128)	avg: 0.56% max: 1.55%	Not applicable	
MATHLIB_RECIP_int24_fl8n16, MATHLIB_RECIP_VEC_int24_fl8n16	[-2^{23}, 2^{23})	avg: 0.56% max: 1.55%	Not applicable	
MATHLIB_RECIP_fl8n16_fl8n16, MATHLIB_RECIP_VEC_fl8n16_fl8n16	[-2^{127}, -2^{-127}] + [2^{-127}, 2^{127})	avg: 0.76% max: 1.55%	Not applicable	Overflow if input in range (-2^{-127}, 2^{-127})
MATHLIB_RECIP_POS_frac48_fl8n16, MATHLIB_RECIP_POS_VEC_frac48_fl8n16	(0, 1)	avg: 0.56% max: 1.55%	Not applicable	Results and absolute errors are huge for inputs close to 0
MATHLIB_RECIP_POS_m8p16_fl8n16, MATHLIB_RECIP_POS_VEC_m8p16_fl8n16	(0, 128)	avg: 0.56% max: 1.55%	Not applicable	
MATHLIB_RECIP_POS_int24_fl8n16, MATHLIB_RECIP_POS_VEC_int24_fl8n16	(0, 2^{23})	avg: 0.56% max: 1.55%	Not applicable	
MATHLIB_RECIP_POS_fl8n16_fl8n16, MATHLIB_RECIP_POS_VEC_fl8n16_fl8n16	[2^{-127}, 2^{127})	avg: 0.70% max: 1.55%	Not applicable	Overflow if input smaller than 2^{-127}

7.5.5 Division

The DIVIDE macros are implemented based on the division-step instruction of the CFX. These macros use the division-step instruction consecutively as many times as needed (depending on the number of bits in the data format).

Three macros are provided for division. Considering the heavy calculation load in such an operation, it is assumed that the division operation will not be run iteratively. Therefore, only scalar inputs are supported. Any kind of 24-bit user-defined fixed-point format is acceptable in input, while the output is either 24-bit integer, 48-bit mixed or 24-bit fractional.

```
MATHLIB_DIVIDE_int24            // Integer output scalars
MATHLIB_DIVIDE_m24p24           // Mixed output scalars
MATHLIB_DIVIDE_frac24           // Frac output scalars
```

All divide macros call the core function `_MATHLIB_DIVIDE`. They expect positive dividend and divisor numbers, represented in the same fixed-point format. The dividend is expected in location `a0.h`, while the divisor must be placed in `x0`. For `MATHLIB_DIVIDE_int24` and `MATHLIB_DIVIDE_frac24`, the quotient is returned in the lower part of the accumulator, `a0.l`. `MATHLIB_DIVIDE_m24p24`, on the other hand, returns the 48-bit quotient in `a0`. In the case of `MATHLIB_DIVIDE_int24`, the remainder of the division can be optionally determined by executing the instruction `add a0, x0, lt` after the macro.

The relative error is as small as 0.0016% and 0.00063% for the macros providing mixed and fractional outputs, respectively. This represents absolute errors of order $\pm10^{-7}$ and $\pm10^{-8}$ respectively. The error is zero for the int24 version of the macro, while the precision of the remainder reaches $\pm10^{-16}$.

Table 73. Properties for Division macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_DIVIDE_int24	User-defined, positive	0%	0	Remainder error smaller than 10^{-16}
MATHLIB_DIVIDE_m24p24	User-defined, positive	avg: 1.67×10^{-5}% max: 1.60×10^{-3}%	avg: $\pm 3.0 \times 10^{-8}$ max: $\pm 6.0 \times 10^{-8}$	Relative error and absolute error provided here assume fractional input data
MATHLIB_DIVIDE_frac24	User-defined, positive	avg: 2.1×10^{-5}% max: 6.3×10^{-4}%	avg: $\pm 5.7 \times 10^{-8}$ max: $\pm 1.2 \times 10^{-7}$	

7.5.6 Linear Interpolation

Based on a fractional ratio *x*, this macro performs the linear interpolation of two values *t0* and *t1*:

$$y = t0 + x \cdot (t1 - t0)$$

When x=0.5, the middle point between *t0* and *t1* is returned (arithmetic average).

The primary application of this function is to extract a value from a lookup table, when the input to the table is situated between two consecutive indexes. The ratio *x* is the ratio of the distance between the first value and the input value to the distance between the first value and the next value in the lookup table. (It is assumed that the first index is associated to the element *t0*, while the next index is associated to *t1*).

A single macro is provided for the linear interpolation. Its input parameter `XP_IN` is a pointer that addresses the location where *t0* is located in X memory. The value *t1* must be stored in the next memory word, and the 24-bit ratio *x* is expected in the high part of the accumulator (`a0.h`):

```
MATHLIB_LIN_INTERP(XP_IN)
```

The numerical format of the table elements *t0* and *t1* can be any fixed-point format. However, (*t1-t0*) must not exceed the range of the format used. Normally, when using a fractional format, (*t1-t0*) must be in range [-1, 1). This means that the table must contain smoothly organized elements (which is generally the case). *x* is a 24-bit fractional, usually in the range [0,1), although negative values are supported as well.

The relative error measured on the interpolation output can reach 0.0003% at maximum, which makes the absolute error smaller than $\pm 1.2 \times 10^{-7}$.

Table 74. Properties for the Linear Interpolation Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_LIN_INTERP	User-defined	avg: 1.9×10^{-5}% max: 3×10^{-4}%	avg: $\pm 5.9 \times 10^{-8}$ max: $\pm 1.2 \times 10^{-7}$	Relative error and absolute error provided here assume fractional input data

7.5.7 Exponential Average Filter

The first order exponential averaging filter can be expressed as the following difference equation:

$$y(n) = a \cdot x(n) + (1 - a) \cdot y(n - 1) = a \cdot (x(n) - y(n - 1)) + y(n - 1)$$

In actual implementation, the second form is used, where a is the coefficient of the filter and *x(n)* is the current input sample. *y(n)* and *y(n-1)* are the current and previous output samples respectively.

Two macros are provided for the exponential averaging operation:

```
MATHLIB_EXP_AVG(XP_OUT, XP_COEFF)                          // input scalar
MATHLIB_EXP_AVG_VEC(XP_IN, XP_OUT, XP_COEFF, VEC_LENGTH)   // input vector
```

The first macro expects a 24-bit scalar value in `a0.h` for the input *x(n)*, while the second accepts a vector in which the first element is addressed by `XP_IN`. In both cases, `XP_OUT` is a pointer to the previous output *y(n-1)*, and `XP_COEFF` is the pointer to the coefficient *a*. For the vector macro, there must be `VEC_LENGTH` number of values stored in memory starting from locations `XP_OUT` and `XP_COEFF`, specifying one previous output y(n-1) and one filter coefficient a for each input vector element. This allows multiple averaging filters to be applied independently to different input data. The result of the filtering is returned in `a0.h` for the scalar macro, and in the locations addressed by `XP_OUT` when using the vector macro.

The format of the input *x(n)* (and consequently of the output *y(n)*) can be any type of 24-bit fixed-point numerical representation. The filter coefficient(s) must be positive frac24 numbers. It is assumed that *(x(n)-y(n-1))* does not exceed the range of the format used. For example, *(x(n)-y(n-1))* must be in the range [-1, 1) for frac24 inputs.

The relative error measured on the filtered output can reach 0.056% at maximum, which makes the absolute error smaller than $\pm 1.2 \times 10^{-7}$.

Table 75. Properties for the Exponential Average Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_EXP_AVG, MATHLIB_EXP_AVG_VEC	User-defined	avg: 7.9×10^{-5}% max: 5.6×10^{-2}%	avg: $\pm 5.9 \times 10^{-8}$ max: $\pm 1.2 \times 10^{-7}$	Relative error and absolute error provided here assume fractional input data

7.5.8 Attack-Release Filter

First order attack-release filters are widely used in audio processing, for the smoothing of frame-by-frame sequences (like signal energy). The expression for the output of the filter *y(n)* is as follows:

$$y(n) = coeff \cdot x(n) + (1 - coeff) \cdot y(n - 1) = coeff \cdot (x(n) - y(n - 1)) + y(n - 1)$$

where:

coeff = a if *x(n) ≥ y(n-1)*

coeff = b if *x(n) < y(n-1)*.

In this expression, *x(n)* is the input sample, *y(n-1)* is the previous output, while *a* and *b* are the coefficient of the filter, used in case of attack or release respectively.

Two macros are provided for the attack-release filtering operation:

```
// input scalar
MATHLIB_ATTACK_RELEASE(XP_OUT, XP_COEFF)
// input vector
MATHLIB_ATTACK_RELEASE_VEC(XP_IN, XP_OUT, XP_COEFF, VEC_LENGTH)
```

The first macro expects a 24-bit scalar value in `a0.h` for the input *x(n)*. `XP_OUT` is a pointer to the previous output *y(n-1)*, and `XP_COEFF` is the pointer to the filter coefficient *a*. This coefficient must be immediately followed by coefficient *b*. The result of the filtering is returned. The second macro accepts a vector of inputs *x(n)*, whose first element is addressed by `XP_IN`. There must be `VEC_LENGTH` values stored in memory starting from location `XP_OUT` (one previous output *y(n-1*) per input vector element), and 2*`VEC_LENGTH` values stored in memory starting from location `XP_COEFF` (one coefficient *a* followed by one coefficient *b* per input vector element). The outputs *y(n)* are stored at the locations addressed by `XP_OUT`.

The format of the input *x(n*) (and consequently of the output *y(n)*) can be any type of 24-bit fixed-point numerical representation. The filter coefficients must be positive frac24 numbers. It is assumed that *(x(n)-y(n-1))* does not exceed the range of the used format. For example, *(x(n)-y(n-1))* must be in range [-1, 1) for frac24 inputs.

The relative error measured on the filtered output can reach 0.07% at maximum, which makes the absolute error smaller than $\pm 1.2 \times 10^{-7}$.

Table 76. Properties for the Attack-Release Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_ATTACK_RELEASE, MATHLIB_ATTACK_RELEASE_VEC	User-defined	avg: 7.3×10^{-5}% max: 7.0×10^{-2}%	avg: $\pm 5.9 \times 10^{-8}$ max: $\pm 1.2 \times 10^{-7}$	Relative error and absolute error provided here assume fractional input data

7.5.9 RMS-to-Decibel Conversion

Conversions from RMS (root-mean-square) to decibels are often required in audio processing, for calculating signal energy levels. The expression for the conversion is as follows:

$$y = 20 \cdot \log_{10} x = 20 \cdot \log_{10} 2 \cdot \log_2 x$$

In the implementation, the second form is used, taking advantage of the available log2 function.

Two macros are provided for the conversion operation. Only non-zero positive input numbers in the range (0,1) are supported. Scalar or vector inputs must be expressed in frac48, while the output is returned in m8p16 format, well-adapted for representing numbers in decibel levels:

```
// input scalar
MATHLIB_RMS_DB10_frac48_m8p16
// input vector
MATHLIB_RMS_DB10_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_LOG2_frac48_m8p16` is called by the macros to calculate the base 2 logarithm involved.

As the output format (m8p16) cannot represent values less than -128, overflow of the result returned in `a0.h` will occur when the input is smaller than $10^{(-128)/20}$. Within the valid input range, the absolute error of the conversion is ±0.050 on average, and always lower than ±0.133 dB. The relative error is not meaningful because the results of the logarithm can be very close to zero.

Table 77. Properties for the RMS-to-Decibel Conversion Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_RMS_DB10_frac48_m8p16, MATHLIB_RMS_DB10_VEC_frac48_m8p16	$[10^{-128/20}, 1)$	Not applicable	avg: ±0.050 max: ±0.133	Overflow if input smaller than $10^{-128/20}$

7.5.10 Decibel-to-RMS Conversion

The expression for the decibel-to-RMS (root-mean-square) conversion is as follows:

$$y = 10^{\frac{x}{20}} = 2^{\frac{x}{20 \cdot \log_{10} 2}}$$

In the implementation, the second form is used, taking advantage of the available exp2 function.

Two macros are provided for the conversion operation supporting scalar or vector inputs. The input level is expressed in m8p16 format.

```
// input scalar
MATHLIB_DB10_RMS_m8p16_frac48
// input vector
MATHLIB_DB10_RMS_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_EXP2_m8p16_frac48` is called by the macros to calculate the exponent involved.

As the output format (frac48) cannot represent values equal to or higher than 1, the input data must be negative. Overflow occurs if the input is zero (since $2^0 = 1$), unless saturation mode is set. Within the valid input range, the maximum absolute error of the conversion is lower than ±0.011. Highest absolute errors occur for outputs very close to 1. The relative error is smaller than 1.1%.

Table 78. Properties for the Decibel-to-RMS Conversion Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_DB10_RMS_m8p16_frac48, MATHLIB_DB10_RMS_VEC_m8p16_frac48	[-128, 0)	avg: 0.5% max: 1.1%	avg: $\pm 4.2 \times 10^{-4}$ max: ±0.011	Overflow / saturation if input is zero or positive

7.5.11 Power-to-Decibel Conversion

Power-to-decibel conversions are also widely needed in audio processing for calculating signal energy levels. The expression for the conversion is as follows:

$$y = 10 \cdot \log_{10} x = 10 \cdot \log_{10} 2 \cdot \log_2 x$$

In the implementation, the second form is used, taking advantage of the available log2 function.

Two macros are provided for the conversion operation. Only non-zero positive input numbers in the range (0,1) are supported. Scalar or vector inputs must be expressed in frac48, while the output is returned in m8p16 format:

```
// input scalar
MATHLIB_PWR_DB10_frac48_m8p16
// input vector
MATHLIB_PWR_DB10_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_LOG2_frac48_m8p16` is called by the macros to calculate the logarithm involved.

As the output format (m8p16) cannot represent values less than -128, overflow of the result returned in a0.h will occur when the input is smaller than $10^{-128/10}$. Within the valid input range, the absolute error of the conversion is ±0.024 on average, and always lower than ±0.067 decibels. The relative error is not meaningful because the results of the logarithm can be very close to zero.

Table 79. Properties for the Power to dB Conversion Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_PWR_DB10_frac48_m8p16, MATHLIB_PWR_DB10_VEC_frac48_m8p16	$[10^{-128/10}, 1)$	Not applicable	avg: ±0.024 max: ±0.067	Overflow if input smaller than $10^{-128/10}$

7.5.12 Decibel-to-Power Conversion

The expression for the conversion from decibel to power is as follows:

$$y = 10^{\frac{x}{10}} = 2^{\frac{x}{10 \cdot \log_{10} 2}}$$

In the implementation, the second form is used, taking advantage of the available exp2 function.

Two macros are provided for the conversion operation supporting scalar or vector inputs. The input level is expressed in m8p16 format, well adapted for representing numbers in decibel levels. The output is in frac48.

```
// input scalar
MATHLIB_DB10_PWR_m8p16_frac48
// input vector
MATHLIB_DB10_PWR_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_EXP2_m8p16_frac48` is called by the macros to calculate the exponent involved.

As the output format (frac48) cannot represent values equal or higher than 1, the input data must be negative. Overflow occurs if the input is zero (since $2^0 = 1$), unless saturation mode is set. Within the valid input range, the maximum absolute error of the conversion is lower than ±0.011. Highest absolute errors occur for outputs very close to 1. The relative error is smaller than 5%.

Table 80. Properties for the Decibel-to-Power Conversion Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_DB10_PWR_m8p16_frac48, MATHLIB_DB10_PWR_VEC_m8p16_frac48	[-128, 0)	avg: 0.59% max: 5.0%	avg: $\pm 2.3 \times 10^{-4}$ max: ±0.011	Overflow / saturation if input is zero or positive

7.5.13 RMS-to-Decibel (Base 2) Conversion

The expression for the RMS-to-decibel (base 2) conversion is as follows:

$$y = 20 \cdot \log_2 x$$

Two macros are provided for the conversion operation. Only non-zero positive input numbers in the range (0,1) are supported. Scalar or vector inputs must be expressed in frac48, while the output is returned in m8p16 format:

```
// input scalar
MATHLIB_RMS_DB2_frac48_m8p16
// input vector
MATHLIB_RMS_DB2_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_LOG2_frac48_m8p16` is called by the macros to calculate the logarithm involved.

As the output format (m8p16) cannot represent values less than -128, overflow of the result returned in `a0.h` will occur when the input is smaller than $2^{-128/20}$. Within the valid input range, the absolute error of the conversion is ±0.157 on average, and always lower than ±0.442. The relative error is not meaningful because the results of the logarithm can be very close to zero.

Table 81. Properties for the RMS-to-Decibel (Base 2) Conversion Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_RMS_DB2_frac48_m8p16, MATHLIB_RMS_DB2_VEC_frac48_m8p16	$[2^{-128/20}, 1)$	Not applicable	avg: ±0.157 max: ±0.442	Overflow if input smaller than $2^{-128/20}$

7.5.14 dB (Base 2)-to-RMS Conversion

The expression for the conversion from base-2 decibels to RMS (root-mean-square) is as follows:

$$y = 2^{\frac{x}{20}}$$

Two macros are provided for the conversion operation supporting scalar or vector inputs. The input level is expressed in m8p16 format. The output is in frac48.

```
// input scalar
MATHLIB_DB2_RMS_m8p16_frac48
// input vector
MATHLIB_DB2_RMS_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_EXP2_m8p16_frac48` is called by the macros to calculate the exponent involved.

As the output format (frac48) cannot represent values equal or higher than 1, the input data must be negative. Overflow occurs if the input is zero (since $2^0 = 1$), unless saturation mode is set. Within the valid input range, the maximum absolute error of the conversion is lower than ±0.011. Highest absolute errors occur for outputs very close to 1. The relative error is smaller than 1.1%.

Table 82. Properties for the dB (Base 2)-to-RMS Conversion Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_DB2_RMS_m8p16_frac48, MATHLIB_DB2_RMS_VEC_m8p16_frac48	[-128, 0)	avg: 0.54% max: 1.1%	avg: ±0.0012 max: ±0.011	Overflow / saturation if input is non-negative

7.5.15 Power-to-Decibel (Base 2) Conversion

The expression for the power-to-decibel (base 2) conversion is as follows:

$$y = 10 \cdot \log_2 x$$

Two macros are provided for the conversion operation. Only non-zero positive input numbers in the range (0,1) are supported. Scalar or vector inputs must be expressed in frac48, while the output is returned in m8p16 format:

```
// input scalar
MATHLIB_PWR_DB2_frac48_m8p16
// input vector
MATHLIB_PWR_DB2_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_LOG2_frac48_m8p16` is called by the macros to calculate the logarithm involved.

As the output format (m8p16) cannot represent values less than -128, overflow of the result returned in `a0.h` will occur when the input is smaller than $2^{-128/10}$. Within the valid input range, the absolute error of the conversion is ±0.077 on average, and always lower than ±0.22. The relative error is not meaningful because the results of the logarithm can be very close to zero.

Table 83. Properties for the Power to dB (Base 2) Conversion Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_PWR_DB2_frac48_m8p16, MATHLIB_PWR_DB2_VEC_frac48_m8p16	$[2^{-128/10}, 1)$	Not applicable	avg: ±0.077 max: ±0.22	Overflow if input smaller than $2^{-128/10}$

7.5.16 dB (Base 2)-to-Power Conversion

The expression for the base-2 decibels-to-power conversion is as follows:

$$y = 2^{\frac{x}{10}}$$

Two macros are provided for the conversion operation supporting scalar or vector inputs. The input level is expressed in m8p16 format, well-adapted to decibels. The output is in frac48.

```
// input scalar
MATHLIB_DB2_PWR_m8p16_frac48
// input vector
MATHLIB_DB2_PWR_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_EXP2_m8p16_frac48` is called by the macros to calculate the exponent involved.

As the output format (frac48) cannot represent values equal or higher than 1, the input data must be negative. Overflow occurs if the input is zero (since $2^0 = 1$), unless saturation mode is set. Within the valid input range, the maximum absolute error of the conversion is lower than ±0.011. Highest absolute errors occur for outputs very close to 1. The relative error is smaller than 1.1%.

Table 84. Properties for the dB (Base 2) to Power Conversion Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_DB2_PWR_m8p16_frac48, MATHLIB_DB2_PWR_VEC_m8p16_frac48	[-128, 0)	avg: 0.54% max: 1.1%	avg: $\pm 6.6 \times 10^{-4}$ max: ±0.011	Overflow / saturation if input is positive

7.5.17 Squared Magnitude

Calculation of the squared magnitude *y* of a complex number *x* is often needed, notably after a frequency-domain analysis. The operation is:

$$y = \text{real}(x)^2 + \text{imag}(x)^2$$

Two macros are provided for calculating the squared magnitude, one for scalar and the other for vector inputs. Both macros expect 24-bit fractional input data:

```
// input scalar
MATHLIB_SQMAG_frac24_m2p46
// input vector
MATHLIB_SQMAG_VEC_frac24_m2p46(XP_IN, XP_OUT, VEC_LENGTH)
```

The format of the output results (m2p46, see Section 7.4, “Numeric Types” on page 78) was specially defined for this operation, since the squared magnitude of fractional complex numbers is between 0 and 2. Using such a format, the calculation is performed without any quantization error. Since the input has an unsigned precision of only 23 bits and the output uses 46 bits to represent the fractional part of the result, the absolute error is zero.

Table 85. Properties for the SQMAG Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_SQMAG_frac24_m2p46, MATHLIB_SQMAG_VEC_frac24_m2p46	[-1, 1)	0%	0	Overflow occurs for input = -1-1j unless saturation mode is set

NOTE: As a special case, the squared magnitude of the number -1-1j (that can be represented as a 24-bit fractional) is equal to 2. Unfortunately, 2 just overflows the m2p46 format, and the result would be -2. Hence, saturation mode should be used to avoid this special situation.

7.5.18 Magnitude

The expression to calculate the magnitude *y* of a complex number *x* is as follows:

$$y = \sqrt{\text{real}(x)^2 + \text{imag}(x)^2}$$

Two macros are provided for calculating the magnitude, one for scalar and the other for vector inputs. Both macros expect 24-bit fractional input data:

```
// input scalar
MATHLIB_MAG_frac24_m2p46
// input vector
MATHLIB_MAG_VEC_frac24_m2p46(XP_IN, XP_OUT, VEC_LENGTH)
```

The format of the output results (m2p46, see Section 7.4, “Numeric Types” on page 78) was specially defined for this operation, since the magnitude of fractional complex numbers can be higher than 1.

The macro uses the core SQRT function named `_MATHLIB_SQRT_frac48_frac48`. As this function uses a table lookup for the determination of the square root, the magnitude result is an approximation. The relative error of the approximation is 0.57% on average, and 1.55% at maximum. The average absolute error is ±0.0044, while its maximum reaches ±0.022 for the highest output value.

IMPORTANT: The input value x=0+0j is not allowed because the SQRT function does not support 0 inputs. An undefined result would be returned. If necessary, the user must test the input value before launching the function.

IMPORTANT: The MAG function calls the SQMAG function, so the input value x=-1-j causes the same issue for MAG as it does for SQMAG. Specifically, it will overflow if saturation mode is not set.

Table 86. Properties for the MAG Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_MAG_frac24_m2p46, MATHLIB_MAG_VEC_frac24_m2p46	[-1, 1)	avg: 0.57% max: 1.55%	avg: ±0.0044 max: ±0.022	The input value 0+0j is not allowed; the input value -1-j will cause overflow unless saturation mode is set

7.5.19 Sine Function

The sine function:

$$y = \sin(x)$$

is implemented by means of a table lookup. The supported input and output formats are 24-bit fractional. Both scalar and vector versions are implemented. Input data are expected in normalized radian units, where the radian range [-π, +π) is re-mapped to the fractional range [-1, 1).

```
// input scalar
MATHLIB_SIN_frac24_frac24
// input vector
MATHLIB_SIN_VEC_frac24_frac24(XP_IN, XP_OUT, VEC_LENGTH)
```

The average absolute error of the sine function is ±0.004, while its maximum reaches ±0.013. This occurs for outputs very close to zero. The relative error is not meaningful, as results can be very close to zero.

Table 87. Properties for the SIN Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_SIN_frac24_frac24, MATHLIB_SIN_VEC_frac24_frac24	[-1, 1)	Not applicable	avg: ±0.004 max: ±0.013	The relative error is not meaningful, as results can be very close to zero

7.5.20 Cosine Function

The cosine function:

$$y = \cos(x)$$

is implemented by means of a table lookup. The supported input and output formats are 24-bit fractional. Both scalar and vector versions are implemented. Input data are expected in normalized radian units, where the radian range [-π, +π) is re-mapped to the fractional range [-1, 1).

```
// input scalar
MATHLIB_COS_frac24_frac24
// input vector
MATHLIB_COS_VEC_frac24_frac24(XP_IN, XP_OUT, VEC_LENGTH)
```

The average absolute error of the cosine function is ±0.004, while its maximum reaches ±0.013 This occurs for outputs very close to zero. The relative error is not meaningful, as results can become very close to zero.

Table 88. Properties for the COS Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_COS_frac24_frac24, MATHLIB_COS_VEC_frac24_frac24	[-1, 1)	Not applicable	avg: ±0.004 max: ±0.013	The relative error is not meaningful, as results can be very close to zero

7.5.21 Fixed-Point-to-Floating-Point Conversion

Several conversion routines are provided to convert data represented in one of the fixed-point formats (frac24, frac48, m8p16, m24p24, int24) to the floating-point format fl8n16. Such routines are particularly useful when the

programmer wishes to use one of the library routines with floating-point input data. All the provided conversion routines make use of the log2abs instruction available in the CFX.

IMPORTANT: For fixed-point-to-floating-point conversion macros, enable Flag Updates mode.

The following macros are available, depending on the input format and the form of the input (scalar or vector):

```
// mixed 24.24 input scalar
MATHLIB_CONV_m24p24_fl8n16
// mixed 24.24 input vector
MATHLIB_CONV_VEC_m24p24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// mixed 8.16 input scalar
MATHLIB_CONV_m8p16_fl8n16
// mixed 8.16 input vector
MATHLIB_CONV_VEC_m8p16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// integer input scalar
MATHLIB_CONV_int24_fl8n16
// integer input vector
MATHLIB_CONV_VEC_int24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// frac48 input scalar
MATHLIB_CONV_frac48_fl8n16
// frac48 input vector
MATHLIB_CONV_VEC_frac48_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// frac24 input scalar
MATHLIB_CONV_frac24_fl8n16
// frac24 input vector
MATHLIB_CONV_VEC_frac24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
```

All the conversions provide good precision as the relative error is maintained below 0.006%.

Table 89. Properties for Fixed-Point-to-Floating-Point Conversion Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_CONV_m24p24_fl8n16, MATHLIB_CONV_VEC_m24p24_fl8n16	$[-2^{23}, 2^{23})$	avg: 0.002% max: 0.006%	avg: ±84.7 max: ±255	Large absolute errors are caused by large input values
MATHLIB_CONV_m8p16_fl8n16, MATHLIB_CONV_VEC_m8p16_fl8n16	[-128, 128)	avg: 0.002% max: 0.006%	avg: ±0.001 max: ±0.004	
MATHLIB_CONV_int24_fl8n16, MATHLIB_CONV_VEC_int24_fl8n16	$[-2^{23}, 2^{23})$	avg: 0.002% max: 0.006%	avg: ±74.4 max: ±255	Large absolute errors are caused by large input values
MATHLIB_CONV_frac48_fl8n16, MATHLIB_CONV_VEC_frac48_fl8n16	[-1, 1)	avg: 0.002% max: 0.006%	avg: $\pm 1.1 \times 10^{-5}$ max: $\pm 3.1 \times 10^{-5}$	
MATHLIB_CONV_frac24_fl8n16, MATHLIB_CONV_VEC_frac24_fl8n16	[-1, 1)	avg: 0.002% max: 0.006%	avg: $\pm 1.1 \times 10^{-5}$ max: $\pm 3.1 \times 10^{-5}$	

7.5.22 Floating-Point-to-Fixed-Point Conversion

Several conversion routines are provided to convert floating-point data to one of the fixed-point formats (m24p24, frac48, m8p16). Such routines are particularly useful when floating-point results are returned by a library routine (for example, the reciprocal operation) and need to be converted to fixed-point format for a subsequent ALU operation. The following macros are available, depending on the input format and the form of the input (scalar or vector):

```
// mixed 8.16 output scalar
MATHLIB_CONV_fl8n16_m8p16
// mixed 8.16 output vector
MATHLIB_CONV_VEC_fl8n16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
// frac48 output scalar
MATHLIB_CONV_fl8n16_frac48
// frac48 output vector
MATHLIB_CONV_VEC_fl8n16_frac48(XP_IN, XP_OUT, VEC_LENGTH)
// mixed 24.24 output scalar
MATHLIB_CONV_fl8n16_m24p24
// mixed 24.24 output vector
MATHLIB_CONV_VEC_fl8n16_m24p24(XP_IN, XP_OUT, VEC_LENGTH)
```

When converting from floating-point-to-fixed-point format, the range of the resulting data is obviously reduced compared to the range of the input. Hence, one must be careful not to overflow the target format. Typically, the absolute value of the input must not be higher than the following for conversions:

- 128 for the m8p16 format
- 2^{23} for the m24p24 format
- 1 for the frac48 format

Similarly, one must be aware that very small input values approaching zero will be quantized to the precision of the fixed-point format, which increases the conversion error for these very small input values. To avoid the quantization effects, the absolute value of the input must not be lower than the following for conversions:

- 2^{-16} for the m8p16 format
- 2^{-24} for the m24p24 format
- 2^{-47} for the frac48 format

If an input is below the lower bound in any of these cases, the output of the conversion macro will be zero. Furthermore, for any floating-point input that has an exponent less than -56, the output of the macro is undefined.

Table 90. Properties for Floating-Point-to-Fixed-Point Conversion Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_CONV_fl8n16_m8p16, MATHLIB_CONV_VEC_fl8n16_m8p16	[-128, 128)	Not applicable	avg: $\pm 2.8 \times 10^{-6}$ max: $\pm 1.6 \times 10^{-5}$	Relative error is not meaningful because the floating-point input can be very close to zero compared to the output format

Table 90. Properties for Floating-Point-to-Fixed-Point Conversion Macros

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_CONV_fl8n16_frac48, MATHLIB_CONV_VEC_fl8n16_frac48	[-1, 1)	avg: 8.2×10^{-16}% max: 2.3×10^{-14}%	avg: $\pm 1.5 \times 10^{-22}$ max: $\pm 6.8 \times 10^{-21}$	
MATHLIB_CONV_fl8n16_m24p24, MATHLIB_CONV_VEC_fl8n16_m24p24	$[-2^{23}, 2^{23})$	avg: 6.5×10^{-16}% max: 2.3×10^{-14}%	avg: $\pm 2.2 \times 10^{-22}$ max: $\pm 1.4 \times 10^{-20}$	

7.6 EXAMPLES

This section shows typical code examples that use the math library macros and the different numerical formats. The numerical results given for these examples were generated with the rounding modes in the SR set for biased rounding.

7.6.1 Squared Magnitude

The first example calculates the squared magnitude of the complex number 0.5 + *j**0.5. The macro which expects a single complex input would be typically used in that situation. Its name is MATHLIB_SQMAG_frac24_m2p46. The complex input number must be made of a 24-bit fractional real part and a 24-bit fractional imaginary part. Both components must be available as single words in consecutive locations of the X memory, the first of it being addressed by a register (xp0). Hence, the first task of the code is to place both real and imaginary parts in memory, and set the pointer accordingly. Then, the macro can be called after setting the saturation mode.

```
Sys_Reg_frac24(x0, 5000000)      // Load real part (0.5) into x0
Sys_Reg_frac24(x1, 5000000)      // Load imaginary part (0.5) into x1
load xp0, 0x1000                 // Set address register xp0 to location
                                 //   X:0x1000
nop ||| store x[xp0++], x0       // Place real part into the location
                                 //   addressed by xp0
nop ||| store x[xp0--], x1       // Place imaginary part into the next
                                 //   memory location
Sys_CFX_SaturateEnable           // Set saturation mode
MATHLIB_SQMAG_frac24_m2p46(xp0)  // Call the macro
Sys_CFX_SaturateDisable          // Unset saturation mode
```

After the macro execution, the result for the squared magnitude (which is 0.5) is returned into the accumulator in the format m2p46. Hence the accumulator holds 0x200000|000000, which corresponds to the expected value divided by 2 if interpreted in the usual double fractional format. Any subsequent steps in the calculation should take account of this special format. A left shift might be possible later to restore the scale, but care should be taken with respect to the data range. In this example, the overflow protection is set before the macro call (and reset afterwards) in order to saturate the result of the calculation in the special case where the input is -1-j.

7.6.2 Square Root

This example demonstrates calculating the square roots of the elements of a vector using the macro MATHLIB_SQRT_VEC_frac24_frac48. The input vector contains four fractional values [0.01, 0.25, 0.5, 1] that are converted to 24-bit fractional form using Sys_Reg_frac24. The code below places those four values in X memory (from location X:0x1000 as controlled by the input pointer xp1), and runs the macro with its parameters. Since the outputs are 48-bit numbers, the output register xp2 must point somewhere in the XY memory. The register xp0 is used internally by the macro, so it cannot be used as an input/output parameter.

```
load xp1, 0x1000                 // Set address register xp1 to location
                                 //   X:0x1000
```

```
Sys_Reg_frac24(x0, 100000)       // Load first element into x0
nop ||| store x[xp1++], x0       // Move it to memory (first location)
Sys_Reg_frac24(x0, 2500000)      // Load second element into x0
nop ||| store x[xp1++], x0       // Move it to memory (second location)
Sys_Reg_frac24(x0, 5000000)      // Load third element into x0
nop ||| store x[xp1++], x0       // Move it to memory (third location)
Sys_Reg_frac24(x0, 10000000)     // Load fourth element into x0
nop ||| store x[xp1], x0         // Move it to memory (fourth location)
load xp1, 0x1000                 // Set register xp1 to the first element
                                 //   in the input vector
load xp2, 0x500                  // Set register xp2 to the output
                                 //   location XY:0x500
MATHLIB_SQRT_VEC_frac24_frac48(xp1, xp2, 4)    // Call the macro
```

After execution of this code, the result vector can be read from XY memory, at location 0x500. The output values are [0x0CCE66400000, 0x40FE08000000, 0x5BE9BB000000, 0x7EFEFE000000] or [0.10005, 0.50775, 0.71807, 0.99216]. Their precision is not perfect because of the approximation (table lookup) used in the calculation, but they effectively correspond to the expected results [0.1, 0.5, 0.70711, 1].

You can use the SQRT macros for integer input as well. The integer input must be represented in one of the fixed-point numeric formats that has integer bits, such as m24p24, so that the SQRT macro's input value is in the fractional range. Also, since the fractional formats have a single integer bit while the mixed formats have an even number of integer bits, the operations are simpler if the input value is pre-scaled by an odd power of two.

The following example calculates the square root of 25. The input is stored in m24p24 format and then pre-scaled by 2, so the square root produced by MATHLIB_SQRT_frac24_frac48 is scaled by 2^{11} when interpreted as a fraction. Dividing by 2^{12} produces the result in m24p24 format.

```
load a0, 25                      // 25 as m24p24 or 25/(2^23) as frac24
shll a0, 1 ||| nop               // a0 is now 25/(2^22) in frac24 format
MATHLIB_SQRT_frac24_frac48       // Get result 5/(2^11) as frac48 in a0
shra a0, 12 ||| nop              // Divide result by 2^12 to get 5/(2^23)
                                 //   as frac24, or 5 in m24p24 format
```

The result in a0 after the division (the shift right) is 0x0000050CBC90, which represents 5.04975.

7.6.3 Cosine

The cosine of the angle $\pi/4$ is calculated in this example, using the scalar macro MATHLIB_COS_frac24_frac24. The input angle is expected in a0, and it must be normalized by π (that is with the range $[-\pi, \pi)$ being mapped to [-1, 1)). Hence, an original angle expressed in radians should be normalized before running the macro. Typically, this can be done as shown below. In this case, the original angle $\pi/4$ is smaller than one, so it can be represented as a fractional. If the original radian angle is larger than 1, then it would be represented using a mixed format, and the normalization procedure should take this into account (see Section 7.6.4, "Sine" for the sine calculation).

```
Sys_Reg_frac24(x0, 31415927 / 4) // The input value (pi/4) is made
                                 //   available in x0 as a fractional
Sys_Reg_frac24(x1, 3183099)      // Load the normalization factor 1/pi in
                                 //   x1 as a fractional
mul a0, x0, x1 ||| nop           // Perform a multiplication and get the
                                 //   normalized value in a0
nop                              // Delay, waiting for 2nd cycle of
                                 //   the multiplication
```

Here the normalized angle 1/4 (0x200000) is obtained in a0, and the cosine macro can be run, providing its fractional result (0.70711 or 0x5A827A) in a0 :

```
MATHLIB_COS_frac24_frac24        // Calculate cosine
```

7.6.4 Sine

This example shows the calculation of sin(-π/2) using the macro MATHLIB_SIN_frac24_frac24. Since the input angle is larger than 1, it is originally represented with the mixed format m4p20, where 4 bits are allocated to the signed integer part, and 20 bits to the unsigned fractional part (which is always positive). The frac24 format is m1p23, so the value of -π/2 in m4p20 format is computed using Sys_Reg_frac24 to convert the desired value divided by 8, or -π/16. The code is shown below:

```
Sys_Reg_frac24(x0, -31415927/16) // -pi/16 in frac24 is -pi/2 in m4p20
Sys_Reg_frac24(x1, 3183099)      // This is 1/pi in fractional format
mul a0, x0, x1 ||| nop           // Perform the product and get the
                                 //   normalized value -1/2 (0xF80000 as
                                 //   m4p20) in a0
nop                              // Delay, waiting for 2nd cycle of mul
shll a0, (4-1) ||| nop           // Convert the normalized angle -1/2 to
                                 //   fractional (a0 becomes 0xC00000)
MATHLIB_SIN_frac24_frac24        // Run the macro, which returns 0x800001
                                 //   (that is, about -1) in a0
```

7.6.5 Format Conversion

These examples demonstrate two types of format conversion. The first one illustrates a conversion from the 24-bit fractional fixed-point format (frac24) to the floating-point format (fl8n16). The vector macro MATHLIB_CONV_VEC_frac24_fl8n16 is used, with a 4-element input vector made of the values [0.5, 0.25, -1, -0.5]. After conversion, the outputs are [0x004000, 0xFF4000, 0x008000, 0xFF8000]. The pointer registers xp0 and xp1 are used to address the input and output memory locations, respectively, in X memory. The code is shown below:

```
load xp0, 0x1000                 // Set address register xp0 to location
                                 //   X:0x1000
Sys_Reg_frac24(x0, 5000000)      // Load first element into x0
nop ||| store x[xp0++], x0       // Move it to memory (first location)
Sys_Reg_frac24(x0, 2500000)      // Load second element into x0
nop ||| store x[xp0++], x0       // Move it to memory (second location)
Sys_Reg_frac24(x0, -10000000)    // Load third element into x0
nop ||| store x[xp0++], x0       // Move it to memory (third location)
Sys_Reg_frac24(x0, -5000000)     // Load fourth element into x0
nop ||| store x[xp0], x0         // Move it to memory (fourth location)
load xp0, 0x1000                 // Set register xp0 to the first element
                                 //   in the input vector
load xp1, 0x1010                 // Set register xp1 to the output
                                 //   location X:0x1010
MATHLIB_CONV_VEC_frac24_fl8n16(xp0, xp1, 4)    // Call the macro
```

In the second example, let's perform the opposite conversion, but this time, a 48-bit fractional result is desired. The macro MATHLIB_CONV_VEC_fl8n16_frac48 is used as follows. The input register is xp1, and the output register is xp2, pointing at a 48-bit XY memory location.

```
load xp1, 0x1010                 // Set register xp1 to the first element
                                 //   in the input vector
```

```
load xp2, 0x500                  // Set register xp2 to the output
                                 //   location XY:0x500
MATHLIB_CONV_VEC_fl8n16_frac48(xp1, xp2, 4)    // Call the macro
```

The returned output vector is [0x400000000000, 0x200000000000, 0x800000000000, 0xC00000000000], as expected.

7.6.6 Logarithm

This example considers the floating-point vector generated in the previous example, and calculates its logarithm in base 2 using the macro MATHLIB_LOG2_VEC_fl8n16_m8p16. Only the positive values 0x004000 (0.5) and 0xFF4000 (0.25) are processed, addressed by xp1. The output is obtained at the location addressed by xp2 in X memory, and the vector length is set to 2:

```
load xp1, 0x1010                 // Set register xp1 to the first element
                                 //   in the input vector
load xp2, 0x1020                 // Set register xp2 to the output
                                 //   location X:0x1020
MATHLIB_LOG2_VEC_fl8n16_m8p16(xp1, xp2, 2)    // Call the macro
```

The resulting vector, expressed in mixed format m8p16, is [0xFF05BA, 0xFE05BA]. In this vector, the first number represents -1+0.0224 = -0.9776 since 0xFF is -1 (signed integer part) and 0x05BA is 0.0224 (unsigned 16-bit fractional value). The result is an approximation because of the table used to estimate the logarithm (log2(0.5) = -1). Similarly, the second number in the vector is -2+0.2224 = -1.9776 since 0xFE is -2.

7.6.7 Exponential

Using the macro MATHLIB_EXP2_VEC_m8p16_m8p16, this example calculates the base-2 exponent of an input vector. The input vector is the result from the previous example, [0xFF05BA, 0xFE05BA], so that the logarithm operation will be reversed (expecting [0.5, 0.25]). However, the result of the operation is now expressed in the mixed format m8p16. The input vector is addressed by xp1, while the output uses xp2. The call is as follows:

```
load xp1, 0x1020                  // Set register xp1 to the first
                                  //   element in the input vector
load xp2, 0x1030                  // Set register xp2 to the output
                                  //   location X:0x1030
MATHLIB_EXP2_VEC_m8p16_m8p16(xp1, xp2, 2)    // Call the macro
```

The resulting vector, expressed in mixed format m8p16, is [0x008165, 0x0040B2] = [0.5054, 0.2527].

7.6.8 Attack Release

This example illustrates an attack-release filtering operation (*y(n) = coeff*x(n) + (1-coeff)*y(n-1)* with *coeff = alpha* or *beta*), assuming a 4-channel parallel process (4-element input and output vectors). The input vector is addressed by xp0, while xp1 points to the output. The macro MATHLIB_ATTACK_RELEASE_VEC is used, with xp2 pointing to the memory location that holds the value of alpha for the first channel. Note that alpha and beta must be written as consecutive pairs (beta following alpha), one pair per channel. In this example, they are written from location X:0xFF0, and the same pairs are written four times since the same alpha and beta values are used in every channel. Assuming an initial input vector *x(n)* = [0x7FFFFF, 0x400000, 0x200000, 0x100000] available in location X:0x1000, the first call to the macro would be as follows, requiring some initialization steps (output vector y(n-1) and filter parameters alpha and beta):

```
// Initialize y(n-1) to zero for all channels
loop.2d 4, Init_Output_Label     // Set loop to repeat 4 times
load xp1, 0x1010                 // [ds] Set pointer xp1 to X:0x1010
nop                              // [ds]
Init_Output_Label:
    nop ||| clr x[xp1++]         // Clear the previous output y(n-1)

// Initialize alpha
loop.2d 4, Alpha_Init_Label      // Set loop to repeat 4 times
Sys_Reg_frac24(x0, 9843750)      // [ds] Set alpha to 0.984375
load xp2, 0x0FF0                 // [ds] Set pointer xp2 to X:0x0FF0
                                 //        (first alpha)
Alpha_Init_Label:
    nop ||| store x[xp2+=2], x0  // Set each channel's alpha in memory

// Initialize beta
loop.2d 4, Beta_Init_Label       // Set loop to repeat 4 times
Sys_Reg_frac24(x0, 7500000)      // [ds] Set beta to 0.75
load xp2, 0x0FF1                 // [ds] Set pointer xp2 to X:0x0FF1
                                 //        (first beta)
Beta_Init_Label:
    nop ||| store x[xp2+=2], x0  // Set each channel's beta in memory

// Set address registers and run macro
load xp0, 0x1000                 // Set pointer xp0: input vector x(n)
load xp1, 0x1010                 // Set pointer xp1: output vector y(n-1)
load xp2, 0x0FF0                 // Set pointer xp2: alpha and beta
MATHLIB_ATTACK_RELEASE_VEC(xp0, xp1, xp2, 4)    // Call the macro
```

The output found at location xp1 would be [0x7DFFFF, 0x3F0000, 0x1F8000, 0x0FC000]. A subsequent call to the same macro with the new input vector [0x100000, 0x200000, 0x400000, 0x7FFFFF] would be simply:

```
// Set address registers and run macro
load xp0, 0x1000                 // Set pointer xp0: input vector x(n)
load xp1, 0x1010                 // Set pointer xp1: output vector y(n-1)
load xp2, 0x0FF0                 // Set pointer xp2: alpha and beta
MATHLIB_ATTACK_RELEASE_VEC(xp0, xp1, xp2, 4)    // Call the macro
```

In this particular case, since the values in the first two channels are now decreasing (release), beta is used in the calculation. On the other hand, the other two channels are in attack, and alpha is used. The output vector y(n) becomes [0x2B8000, 0x27C000, 0x3F7E00, 0x7E3EFF].

7.6.9 Reciprocal

Using the macro MATHLIB_RECIP_VEC_int24_fl8n16, this example calculates the reciprocal of the integer input vector [4, -2]. The input vector is addressed by xp1, while the output uses xp2. The call is as follows:

```
// Create 24-bit integer input vector [-4, 2] at location X:0x1000
store x[0x1000], 4
store x[0x1001], -2

// Set address registers and run macro
load xp1, 0x1000                 // Set register xp1 to the first element
                                 //   in the input vector
```

```
load xp2, 0x1010                // Set register xp2 to the output
                                //   location X:0x1010
MATHLIB_RECIP_VEC_int24_fl8n16(xp1, xp2, 4)    // Call the macro
```

The resulting vector, expressed in floating-point format, is [0xFE7E08, 0xFF81F8]. The first result is made of the signed exponent 0xFE = -2 and signed fractional mantissa 0x7E08 = 0.9846, which turns into the number $0.9846 * 2^{-2} = 0.2462$. The second result is made of the signed exponent 0xFF = -1 and signed mantissa 0x81F8 = -0.9846. This represents $-0.9846 * 2^{-1} = -0.4923$. Since the reciprocal operation uses lookup tables, these results are approximations. A conversion routine, such as `MATHLIB_CONV_VEC_fl8n16_frac48`, can be used afterwards to convert the result to the desired format.

7.6.10 RMS to db10

This example shows the conversion from RMS values to decibels, using the macro MATHLIB_RMS_DB10_frac48_m8p16. This particular case converts the value 0.5 into 20*log10(0.5) = -6.02. After running the macro, the 48-bit input value in the accumulator is replaced with the result (0xFA1D34 = -6 + 0.114 = -5.89), expressed in the mixed format m8p16. The result is an approximation since the calculation of the logarithm in base 2 is performed by means of a lookup table.

```
Sys_Reg_frac24(a0, 5000000)      // Load accumulator with 0.5 in frac48
                                 //   format; the bottom 24 bits are zero
MATHLIB_RMS_DB10_frac48_m8p16    // Call the macro
```

7.6.11 Division

In this example, the use of the divide macros is demonstrated. The macro MATHLIB_DIVIDE_frac24 first calculates the ratio 0.125 / 0.5 = 0.25. The dividend is expected in a0.h, while the divisor must be available in register x0. The quotient 0x200000 is returned into a0.l. The code is as follows:

```
Sys_Reg_frac24(a0, 1250000)      // Load accumulator with dividend 0.125
Sys_Reg_frac24(x0, 5000000)      // Load register x0 with divisor 0.5
MATHLIB_DIVIDE_frac24            // Call the macro
```

Note that this macro expects a dividend smaller than the divisor (since the returned value is fractional). If this is not the case, the other macro, `MATHLIB_DIVIDE_m24p24`, must be used, as shown below. Here, calculation of the division 0.5 / 0.125 = 4 is performed. The result (4) is returned into the accumulator a0 in the form of the mixed m24p24 0x000004|000000.

```
Sys_Reg_frac24(a0, 5000000)      // Load accumulator with dividend 0.5
Sys_Reg_frac24(x0, 1250000)      // Load register x0 with divisor 0.125
MATHLIB_DIVIDE_m24p24            // Call the macro
```

Finally, another macro can be used to divide integers: MATHLIB_DIVIDE_int24. This macro provides the integer part of the result in a0.l. The example below illustrates the division 5/2:

```
load a0, 5                       // Load accumulator with dividend 5
load x0, 2                       // Load register x0 with divisor 2
MATHLIB_DIVIDE_int24             // Call the macro
```

The value 2 is returned into a0.l. Additionally, the instruction

```
add a0, x0, lt ||| nop
```

can be run after the macro. This is the final step to compute, in a0.h, the remainder of the division after multiplication by the divisor. The quotient in a0.l is not modified. Thus a0.h would get the value 1 in this example.

7.6.12 Linear Interpolation

This example illustrates the behavior of the linear interpolation macro MATHLIB_LIN_INTERP. The linear interpolation is performed between two data points available in two consecutive memory words. An address register (xp0 in this particular case) is expected to contain the address of the first point. The positive interpolation ratio is placed into a0 before running the macro. In the following code, the data points to be interpolated are t0 = 0.125 (0x100000) and t1 = 0.625 (0x500000), while the interpolation ratio is x = 0.75 (0x600000). The result is t0 + x*(t1-t0) = 0.5 (0x400000), returned in a0.

```
load xp0, 0x1000                // Set address register
Sys_Reg_frac24(x0, 1250000)     // Load 0.125 into x0
nop ||| store x[xp0++], x0      // Move it to memory
Sys_Reg_frac24(x0, 6250000)     // Load 0.625 into x0
nop ||| store x[xp0--], x0      // Move it to memory
Sys_Reg_frac24(a0, 7500000)     // Load 0.75 into a0
MATHLIB_LIN_INTERP(xp0)         // Run macro
```

CHAPTER 8
ARM Cortex-M3 Processor Math Library

8.1 Overview

The ARM Cortex-M3 processor math library contains a set of fixed-point and floating-point functions that can be used for common time-domain data processing.

The fixed-point module supports signed 32-bit fractional numbers, whereas the floating-point module supports 32-bit floating-point numbers. In the fixed-point module, all addition, subtraction and multiplication operations include both positive and negative saturation. In fixed-point fractional notation, $1 - 2^{-31}$ is returned if positive overflow occurs, and -1 is returned if negative overflow occurs.

Both the fixed-point and floating-point modules provide the following functions:

- Exponential averaging filter
- Dual-time constant attack-release averaging filter
- Linear interpolation

In addition, the floating-point module provides a single-variable linear regression function, and the fixed-point module provides the following basic arithmetic operations:

- Addition with saturation
- Subtraction with saturation
- Multiplication with saturation

8.2 Data Types

Two data types are supported by the math library:

fixed-point — The fixed-point module supports a signed 32-bit fractional type (`frac32`), which is equivalent to `signed long` or `signed int` in standard C divided by a scale of 2^{31}. Data in [-1, 1) is represented using a 32-bit two's-complement value. The implied decimal point is between the MSB (the sign bit) and the next bit.

floating-point — The floating-point module supports a signed 32-bit floating point type, which is the same as `float` in standard C.

8.2.1 Fixed-point Data Saturation

All fixed-point arithmetic operations (add, subtract and multiply) in the math library include saturation. Specifically, the cases are:

- No overflow: the direct result of the arithmetic operation is returned.
- Positive Overflow: the maximum 32-bit fractional number $1-2^{-31}$ (0x7FFFFFFF) is returned.
- Negative Overflow: the minimum 32-bit fractional number -1 (0x80000000) is returned.

8.3 Content of the Library

The library provides the mathematical and digital signal processing operations listed in Table 91.

Table 91. Operations Included in the ARM Cortex-M3 Processor Math Library

Function	Operation
$z = x + y$	Add two numbers (fixed point only)
$z = x - y$	Subtract two numbers (fixed point only)
$z = x \times y$	Multiply two numbers (fixed point only)
$y(n) = a*x(n) + (1 - a)*y(n - 1)$	Exponential averaging filter (first order)
$y(n) = a*x(n) + (1 - a)*y(n - 1)$ if $x(n) \geq y(n - 1)$ $y(n) = b*x(n) + (1 - b)*y(n - 1)$ if $x(n) < y(n - 1)$	Dual-time constant attack-release averaging filter with *a* controlling the attack time and *b* the release time
$y = t0 + x*(t1 - t0)$	Linear interpolation (typically for table lookup operations)
$y_i = a_0 + a_1 \cdot x_i + \varepsilon_i$, i = 1, 2, ..., N	Calculate the least-squares solution for a single variable linear regression model (floating point only)

All math library functions are listed in Table 92.Each macro and function is cross-referenced to the page with detailed information, including supported formats, calling parameters, modified registers, returned values and execution time.

Table 92. Math Library Functions

Function	Description	Reference
Math_Add_frac32	Add two 32-bit signed fractional numbers, then saturate	19.1 on p. 741
Math_AttackRelease	Calculate a first-order attack-release filter	19.2 on p. 742
Math_AttackRelease_frac32	Calculate a fixed-point first-order attack-release filter	19.3 on p. 743
Math_ExpAvg	Calculate a first-order exponential average	19.4 on p. 744
Math_ExpAvg_frac32	Calculate a fixed-point first-order exponential average	19.5 on p. 745
Math_LinearInterp	Calculate linear interpolation on the interval [x0, x1)	19.6 on p. 746
Math_LinearInterp_frac32	Calculate fixed-point linear interpolation on the interval [0, 1)	19.7 on p. 747
Math_Mult_frac32	Multiply two 32-bit signed fractional numbers, then saturate	19.8 on p. 748
Math_SingleVar_Reg	Find the least-squares solution for a single variable linear regression model	19.9 on p. 749
Math_Sub_frac32	Subtract one 32-bit signed fractional number from another, then saturate	19.10 on p. 750

8.4 MATH LIBRARY USAGE

To use any function in the math library, you need to link the compiled library (*sk4_mathlib.a*) in project configurations, and include the math library include file, *sk4_math.h*, in the source code. This include file contains defines and function declarations for the math library.

8.4.1 Naming Conventions

Math library functions are named in mixed case preceded with a prefix of `Math_`. The operations follow the prefix. Floating-point functions have no suffix. Fixed-point functions have a suffix of `_frac32`.

8.5 MATH LIBRARY FUNCTIONS

8.5.1 32-Bit Fixed-Point Basic Arithmetic Operations

Three basic arithmetic operations for the signed 32-bit fractional types are provided by the math library: add, subtract and multiply. All of these operations include saturation. The fixed-point multiplication operation performs a full 32x32 bit multiply, and then truncates the additional fractional bits to return only a 32-bit fractional fixed-point result.

8.5.2 Exponential Averaging Filter

The exponential averaging filter is provided in two versions: a fixed-point function and a floating-point function.

The exponential averaging filter implements the function:

$$y[n] = \alpha \cdot x[n] + (1 - \alpha) \cdot y[n - 1]$$

where:

- $x[n]$ is the input
- α is the filter coefficient, and where α is in the range (0,1)
- n is the time index

8.5.3 Dual-Time Constant Attack-Release Averaging Filter

The dual-time constant attack-release averaging filter is provided in two versions: a fixed-point function and a floating-point function.

The dual-time constant attack-release averaging filter implements the function:

$$y[n] = \beta \cdot x[n] + (1 - \beta) \cdot y[n - 1]$$

where:

- $x[n]$ is the input
- $\beta = \begin{cases} a, \text{ if } x[n] \geq y[n-1] \\ b, \text{ if } x[n] < y[n-1] \end{cases}$ is the dual-time constant, and where *a* is in the range (0,1) and *b* is in the range (0,1)
- n is the time index

8.5.4 Linear Interpolation

Linear interpolation is provided in two versions: a fixed-point function and a floating-point function.

The fixed-point linear interpolation implements the function:

$$y = y_0 + x \cdot (y_1 - y_0)$$

where:

- $x \in \Re$ is the input, and where *x* is in the range [0,1)

- $(0,y_0)$ and $(1,y_1)$ are the two boundary points for the interpolation

The floating-point linear interpolation implements the function:

$$y = y_0 + \frac{(y_1 - y_0)}{(x_1 - x_0)} \cdot (x - x_0)$$

where:

- $x \in \Re$ is the input, and where *x* is in the range $[x_0, x_1)$
- (x_0,y_0) and (x_1,y_1) are the two boundary points.

8.5.5 Single-Variable Linear Regression

For the single-variable linear regression model with *N* observations:

$$y_i = a_0 + a_1 \cdot x_i + \varepsilon_i, \quad i = 1, 2, ..., N$$

The single-variable linear regression function (provided only as a floating-point function) with *N* observations provides a linear least squares solution for the coefficients a_0 and a_1 as follows:

$$\hat{a}_0 = \frac{\bar{y}\left(\sum_{i=1}^{N} x_i^2\right) - \bar{x}\left(\sum_{i=1}^{N} x_i y_i\right)}{\sum_{i=1}^{N} x_i^2 - N\bar{x}^2},$$

$$\hat{a}_1 = \frac{\left(\sum_{i=1}^{N} x_i y_i\right) - N\bar{x}\bar{y}}{\sum_{i=1}^{N} x_i^2 - N\bar{x}^2}.$$

In the above expressions, $\bar{x}$ and $\bar{y}$ are the arithmetic means of x_i and y_i respectively.

CHAPTER 9
EEPROM Library

9.1 DESCRIPTION

The EEPROM library is provided as part of the firmware. The EEPROM library provides access to an EEPROM connected to the Ezairo 7100 device through the CFX SPI port. The library is designed to work with the following EEPROM or non-volatile memories:

- 16-bit addressed devices:
  a. ON Semiconductor CAT25256 and CAT25512
  b. Microchip 25AA256 and 25AA512
  c. Atmel AT25256
  d. ON Semiconductor custom E2M2 ASIC

  These devices are 256 kb or 512 kb serial EEPROMs with related command sets.

- 24-bit addressed devices:
  a. ON Semiconductor CAT25M01, CAT25M02
  b. Microchip 25AA1024

  These devices are 1 Mb or 2 Mb serial EEPROMs with related command sets.

All EEPROM or non-volatile memories with command set and addressing requirements that are compatible with the above devices can be used with this EEPROM library. This includes using the EEPROM library for reading from EEPROM or other non-volatile memory devices that exceed 2 Mb in size.

NOTE: Writing to these devices might not be supported as the EEPROM library assumes that destination EEPROM and non-volatile memories support individual word writes (i.e., they are not limited to page writes). Refer to the datasheet for your non-volatile memory device to confirm whether your device supports this functionality.

The EEPROM library provides high-level functions for configuring, reading from and writing to the EEPROM. Low-level functions are internal to the EEPROM library and are not intended to be used by the user application.

9.2 EEPROM LIBRARY USAGE

EEPROM library functions are contained in a compiled library. Both RAM-based and ROM-based versions are available. For more information, see Section 2.4.1, “ROM-Based Libraries” on page 30.

To use the EEPROM library, include the associated system library include file, *sk4_eeprom.cfi*, in the source code. This include file contains the function declarations for the EEPROM library as well as associated macros. You must then configure your application settings to link in the appropriate compiled library.

The EEPROM library functions are designed to be called from within a user application. The user application initializes the EEPROM library and executes the desired functions. Parameters are passed to the EEPROM functions using registers. Return codes, if any, are usually provided through registers. The function interfaces are listed in the EEPROM function reference.

The library is designed to be used while interrupts are enabled or disabled. That is, the EEPROM library functions are interruptible. The EEPROM library itself does not use interrupts. This design allows the calling application to handle any other system tasks, such as signal processing, within ISRs during EEPROM operations.

9.2.1 SPI Interface Configuration

The EEPROM library for the Ezairo 7100 chip does not configure the SPI interface pads for use by the EEPROM library. The EEPROM library assumes that the CFX SPI interface has been configured by the calling application.

To support an additional EEPROM, the calling application can reconfigure one or more of the SPI interface pins (typically the SPI chip select) to select the additional EEPROM.

IMPORTANT: The EEPROM library does not monitor the SPI interface configuration, so be careful in the calling application that this interface is configured as desired before calling EEPROM library functions.

9.2.2 EEPROM Library Addressing Mode

The EEPROM library supports both 16- and 24-bit addresses when reading and writing the non-volatile memory array.

The EEPROM library initialization routine (see Section 20.4, “EELIB_Initialize” on page 754) sets the address mode of the EEPROM library to support 16-bit addressed EEPROMs by default.

The EEPROM library includes two methods to set the EEPROM library address mode:

- If the required address mode for the attached EEPROM is known, the EEPROM library addressing mode can be set to this mode using the `EELIB_Set_AddressMode` macro (see Section 20.30, “EELIB_Set_AddressMode” on page 780)
- If the required address mode for the attached EEPROM is not known, the EEPROM library addressing mode can be set using the `EELIB_AutoSet_AddressMode` function (see Section 20.1, “EELIB_AutoSet_AddressMode” on page 751)

The `EELIB_AutoSet_AddressMode` function attempts to determine if the EEPROM is 16-bit or 24-bit addressable using several mechanisms. If the EEPROM is 16-bit addressable and the first byte of the EEPROM contains either 0x00 or 0xFF, the EEPROM library temporarily modifies the contents of this address as part of this process.

CAUTION: If the `EELIB_AutoSet_AddressMode` function is aborted (due to a reset or an interrupt handler that does not return) after modifying the EEPROM contents but before restoring the previous value, the contents of the first byte of the EEPROM will be corrupted.

9.2.3 EEPROM Library Naming Conventions

EEPROM library functions are named in mixed case with a prefix of `EELIB_`. The operation follows the prefix. The operation specifies either a read operation, a write operation or another operation. Other operations include the library initialization, setup and configuration, as well as EEPROM write enabling and disabling.

Several functions have alternate functions associated with them that perform the same core operation but with different word sizes. These alternate functions are differentiated by the word size (in bits) following the function name. For example, `EELIB_ReadBlock_X` operates with the default word size (24 bits in this case) whereas `EELIB_ReadBlock_X16` operates with a word size of 16 bits. For more information on default word sizes, see Section 9.4, “Word Sizes”.

9.3 Write Protection

For protection against accidental EEPROM corruption or erasure, the EEPROM device has built-in block write protection (BWP). To allow the EEPROM library to write to the EEPROM, the application must specifically enable writing in the library. This is accomplished by calling the EEPROM library function `EELIB_LibWriteEnable` with EEPROM write-enable opcode (`AT_WREN`) as the parameter.

IMPORTANT: To ensure maximum protection against accidental EEPROM corruption or erasure, do not store the parameter passed into the `EELIB_LibWriteEnable` function in memory. Instead, pass the value into the device from an external system, where possible.

9.4 Word Sizes

The EEPROM library supports multiple word sizes: 8 bits (single byte), 16 bits (two bytes), 24 bits (three bytes) and 32 bits (four bytes). This design facilitates multiple word sizes on the Ezairo 7100 device.

Functions that operate through a specific memory space use that memory space's word size as the default. For example, `EELIB_ReadBlock_P` operates on program memory and thus uses a 32-bit word size.

Functions that use a smaller word size than the nominal word size for a particular memory space operate on the lower significant bits of the nominal word. When reading from the EEPROM to a memory space or register, any bits that are not set by the read are cleared to 0.

IMPORTANT: We recommend that user applications ensure that single-word EEPROM writes of any word size are contained within the boundaries of a single EEPROM page. This affects all `EELIB_WriteWord` and `EELIB_WriteNextWord` function calls. If a user application calls these functions, and the number of bytes written by any one function call or sequence of function calls exceeds the bytes remaining in the currently selected EEPROM page, the excess bytes will wrap to the beginning of the EEPROM page buffer and be written to the beginning of the currently selected EEPROM page.

9.5 Page Sizes

The EEPROM library supports a configurable page size. The library supports all page sizes that are a power of two. **The default page size for the EEPROM library is 64 bytes**. The page size used by the EEPROM library is reset by the `EELIB_Initialize` function, and can be set using the `EELIB_Set_PageSize` macro.

The page size used by the EEPROM library should be set by an application if it is going to be writing blocks of data. This will prevent EEPROM write errors caused by block writes that wrap across page boundaries for EEPROMs that have page sizes smaller than 64 bytes. This will also eliminate the overhead introduced by unnecessary extra commit cycles for EEPROMs that have page sizes larger than 64 bytes.

9.6 Cyclic Redundancy Check (CRC)

The EEPROM library provides a convenient and robust method to validate data read from or written to the EEPROM. All the EEPROM read and write functions automatically update a 16-bit CRC value. The user application must set the initial CRC value. The CRC value is stored in the `D_CRC_VALUE` register. The application can use this value to write the final CRC value of a block of data or to verify data when it is read from the EEPROM.

The recommended initial value for the CRC is 0xFFFF. The *Hardware Reference Manual for Ezairo 7100* has more information about the CRC.

9.7 EEPROM Library Functions

The EEPROM library macros are listed in Table 93 and the functions are listed in Table 94.

Table 93. EEPROM Library Macros

Macro	Description	EEPROM Library Reference
EELIB_Set_AddressMode	Set the address mode for the EEPROM library	20.30 on p. 780
EELIB_Set_PageSize	Set the page size for the EEPROM library	20.31 on p. 781

Table 94. EEPROM Library Functions

Function	Description	EEPROM Library Reference
EELIB_AutoSet_AddressMode	Determine the address mode of the EEPROM attached to the SPI interface, and set the address mode accordingly	20.1 on p. 751
EELIB_Close	Close the EEPROM library	20.2 on p. 752
EELIB_Commit	Commit EEPROM buffer to EEPROM non-volatile memory	20.3 on p. 753
EELIB_Initialize	Initialize the EEPROM library	20.4 on p. 754
EELIB_InitWriteWord	Initialize the EEPROM for writing at the specified address	20.5 on p. 755
EELIB_LibWriteDisable	Write-disable the EEPROM library	20.6 on p. 756
EELIB_LibWriteEnable	Write-enable the EEPROM library	20.7 on p. 757
EELIB_PowerDown	Attempt to place the EEPROM in power-down mode	20.8 on p. 758
EELIB_PowerUp	Place the EEPROM into power-up mode	20.9 on p. 759
EELIB_ReadBlock_P	Read a block of data from EEPROM to PMEM	20.10 on p. 760
EELIB_ReadBlock_P16	Read a block of data from EEPROM to PMEM in 16-bit words	20.11 on p. 761
EELIB_ReadBlock_X	Read a block of data from EEPROM to XMEM	20.12 on p. 762
EELIB_ReadBlock_X16	Read a block of data from EEPROM to XMEM in 16-bit words	20.13 on p. 763
EELIB_ReadBlock_Y	Read a block of data from EEPROM to YMEM	20.14 on p. 764
EELIB_ReadNextBlock_P	Read the next block of data from a queued EEPROM to PMEM	20.15 on p. 765
EELIB_ReadNextBlock_P16	Read the next block of data from a queued EEPROM to PMEM in 16-bit words	20.16 on p. 766
EELIB_ReadNextBlock_X	Read the next block of data from a queued EEPROM to XMEM	20.17 on p. 767
EELIB_ReadNextBlock_X16	Read the next block of data from a queued EEPROM to XMEM in 16-bit words	20.18 on p. 768

Table 94. EEPROM Library Functions (Continued)

Function	Description	EEPROM Library Reference
EELIB_ReadNextBlock_Y	Read the next block of data from a queued EEPROM to YMEM	20.19 on p. 769
EELIB_ReadNextWord16	Read a 16-bit word from the queued EEPROM	20.20 on p. 770
EELIB_ReadNextWord24	Read a 24-bit word from the queued EEPROM	20.21 on p. 771
EELIB_ReadNextWord32	Read a 32-bit word from the queued EEPROM	20.22 on p. 772
EELIB_ReadNextWord8	Read an 8-bit word from the queued EEPROM	20.23 on p. 773
EELIB_ReadStatusRegister	Read the EEPROM status register	20.24 on p. 774
EELIB_ReadWord16	Initialize the EEPROM and read a 16-bit word	20.25 on p. 775
EELIB_ReadWord24	Initialize the EEPROM and read a 24-bit word	20.26 on p. 776
EELIB_ReadWord32	Initialize the EEPROM and read a 32-bit word	20.27 on p. 777
EELIB_ReadWord8	Initialize the EEPROM and read an 8-bit word	20.28 on p. 778
EELIB_SelectSegment	Selects EEPROM segment 0 or 1 (address bit 17)	20.29 on p. 779
EELIB_VerifyEEPROMConnection	Verify that an EEPROM is connected to the SPI port	20.32 on p. 782
EELIB_WriteBlock_Const32	Writes a block of data from one constant PMEM word to EEPROM	20.33 on p. 783
EELIB_WriteBlock_P	Writes a block of data from PMEM to EEPROM	20.34 on p. 784
EELIB_WriteBlock_X	Writes a block of data from XMEM to EEPROM	20.35 on p. 785
EELIB_WriteBlock_Y	Writes a block of data from YMEM to EEPROM	20.36 on p. 786
EELIB_WriteDisable	Write-disable the EEPROM	20.37 on p. 787
EELIB_WriteEnable	Write-enable the EEPROM	20.38 on p. 788
EELIB_WriteNextWord16	Writes a 16-bit word to the EEPROM buffer	20.39 on p. 789
EELIB_WriteNextWord24	Writes a 24-bit word to the EEPROM buffer	20.40 on p. 790
EELIB_WriteNextWord32	Writes a 32-bit word to the EEPROM buffer	20.41 on p. 791
EELIB_WriteNextWord8	Writes a 8-bit word to the EEPROM buffer	20.42 on p. 792
EELIB_WriteStatusRegister	Write the EEPROM status register	20.43 on p. 793
EELIB_WriteWord16	Initialize the EEPROM for writing and send a 16-bit word to be written to the EEPROM buffer	20.44 on p. 794

Table 94. EEPROM Library Functions (Continued)

Function	Description	EEPROM Library Reference
EELIB_WriteWord24	Initialize the EEPROM for writing and send a 24-bit word to be written to the EEPROM buffer	20.45 on p. 795
EELIB_WriteWord32	Initialize the EEPROM for writing and send a 32-bit word to be written to the EEPROM buffer	20.46 on p. 796
EELIB_WriteWord8	Initialize the EEPROM for writing and send a 8-bit word to be written to the EEPROM buffer	20.47 on p. 797

9.8 EEPROM Library Operation

The EEPROM library supports block read, word read, and status register read operations. Figure 2 illustrates how the EEPROM library can be used to perform a read operation or multiple read operations.

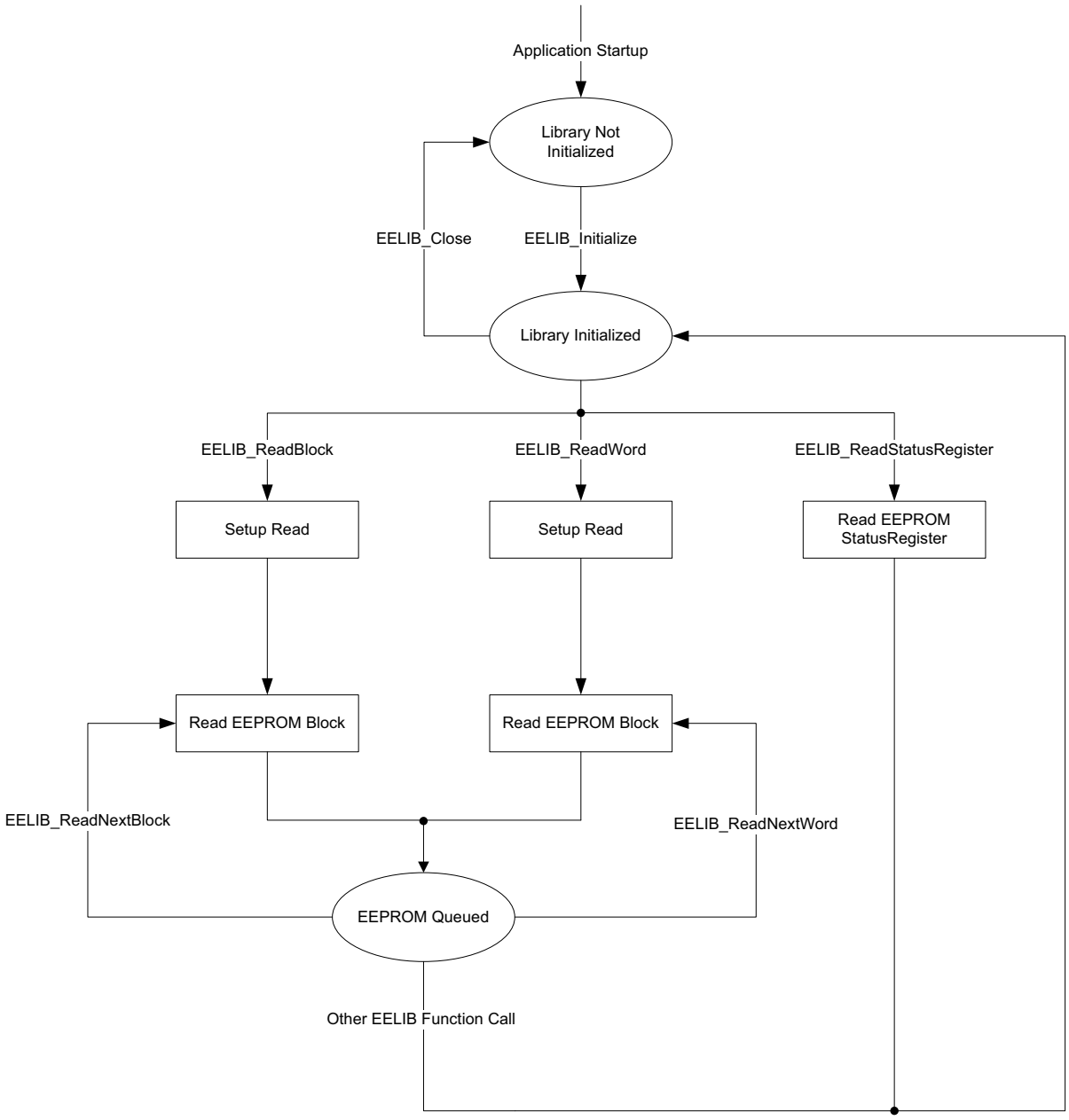


Figure 2. EEPROM Library Read Operation

The EEPROM library supports block write, word write, and status register write operations. Figure 3 illustrates how the EEPROM library can be used to perform a write operation or multiple write operations.



Figure 3. EEPROM Library Write Operation

CHAPTER 10
EEPROM File System

10.1 Description

The file system securely and efficiently stores system data, system code, application code and application data in an EEPROM or similar non-volatile memory device. The data includes:

- Specific calibration settings, manufacturing information, application selection and other system data
- A bootloader
- One or more applications

Multiple stored applications that contain both independent and shared data and code areas are supported. Security is provided through AES encryption.

The file system is designed for a non-volatile memory device (EEPROM) connected through the CFX SPI port. The EEPROM must provide byte read and write capability. The EEPROM sizes supported include, but are not limited to, 128 kb, 256 kb, 512 kb, 1 Mb and 2 Mb.

NOTE: The EEPROM file system can be used with any non-volatile memory provided an appropriate read and write sequence is available. As this library is typically used with EEPROM-type non-volatile memories, it will be referred to generally within this chapter as the “EEPROM file system” and not referred to using the more generic, but equally correct, “NVM file system”.

10.2 Version

The file system version is identified with a 16-bit identifier. The identifier format is shown in Table 95.

Table 95. File System Version

Bits 15-8	Bits 7-0
Major Version	Minor Version

Minor changes to the file system will increase the lower byte of the file system version. Major changes to the file system will increase the upper byte of the file system version.

The EEPROM contains the file system version variable in the system area. The value stored in the EEPROM identifies the format of the accompanying data. The value is checked by the bootloader to ensure compatibility between the installed bootloader and file system. If the bootloader does not support the corresponding file system version, an error is generated.

The file system format described here corresponds to file system version 2.1.

10.3 Word Size and Addressing

The file system supports multiple word sizes depending on the type of data. The default word size is 16 bits for data and 32 bits for addresses. The EEPROM library supports file system word sizes by providing functions to read or write 8-bit, 16-bit, 24-bit or 32-bit words.

Block reads and writes generally require the same word size as the memory space the function is trying to read from or write to. Data words that will be placed into memory on the CFX X memory and Y memory buses are 24 bits

wide. Data words that are destined for the CFX program memory bus are 32 bits wide. To support these various widths, several block read and write macros are provided.

All EEPROM addressing is byte based. File system address vectors—which point to the start of sections and application files in the system—correspond to absolute byte addresses on the EEPROM. Address vectors are required to support both 16-bit and 24-bit addressable non-volatile memories, and use 32-bit words to ensure compatibility with all EEPROM memories. A single address vector provides an addressable range between 0x00000000 and 0xFFFFFFFF. EEPROMs of various sizes can be addressed in the ranges shown in Table 96.

IMPORTANT: To correctly access EEPROMs that are 1 Mb or larger, ensure that the EEPROM address mode is set to use 24-bit addresses.

IMPORTANT: When writing EEPROMs that are smaller than 128 kb, ensure that the EEPROM page mode is set appropriately to allow correct writing of the smaller page sizes these EEPROMs use. When accessing EEPROMs that are larger than 256 kb the page size should be set to allow for optimal write-time efficiency. For more information see Section 9.5, “Page Sizes” on page 111.

Table 96. EEPROM Address Ranges for Typical EEPROMs

Size	Addressable Locations	Start Address	End Address
128 kb	16384	0x00000000	0x00003FFF
256 kb	32768	0x00000000	0x00007FFF
512 kb	65536	0x00000000	0x0000FFFF
1 Mb	131072	0x00000000	0x0001FFFF
2 Mb	262144	0x00000000	0x0003FFFF

10.4 File System Integrity

Each application file, code section, and data section is always accompanied by a 16-bit CRC value. The CRC value helps to ensure file system integrity. The CRC value corresponds to the CRC-CCITT value calculated over the preceding data. The CRC-CCITT parameters are listed in Table 97.

Table 97. File System CRC Parameters

CRC Parameter	Value
Bits	16
Polynomial	x^16 + x^12 + x^5 + 1 (0x1021)
Initial Value	0xFFFF
Final XOR Value	0x0000 (No change due to XOR)

Hardware support is available for calculating the CRC values that accompany each section; see the *Hardware Reference Manual for Ezairo 7100* for more details.

In addition to the CRC value accompanying each section, the file system integrity is ensured by verifying that certain integer and address values are within specified ranges.

10.5 SECURITY

The file system provides optional security through the use of AES encryption. Encrypted sections are unreadable unless the shared user key is known. The user key is stored in the file system. For more information on encrypted sections see Section 10.11.5, “Encrypted Sections” on page 129.

10.5.1 Level 1 Security

Level 1 security is recommended when the device package does not physically prevent direct access to the EEPROM. In this configuration, an external system may read the user key data directly. Level 1 security provides a two-stage encryption process to secure the application code. To prevent unauthorized decryption of the section data, a user key (i.e., one created by the developer) is used to encrypt the section data. This key is, in turn, encrypted with a secret shared key. This shared key is assigned by ON Semiconductor and is unique for each customer. The shared key is used to encrypt the user key and bootloader. The index to the shared key is set in the BOOT_INFO variable in the system area.

During boot, the bootloader and user key are decrypted with the shared key which is only accessible by the Program ROM. The shared key cannot be accessed by user applications and hence the shared key remains secret.

Once decrypted, the user key is used by the bootloader to decrypt encrypted sections of the application. The bootloader itself is encrypted (with the shared key) in order to keep the user key secret.

IMPORTANT: Shared keys for Ezairo 7100 are acquired from ON Semiconductor. For more information please contact your ON Semiconductor representative.

10.5.2 Level 2 Security

Level 2 security is suitable when the device package physically restricts direct access to the EEPROM. In this configuration, the user key is stored in the EEPROM in plain text. The user key is used by the bootloader to decrypt all encrypted sections. Since the EEPROM is not externally accessible, any attempts to read the user key data must go through the debug port. By default, the debug port is in restricted mode, which prevents access to the content of the EEPROM from an external device. Switching the debug port out of restricted mode requires that the contents of the key area be wiped.

Once the user key has been wiped, the remaining data cannot be deciphered, since the user key is unavailable. In this mode, the system will not boot properly and the user key must be rewritten to the EEPROM in order to boot the EEPROM application.

10.6 FILE SYSTEM COMPONENTS

The file system consists of a number of different components:

- System area
- Manufacturing area
- Key area
- Application files
- Code sections
- Data sections

The system area and manufacturing area contain high-level application information as well as system-specific information. The application files describe the applications stored on the EEPROM. Application files contain pointers to their respective code and data sections. Code and data sections may be shared amongst different applications.

Code sections are blocks of data consisting of 32-bit words destined for program memory. Data sections are blocks of either 24-bit or 32-bit words destined for X memory, Y memory or program memory. The words in the system area, manufacturing area, key area and application files are all 16-bit, with EEPROM addresses stored in these areas spanning two 16-bit words.

The system area, manufacturing area and key area addresses have fixed locations in the file system memory map, shown in Table 98. The application files, code sections and data sections may be placed at any available location in the EEPROM. The address vectors for the application files are stored in the system area. All components of the file system memory map are described in *sk4_map_nvm.cfi*.

Table 98. File System Map

EEPROM Byte Address	Size (Bytes)	Component
E:0x00000000-E:0x0000003F	64	System area
E:0x00000040-E:0x000000FF	192	Manufacturing area
E:0x00000100-E:0x0000017F	128	Key area
E:0x00000180-EEPROM end address	Minimum 16000 (given a minimum 128 kb EEPROM)	Available for bootloader and applications

The total size of the reserved EEPROM space is 384 bytes (from 0x00000000 to 0x0000017F). User code and data and the bootloader are free to use any address above 0x0000017F. Generally, the bootloader will be placed at address 0x0180 (the first available address).

An overview of the file system is shown in Figure 4.

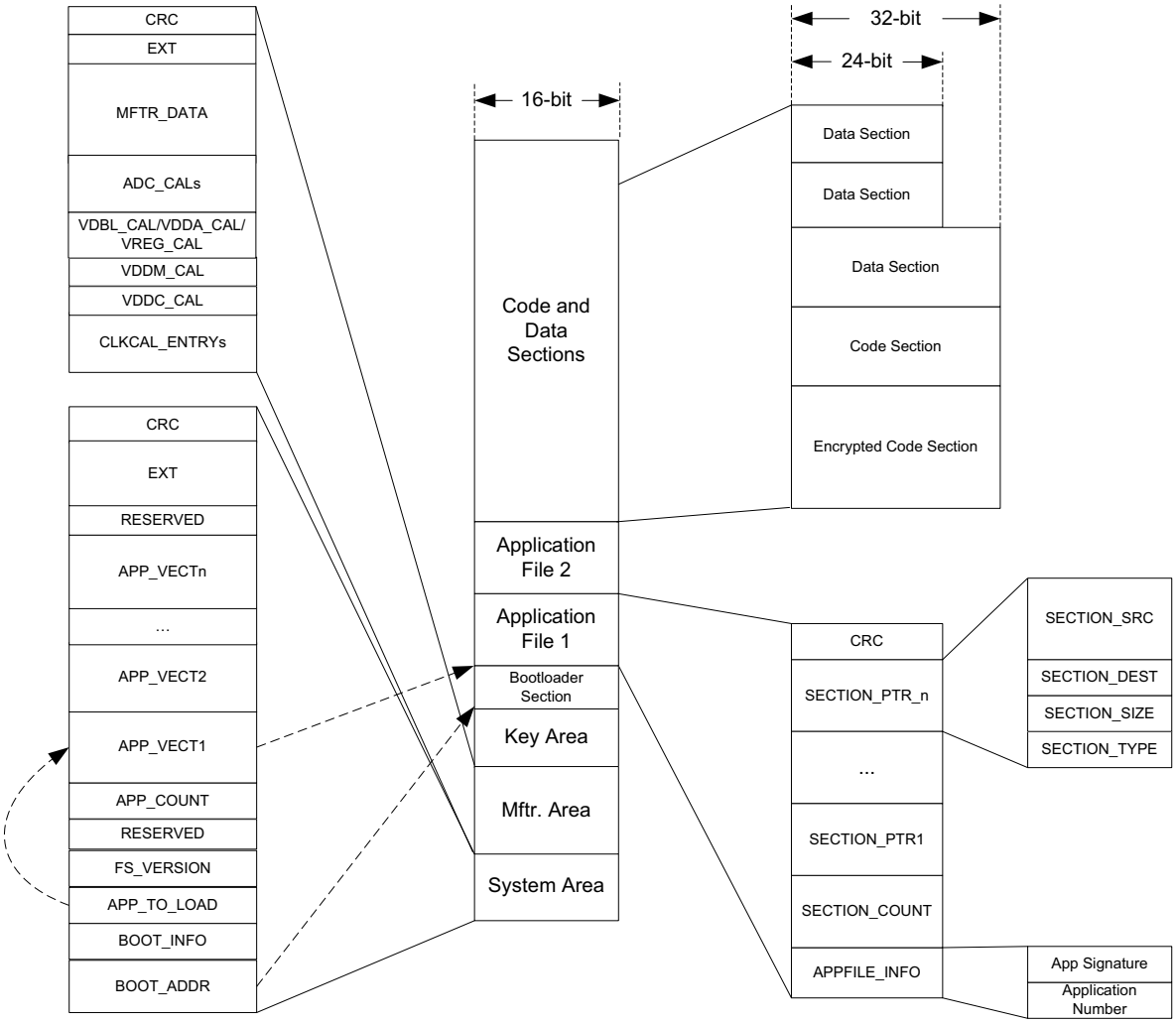


Figure 4. File System Overview

10.7 SYSTEM AREA

The system area contains a boot signature, system variables, and address vectors for the bootloader and the applications stored to the EEPROM. The variables specify the file system version, boot speed, encryption key index, and information on the stored applications. The address vectors indicate where in the EEPROM the application files and the bootloader are stored.

Details of the contents of the system area are provided in Table 99.

Table 99. System Area Contents

Address	Size in Bytes	System Variable	Description
0x0000	4	BOOT_ADDR	Address of the Bootloader in EEPROM; includes a boot signature in the high-byte.
0x0004	2	BOOT_INFO	Bit 15: EEPROM Boot Speed, see Section 10.7.1, “Boot Variables” on page 122 Bits 7:0: Encrypted Bootloader key index
0x0006	2	APP_TO_LOAD	Application to load
0x0008	2	FS_VERSION	File System Version
0x000A	4	RESERVED	-
0x000E	2	APP_COUNT	Number of applications in EEPROM
0x0010	40	APP_VECT	Application Vectors (maximum 10 vectors)
0x0038	2	RESERVED	-
0x003A	4	SYS_AREA_EXT	System Area Extension vector
0x003E	2	SYS_AREA_CRC	CRC of System Area

10.7.1 Boot Variables

The system area contains several words required for booting the system. The address of the bootloader is stored as the first word of the system area. BOOT_ADDR specifies the byte address of the bootloader. The boot ROM code uses this address to load the bootloader from EEPROM to RAM at system start-up. The bootloader code is organized as a special code section; it may also be encrypted.

A 1-byte signature (0x5A) will be included at the top of the boot address in any valid system area. A 16-bit read of this value from an EEPROM addressed with a 16-bit address value results in one of the following values being read:

- 0x5A**, for a 16-bit addressable EEPROM device
- 0x005A or 0xFF5A for a 24-bit addressable EEPROM device (the EEPROM device will not transmit data while it is waiting to receive the remaining 8 address bits)

The boot speed and encryption key index are stored in the BOOT_INFO word. The high bit (bit 15) specifies whether the bootloader should be read with a fast or slow SPI clock. By default, the EEPROM is accessed with a slow SPI clock. After reading BOOT_INFO, the SPI clock speed can be increased to speed up overall booting speed. Since the on-chip oscillator is not calibrated until later in the boot process, the typical SPI speed may vary. Table 100 describes the settings for the BOOT_INFO speed bit.

Table 100. Boot Info Speed Bit Configuration

Bit 15	Description	Typical SPI Clock Frequency
0	Slow SPI clock frequency	160 kHz
1	Fast SPI clock frequency	1.28 MHz

The low 8 bits of BOOT_INFO specify whether the bootloader code section and user key are encrypted. A value of 0 indicates that the code section and user key are not encrypted. Otherwise, the value indicates the key index of the secret shared key to use for decryption. For more information on file system security see Section 10.5, “Security” on page 119.

10.7.2 File System Version

The system area includes the FS_VERSION word, which indicates the file system version of the data in the EEPROM. This value may be used by the firmware to distinguish between various file system formats. See Section 10.2, “Version” on page 117 for more information.

10.7.3 Application Variables

The system area contains application variables that store information about the applications stored in the EEPROM. These variables are used by the bootloader to load the applications from EEPROM to RAM.

Applications are identified by sequential integer values. The first application stored in the EEPROM is application 1. The second application is application 2, and so forth.

The APP_COUNT variable specifies the total number of applications. The APP_TO_LOAD variable specifies the default application to boot by its application number. Each application stored has an associated APP_VECT in the system area. The APP_VECTn variables contain pointers to the associated application files stored in the EEPROM. For example, application 1 would contain an application file, and a pointer to that application file would be stored at APP_VECT1 (E:0x0010). Application 2 would contain a different application file from application 1. A pointer to application 2’s application file would be stored at APP_VECT2 (E:0x0014).

The system area supports a maximum of 10 separate applications. For more information on applications, see Section 10.11, “Applications” on page 127.

10.7.4 System Area Extension Vector

The system area includes a reserved vector at SYS_AREA_EXT (E:0x003A) for possible future extensions of the system area. This reserved vector is not used for file system version 2.1, and for this file system version, must be left as 0x0000.

10.8 Manufacturing Area

The manufacturing area contains calibration information and other manufacturing-specific data. This data is usually set during manufacturing and it should not be modified by users or applications. Specifically, the manufacturing area includes:

- Calibration configuration information for:
  - The internal oscillator
  - VDDC, VDDM, VDBL, VDDA and VREG
  - DC offset calibration entries for each ADC and each pre-amplifier gain configuration

- Manufacturing information
- A 4-byte address for the manufacturing area extension vector
- A CRC calculated over the rest of the manufacturing area

The default clock calibration and power supply calibration information are loaded from EEPROM to the appropriate system registers. All clock calibration entries and ADC calibration data are loaded to the system variables located in X memory and Y memory respectively during the normal boot procedure by the bootloader. For more information about the loaded system variables see Section 2.5.2, “System Variables” on page 33.

Details of the contents of the manufacturing area are provided in Table 101.

Table 101. Manufacturing Area Contents

Address	Size in Bytes	System Variable	Description
0x0040	3	CLKCAL_DEFAULT	Default clock calibration entry, see 10.8.1 on p. 124
0x0043	33	CLKCAL_ENTRY*	Additional clock calibration entries (a maximum of 11 additional entries), see 10.8.1 on p. 124
0x0064	4	VDDC_CAL	VDDC calibration entry, see 10.8.3 on p. 125
0x0068	4	VDDM_CAL	VDDM calibration entry, see 10.8.3 on p. 125
0x006C	1	VDBL_CAL	VDBL calibration entry, see 10.8.4 on p. 126
0x006D	1	VDDA_CAL	VDDA calibration entry, see 10.8.4 on p. 126
0x006E	2	VREG_CAL	VREG calibration entry, see 10.8.4 on p. 126
0x0070	16	ADC0_CAL	ADC0 offset calibration entries, see 10.8.2 on p. 125
0x0080	16	ADC1_CAL	ADC1 offset calibration entries, see 10.8.2 on p. 125
0x0090	16	ADC2_CAL	ADC2 offset calibration entries, see 10.8.2 on p. 125
0x00A0	16	ADC3_CAL	ADC3 offset calibration entries, see 10.8.2 on p. 125
0x00B0	56	MFTR_DATA	Manufacturing Data
0x00E8	4	VDDC_CAL_ALT	VDDC alternate calibration entry
0x00EC	14	MFTR_DATA	Manufacturing Data continued
0x00FA	4	MFTR _AREA_EXT	Manufacturing Area Extension vector
0x00FE	2	MFTR _AREA_CRC	CRC of Manufacturing Area

10.8.1 Clock Calibration

The manufacturing area contains up to 12 clock calibration entries. The clock calibration entries are specific to individual chips. During manufacturing the clock calibration values for all desired operating frequencies should be

determined and stored to the manufacturing area. The first clock calibration entry is the default entry and is used by the bootloader to initially calibrate the clock. Unused clock calibration entries should be set to 0x000000.

For Ezairo 7100 calibration of the internal oscillator requires configuration of two registers:

A_OSC_CTRL_0
Primary frequency adjustment

A_OSC_CTRL_1
Oscillator multiplier configuration, enable

To properly calibrate a device's internal oscillator, the `OSC_CTRL_0_FREQ_CTRL` field from `A_OSC_CTRL_0` and the `OSC_CTRL_1_MULT` field from `A_OSC_CTRL_1` need to be set. These fields are stored along with an operating frequency indicator in a 24-bit field as shown in Figure 5. For a list of valid operating frequencies, see Section 2.9, "Operating Frequencies" on page 36.

Bits 23:18	Bits 17:16	Bits 15:0
Operating Frequency Index (SYS_FREQ_*)	Multiplier setting for bits 5:4 of A_OSC_CTRL_1	Frequency setting for bits 15:0 of A_OSC_CTRL_0

Figure 5. Clock Calibration Bit Fields

10.8.2 ADC Offset Calibration

The manufacturing area contains an ADC DC offset calibration setting for each of the 13 preamplifier configurations that each input channel can be configured for. A calibration table containing ADC DC offset configurations for all four ADCs is created by the calibration library for a given sampling frequency configuration and stored to EEPROM. Each ADC's calibration data is stored to a 16-byte block of ADC calibration data as:

Byte 0 The SFCR value if calibration settings are available; 0x55 otherwise

Bytes 1 to 14: ADC offset calibration settings for a preamplifier configured to provide 0dB, 3dB, …, or 36dB of preamplifier gain

Byte 15: Reserved

The bootloader loads the resulting calibration table from EEPROM to firmware-reserved Y memory where it is used by the `Sys_Set_ADC_CTRL`, `Sys_SetReg_ADC_CTRL` and `Sys_CalibrateADCs` macros, and the `SYSLIB_CalibrateADC` and `SYSLIB_CalibrateADCs` functions. For more information on the system macros see Section 3.12, "Analog Block Configuration" on page 54. For more information on the system library macros and functions see Section 4.4, "System Library Macros" on page 61 and Section 4.3, "System Library Functions" on page 61. The calibration table for an ADC is considered valid if byte 0 of the ADC calibration data contains a valid SFCR value. This byte should be set to 0x55, as described above, if the ADC calibration data is invalid.

10.8.3 VDDC and VDDM Calibration

The Ezairo 7100 device has two digital supply voltages that will need to be calibrated for optimal power efficiency. The VDDC supply voltage is the primary digital supply voltage supplying all of the digital components except the memories, which are supplied by VDDM. The EEPROM file system includes configuration for the two registers that are used to trim these supply voltages. This information is stored in a 4-byte array. Bit 7 of byte 0 is used as a valid marker for these calibration settings. This bit will be set to 0x0 if no calibration settings are available, and to 0x1 otherwise.

10.8.4 VREG, VDDA and VDBL Calibration

The Ezairo 7100 device uses a bandgap regulator to provide a reference voltage used to create the VREG output voltage. This voltage is the primary reference voltage for the analog supply voltage (VDDA) and the various analog system components. The EEPROM file system includes a configuration value for the register used to trim this bandgap voltage.

The Ezairo 7100 device provides two separate charge pumps, supporting VDDA and the regulated doubled supply voltage (VDBL). These supply voltages can be calibrated, and the EEPROM file system includes configuration values for the registers used to trim the output voltage for these charge pumps.

The information for these three calibrations will be stored in a 4-byte array as:

Bytes 0 and 1 **Bit 15** - Valid marker for VREG trim (0x0 if no calibration settings are available, 0x1 otherwise)

Bits 12:8, 4:0 - Settings for `A_BG_CTRL`

Byte 2 **Bit 7** - Valid marker for VDDA trim (0x0 if no calibration settings are available, 0x1 otherwise)

Bits 5:0 - Settings for the `CP_VDDA_CTRL_TRIM` bit-field from `A_CP_VDDA_CTRL`

Byte 3 **Bit 7** - Valid marker for VDBL trim (0x0 if no calibration settings are available, 0x1 otherwise)

Bits 5:0 - Settings for the `CP_VDBL_CTRL_VTRIM` bit-field from `A_CP_VDBL_CTRL`

10.8.5 Manufacturing Data Area

The manufacturing area includes an area reserved for other manufacturing data at MFTR_DATA (E:0x0000B0).

IMPORTANT: To avoid over writing traceability data please contact your ON Semiconductor representative before storing customer manufacturing data to this block.

10.8.6 Manufacturing Area Extension Vector

The manufacturing area includes a reserved vector at MFTR_EXT (E:0x0000FA) for possible future extensions of the manufacturing area. This reserved vector is not used for file system version 2.1, and for this file system, must be left as 0x00000000.

10.9 KEY AREA

The key area is reserved exclusively to store user encryption keys and other secure data. The keys are used by the bootloader to decrypt encrypted code sections in user applications. For increased security, the keys themselves may be encrypted using the ROM-based shared key. For more information on file system security see Section 10.5, “Security” on page 119. Table 102 lists the entries in the key area.

Table 102. Key Area

EEPROM Byte Address	Size (Bytes)	System Variable	Description
E:0x00000100	2	KEY_INFO	User key information

Table 102. Key Area

EEPROM Byte Address	Size (Bytes)	System Variable	Description
E:0x00000102-E:0x00000121	32	KEY_DATA[n]	User encryption keys
E:0x00000122-E:0x0000017F	94	RESERVED	Reserved

The KEY_INFO variable indicates whether KEY_DATA is encrypted. Table 103 lists the possible KEY_INFO values.

Table 103. KEY_INFO Values

KEY_INFO	Description
0	No user key
1	Encrypted user key
2	Plain-text user key

The key area is bounded on a 128-byte page boundary and is 128 bytes in size. This is required for compatibility with some 512-kb and larger EEPROMs that only support page writes.

10.10 BOOTLOADER

The bootloader is a special code section stored in the EEPROM. The bootloader code is automatically loaded from EEPROM to RAM at boot time by the boot ROM. The bootloader can be placed at any available location in the EEPROM. Generally, the bootloader is placed at E:0x00000180 (the first available address after the reserved areas). The BOOT_VECT word in the system area points to the first word of the bootloader section. The bootloader consists of a specially formatted single code section. Table 104 lists the format of the section. The bootloader section contains the bootloader size, destination address, data and a CRC. The data may be encrypted.

Table 104. Bootloader Section

Byte Offset	Bootloader Variable	Description
0	BOOT_SIZE	Size of the bootloader data (in 32-bit words)
2	BOOT_DEST	Destination address
4	Data	Bootloader data
4 + 4 * BOOT_SIZE	CRC	CRC of the bootloader section

For more information on the bootloader, see Chapter 13, “Bootloader” on page 145.

10.11 APPLICATIONS

The file system may store up to 10 different applications. Each application consists of an application vector, application file, and sections. The application vector is stored in the system area and contains a pointer to the associated application file. The application file contains section pointers to the associated sections. The application sections may contain code or data and are loaded into X memory, Y memory or program memory.

Multiple applications may share section data. This reduces overall EEPROM usage as it allows common parameters and code to be shared between different applications.

10.11.1 Application Files

The application file describes each of the sections associated with a particular application. The sections may be code or data destined for X memory, Y memory or program memory. Each section is described in the application file by a section pointer. The application file is made up of a header and multiple section pointers. The header includes a signature, application number and a count indicating the total number of sections in the application. The section pointers are continuous in the application file. The application file contains a CRC calculated based on all the preceding words in the application file.

Table 105. Application File Format

Byte Offset	Variable	Description
0	APPFILE_INFO	Application number and signature
2	SECTION_COUNT	Number of sections in the application
4	SECTION_PTR[n]	Section pointers
4 + SECTION_COUNT*10	CRC	CRC of the application file

The APPFILE_INFO word contains two fields. The high byte of the word contains an application signature. The application signature value is a constant value of 0xAA. The application signature is used to further verify file system integrity. The low byte of APPFILE_INFO is the application number.

Applications are limited to a maximum of 256 sections. An application specifying a SECTION_COUNT exceeding 256 is considered invalid.

Table 106 lists the contents of each section pointer in the application file.

Table 106. Section Pointer Format

Byte Offset	Variable	Size (bytes)	Description
0	SECTION_TYPE	2	Indicates the type of data
2	SECTION_SIZE	2	Size of the section (in words)
4	SECTION_DEST	2	Destination address for the section data
6	SECTION_SRC	4	EEPROM address of the section data

The section type describes the type of data associated with the section. The section types are listed in Table 107. The size of the section is specified in the native word size depending on the section type.

Table 107. Section Types

SECTION_TYPE	Identifier	Word Size	Description
0	STD_X	24-bit	Data destined for X memory. The CRC must be valid.
1	VOL_X	24-bit	Data destined for X memory. The CRC is calculated but its validity is not critical.
2	STD_Y	24-bit	Data destined for Y memory. The CRC must be valid.
3	VOL_Y	24-bit	Data destined for Y memory. The CRC is calculated but its validity is not critical.

Table 107. Section Types

SECTION_TYPE	Identifier	Word Size	Description
4	STD_P	32-bit	Data destined for program memory. The CRC must be valid.
5	ENC_P	32-bit	Data destined for program memory. The data is encrypted with the user key. The CRC must be valid.

10.11.2 Application Execution

During the boot process, the final step of the bootloader is to execute the loaded application. The bootloader selects the start address of the application using the first program memory section pointer in the application file. Hence, when organizing an application in the file system, the first program memory section pointer must correspond to the main entry point of the application.

10.11.3 Sections

The section pointer indicates the EEPROM address for the section data. The section pointer contains all of the information about the data. The section data itself only contains the data to be loaded into XMEM, YMEM or PMEM, and a CRC. Table 108 shows the section format.

The size of the data in words is specified in the SECTION_SIZE word of the section pointer. The size in EEPROM of the section data is determined based on the size in words and the word size. The word size is based on the destination memory space.

The last word of the section contains a 16-bit CRC value, which is calculated over the actual EEPROM data.

Table 108. Section Format

Byte Offset	Description
0	Section Data
SECTION_SIZE * WORD_SIZE	CRC of section

10.11.4 Volatile Sections

The file system has provisions for data sections that may have an incorrect CRC. The volatile section type indicates a non-critical error should be generated when the CRC does not match the data. A non-critical error does not prevent the rest of the application from being loaded and subsequently executed. This section type is useful for parameters that may be updated in the field. If, during a programming cycle, the EEPROM loses power or otherwise does not program correctly, the data being written may become corrupt. In this case, the bootloader will detect the incorrect CRC on the subsequent boot. For non-volatile sections, an error will be generated and the bootloader will not start the application. For volatile sections, an error is generated but the bootloader will start the application. The application must check the error code and contain provisions for re-initializing the corrupt section.

10.11.5 Encrypted Sections

The file system supports sections that are encrypted. Encrypted sections are denoted in the section pointer with the ENC_P section type. The encryption key used for encrypted sections is stored in the key area. During the boot process, the key is read from EEPROM and used to decrypt all encrypted sections.

Encrypted sections have two associated CRCs. First, the section data CRC is calculated based on the encrypted data stored in the EEPROM. This CRC must be verified before the data is decrypted. Further, the plain-text data contains an additional CRC used to verify the plain-text data after decryption.

Encrypted sections are only valid for code and data destined for program memory. **If encrypted X memory or Y memory data is desired, the data must be encoded into a program memory data section and later manually copied by the application into the appropriate memory space.**

Encrypted sections must be padded to 128 bits. Since encrypted sections are always destined for program memory, this corresponds to four 32-bit words. The last word of the plain-text data is the plain-text CRC. The decryption is done in place in program memory.

IMPORTANT: A program memory block will be padded with up to 3 additional words if the section is encrypted. This is required because AES encryption and decryption operate on 128-bit (four 32-bit word) data blocks. Take care to ensure that these added program memory words do not overwrite other program segments in program memory when the data is being loaded from the EEPROM. One possible way to ensure this is to space out the program memory sections by three or more words to account for this padding. Alternatively, the section pointers in the application file can be set up in increasing order based on the destination address to ensure that, even if sections overlap, only the padding data is overwritten.

CHAPTER 11
Calibration Library

11.1 Introduction

The calibration library enables users to calibrate their devices for optimal performance using standard, verified calibration code.

The library contains macros and functions that can be used to:

- Calibrate the current voltages of various power supplies
- Calibrate the current system clock provided by the internal oscillator to a specified frequency
- Calibrate the ADC DC offsets to place audio artifacts outside of the audible band
- Create a manufacturing area containing the information gathered from the calibrations that have been completed on a device and write this manufacturing area to an EEPROM (or compatible NVM device)

11.2 Power Supply Calibration

The calibration library provides functions that enable a user to calibrate the power supply components listed in Table 109. To obtain the performance specified in the Ezairo 7100 datasheet, calibrate these voltages to the target voltages indicated in this table.

Table 109. Power Supplies

Power Supply	Target Voltage
VREG	0.97 V
VDBL	2.0 V
VDDA	2.0 V
VDDC	0.82 V (appropriate for $f_{SYSCLK} \leq 10.24$ MHz)
VDDC_ALT	0.88 V (appropriate for $f_{SYSCLK} \leq 15.36$ MHz, required for $f_{SYSCLK} > 10.24$ MHz)
VDDM	0.90 V

For more information about each of these power supplies, refer to the *Hardware Reference Manual for Ezairo 7100*. For the optimal VDDC and VDDM voltages for each SYSCLK frequency (including functional configurations for cases where $f_{SYSCLK} > 10.24$ MHz), refer to the Ezairo 7100 datasheet.

To calibrate the VREG supply, the standard calibration procedure requires that VBAT is supplied with a known voltage of 1.25 V ±1%. An alternate function is provided to support calibrating the VREG supply relative to an alternate VBAT supply setting. If using an alternate VBAT supply setting to calibrate VREG, the VBAT voltage setting passed to the alternate function is encoded using:

$$\text{Setting}_{\text{ALT_VBAT}} = \frac{V_{VBAT}}{V_{VREG}} \times 0\text{x}100$$

To perform a valid calibration of the other power supplies, the VREG voltage must be calibrated to 0.97 V. Inaccurate calibrations will occur if VREG has not been calibrated previously, as all other power supplies are calibrated relative to VREG. To perform a calibration of VDBL, VDDC, or VDDM that does not adhere to the standard target voltages listed in Table 109, a set of alternate calibration functions is provided where the user needs to provide the LSAD measurement equivalent to the desired voltage.

For an alternate VDBL setting, the value passed in can be calculated using:

$$\text{Setting}_{\text{ALT_VDBL}} = \left(\frac{V_{\text{VDBL}}}{V_{\text{VREG}}} \times 0\text{x}100\right) - 0\text{x}100$$

For an alternate VDDC or VDDM setting, the value passed in can be calculated using:

$$\text{Setting}_{\text{ALT_VDD*}} = \frac{V_{\text{VDD*}}}{V_{\text{VREG}}} \times 0\text{x}100$$

IMPORTANT: The VDDC and VDDM fractional charge pumps are only calibrated if the VBAT supply voltage exceeds the target VDDC or VDDM to a supply voltage by 5/4.

For example, to calibrate the fractional charge pump for VDDM to 1.1 V (as required for a VDDO supply that is greater than 2.1 V) the VBAT supply voltage must be at least 1.375 V. If the VBAT supply voltage is below 1.375 V, only the LDO is calibrated for VDDM.

To calibrate a set of power supplies, first run the `CALIBRATE_Power_Initialize` function, followed by the `CALIBRATE_Power_VREGCalibrate` function or `CALIBRATE_Power_VREGAltCalibrate` function if calibrating VREG. After calibrating VREG, all of the other power supplies can be calibrated in any order by calling the appropriate `CALIBRATE_Power_VD**Calibrate` (or `CALIBRATE_Power_VD**AltCalibrate`) functions.

Table 110. Power Supply Calibration Library Functions

Function	Description	Reference
CALIBRATE_Power_Initialize	Initialize the system for power supply (VREG, VDDC, VDDM, VDDA and VDBL) calibration	21.18 on p. 815
CALIBRATE_Power_VDBLAltCalibrate	Calibrate the interface charge pump (VDBL) to a specified target voltage	21.19 on p. 816
CALIBRATE_Power_VDBLCalibrate	Calibrate the interface charge pump (VDBL)	21.20 on p. 817
CALIBRATE_Power_VDDACalibrate	Calibrate the analog charge pump (VDDA)	21.21 on p. 818
CALIBRATE_Power_VDDCAltCalibrate	Calibrate the digital core voltage (VDDC) to a specified target voltage	21.22 on p. 819
CALIBRATE_Power_VDDCCalibrate	Calibrate the digital core voltage (VDDC)	21.23 on p. 820
CALIBRATE_Power_VDDMAltCalibrate	Calibrate the digital memory voltage (VDDM) to a specified target voltage	21.24 on p. 821
CALIBRATE_Power_VDDMCalibrate	Calibrate the digital memory voltage (VDDM)	21.25 on p. 822
CALIBRATE_Power_VREGAltCalibrate	Calibrate the bandgap voltage (VREG) against a specified VBAT supply voltage	21.26 on p. 823
CALIBRATE_Power_VREGCalibrate	Calibrate the bandgap voltage (VREG)	21.27 on p. 824

11.3 Clock Calibration

The calibration library provides code for calibrating the system clock to any supported clock frequency when it uses the internal oscillator. This calibration procedure requires a 128-Hz square-wave reference clock to be provided at a

user-specified DIO (from the appropriate VDDO* to DGND). Based on this reference clock, the procedure calibrates the internal oscillator to any user-specified clock frequency.

IMPORTANT: To perform a valid calibration of the internal oscillator, you must first calibrate the VREG voltage. Changes to the VREG voltage from that used when calibrating the internal oscillator will affect the frequency of the internal oscillator.

Before calibrating the internal oscillator, use the `CALIBRATE_Clock_Initialize` function to initialize the system components required. After calling the initialization function, the `Calibrate_Clock` macro or its underlying `CALIBRATE_Clock` function can be used to calibrate the internal oscillator to a desired frequency.

If calibrating a set of system clock frequencies, each clock configuration can be stored to the local manufacturing area using the `Calibrate_AddClockCalibration` macro. This macro sets the current oscillator configuration as the setting for a specified operating frequency. For more information see Section 11.5, “Storing Calibration Settings”.

Table 111. Oscillator Calibration Library Macros and Functions

Function	Description	Reference
CALIBRATE_Clock	Calibrate the system clock	21.13 on p. 810
Calibrate_Clock	Calculate the calibration routine's reference clock cycle target for the given clock frequency	21.14 on p. 811
CALIBRATE_Clock_Initialize	Initialize the system to support the clock calibration	21.15 on p. 812

11.4 ADC Calibration

The calibration library provides functions for calibrating the DC offset observed by each ADC to obtain a noise floor that does not contain any audible artifacts caused by this DC offset. Each calibration setting is valid for one ADC input channel, at one preamplifier gain, and for one sampling frequency configuration register setting (the `SAMP_FREQ` bit-field from the `A_ADC_SAMP_FREQ_CFG` register). The ADC calibration settings are used to configure the `OFFSET_SIGN` and `OFFSET_MAGN` bit-fields from `A_ADC_CFG_*`.

NOTE: The manufacturing area and system variables are designed to store a complete set of ADC calibration values for a single `SAMP_FREQ` setting. If more than one SAMP_FREQ setting is required by your applications, your application will be required to store the additional ADC calibration tables elsewhere. By default, the ADCs will be calibrated assuming the default SAMP_FREQ setting of 2.

IMPORTANT: To perform a valid calibration of the ADC DC offsets, you must first calibrate the VREG voltage. Changes to the VREG voltage from that used when calibrating the ADC DC offsets could result in audible artifacts that are in-band.

NOTE: No external connections are required to perform an ADC calibration, as each of the ADC inputs can be grounded internally.

Prior to calibrating the ADCs, configure the `SAMP_FREQ` bit field from the `A_ADC_SAMPLE_FREQ_CFG` register to match the configuration of this register that your applications will use. If the `SAMP_FREQ` setting for the system is updated, the calibration will need to be repeated. The application should then use the `CALIBRATE_ADC_Initialize` function. This function is used to initialize the system to perform an ADC calibration; it configures the input signal

chain, FIFOs, IOC, and decimation filter to allow for DC measurements. This function takes a parameter, representing the tone placement of the ADC DC offset artifact as a multiple of Fs, encoded in the m4p20 format.

The library default for this parameter places the fundamental audio artifact caused by the ADC DC offset at -16/7 times the sampling frequency. A define exists in *sk4_calibrate.cfi* named DEFAULT_PLACEMENT_FS_MULTIPLE for this recommended target. For this default placement all calibrations that are within $\pm 1.92\text{mV}$ are valid. It is recommended to calibrate the DC offset to an accuracy of $\pm 1.51\text{mV}$, to provide some margin for possible errors in the VREG calibration. Defines in *sk4_calibrate.cfi* are provided for both accuracy targets as CALIBRATION_ACCURACY_1P92_MV and CALIBRATION_ACCURACY_1P51_MV. These defines can be used when calling the initialize function as follows:

```
load y0, CALIBRATION_ACCURACY_1P51_MV
call.1d CALIBRATE_ADC_Initialize
load y1, DEFAULT_PLACEMENT_FS_MULTIPLE
```

Once the system is initialized for ADC calibration, each ADC must be calibrated for each preamplifier gain setting that will be used. The ADCs can be calibrated for the current set of preamplifier gain settings using the CALIBRATE_ADC_CalibrateCurrentConfiguration function, which calibrates all four ADCs in parallel, or they can be calibrated for all preamplifier gains using the CALIBRATE_ADC_CalibrateCompleteSystem function.

The output from the CALIBRATE_ADC_CalibrateCurrentConfiguration function returns an unpacked structure in X memory containing the ADC0, ADC1, ADC2 and ADC3 calibrations for the current preamplifier configuration.

The output from CALIBRATE_ADC_CalibrateCompleteSystem is a table that contains an unpacked structure in X memory containing calibration data for all four ADCs and all 13 preamplifier gain settings, ordered as follows:

```
<ADC0 0dB><ADC0 3dB>...<ADC0 36dB><ADC1 0dB>...<ADC1 36dB><ADC2 0dB>...
<ADC2 36dB><ADC3 0dB>...<ADC3 36dB>
```

This is the same format expected by the Calibrate_AddADCCalibration macro, which can be used to pack data from an ADC calibration into the manufacturing area—see Section 11.5, “Storing Calibration Settings”.

Table 112. ADC Calibration Library Functions

Function	Description	Reference
CALIBRATE_ADC_CalibrateCompleteSystem	Builds a table of all ADC DC offset values (starting at xp0)	21.1 on p. 798
CALIBRATE_ADC_CalibrateCurrentConfiguration	Given the current PGA gain setting and the target DC offset, an 8-bit calibration value is calculated for each ADC channel	21.2 on p. 799
CALIBRATE_ADC_Initialize	Initialize the system to support the ADC calibration	21.3 on p. 800

11.5 Storing Calibration Settings

The calibration settings that are calculated using the calibration library or set through manual calibration are generally are stored in their respective configuration registers. The calibration library contains a number of macros that can use these values to create a local manufacturing area data structure that can be stored to the EEPROM, to be used by the bootloader and user applications.

Before accessing the manufacturing area on an EEPROM, the DIOs for the SPI interface used to access the EEPROM must be selected and the EEPROM library must be initialized as required for the read or write accesses that will be performed. For more information on the contents of the manufacturing area of the file system see Section 10.8, “Manufacturing Area” on page 123. For more information on setting up the EEPROM library to read or write a manufacturing area see Chapter 9, “EEPROM Library” on page 109.

Before storing any calibration settings, a local version of the manufacturing area must be created in X memory to store the calibration data. The local version of the manufacturing area can be created and initialized to a default blank state using the `Calibrate_InitializeMA` macro. If the existing manufacturing area in EEPROM contains data that should be preserved, this data can be loaded from the EEPROM to the local manufacturing area instance using the `Calibrate_LoadMA` macro.

The following macros update the local copy of the manufacturing area with power supply calibration information based on the current power supply configuration. They store the specified power supply configuration as calibrated settings for that power supply component:

- `Calibrate_AddVREGCalibration`
- `Calibrate_AddVDBLCalibration`
- `Calibrate_AddVDDACalibration`
- `Calibrate_AddVDDCCalibration`
- `Calibrate_AddVDDMCalibration`

To `Calibrate_AddClockCalibration` macro can be used to update the local copy of the manufacturing area with oscillator calibration information. It sets the current oscillator configuration as the setting for a specified operating frequency (see Section 2.9, “Operating Frequencies” on page 36).

The `Calibrate_AddADCCalibration` macro can be used to update the local copy of the manufacturing area with a set of calibration information for one ADC. It copies a table of previously calculated ADC DC offset calibration settings and can be used to update the local manufacturing area structure with data for the ADCs that are of interest to a user.

The following macros support updating the manufacturing data area within the manufacturing area with user-defined 8-bit, 24-bit and 32-bit data. These macros are:

- `Calibrate_UpdateMA8`
- `Calibrate_UpdateMA24`
- `Calibrate_UpdateMA32`

Once the local manufacturing area data structure contains all data that should be stored to the EEPROM, this data can be stored using the `Calibrate_StoreMA` macro or its underlying `CALIBRATE_StoreMA` function. This function calculates a CRC over the local manufacturing area data structure, and stores the contents of that image to the manufacturing area of the EEPROM file system, overwriting any existing data.

IMPORTANT: If your Ezairo 7100 device is supplied as part of a SiP that contains an EEPROM, it is likely that your device will be pre-calibrated for you. If you have questions about the EEPROM contents for an ON Semiconductor supplied SiP, contact your ON Semiconductor representative.

Macros that support populating a local manufacturing area instance are listed in Table 113. The macro (and underlying function) provided by the calibration library that can be used to store this manufacturing area back to an EEPROM can be found in Table 114.

Table 113. Local Manufacturing Area Support Macros

Macro	Description	Reference
Calibrate_ADCCopyToRAM	Copy the ADC calibration data from the local manufacturing area to the system variables in low Y RAM	21.4 on p. 801
Calibrate_AddADCCalibration	Update the ADC calibration entry specified in the local manufacturing area using the current SAMP_FREQ and the data pointed to by xpreg	21.5 on p. 802
Calibrate_AddClockCalibration	Update the specified clock calibration entry in the local manufacturing area using the current clock configuration	21.6 on p. 803
Calibrate_AddVDBLCalibration	Update the VDBL calibration entry in the local manufacturing area using the current VDBL configuration	21.7 on p. 804
Calibrate_AddVDDACalibration	Update the VDDA calibration entry in the local manufacturing area using the current VDDA configuration	21.8 on p. 805
Calibrate_AddVDDCCalibration	Update the VDDC calibration entry in the local manufacturing area using the current VDDC configuration	21.10 on p. 807
Calibrate_AddVDDCAltCalibration	Update the VDDC alternate calibration entry in the local manufacturing area using the current VDDC configuration	21.9 on p. 806
Calibrate_AddVDDMCalibration	Update the VDDM calibration entry in the local manufacturing area using the current VDDM configuration	21.11 on p. 808
Calibrate_AddVREGCalibration	Update the VREG calibration entry in the local manufacturing area using the current VREG configuration	21.12 on p. 809
Calibrate_InitializeMA	Reset the contents of the local manufacturing area	21.16 on p. 813
Calibrate_LoadMA	Load the local manufacturing area instance from an attached EEPROM	21.17 on p. 814
Calibrate_UpdateMA24	Update the local manufacturing area by writing at the table offset specified	21.30 on p. 827
Calibrate_UpdateMA32	Update the local manufacturing area by writing at the table offset specified	21.31 on p. 828
Calibrate_UpdateMA8	Update the local manufacturing area by writing at the table offset specified	21.32 on p. 829

Table 114. Manufacturing Area Store Support

Function	Description	Reference
CALIBRATE_StoreMA	Store the contents of the local manufacturing area to an attached EEPROM	21.28 on p. 825
Calibrate_StoreMA	Store the contents of the local manufacturing area to an attached EEPROM	21.29 on p. 826

11.6 Calibration Library Errors

When attempting to calibrate a system, the calibration could fail for a variety of reasons. To identify why a calibration may have failed, the calibration library sets the `SYSVAR_CAL_ERRNO` system variable using one of the error codes listed in Table 115. For more information on system variables see Section 2.5.2, “System Variables” on page 33.

Table 115. Calibration Error Codes

Error code	Symbol	Description
0x000	ERRNO_NO_ERROR	No error during calibration
0x011	ERRNO_ADC_DID_NOT_CONVERGE	One or more ADC calibrations attempts did not converge on an appropriate ADC DC offset
0x021	ERRNO_NO_CLK_REFERENCE	No reference clock was detected on the specified DIO
0x022	EERNO_TIMEOUT_DURING_CALIBRATE	A time out occurred while attempting to calibrate the clock
0x031	ERRNO_VREG_CAL_ERROR	VREG failed to calibrate correctly
0x032	ERRNO_VDDA_CAL_ERROR	VDDA failed to calibrate correctly
0x033	ERRNO_VDBL_CAL_ERROR	VDBL failed to calibrate correctly
0x034	ERRNO_VDDC_CAL_ERROR	VDDC failed to calibrate correctly
0x035	ERRNO_VDDM_CAL_ERROR	VDDM failed to calibrate correctly

To troubleshoot calibration failures consult the following lists of common causes for calibration failures:

Power supply calibration failures (VREG)

- The supplied VBAT as observed at the Ezairo 7100's VBAT pad is not 1.25 V
- VREG, VDBL or VDDA is being over-driven externally

Power supply calibration failures (Other)

- VREG has not been previously calibrated
- VREG, VDBL or VDDA is being over-driven externally
- The power supply to be calibrated is being over-driven externally

Clock calibration failures

- VREG has not been previously calibrated
- No reference clock has been supplied
- The voltage level of the supplied reference clock does not match the appropriate VDDO* supply voltage
- The supplied reference clock is intermittent
- It is not possible to calibrate the Ezairo 7100 oscillator to the requested frequency; i.e., the requested frequency is outside of the frequency range supported by the Ezairo 7100 oscillator.

ADC calibration failures

- VREG has not been previously calibrated
- The specified tolerance for ADC calibration is too narrow
- The specified DC offset to calibrate to is out of range

CHAPTER 12
Program ROM

12.1 DESCRIPTION

The Program ROM is a ROM mapped into program memory. It contains various elements of firmware code. The primary purpose of the Program ROM is to facilitate booting the system, but it also contains firmware code required for security. In addition, many of the firmware libraries are duplicated in the Program ROM so that ROM-based functions can be used instead of RAM-based versions, reducing the firmware footprint in the RAM and EEPROM memory spaces. The Program ROM provides access to the ROM instances of the firmware code, internally through function tables and externally through the debug port's JumpROM facility. For more information about the JumpROM facility and file system security, see the *Communication Protocols Manual for Ezairo 7100*.

12.1.1 Versions

The Program ROM version is a 16-bit symbol that can be read from the `SYSVAR_PROGROM_VER` variable. This value can be used directly in the application code to identify the version of the ROM available on your device. An external application or device can also read the ROM version from the `SYSVAR_PROGROM_VER` variable or through the JumpROM command. See Section 2.8, "Versions" on page 35 for more information about version symbols. Table 116 lists the versions of the Program ROM referenced by this manual.

Table 116. Program ROM Versions

Device	Version
Ezairo 7100 version 1.01.xx	6.0A

12.2 BOOT ROM

After Power-on-Reset (POR), program execution begins in the Program ROM. The start of the Program ROM contains the boot ROM code.

12.2.1 Boot Initialization

The boot ROM performs basic system initialization required for booting the system. The system initialization consists of the following:

- Resetting the CFX status register
- Resetting the CFX loop stack
- Initializing the CFX stack pointer
- Pausing the HEAR Configurable Accelerator, ARM Cortex-M3 processor and the Filter Engine
- Setting the Watchdog timeout to maximum (nominally 3.2 seconds)
- Restoring the memory component configuration to the default state
- Reconfiguring all DIO pins:
  - Configuring DIO 0 as SPI_CLK
  - Configuring DIO 1 as SPI_CS
  - Configuring DIO 2 as SPI_SERI
  - Configuring DIO 3 as SPI_SERO
  - Configuring DIOs 4 to 29 as LSAD inputs

- Power supply to analog components enabled (if disabled)
- Other power supply components enabled
- Resetting the internal oscillator to its default configuration

- Configuring clocking to select the internal oscillator
- Initializing firmware global variables
- Configuring the ARM Cortex-M3 processor with a minimal application to prevent corruption of the ARM Cortex-M3 processor state during ARM Cortex-M3 debug events
- Resetting the ARM Cortex-M3 processor to ensure it is in a good state, and will run within the configured application

12.2.2 Stabilization Delay

After system initialization, the boot ROM implements a power stabilization delay. This delay is intended to prevent spurious system resets due to voltage transients when connecting a battery. The power stabilization delay is nominally 140 ms (minimum 100 ms, maximum 233 ms). If desired, the application may introduce an additional delay before starting audio output.

12.2.3 Loading the Bootloader

The boot ROM proceeds to load the provided bootloader from EEPROM. During the boot process the boot ROM checks various file system parameters and the bootloader CRC to verify data integrity. If the bootloader is encrypted, the boot ROM executes the appropriate decryption functions in the Program ROM. The boot variable used to indicate the selected application (APP_TO_LOAD) is also loaded from EEPROM and stored in RAM. Finally, the boot ROM releases control of the system to the loaded bootloader.

12.2.4 Boot Failures

A boot failure may occur if the file system does not contain a valid bootloader image. In the case of a failed boot, the boot ROM implements a delay and proceeds to reboot the system. The system error code indicates the particular error. The delay before resetting the system is nominally 1400 ms (minimum 1000 ms, maximum 2333 ms). See Section 2.10.2, “Boot Errors” on page 38 for more information on boot errors, and Section 2.5.2, “System Variables” on page 33 for more information on the `SYSVAR_SYS_ERRNO` (all boot failures) and `SYSVAR_BOOT_ERRNO` (boot failures within the boot ROM) variables that will hold the reported error codes for any errors that result in a boot failure.

For security, the Ezairo 7100 debug port is initially in restricted mode. Program ROM functions are provided to wipe the EEPROM key area and then unlock the device. However, for convenience, the boot ROM will automatically attempt to unlock the device in the special case when the value of the first word of the EEPROM does not contain an application signature. In that case, the boot ROM will check the contents of the key area. If the key area has previously been wiped, by writing it with all zeros or all ones, the device is automatically unlocked. This feature allows developers to easily connect to an un-initialized device for development.

12.3 Function Tables

The Program ROM contains the implementation of many firmware functions. Most of the functions are available in firmware libraries. To reduce EEPROM and RAM usage, specialized versions of the libraries are available that make use of the code in the Program ROM. See Section 2.4.1, “ROM-Based Libraries” on page 30 for more information on ROM-based libraries.

To avoid dependencies on the particular addresses of the Program ROM functions, the Program ROM contains a function table consisting of `goto.0d` vectors for each of the embedded functions. The specialized ROM-based libraries use the function table instead of absolute addresses.

In the function table, each function is assigned a function number. The function number is used to calculate an offset into the function table. The firmware include file, *sk4_romvect.cfi* defines the function table entries for available Program ROM functions. A list of the function table vectors that are available is in Table 117.

Table 117. Program ROM Functions in the Function Table

Function Name	Function Number	Notes
ROMVECT_PopLR_Return	0	-
ROMVECT_SystemInitialize	1	-
ROMVECT_BootAppEntry	2	Used for JumpROM BootApp
ROMVECT_PushLoopsToStack	3	Push all the hardware loops to the stack
ROMVECT_PopLoopsFromStack	4	Pop all the hardware loops from the stack
ROMVECT_AESInitialize	5	See the *Communication Protocols Manual for Ezairo 7100*
ROMVECT_AESExpandKey	6	See the *Communication Protocols Manual for Ezairo 7100*
ROMVECT_AESDecrypt	7	See the *Communication Protocols Manual for Ezairo 7100*
ROMVECT_AESCleanUp	8	See the *Communication Protocols Manual for Ezairo 7100*
ROMVECT_SYSLIB_Reset	9	Implements SYSLIB_Reset
ROMVECT_SYSLIB_Delay	10	Implements SYSLIB_Delay
ROMVECT_SYSLIB_DelayBase	11	-
ROMVECT_SYSLIB_WaitForSPIDone	12	Implements SYSLIB_WaitForSPIDone
ROMVECT_SYSLIB_SetOperatingFrequency	13	Implements SYSLIB_SetOperatingFrequency
ROMVECT_SYSLIB_SetOperatingFrequency_Throttle	14	Implements SYSLIB_SetOperatingFrequency_Throttle
ROMVECT_SYSLIB_SoftPowerdownEnable	15	Implements SYSLIB_SoftPowerdownEnable
ROMVECT_SYSLIB_SoftPowerdownDisable	16	Implements SYSLIB_SoftPowerdownDisable
ROMVECT_SYSLIB_CalibrateADC	17	Implements SYSLIB_CalibrateADC
ROMVECT_MATHLIB_SQRT_frac48_frac48	18	Implements MATHLIB_SQRT_frac48_frac48
ROMVECT_MATHLIB_LOG2_frac48_m8p16	19	Implements MATHLIB_LOG2_frac48_m8p16
ROMVECT_MATHLIB_EXP2_m8p16_frac48	20	Implements MATHLIB_EXP2_m8p16_frac48
ROMVECT_MATHLIB_RECIP_POS_frac48_fl8n16	21	Implements MATHLIB_RECIP_POS_frac48_fl8n16
ROMVECT_MATHLIB_RECIP_GEN_frac48_fl8n16	22	Implements MATHLIB_RECIP_GEN_frac48_fl8n16
ROMVECT_MATHLIB_DIVIDE	23	Implements MATHLIB_DIVIDE
ROMVECT_MATHLIB_DIVIDE_LOOP_START	24	-
ROMVECT_EELIB_Initialize	25	Implements EELIB_Initialize
ROMVECT_EELIB_Close	26	Implements EELIB_Close
ROMVECT_EELIB_VerifyEEPROMConnection	27	Implements EELIB_VerifyEEPROMConnection
ROMVECT_EELIB_LibWriteEnable	28	Implements EELIB_LibWriteEnable
ROMVECT_EELIB_LibWriteDisable	29	Implements EELIB_LibWriteDisable
ROMVECT_EELIB_WriteDisable	30	Implements EELIB_WriteDisable

Table 117. Program ROM Functions in the Function Table

Function Name	Function Number	Notes
ROMVECT_EELIB_WriteEnable	31	Implements EELIB_WriteEnable
ROMVECT_EELIB_ReadStatusRegister	32	Implements EELIB_ReadStatusRegister
ROMVECT_EELIB_WriteStatusRegister	33	Implements EELIB_WriteStatusRegister
ROMVECT_EELIB_ReadWord16	34	Implements EELIB_ReadWord16
ROMVECT_EELIB_ReadNextWord16	35	Implements EELIB_ReadNextWord16
ROMVECT_EELIB_ReadWord8	36	Implements EELIB_ReadWord8
ROMVECT_EELIB_ReadNextWord8	37	Implements EELIB_ReadNextWord8
ROMVECT_EELIB_ReadWord24	38	Implements EELIB_ReadWord24
ROMVECT_EELIB_ReadNextWord24	39	Implements EELIB_ReadNextWord24
ROMVECT_EELIB_ReadWord32	40	Implements EELIB_ReadWord32
ROMVECT_EELIB_ReadNextWord32	41	Implements EELIB_ReadNextWord32
ROMVECT_EELIB_InitWriteWord	42	Implements EELIB_InitWriteWord
ROMVECT_EELIB_WriteWord16	43	Implements EELIB_WriteWord16
ROMVECT_EELIB_WriteNextWord16	44	Implements EELIB_WriteNextWord16
ROMVECT_EELIB_WriteWord8	45	Implements EELIB_WriteWord8
ROMVECT_EELIB_WriteNextWord8	46	Implements EELIB_WriteNextWord8
ROMVECT_EELIB_WriteWord24	47	Implements EELIB_WriteWord24
ROMVECT_EELIB_WriteNextWord24	48	Implements EELIB_WriteNextWord24
ROMVECT_EELIB_WriteWord32	49	Implements EELIB_WriteWord32
ROMVECT_EELIB_WriteNextWord32	50	Implements EELIB_WriteNextWord32
ROMVECT_EELIB_WriteBlock_P	51	Implements EELIB_WriteBlock_P
ROMVECT_EELIB_WriteBlock_X	52	Implements EELIB_WriteBlock_X
ROMVECT_EELIB_WriteBlock_Y	53	Implements EELIB_WriteBlock_Y
ROMVECT_EELIB_WriteBlock_Const32	54	Implements EELIB_WriteBlock_Const32
ROMVECT_EELIB_WriteBlock	55	-
ROMVECT_EELIB_CommitAndWrite	56	-
ROMVECT_EELIB_ReadBlock_P	57	Implements EELIB_ReadBlock_P
ROMVECT_EELIB_ReadBlock_P16	58	Implements EELIB_ReadBlock_P16
ROMVECT_EELIB_ReadNextBlock_P	59	Implements EELIB_ReadNextBlock_P
ROMVECT_EELIB_ReadNextBlock_P16	60	Implements EELIB_ReadNextBlock_P16
ROMVECT_EELIB_ReadBlock_X	61	Implements EELIB_ReadBlock_X
ROMVECT_EELIB_ReadBlock_X16	62	Implements EELIB_ReadBlock_X16
ROMVECT_EELIB_ReadNextBlock_X	63	Implements EELIB_ReadNextBlock_X
ROMVECT_EELIB_ReadNextBlock_X16	64	Implements EELIB_ReadNextBlock_X16
ROMVECT_EELIB_ReadBlock_Y	65	Implements EELIB_ReadBlock_Y

Table 117. Program ROM Functions in the Function Table

Function Name	Function Number	Notes
ROMVECT_EELIB_ReadNextBlock_Y	66	Implements EELIB_ReadNextBlock_Y
ROMVECT_EELIB_Commit	67	Implements EELIB_Commit
ROMVECT_EELIB_InitAddrSPIWrite	68	-
ROMVECT_EELIB_Init8BitSPIWrite	69	-
ROMVECT_EELIB_InitSPIWriteOperation	70	-
ROMVECT_EELIB_PowerDown	71	Implements EELIB_PowerDown
ROMVECT_EELIB_PowerUp	72	Implements EELIB_PowerUp
ROMVECT_EELIB_SelectSegment	73	Implements EELIB_SelectSegment
ROMVECT_EELIB_AutoSet_AddressMode	74	Implements EELIB_AutoSet_AddressMode

12.4 File System Decryption Support

The Ezairo 7100 series supports the use of encrypted data in the non-volatile memory to provide protection to software intellectual property for copyright and related purposes. A key component of this support is provided through the AES function set provided by the Program ROM. The Program ROM uses these functions to decrypt AES-encrypted data blocks in P memory for Level 1 security (see Section 10.5.1, “Level 1 Security” on page 119), and the bootloader uses these functions for encrypted application sections (see Section 10.11.5, “Encrypted Sections” on page 129).

For a complete view of all of the security components provided in the system, see the file system security section of the *Communication Protocols Manual for Ezairo 7100*.

12.4.1 Advanced Encryption Standard (AES) Functions

The ROM-based implementation of the AES standard contains a function set that can be used to decrypt AES pages. The functions provided are:

AESInitialize Load the look-up tables required for a standard table-driven implementation of the AES standard

AESExpandKey Expand a 128-bit AES key to the full expanded key—including both the encryption form (needed for the expansion of the decryption key), and the decryption form (needed for decrypting blocks of AES encrypted data)

AESDecrypt Decrypt one or more blocks of contiguous data using the specified expanded key

AESCleanup Clear the keys from memory—use this before the application switches the access mode to unrestricted, if it is important to maintain the security of the current key

12.4.1.1 Memory, Register and Other Resource Usage by AES Routines

The AES routines that are provided in the Program ROM for the Ezairo 7100 series use a standard table-driven implementation for both expansion of a provided AES key and the decryption of any blocks of encrypted data. A list of the memory sections and their uses by the AES routines is provided in Table 118.

Table 118. AES Memory Usage

Memory	Start Address	End Address	Used for
X	0x8000	0x83FF	AES S-Box, Inverted S-Box and AES lookup tables
X	0xA400	0xA5FF	AES S-Box, Inverted S-Box
P	0xDF80	0xDFD4	Full AES key expansion
P	Block start address	Block end address	Temporary space for decryption (blocks are decrypted in place)

The resources used by the AES routines have been selected to allow encrypted sections to be loaded at any point during the loading of an application without concern for previously loaded values.

AES routines are not designed to save and restore any registers that they use, as they generally are not intended to be used during normal program execution. A list of register modifications can be found in Table 119.

Table 119. AES Function Summary

Function Vector	Inputs	Outputs	Register Mods
ROMVECT_AESInitialize	-	Loaded AES lookup tables	x0, y0, a0, a1, xp0, xp1, xp2, xp3, yp0, yp1, yp2, yp3, xstep0, xstep1, xstep2, xstep3
ROMVECT_AESExpandKey	yp1 - Key pointer	Expanded key	x0, x1, y0, y1, a0, a1, b0, b1, xp0, xp1, yp0, yp2, xstep0, xstep1, xstep2, ystep1, xcirc0
ROMVECT_AESDecryrpt	yp0 - Pointer to encrypted data yp1 - Key pointer x0 - Number of encrypted blocks	Decrypted data	x1, y0, y1, a0, a1, b0, b1, xp0, xp1, xp2, yp2, yp3, xstep0, ystep1, ystep2, ystep3, xcirc0, ycirc1, ycirc2, ycirc3
ROMVECT_AESCleanUp	yp1 - Key pointer	-	b0, yp1, ystep1

All AES routines are blocking routines that execute on the CFX DSP core, but do not prevent interruption through the interrupt controller.

12.4.2 Encryption Blocks

AES encryption is a block-based encryption and decryption scheme that operates on 128-bit data blocks. The implementation used on the Ezairo 7100 chip can be used to decrypt data to be loaded to P memory in four-word blocks. P memory sections that are not a multiple of four words in length are padded out to 128 bits before they are encrypted.

CAUTION: P memory sections that have been encrypted are decrypted in place, so it is possible for the padding to the 128-bit block size to overlap with the segment that is loaded to the immediately following P memory address.

To ensure that padded P memory data does not result in code or data being lost in P memory due to the padding overlapping with a subsequent function, take one of the following approaches:

- Load P memory sections from the section with the lowest starting address to the section with the highest starting address.
- Space P memory sections that are padded so that the padding does not overlap with any other sections.

CHAPTER 13
Bootloader

13.1 Description

The bootloader is a small system program that is included with the firmware. The bootloader's purpose is to load full applications from EEPROM to RAM (providing decryption as required), and to verify the integrity of the file system. This corresponds to the second stage of the two-stage boot process. The bootloader is stored in EEPROM and is loaded to RAM by the Boot ROM portion of the Program ROM. The bootloader also:

- Loads the calibration values for the clock and ADCs from EEPROM to RAM
- Sets up the power supply to its default calibrated configuration
- Sets up the clock to its default calibrated operating frequency

13.2 Installation

The bootloader is distributed in both source and binary forms. The binary form is a standard *.cfe* file, while the source form is a standard IDE project. The binary form is generally used when creating an EEPROM image to boot an application.

To use the bootloader, it must be loaded into the EEPROM. The bootloader may be located at any EEPROM address after the Key Area of the file system. By default, the bootloader is stored in EEPROM at E:0x0180. The System Area at the base of the file system contains a vector indicating the location of the bootloader in EEPROM.

13.3 Bootloader Version

The bootloader contains a 16-bit version identifier. The version identifier is stored in the binary image and is loaded to RAM during the bootloader operation. For more information see Section 2.8, "Versions" on page 35.

13.4 Memory Usage

The bootloader executes from program memory once it has been loaded by the Program ROM. By default, the bootloader is loaded into reserved program memory and allocates a stack area at the top of X memory.

The bootloader also loads variables into the reserved areas in X memory and Y memory. For more information see Section 2.5, "Memory Map" on page 30.

13.5 Operation

The bootloader loads a full application from the EEPROM to RAM. Multiple applications can be stored on the EEPROM. The bootloader will select a single application as specified through a system variable. This system variable is loaded by the Program ROM from the EEPROM during boot. The applications must be stored in the file system format described in Chapter 10, "EEPROM File System" on page 117. The bootloader verifies the integrity of the selected application and all system areas.

The bootloader begins operation by performing initialization. After the bootloader initialization, the bootloader proceeds to:

- Check the boot information (32-bit word starting at address 0x0000) and set the SPI speed appropriately
- Verify the System area
- Verify the Manufacturing area
- Load the clock calibration table to memory and set the default operating frequency

- Load the VDDC, VDDM, VDDA, VDBL and VREG power supply calibration settings to their appropriate registers, if calibration information is available
- Load the ADC calibration table to memory
- Load and decrypt the user key, if required
- Load the specified application
- Verify the application header
- Decrypt the application sections, if required
- Verify the application start pointer has been set
- Execute the application

The verification steps are performed using CRCs and range checks. If a CRC or range check fails, the bootloader executes its error handler. In the case of an invalid CRC for a volatile section, the boot process may continue (See Section 10.11.4, “Volatile Sections” on page 129 for more information). For all other errors the boot process is terminated.

13.6 Bootloader Errors

When a bootloader error occurs, the boot process is terminated and an error handler is executed. The bootloader error handler sets a system variable indicating the error that occurred. The error handler implements a delay and then proceeds to reboot the system using a watchdog reset. The delay is nominally 3000 ms. The actual delay may vary depending on the default clock frequency and whether the default operating frequency was modified by the bootloader.

The bootloader error handler uses a logical OR to combine the error code corresponding to the boot failure with the bootloader error constant, 0x100. The resulting code is stored in a system variable. The bootloader error constant indicates that the error occurred in the bootloader. For more information on boot errors, including a list of error codes, see Section 2.10.2, “Boot Errors” on page 38.

CHAPTER 14
CFX System Macro Reference

This reference chapter presents a detailed description of all the macros defined in the CFX system include files. For each macro, it describes calling parameters, modified registers, returned values and execution time.

14.1 SYS_ACB_CLEAR_STATE

Clear the internal state of all of the audio codec blocks

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_acb.cfi
Template	`Sys_ACB_Clear_State`
Description	Clear the internal state of all of the audio codec blocks
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	The G.722 and CVSD components are idle
Clock Cycles	1
Example	`// Clear the current state for the audio codec blocks` `Sys_ACB_Clear_State`

14.2 SYS_ACB_DISABLEALL

Disable all of the audio codec blocks

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_acb.cfi
Template	`Sys_ACB_DisableAll`
Description	Disable all of the audio codec blocks
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Disable the audio codec blocks` `Sys_ACB_DisableAll`

14.3 SYS_ACB_INT_ACK

Acknowledge all audio codec block interrupt sources

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_acb.cfi
Template	`Sys_ACB_Int_Ack`
Description	Acknowledge all audio codec block interrupt sources
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Acknowledge the audio codec block interrupts` `Sys_ACB_Int_Ack`

14.4 SYS_ACB_INT_ENABLE

Enable/disable a set of audio codec block interrupts

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_acb.cfi
Template	`Sys_ACB_Int_Enable( src )`
Description	Enable/disable a set of audio codec block interrupts
Inputs	src = Audio codec block interrupt sources to enable/disable; use INT_EBL_* and INT_DIS_*
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if src can be sign-extended from 8 bits; 2 if src can be sign-extended from 24 bits; 3 otherwise

Example	`// Enable the CVSD audio codec block interrupts (implicitly`
	`// disabling all other interrupts)`
	`Sys_ACB_Int_Enable( INT_EBL_CVSD_ENC_IN_ERR |`
	`INT_EBL_CVSD_DEC_IN_ERR )`

14.5 SYS_AUDIOMUX_CONFIG

Configure the audio multiplexers and transfer formats

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ioc.cfi
Template	`Sys_AudioMux_Config( config )`
Description	Configure the audio multiplexers and transfer formats. This selects the audio source for each input channel, selecting between the ADC, bypass register, and DMIC inputs. This also selects the IOC input sources, selecting between the input source and FENG. Finally this selects the output driver sources, selecting between the interpolation filter and the FENG. This macro also configures audio data rounding and truncation.
Inputs	config = Audio mux configuration; use a combination of DMIC_CLK_SRC_[EXTCLK | SLOWCLK], ROUND_[NORMAL | AVOID_ALL_ZEROS], SDM*_[ROUND | TRUNCATE], INT_FILTER*_[ROUND | TRUNCATE], [ADC*_OUT | BYPASS_DATA_REG* | DMIC*_DATA_RE | DMIC*_DATA_FE], IOC_SRC*_[DEC_FILTER | FENG], INT_FILTER*_[IOC | FENG] and SDM*_[INT_FILTER | FENG]
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if config can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure audio filtering and multiplexing, supporting one input`
	`// and one output from the Filter Engine`
	`#define AUDIO_MUX_CFG_VAL ( ADC3_OUT | \`
	`ADC2_OUT | \`
	`ADC1_OUT | \`
	`ADC0_OUT | \`
	`IOC_SRC3_DEC_FILTER | \`
	`IOC_SRC2_DEC_FILTER | \`
	`IOC_SRC1_DEC_FILTER | \`
	`IOC_SRC0_FENG | \`
	`INT_FILTER1_FENG | \`
	`INT_FILTER0_IOC | \`
	`SDM1_INT_FILTER | \`
	`SDM0_INT_FILTER )`
	`Sys_AudioMux_Config( AUDIO_MUX_CFG_VAL )`

14.6 SYS_BOOTAPP

Boot the specified application

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_system.cfi
Template	`Sys_BootApp( reg )`
Description	Boot the specified application. The boot ROM is used to load the bootloader and application from EEPROM. The APPNUM argument overrides the value set in the file system. If APPNUM is invalid, the system will reboot and use the default application specified in the system data area of the EEPROM file system.
Inputs	reg = Register specifying application number to boot (APPNUM)
Outputs	None
Register Modifications	Not applicable because the application that calls Sys_BootApp is terminated
Assumptions	None
Clock Cycles	Not applicable

Example	`// Load x0 with the number of the application we'd like to run (2)`
	`// and reboot into that application using the boot ROM.`
	`load x0, 2`
	`Sys_BootApp( x0 )`

14.7 SYS_C_I2C_ACK

Manually ACK the latest ARM Cortex-M3 subsystem's I^{2}C interface byte

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_C_I2C_ACK`
Description	Manually ACK the latest ARM Cortex-M3 subsystem's I^{2}C interface byte
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Manually ACK the latest I2C byte` `Sys_C_I2C_ACK`

14.8 SYS_C_I2C_CONFIG

Configure the ARM Cortex-M3 subsystem's I²C interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_C_I2C_Config( cfg )`
Description	Configure the ARM Cortex-M3 subsystem's I²C interface
Inputs	cfg = Combination of C_I2C_SLAVE_[ENABLE | DISABLE], C_I2C_SAMPLE_CLK_[ENABLE | DISABLE], C_I2C_AUTO_ACK_[ENABLE | DISABLE], C_I2C_STOP_INT_[ENABLE | DISABLE], C_I2C_CONTROLLER_[CM3 | DMA], C_I2C_MASTER_SPEED_*, and an I2C_CTRL0_SLAVE_ADDRESS setting
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if cfg can be sign-extended from 8 bits; 2 otherwise

Example	`// Enable the ARM Cortex-M3 I2C slave and set the address to 0x10.`
	`// Also enable auto-acknowledge mode.`
	`Sys_C_I2C_Config( C_I2C_SLAVE_ENABLE | \`
	`C_I2C_AUTO_ACK_ENABLE | \`
	`(0x10 << I2C_CTRL0_SLAVE_ADDRESS_POS) )`

14.9 SYS_C_I2C_NACK

Manually NACK the latest ARM Cortex-M3 subsystem's I^2C interface byte

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_C_I2C_NACK`
Description	Manually NACK the latest ARM Cortex-M3 subsystem's I^2C interface byte
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Manually NACK the latest I2C byte` `Sys_C_I2C_NACK`

14.10 SYS_C_I2C_NACK_AND_STOP

Manually NACK the last ARM Cortex-M3 subsystem's I²C interface byte and issue a stop condition (applies to I²C master mode transfers only)

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_C_I2C_NACK_And_Stop`
Description	Manually NACK the last ARM Cortex-M3 subsystem's I²C interface byte and issue a stop condition (applies to I²C master mode transfers only)
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Manually NACK the latest I2C byte and issue a stop condition` `Sys_C_I2C_NACK_And_Stop`

14.11 SYS_C_I2C_RESET

Reset the ARM Cortex-M3 subsystem's I^{2}C interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_C_I2C_Reset`
Description	Reset the ARM Cortex-M3 subsystem's I^{2}C interface.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Reset the ARM Cortex-M3 subsystem's I2C interface.` `Sys_C_I2C_Reset`

14.12 SYS_C_I2C_SLAVE_DISABLE

Disable the ARM Cortex-M3 subsystem's I²C interface by clearing the C_I²C_SLAVE_ENABLE bit

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_C_I2C_Slave_Disable`
Description	Disable the ARM Cortex-M3 subsystem's I²C interface by clearing the C_I²C_SLAVE_ENABLE bit. The I²C interface configuration is unchanged.
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	3

Example	`// Disable the ARM Cortex-M3 I2C interface slave mode, preventing`
	`// slave mode transfers`
	`Sys_C_I2C_Slave_Disable`

14.13 SYS_C_I2C_SLAVE_ENABLE

Enable the ARM Cortex-M3 subsystem's I²C interface by setting the C_I²C_SLAVE_ENABLE bit

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_C_I2C_Slave_Enable`
Description	Enable the ARM Cortex-M3 subsystem's I²C interface by setting the C_I²C_SLAVE_ENABLE bit. The I²C interface configuration is unchanged.
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	3
Example	`// Enable the ARM Cortex-M3 I2C interface for slave mode transfers` `Sys_C_I2C_Slave_Enable`

14.14 SYS_C_I2C_START_READ

Start an I²C read transfer (applies to I²C master mode transfers only) using the ARM Cortex-M3 subsystem's I²C interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_C_I2C_Start_Read( addr )`
Description	Start an I²C read transfer (applies to I²C master mode transfers only) using the ARM Cortex-M3 subsystem's I²C interface
Inputs	addr = Address of remote I²C slave device
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Start an I2C master mode transfer to read from a device connected`
	`// to the ARM Cortex-M3 I2C bus at I2C_ADDRESS.`
	`Sys_C_I2C_Start_Read( I2C_ADDRESS )`

14.15 SYS_C_I2C_START_WRITE

Start an I²C write transfer (applies to I²C master mode transfers only) using the ARM Cortex-M3 subsystem's I²C interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_C_I2C_Start_Write( addr )`
Description	Start an I²C write transfer (applies to I²C master mode transfers only) using the ARM Cortex-M3 subsystem's I²C interface
Inputs	addr = Address of remote I²C slave device
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Start an I2C master mode transfer to write to any devices`
	`// connected to the ARM Cortex-M3 I2C bus at I2C_ADDRESS.`
	`Sys_C_I2C_Start_Write( I2C_ADDRESS )`

14.16 SYS_C_PCM_CONFIG

Configure the ARM Cortex-M3 subsystem's PCM interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_pcm.cfi
Template	`Sys_C_PCM_Config( pcm_cfg )`
Description	Configure the ARM Cortex-M3 subsystem's PCM interface
Inputs	pcm_cfg = Configuration consisting of: C_PCM_SELECT_[MASTER | SLAVE], C_PCM_[ENABLE | DISABLE], C_PCM_CONTROLLER_[DMA | CM3], C_PCM_SUBFRAME_[ENABLE | DISABLE], C_PCM_MULTIWORD_x, C_PCM_FRAME_WIDTH_[SHORT | LONG], C_PCM_FRAME_ALIGN_[LAST | FIRST], C_PCM_WORD_SIZE_x, C_PCM_TX_ALIGN_[MSB | LSB], C_PCM_BIT_ORDER_[LSB | MSB]_FIRST
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if pcm_cfg can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure the ARM Cortex-M3 PCM interface as a slave receiver.`
	`// The transfer is configured for:`
	`// - 16 bits per word (sent MSB first)`
	`// - Pulse frame signal aligned with the first bit every 2 words`
	`#define cfg ( C_PCM_SELECT_SLAVE | C_PCM_ENABLE | \`
	`C_PCM_CONTROLLER_CM3 | C_PCM_SUBFRAME_DISABLE | \`
	`C_PCM_MULTIWORD_2 | C_PCM_FRAME_WIDTH_SHORT | \`
	`C_PCM_FRAME_ALIGN_FIRST | C_PCM_WORD_SIZE_16 | \`
	`C_PCM_TX_ALIGN_LSB | C_PCM_BIT_ORDER_MSB_FIRST )`
	`Sys_C_PCM_Config( cfg )`

14.17 SYS_C_PCM_DISABLE

Disable the ARM Cortex-M3 subsystem's PCM interface by clearing the PCM_CTRL_ENABLE bit

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_pcm.cfi
Template	`Sys_C_PCM_Disable`
Description	Disable the ARM Cortex-M3 subsystem's PCM interface by clearing the PCM_CTRL_ENABLE bit. The PCM interface configuration is unchanged.
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	3
Example	`// Disable the ARM Cortex-M3 PCM interface` `Sys_C_PCM_Disable`

14.18 SYS_C_PCM_ENABLE

Enable the ARM Cortex-M3 subsystem's PCM interface by setting the PCM_CTRL_ENABLE bit

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_pcm.cfi
Template	`Sys_C_PCM_Enable`
Description	Enable the ARM Cortex-M3 subsystem's PCM interface by setting the PCM_CTRL_ENABLE bit. The PCM interface configuration is unchanged.
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	3
Example	`// Enable the ARM Cortex-M3 PCM interface` `Sys_C_PCM_Enable`

14.19 SYS_C_SPI_CS_HIGH

Set the ARM Cortex-M3 subsystem's SPI_CS line high

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_C_SPI_CS_High`
Description	Set the ARM Cortex-M3 subsystem's SPI_CS line high
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	ARM Cortex-M3 subsystem's SPI interface is enabled ARM Cortex-M3 subsystem's SPI interface is configured as an SPI master
Clock Cycles	1
Example	`// Set the ARM Cortex-M3 SPI_CS line high` `Sys_C_SPI_CS_High`

14.20 Sys_C_SPI_CS_Low

Set the ARM Cortex-M3 subsystem's SPI_CS line low

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_C_SPI_CS_Low`
Description	Set the ARM Cortex-M3 subsystem's SPI_CS line low
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	ARM Cortex-M3 subsystem's SPI interface enabled ARM Cortex-M3 subsystem's SPI interface configured as an SPI master
Clock Cycles	1
Example	`// Set the ARM Cortex-M3 SPI_CS line low` `Sys_C_SPI_CS_Low`

14.21 SYS_C_SPI_DISABLE

Disable the ARM Cortex-M3 subsystem's SPI interface (clears the SPI configuration)

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_C_SPI_Disable`
Description	Disable the ARM Cortex-M3 subsystem's SPI interface (clears the SPI configuration)
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Disable the ARM Cortex-M3 SPI interface` `Sys_C_SPI_Disable`

14.22 SYS_C_SPI_ENABLE

Configure the ARM Cortex-M3 subsystem's SPI interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_C_SPI_Enable( spi_cfg )`
Description	Configure the ARM Cortex-M3 subsystem's SPI interface
Inputs	spi_cfg = SPI configuration; use C_SPI_PRESCALE_*, C_SPI_MODE_SELECT_[MANUAL | AUTO], C_SPI_CLK_POLARITY_[NORMAL | INVERSE], C_SPI_SELECT_[SLAVE | MASTER], C_SPI_CONTROLLER_[CM3 | DMA], C_SPI_UNDERRUN_INT_[DISABLE | ENABLE], and C_SPI_OVERRUN_INT_[DISABLE | ENABLE]
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if spi_cfg can be sign-extended from 8 bits; 2 otherwise

Example	`// Enable the ARM Cortex-M3 SPI interface in master mode in auto`
	`// mode, with a normal polarity clock prescaled by 4`
	`Sys_C_SPI_Enable( C_SPI_PRESCALE_4 | C_SPI_MODE_SELECT_AUTO | \`
	`C_SPI_CLK_POLARITY_NORMAL | C_SPI_CONTROLLER_CM3 )`

14.23 SYS_C_SPI_READ

Read the specified number of bits over the ARM Cortex-M3 subsystem's SPI interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_C_SPI_Read( bits )`
Description	Read the specified number of bits over the ARM Cortex-M3 subsystem's SPI interface. NOTE: The caller must wait until the current SPI operation is complete before starting another SPI operation; use Sys_C_SPI_WaitDone.
Inputs	bits = Number of bits to receive (between 1-32) Optional: OR in C_SPI_CS_1 to force CS = 1
Outputs	None
Register Modifications	None
Assumptions	SPI interface enabled
Clock Cycles	1

Example	`// Initiate a read of 32 bits from the ARM Cortex-M3 SPI interface.`
	`// Another SPI operation using this interface cannot be initiated`
	`// until this operation has completed.`
	`Sys_C_SPI_Read( 32 )`

14.24 SYS_C_SPI_READWRITE

Read and write the specified number of bits over the ARM Cortex-M3 subsystem's SPI interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_C_SPI_ReadWrite( bits )`
Description	Read and write the specified number of bits over the ARM Cortex-M3 subsystem's SPI interface. NOTE: The caller must wait until the current SPI operation is complete before starting another SPI operation; use Sys_C_SPI_WaitDone.
Inputs	bits = Number of bits to send (between 1-32) Optional: OR in C_SPI_CS_1 to force CS = 1
Outputs	None
Register Modifications	None
Assumptions	SPI interface enabled
Clock Cycles	1

Example	`// Initiate a bi-directional read and write of 32 bits using the ARM`
	`// Cortex-M3 SPI interface. Another SPI operation using this interface`
	`// cannot be initiated until this operation has completed.`
	`Sys_C_SPI_ReadWrite( 32 )`

14.25 SYS_C_SPI_WAITDONE

Wait for the ARM Cortex-M3 subsystem's SPI operation to complete; exit when the SPI interface is no longer busy

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_C_SPI_WaitDone`
Description	Wait for the ARM Cortex-M3 subsystem's SPI operation to complete; exit when the SPI interface is no longer busy
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	ARM Cortex-M3 subsystem's SPI interface is enabled Valid stack pointer
Clock Cycles	Depends on SYSCLK, ARM Cortex-M3 subsystem's SPI prescale and number of bits remaining in the current SPI transaction

Example	`// Block and wait for the read to complete on the ARM Cortex-M3`
	`// SPI interface`
	`Sys_C_SPI_WaitDone`

14.26 SYS_C_SPI_WRITE

Write the specified number of bits over the ARM Cortex-M3 subsystem's SPI interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_C_SPI_Write( bits )`
Description	Write the specified number of bits over the ARM Cortex-M3 subsystem's SPI interface. NOTE: The caller must wait until the current SPI operation is complete before starting another SPI operation; use Sys_C_SPI_WaitDone.
Inputs	bits = Number of bits to send (between 1-32) Optional: OR in C_SPI_CS_1 to force CS = 1
Outputs	None
Register Modifications	y1
Assumptions	SPI interface enabled
Clock Cycles	2

Example	`// Write 24 bits to the ARM Cortex-M3 SPI interface. Another SPI`
	`// operation using this interface cannot be initiated until this`
	`// operation has completed.`
	`Sys_C_SPI_Write( 24 )`

14.27 SYS_CALC_GAIN_FACTOR_VAL

Calculate the decimation gain factor for a desired gain given an adc_cfg_sample_freq field

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_Calc_Gain_Factor_Val( adc_cfg_sample_freq, gain )`
Description	Calculate the decimation gain factor for a desired gain given an adc_cfg_sample_freq field. The developer must pre-multiply the gain by 10^7 before passing it in. The resulting gain factor produces the closest gain possible without going over the specified parameter.
Inputs	adc_cfg_sample_freq = Sampling frequency field in A_ADC_SAMPLE_FREQ_CFG gain = Desired gain multiplied by 10^7. Can be positive or negative. Range is dependent on adc_cfg_sample_freq
Outputs	SYS_CALC_GF = Define containing the rounded value for the desired gain factor; use for the ADC_GF field of A_ADC_DEC_CTRL_*
Register Modifications	None
Assumptions	adc_cfg_sample_freq, gain are within valid ranges
Clock Cycles	0 (evaluated during compile)

Example	`// Calculate the gain factor value that will produce a unity gain`
	`// in channel 1 given a ADC_CFG_SAMPLE_FREQ setting of 0x2`
	`// NOTE: Gain factor value is #set into SYS_CALC_GF`
	`#define ADC_CFG_SAMPLE_FREQ 0x2`
	`#define UNITY_GAIN 10000000`
	
	`Sys_Calc_Gain_Factor_Val( ADC_CFG_SAMPLE_FREQ, UNITY_GAIN )`
	
	`// Configure the decimation filter for input channel 1`
	`Sys_Set_ADC_DEC_CTRL( 1, BAND_SELECT_ADC_0K_8K |`
	`ADC_INTEGER_DELAY_0 | ADC_UNMUTE |`
	`ADC_DEC_ENABLE |`
	`ADC_DC_REMOVE_CUTOFF_10HZ |`
	`SYS_CALC_GF )`

14.28 SYS_CALC_SAMPLE_FREQ_VAL

Calculate the sampling frequency field for A_ADC_SAMPLE_FREQ_CFG

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_Calc_Sample_Freq_Val( slowclk, sampling_freq )`
Description	Calculate the sampling frequency field for A_ADC_SAMPLE_FREQ_CFG
Inputs	slowclk = Slow clock frequency in Hz sampling_freq = Desired sampling frequency in Hz
Outputs	SYS_CALC_SF = Define containing the rounded value for the sampling frequency field in A_ADC_SAMPLE_FREQ_CFG
Register Modifications	None
Assumptions	slowclk, sampling_freq are within valid ranges
Clock Cycles	0 (evaluated during compile)

Example	`// Set the sampling frequency to 16 kHz with a 1.28 MHz slow clock`
	`// NOTE: Value for A_ADC_SAMPLE_FREQ_CFG is #set into SYS_CALC_SF`
	`Sys_Calc_Sample_Freq_Val( 1280000, 16000 )`
	`Sys_Set_ADC_SAMPLE_FREQ_CFG( SYS_CALC_SF | ADC_HIGH_FREQ_DISABLE )`
	
	`// Set the sampling frequency to 24 kHz with a 1.92 MHz slow clock;`
	`// ADC_HIGH_FREQ_ENABLE should be set as SLOWCLK is set to 1.92 MHz`
	`Sys_Calc_Sample_Freq_Val( 1920000, 24000 )`
	`Sys_Set_ADC_SAMPLE_FREQ_CFG( SYS_CALC_SF | ADC_HIGH_FREQ_ENABLE )`

14.29 SYS_CFX_BIASDISABLE

Set the SR_BIAS bit to disable bias

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_BiasDisable`
Description	Set the SR_BIAS bit to disable bias
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Set the SR_BIAS bit to disable bias during data conversions` `Sys_CFX_BiasDisable`

14.30 SYS_CFX_BIASENABLE

Clear the SR_BIAS bit to enable bias

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_BiasEnable`
Description	Clear the SR_BIAS bit to enable bias
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Clear the SR_BIAS bit to enable bias during data conversions` `Sys_CFX_BiasEnable`

14.31 SYS_CFX_CIRCADDRCONFIG

Configure one of the CFX's circular registers

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_CircAddrConfig( reg, config )`
Description	Configure one of the CFX's circular registers
Inputs	reg = Circular register to configure config = Two-part parameter consisting of CIRC_[CIRCULAR | BIT_REVERSE] and 14-bit size of buffer (value between 0 and 16383)
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure xcirc0 to support a circular buffer of 8 words.`
	`Sys_CFX_CircAddrConfig( xcirc0, CIRC_CIRCULAR | 8 )`
	
	`// Configure ycirc0 to support bit-reversed addressing on a`
	`// 128-word buffer`
	`Sys_CFX_CircAddrConfig( ycirc0, CIRC_BIT_REVERSE | 128 )`

14.32 SYS_CFX_FLAGUPDATEDISABLE

Clear the SR_FLAG_UPD bit to disable updates to SR flags

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_FlagUpdateDisable`
Description	Clear the SR_FLAG_UPD bit to disable updates to SR flags
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Clear the SR_FLAG_UPD bit to disable flag updates on arithmetic`
	`// operations`
	`Sys_CFX_FlagUpdateDisable`

14.33 SYS_CFX_FLAGUPDATEENABLE

Set the SR_FLAG_UPD bit to enable updates to SR flags

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_FlagUpdateEnable`
Description	Set the SR_FLAG_UPD bit to enable updates to SR flags
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Set the SR_FLAG_UPD bit to enable flag updates on arithmetic`
	`// operations`
	`Sys_CFX_FlagUpdateEnable`

14.34 SYS_CFX_INITLOOPSTACK

Initialize the loop stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_InitLoopStack`
Description	Initialize the loop stack
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	4
Example	`// Initialize the loop stack, clearing any existing hardware loops` `Sys_CFX_InitLoopStack`

14.35 SYS_CFX_INITSTACK

Initialize the stack pointer

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_InitStack( stack_base_addr )`
Description	Initialize the stack pointer
Inputs	stack_base_addr = Base address of stack
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Initialize stack pointer to point to the base of the stack in X memory` `Sys_CFX_InitStack(X_STACK_BOTTOM)`

14.36 SYS_CFX_INTDISABLE

Clear the SR_IE bit to disable interrupts

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_IntDisable`
Description	Clear the SR_IE bit to disable interrupts
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Clear the SR_IE bit to disable interrupts` `Sys_CFX_IntDisable`

14.37 SYS_CFX_INTENABLE

Set the SR_IE bit to enable interrupts

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_IntEnable`
Description	Set the SR_IE bit to enable interrupts
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	3
Example	`// Set the SR_IE bit to enable interrupts` `Sys_CFX_IntEnable`

14.38 SYS_CFX_ROUNDDISABLE

Clear the SR_ROUND bit to select truncation

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_RoundDisable`
Description	Clear the SR_ROUND bit to select truncation
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Clear the SR_ROUND bit to enable truncation of data during data`
	`// conversions`
	`Sys_CFX_RoundDisable`

14.39 SYS_CFX_ROUNDENABLE

Set the SR_ROUND bit to select rounding

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_RoundEnable`
Description	Set the SR_ROUND bit to select rounding
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Set the SR_ROUND bit to enable rounding of data during data`
	`// conversions`
	`Sys_CFX_RoundEnable`

14.40 SYS_CFX_SATURATEDISABLE

Clear the SR_SATURATE bit to disable saturation

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_SaturateDisable`
Description	Clear the SR_SATURATE bit to disable saturation
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Disable saturation` `Sys_CFX_SaturateDisable`

14.41 SYS_CFX_SATURATEENABLE

Set the SR_SATURATE bit to enable saturation

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_SaturateEnable`
Description	Set the SR_SATURATE bit to enable saturation
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Enable saturation` `Sys_CFX_SaturateEnable`

14.42 SYS_CFX_SRCONFIG

Configure the CFX status register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_SRConfig( arg )`
Description	Configure the CFX status register
Inputs	arg = Five-part parameter consisting of SR_ROUND_[ENABLE | DISABLE], SR_BIAS_[ENABLE | DISABLE], SR_INT_[ENABLE | DISABLE], SR_SATURATE_[ENABLE | DISABLE], SR_FLAG_UPD_[ENABLE | DISABLE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the CFX DSP core (through the status register) for:`
	`// - Rounding enabled`
	`// - Bias disabled`
	`// - Interrupts enabled`
	`// - Saturation disabled`
	`// - Flag updates enabled`
	`Sys_CFX_SRConfig( SR_ROUND_ENABLE | \`
	`SR_BIAS_DISABLE | \`
	`SR_INT_ENABLE | \`
	`SR_SATURATE_DISABLE | \`
	`SR_FLAG_UPD_ENABLE )`

14.43 SYS_CFX_STEPCONFIG

Configure one of the CFX's step registers

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cfx.cfi
Template	`Sys_CFX_StepConfig( reg, config )`
Description	Configure one of the CFX's step registers
Inputs	reg = Step register to configure config = 16-bit signed two's complement value to load into reg
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Configure xstep0 to provide a step size of 2` `Sys_CFX_StepConfig( xstep0, 2 )`

14.44 SYS_CLK_CONFIG

Configure the main clocks used by the system

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_clk.cfi
Template	`Sys_Clk_Config( arg0, arg1 )`
Description	Configure the main clocks used by the system
Inputs	arg0 = Data to be used by clock configuration 0; use HEARCLK_PRESCALE_*_* or a constant, EXTCLK_[LOW | HIGH]_DRIVE, EXTCLK_PRESCALE_*, EXTCLK_[DISABLED | OUT_SYSCLK | OUT_SLOWCLK_OR_SDMCLK | IN_ENABLE], SYSCLK_SEL_[DEFAULT | STANDBY], EXTCLK_PU_[DISABLED | ENABLED], CM3CLK_PRESCALE_*, and EXTCLK_OUT_SRC_[SLOWCLK | SDMCLK] arg1 = Data to be used by clock configuration 1; use SLOWCLK_SRC_[SYSCLK | SDMCLK], UCLK_SRC_[SYSCLK | SDMCLK], DBL_ACCESS_MODE_[DISABLE | ENABLE], UCLK_PRESCALE_*, and SLOWCLK_PRESCALE_*
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	4

Example	`// Configure the clock distribution:`
	`// - Select the default configuration for SYSCLK with no external`
	`//  clock supplied`
	`// - Do not share cycles between the cores but configure slow clock`
	`//  as 1/8 of the system clock (640kHz for a 5.12MHz system clock)`
	`Sys_Clk_Config( SYSCLK_SEL_DEFAULT | EXTCLK_DISABLED, \`
	`DBL_ACCESS_MODE_DISABLE | SLOWCLK_PRESCALE_8 )`

14.45 SYS_CLK_CONFIG0

Configure the HEARCLK, EXTCLK, SYSCLK and CM3CLK clocks

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_clk.cfi
Template	`Sys_Clk_Config0( arg )`
Description	Configure the HEARCLK, EXTCLK, SYSCLK and CM3CLK clocks
Inputs	arg = Data to be used in the clock configuration; use HEARCLK_PRESCALE_*_* or a constant, EXTCLK_[LOW | HIGH]_DRIVE, EXTCLK_PRESCALE_*, EXTCLK_[DISABLED | OUT_SYSCLK | OUT_SLOWCLK_OR_SDMCLK | IN_ENABLE], SYSCLK_SEL_[DEFAULT | STANDBY], EXTCLK_PU_[DISABLED | ENABLED], CM3CLK_PRESCALE_*, and EXTCLK_OUT_SRC_[SLOWCLK | SDMCLK]
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	2 if arg can be sign extended from 24 bits, 3 otherwise

Example	`// Enable an external clock input (scaled by 4). Restore the`
	`// default HEARCLK and CM3CLK configurations.`
	`Sys_Clk_Config0( SYSCLK_SEL_DEFAULT | HEARCLK_PRESCALE_1_000 | \`
	`CM3CLK_PRESCALE_1 | EXTCLK_PU_DISABLED | \`
	`EXTCLK_IN_ENABLE | EXTCLK_PRESCALE_4 )`

14.46 SYS_CLK_CONFIG1

Configure the SLOWCLK and UCLK clocks, and double access mode

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_clk.cfi
Template	`Sys_Clk_Config1( arg )`
Description	Configure the SLOWCLK and UCLK clocks, and double access mode
Inputs	arg = Data to be used in the clock configuration; use SLOWCLK_SRC_[SYSCLK | SDMCLK], UCLK_SRC_[SYSCLK | SDMCLK], DBL_ACCESS_MODE_[DISABLE | ENABLE], UCLK_PRESCALE_*, and SLOWCLK_PRESCALE_*
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2

Example	`// Given a system clock of 5.12 MHz:`
	`// - Select a slow clock of 1.28 MHz`
	`// - Select a user clock of 10.0 kHz`
	`// - No sharing of cycles between the cores`
	`Sys_Clk_Config1( UCLK_SRC_SYSCLK | DBL_ACCESS_MODE_DISABLE | \`
	`SLOWCLK_PRESCALE_4 | UCLK_PRESCALE_512 )`

14.47 SYS_CLK_MULTTHROTTLECONFIG

Configure whether UCLK, SLOWCLK and SDMCLK are prescaled by the VCO multiplier value

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_clk.cfi
Template	`Sys_Clk_MultThrottleConfig( config )`
Description	Configure whether UCLK, SLOWCLK and SDMCLK are prescaled by the VCO multiplier value. If they are prescaled, a user application can throttle the clock using the VCO multiplier setting without affecting the UCLK, SLOWCLK and SDMCLK signals.
Inputs	config = Clock divisor multiplier support; use VCO_MULT_THROTTLE_UCLK_[DISABLE | ENABLE], VCO_MULT_THROTTLE_SDMCLK_[DISABLE | ENABLE] and VCO_MULT_THROTTLE_SLOWCLK_[DISABLE | ENABLE]
Outputs	None
Register Modifications	None
Assumptions	UCLK_PRESCALE, SLOWCLK_PRESCALE and SDMCLK_PRESCALE settings are non-zero (i.e., UCLK, SLOWCLK and SDMCLK are divided from SYSCLK) if you are prescaling them by the VCO multiplier value
Clock Cycles	1

Example	`// Divide both SLOWCLK and the SDMCLK by the multiplier value`
	`// so that the VCO multiplier can be used for clock throttling`
	`Sys_Clk_MultThrottleConfig( VCO_MULT_THROTTLE_SDMCLK_ENABLE | \`
	`VCO_MULT_THROTTLE_SLOWCLK_ENABLE )`

14.48 SYS_CLK_THROTTLE_CONFIG

Configure SYSCLK, SLOWCLK, and the SDMCLK using the clock throttling mechanism

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_clk.cfi
Template	`Sys_Clk_Throttle_Config( sysclk, slowclk, sdmclk )`
Description	Configure SYSCLK, SLOWCLK, and the SDMCLK using the clock throttling mechanism. Settings will be applied the next time that the SLOWCLK divisor resets.
Inputs	sysclk = Value to apply to the A_OSC_CTRL_0 register slowclk = Value to apply to the SLOWCLK_PRESCALE bit field of the D_CLK_CFG_1 register sdmclk = Value to apply to the SDMCLK_PRESSCALE bit field of the A_O_SDMCLK register
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	Not relevant (clock frequency is being adjusted)

Example	`// Reconfigure SYSCLK, SLOWCLK, and SDMCLK using the clock`
	`// throttling mechanism.`
	`//`
	`// Given a current system clock of 5.12 MHz, and a new system clock`
	`// frequency of 2.56 MHz, we need to maintain a slow clock of 1.28 MHz,`
	`// and maintain a SDMCLK at 2.56 MHz`
	`Sys_Clk_Throttle_Config( SYSCLK_2M56,`
	`SLOWCLK_PRESCALE_2,`
	`SDMCLK_PRESCALE_1 )`

14.49 SYS_CLK_THROTTLE_MULTCONFIG

Configure SYSCLK using the VCO multiplier clock throttling mechanism

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_clk.cfi
Template	`Sys_Clk_Throttle_MultConfig( multiplier )`
Description	Configure SYSCLK using the VCO multiplier clock throttling mechanism
Inputs	multiplier = Multiplier value to use for the VCO multiplier; use OSC_MULTIPLY_BY_*
Outputs	None
Register Modifications	None
Assumptions	The VCO multiplier does not apply to one or more of UCLK, SLOWCLK or SDMCLK The internal oscillator should be enabled
Clock Cycles	Not relevant (clock frequency is being adjusted)

Example	`// Change the internal oscillator multiplier to 3, to boost the`
	`// number of clock cycles available for the user application`
	`Sys_Clk_Throttle_MultConfig( OSC_MULTIPLY_BY_3 )`

14.50 SYS_CM3_COMMAND

Issue a command to trigger an ARM Cortex-M3 core interrupt

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cm3.cfi
Template	`Sys_CM3_Command( cmd )`
Description	Issue a command to trigger an ARM Cortex-M3 core interrupt
Inputs	cmd = Command to issue to the ARM Cortex-M3; use CM3_CMDS_CM3_CMD_*_POS or a constant
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Issue a command to the ARM Cortex-M3 processor to trigger handling`
	`// of CFX command 1`
	`Sys_CM3_Command( 1 )`

14.51 SYS_CM3_PAUSE

Pause the ARM Cortex-M3 processor

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cm3.cfi
Template	`Sys_CM3_Pause`
Description	Pause the ARM Cortex-M3 processor
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Pause the ARM Cortex-M3 processor` `Sys_CM3_Pause`

14.52 SYS_CM3_RESET

Reset the ARM Cortex-M3 processor

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cm3.cfi
Template	`Sys_CM3_Reset`
Description	Reset the ARM Cortex-M3 processor
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Reset the ARM Cortex-M3 processor` `Sys_CM3_Reset`

14.53 SYS_CM3_START

Start (or resume) the ARM Cortex-M3 processor

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cm3.cfi
Template	`Sys_CM3_Start`
Description	Start (or resume) the ARM Cortex-M3 processor
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Start the ARM Cortex-M3 processor` `Sys_CM3_Start`

14.54 SYS_CM3ANDSUBSYSTEM_RESET

Reset the ARM Cortex-M3 processor along with its interfaces and peripherals

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_cm3.cfi
Template	`Sys_CM3AndSubsystem_Reset`
Description	Reset the ARM Cortex-M3 processor along with its interfaces and peripherals
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Reset the ARM Cortex-M3 processor and its interfaces and`
	`// peripherals`
	`Sys_CM3AndSubsystem_Reset`

14.55 SYS_CVSD_DECODER_CLEAR_STATE

Clear the internal state of the CVSD decoder

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_acb.cfi
Template	`Sys_CVSD_Decoder_Clear_State`
Description	Clear the internal state of the CVSD decoder
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	The CVSD decoder is idle
Clock Cycles	3
Example	`// Clear the current state for the CVSD decoder block` `Sys_CVSD_Decoder_Clear_State`

14.56 SYS_CVSD_DECODER_CONFIG

Configure the CVSD decoder

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_acb.cfi
Template	`Sys_CVSD_Decoder_Config( config )`
Description	Configure the CVSD decoder
Inputs	config = The CVSD decoder configuration; use CVSD_DEC_[DISABLE | ENABLE], CVSD_DEC_J_*, CVSD_DEC_BETA_*, and CVSD_DEC_H_*
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	3

Example	`// Enable and configure the CVSD decoder`
	`Sys_CVSD_Decoder_Config( CVSD_DEC_ENABLE | \`
	`CVSD_DEC_J_4 | \`
	`CVSD_DEC_BETA_8 | \`
	`CVSD_DEC_H_6 )`

14.57 SYS_CVSD_ENCODER_CLEAR_STATE

Clear the internal state of the CVSD encoder

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_acb.cfi
Template	`Sys_CVSD_Encoder_Clear_State`
Description	Clear the internal state of the CVSD encoder
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	The CVSD encoder is idle
Clock Cycles	2
Example	`// Clear the current state for the CVSD encoder block` `Sys_CVSD_Encoder_Clear_State`

14.58 SYS_CVSD_ENCODER_CONFIG

Configure the CVSD encoder

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_acb.cfi
Template	`Sys_CVSD_Encoder_Config( config )`
Description	Configure the CVSD encoder
Inputs	config = The CVSD encoder configuration; use CVSD_ENC_[DISABLE | ENABLE], CVSD_ENC_J_*, CVSD_ENC_BETA_*, and CVSD_ENC_H_*
Outputs	None
Register Modifications	y1, b1
Assumptions	None
Clock Cycles	6

Example	`// Enable and configure the CVSD encoder`
	`Sys_CVSD_Encoder_Config( CVSD_ENC_ENABLE | \`
	`CVSD_ENC_J_4 | \`
	`CVSD_ENC_BETA_8 | \`
	`CVSD_ENC_H_6 )`

14.59 SYS_DATA_FENG_COEFFICIENTS

Create a data record for one set of Filter Engine coefficients for use with the Filter Engine BIQ and FIR instructions

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Data_FENG_Coefficients( cset, c0, c1, c2, c3, c4 )`
Description	Create a data record for one set of Filter Engine coefficients for use with the Filter Engine BIQ and FIR instructions.
Inputs	cset = Coefficient set number; use a constant between 0 and 31 c* = Coefficient * from the given coefficient set; use constants defined using Sys_m2p22 for BIQ and Sys_frac24 for FIR
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	N/A

Example	`// Statically configure the set of coefficients for a Biquad-IIR`
	`// filter`
	`#set c3_0  Sys_m2p22( -1000000 )`
	`#set c3_1  Sys_m2p22( -500000 )`
	`#set c3_2  Sys_m2p22( 0 )`
	`#set c3_3  Sys_m2p22( 500000 )`
	`#set c3_4  Sys_m2p22( 1000000 )`
	
	`Sys_Data_FENG_Coefficients( 3, c3_0, c3_1, c3_2, c3_3, c3_4 )`

14.60 SYS_DATA_FL8N16

Create a .data directive using the Sys_fl8n16 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Data_fl8n16( const )`
Description	Create a .data directive using the Sys_fl8n16 macro.
Inputs	const = Constant to be passed to Sys_fl8n16
Outputs	Sys_Data_fl8n16 = .data directive with floating-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^63
Clock Cycles	0 (evaluated during compile)

Example	`// Load 7.125 to memory as a <8,16> floating point`
	`Data_Value_fl8n16:`
	`Sys_Data_fl8n16( 71250000 )`

14.61 SYS_DATA_FRAC24

Create a .data directive using the Sys_frac24 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Data_frac24( const )`
Description	Create a .data directive using the Sys_frac24 macro.
Inputs	const = Constant to be passed to Sys_frac24
Outputs	Sys_Data_frac24 = .data directive with fractional approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^46
Clock Cycles	0 (evaluated during compile)

Example	`// Load 0.125 to memory as a <1,23> fractional value`
	`Data_Value_frac24:`
	`Sys_Data_frac24( 1250000 )`

14.62 SYS_DATA_FRAC32

Create a .data directive using the Sys_frac32 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Data_frac32( const )`
Description	Create a .data directive using the Sys_frac32 macro. NOTE: The frac32 data format is layout-compatible with the frac24 format.
Inputs	const = Constant to be passed to Sys_frac32
Outputs	Sys_Data_frac32 = .data directive with <9,23> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^46
Clock Cycles	0 (evaluated during compile)

Example	`// Load 64.0 to memory as a <9,23> fixed-point value`
	`Data_Value_frac32:`
	`Sys_Data_frac32( 640000000 )`

14.63 SYS_DATA_FRAC48

Create a .data directive using the Sys_frac48 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Data_frac48( const )`
Description	Create a .data directive using the Sys_frac48 macro.
Inputs	const = Constant to be passed to Sys_frac48
Outputs	Sys_Data_frac48 = .data directive with <1,47> fractional approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^24
Clock Cycles	0 (evaluated during compile)

Example	`// Load 0.125 to memory as a <1,47> fractional value`
	`Data_Value_frac48:`
	`Sys_Data_frac48( 1250000 )`

14.64 SYS_DATA_M24P24

Create a .data directive using the Sys_m24p24 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Data_m24p24( const )`
Description	Create a .data directive using the Sys_m24p24 macro.
Inputs	const = Constant to be passed to Sys_m24p24
Outputs	Sys_Data_m24p24 = .data directive with <24,24> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^62
Clock Cycles	0 (evaluated during compile)

Example	`// Load 7.125 to memory as a <24,24> fixed-point value`
	`Data_Value_m24p24:`
	`Sys_Data_m24p24( 71250000 )`

14.65 SYS_DATA_M2P22

Create a .data directive using the Sys_m2p22 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Data_m2p22( const )`
Description	Create a .data directive using the Sys_m2p22 macro.
Inputs	const = Constant to be passed to Sys_m2p22
Outputs	Sys_Data_m2p22 = .data directive with <2,22> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^46
Clock Cycles	0 (evaluated during compile)

Example	`// Load 1.125 to memory as a <2,22> fixed-point value`
	`Data_Value_m2p22:`
	`Sys_Data_m2p22( 11250000 )`

14.66 SYS_DATA_M2P46

Create a .data directive using the Sys_m2p46 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Data_m2p46( const )`
Description	Create a .data directive using the Sys_m2p46 macro.
Inputs	const = Constant to be passed to Sys_m2p46
Outputs	Sys_Data_m2p46 = .data directive with <2,46> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^40
Clock Cycles	0 (evaluated during compile)

Example	`// Load 1.125 to memory as a <2,46> fixed-point value`
	`Data_Value_m2p46:`
	`Sys_Data_m2p46( 11250000 )`

14.67 SYS_DATA_M8P16

Create a .data directive using the Sys_m8p16 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Data_m8p16( const )`
Description	Create a .data directive using the Sys_m8p16 macro.
Inputs	const = Constant to be passed to Sys_m8p16
Outputs	Sys_Data_m8p16 = .data directive with fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^53
Clock Cycles	0 (evaluated during compile)

Example	`// Load 7.125 to memory as a <8,16> fixed-point value`
	`Data_Value_m8p16:`
	`Sys_Data_m8p16( 71250000 )`

14.68 SYS_DEBUGPORT_GET_AUDIOMODE

Return the debug port audio mode setting

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_debugport.cfi
Template	`Sys_DebugPort_Get_AudioMode( reg )`
Description	Return the debug port audio mode setting
Inputs	None
Outputs	reg = Register the audio mode setting is stored to
Register Modifications	None
Assumptions	Result will be compared with AUDIO_STREAM_DISABLED or AUDIO_STREAM_ENABLED and hence no shift is required
Clock Cycles	2
Example	`// Load x1 with the current debug port audio mode setting` `Sys_DebugPort_Get_AudioMode( x1 )`

14.69 SYS_DEBUGPORT_SET_ACCESSMODE

Set the debug port access mode

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_debugport.cfi
Template	`Sys_DebugPort_Set_AccessMode( mode )`
Description	Set the debug port access mode
Inputs	mode = ACCESS_[RESTRICTED | UNRESTRICTED]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Unrestrict access through the debug port` `Sys_DebugPort_Set_AccessMode( ACCESS_UNRESTRICTED )`

14.70 SYS_DEBUGPORT_SET_ADDRESS

Set the internally-programmable CFX debug port address

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_debugport.cfi
Template	`Sys_DebugPort_Set_Address( addr )`
Description	Set the internally-programmable CFX debug port address
Inputs	addr = 7-bit I^2C internally-programmable address for the CFX debug port
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Assign 0x50 to the debug port's internally-programmable I2C address` `Sys_DebugPort_Set_Address( 0x50 )`

14.71 SYS_DEBUGPORT_SET_CM3ADDRESS

Set the debug port address for the ARM Cortex-M3 debug port

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_debugport.cfi
Template	`Sys_DebugPort_Set_CM3Address( addr )`
Description	Set the debug port address for the ARM Cortex-M3 debug port
Inputs	addr = 7-bit I^2C address for the ARM Cortex-M3 debug port
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Assign 0x5A to the ARM Cortex-M3 debug port's I2C address` `Sys_DebugPort_Set_CM3Address( 0x5A )`

14.72 SYS_DEBUGPORT_SET_EXTERNALADDRESS

Set the externally-programmable CFX debug port address

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_debugport.cfi
Template	`Sys_DebugPort_Set_ExternalAddress( addr )`
Description	Set the externally-programmable CFX debug port address
Inputs	addr = 7-bit I^2C externally-programmable address for the CFX debug port
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Assign 0x55 to the debug port's externally-programmable I2C address` `Sys_DebugPort_Set_ExternalAddress( 0x55 )`

14.73 SYS_DIO_CONFIG

Configure the specified digital I/O

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_dio.cfi
Template	`Sys_DIO_Config( dio, cfg )`
Description	Configure the specified digital I/O
Inputs	dio = DIO to configure; use a constant between 0 and 29 cfg = DIO configuration; use DIO_[LOW | HIGH]_DRIVE, DIO_LPF_[DISABLE | ENABLE], DIO_WEAK_PU_[DISABLE | ENABLE], DIO_STRONG_PU_[DISABLE | ENABLE], DIO_MODE_*
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if cfg can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure DIO 0 as a GPIO input, using the weak pull-up resistor`
	`// and low-pass filter`
	`Sys_DIO_Config(0, DIO_LPF_ENABLE | \`
	`DIO_LOW_DRIVE | \`
	`DIO_WEAK_PU_ENABLE | \`
	`DIO_STRONG_PU_DISABLE | \`
	`DIO_MODE_GPIO_IN_0)`

14.74 SYS_DIO_CONFIG_C_I2C

Configure two DIOs as the ARM Cortex-M3 I²C interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_dio.cfi
Template	`Sys_DIO_Config_C_I2C( cfg, scl, sda )`
Description	Configure two DIOs as the ARM Cortex-M3 I²C interface
Inputs	cfg = DIO pin configuration for the ARM Cortex-M3 I²C interface pins scl = DIO to configure as the I²C serial clock sda = DIO to configure as the I²C serial data
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	4 if cfg can be sign-extended from 8 bits; 6 otherwise
Example	`// Configure DIOs 8 and 9 as the ARM Cortex-M3 I2C interface` `Sys_DIO_Config_C_I2C( INTERFACE_PAD_CONFIG, 8, 9 )`

14.75 SYS_DIO_CONFIG_C_PCM

Configure four DIOs as the ARM Cortex-M3 PCM interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_dio.cfi
Template	`Sys_DIO_Config_C_PCM( cfg, mode, clk, frame, seri, sero )`
Description	Configure four DIOs as the ARM Cortex-M3 PCM interface
Inputs	cfg = DIO pin configuration for the PCM interface pins mode = PCM interface mode; C_PCM_SELECT_[MASTER | SLAVE] clk = DIO to configure as the PCM clock frame = DIO to configure as the PCM frame seri = DIO to configure as the PCM serial input sero = DIO to configure as the PCM serial output
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	6 if cfg can be sign-extended from 8 bits; 10 otherwise

Example	`// Configure DIOs 10, 11, 12 and 13 as the ARM Cortex-M3 PCM`
	`// interface in slave mode`
	`Sys_DIO_Config_C_PCM( INTERFACE_PAD_CONFIG, C_PCM_SELECT_SLAVE, \`
	`10, 11, 12, 13 )`

14.76 SYS_DIO_CONFIG_C_SPI

Configure four DIOs as the ARM Cortex-M3 SPI interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_dio.cfi
Template	`Sys_DIO_Config_C_SPI( cfg, mode, clk, cs, seri, sero )`
Description	Configure four DIOs as the ARM Cortex-M3 SPI interface
Inputs	cfg = DIO pin configuration for the SPI interface pins mode = SPI interface mode; C_SPI_SELECT_[MASTER | SLAVE] clk = DIO to configure as the SPI clock cs = DIO to configure as the SPI chip select seri = DIO to configure as the SPI serial input sero = DIO to configure as the SPI serial output
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	6 if cfg can be sign-extended from 8 bits; 10 otherwise

Example	`// Configure DIOs 4, 5, 6 and 7 as the ARM Cortex-M3 SPI interface`
	`// in slave mode`
	`Sys_DIO_Config_C_SPI( INTERFACE_PAD_CONFIG, C_SPI_SELECT_SLAVE, \`
	`4, 5, 6, 7 )`

14.77 SYS_DIO_CONFIG_DAC_SCALE

Configure the DIO to be used as the DAC scaling control pin

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_dio.cfi
Template	`Sys_DIO_Config_DAC_SCALE( cfg, ctrl )`
Description	Configure the DIO to be used as the DAC scaling control pin
Inputs	cfg = DIO pin configuration for the DAC scaling control ctrl = DIO to configure as the DAC scaling control pin
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2 if cfg can be sign-extended from 8 bits; 3 otherwise
Example	`// Configure DIO 15 as the DAC scaling controller` `Sys_DIO_Config_DAC_SCALE( INTERFACE_PAD_CONFIG, 15 )`

14.78 SYS_DIO_CONFIG_DMICINPUT

Configure DIOs as the DMIC inputs

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_dio.cfi
Template	`Sys_DIO_Config_DMICInput( cfg, dmic0, dmic1, dmic2, dmic3 )`
Description	Configure DIOs as the DMIC inputs
Inputs	cfg = DIO pin configuration for the DMIC input pins dmic0 = DIO to configure as DMIC input 0; use 30 for a constant low, 31 for a constant high dmic1 = DIO to configure as DMIC input 1; use 30 for a constant low, 31 for a constant high dmic2 = DIO to configure as DMIC input 2; use 30 for a constant low, 31 for a constant high dmic3 = DIO to configure as DMIC input 3; use 30 for a constant low, 31 for a constant high
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 for each non-constant input if cfg can be sign-extended from 8 bits (2 for each non-constant input otherwise), plus 3 additional cycles

Example	`// Configure DIOs 10, 11 as DMIC inputs 0, 1. Configure DMIC inputs`
	`// 2, 3 as constant low since they are unused in this application.`
	`Sys_DIO_Config_DMICInput( INTERFACE_PAD_CONFIG, 10, 11, 30, 30 )`

14.79 SYS_DIO_CONFIG_PCM

Configure four DIOs as the CFX PCM interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_dio.cfi
Template	`Sys_DIO_Config_PCM( cfg, mode, clk, frame, seri, sero )`
Description	Configure four DIOs as the CFX PCM interface
Inputs	cfg = DIO pin configuration for the PCM interface pins mode = PCM interface mode; PCM_[MASTER | SLAVE] clk = DIO to configure as the PCM clock frame = DIO to configure as the PCM frame seri = DIO to configure as the PCM serial input sero = DIO to configure as the PCM serial output
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	6 if cfg can be sign-extended from 8 bits; 10 otherwise

Example	`// Configure DIOs 0, 1, 2 and 3 as the CFX PCM interface in`
	`// master mode`
	`Sys_DIO_Config_PCM( INTERFACE_PAD_CONFIG, PCM_MASTER, \`
	`0, 1, 2, 3 )`

14.80 SYS_DIO_CONFIG_PLL

Configure the DIO to be used as the PLL reference clock input

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_dio.cfi
Template	`Sys_DIO_Config_PLL( cfg, refclk )`
Description	Configure the DIO to be used as the PLL reference clock input
Inputs	cfg = DIO pin configuration for the PLL reference clock pin refclk = DIO to configure as the PLL reference clock; use 30 for a constant low, 31 for EXTCLK
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if refclk is a constant low or EXTCLK; 2 if cfg can be sign-extended from 8 bits; 3 otherwise
Example	`// Configure EXTCLK as the PLL reference clock input` `Sys_DIO_Config_PLL( INTERFACE_PAD_CONFIG, 31 )`

14.81 SYS_DIO_CONFIG_SPI

Configure four DIOs as the CFX SPI interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_dio.cfi
Template	`Sys_DIO_Config_SPI( cfg, clk, cs, seri, sero )`
Description	Configure four DIOs as the CFX SPI interface
Inputs	cfg = DIO pin configuration for the SPI interface pins clk = DIO to configure as the SPI clock cs = DIO to configure as the SPI chip select seri = DIO to configure as the SPI serial input sero = DIO to configure as the SPI serial output
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	5 if cfg can be sign-extended from 8 bits; 9 otherwise
Example	`// Configure DIOs 0, 1, 2 and 3 as the CFX SPI interface` `Sys_DIO_Config_SPI( INTERFACE_PAD_CONFIG, 0, 1, 2, 3 )`

14.82 SYS_DIO_CONFIG_SYNC_SLAVE

Configure the DIO to be used as the synchronization pin for synchronizing with another device

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_dio.cfi
Template	`Sys_DIO_Config_SYNC_SLAVE( cfg, sync_slave )`
Description	Configure the DIO to be used as the synchronization pin for synchronizing with another device.
Inputs	cfg = DIO pin configuration for the slave synchronization pin sync_slave = DIO to configure as the slave synchronization signal
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2 if cfg can be sign-extended from 8 bits; 3 otherwise
Example	`// Configure DIO 15 as the slave synchronization pad` `Sys_DIO_Config_SYNC_SLAVE( INTERFACE_PAD_CONFIG, 15 )`

14.83 SYS_DIO_CONFIG_UART

Configure two DIOs as the CFX UART interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_dio.cfi
Template	`Sys_DIO_Config_UART( cfg, tx, rx )`
Description	Configure two DIOs as the CFX UART interface
Inputs	cfg = DIO pin configuration for the UART interface pins tx = DIO to configure as the UART transmission signal rx = DIO to configure as the UART reception signal
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	3 if cfg can be sign-extended from 8 bits; 5 otherwise
Example	`// Configure DIOs 20 and 21 as the CFX UART interface` `Sys_DIO_Config_UART( INTERFACE_PAD_CONFIG, 20, 21 )`

14.84 SYS_DIO_GET_MODE

Get the DIO modes (DIO or GPIO) of all of the DIOs and store them to the specified accumulator register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_dio.cfi
Template	`Sys_DIO_Get_Mode( reg )`
Description	Get the DIO modes (DIO or GPIO) of all of the DIOs and store them to the specified accumulator register.
Inputs	None
Outputs	reg = Accumulator to store the current DIO/GPIO modes to; compare with DIO*_[IS | IS_NOT]_GPIO
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Query which DIOs are currently GPIOs.` `Sys_DIO_Get_Mode( b1 )`

14.85 SYS_ECB_INT_ACK

Acknowledge all ECB interrupt sources

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ecb.cfi
Template	`Sys_ECB_Int_Ack`
Description	Acknowledge all ECB interrupt sources
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Acknowledge the error correction coding block interrupts` `Sys_ECB_Int_Ack`

14.86 SYS_ECB_INT_ENABLE

Enable/disable a set of error correction coding block interrupts

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ecb.cfi
Template	`Sys_ECB_Int_Enable( src )`
Description	Enable/disable a set of error correction coding block interrupts
Inputs	src = ECB interrupt sources to enable; use INT_EBL_* and INT_DIS_*
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if src can be sign-extended from 8 bits; 2 if src can be sign-extended from 24 bits; 3 otherwise

Example	`// Enable the Reed-Solomon decoder interrupts`
	`Sys_ECB_Int_Enable( INT_EBL_RS_DEC_IN_ERR |  \`
	`INT_EBL_RS_DEC_BLK_END |  \`
	`INT_EBL_RS_DEC_MISS_ERR )`

14.87 SYS_FENG_CONFIG

Configure the inputs to, outputs from, output shadow register enable, and coefficient offset for the Filter Engine

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_FENG_Config( cfg )`
Description	Configure the inputs to, outputs from, output shadow register enable, and coefficient offset for the Filter Engine.
Inputs	cfg = Input/output configuration including selection of the sample ready input, output shadow register enable and coefficient offset; use FENG_SRI_[SELECT_* | DISABLE], FENG_OSR*_[DISABLE | ENABLE], FENG_FIN*_[DISABLE | ENABLE] and a constant shifted to FENG_CFG_COEFF_OFFSET_POS
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if cfg can be sign-extended from 8 bits; 2 if cfg can be sign-extended from 24 bits, but not 8 bits; 3 otherwise

Example	`// Configure the Filter Engine to enable inputs 0, 2; output 0; and`
	`// trigger Filter Engine processing on input 2`
	`Sys_FENG_Config( FENG_FIN0_ENABLE | FENG_FIN2_ENABLE | \`
	`FENG_OSR0_ENABLE | FENG_SRI_SELECT_IN2 )`

14.88 SYS_FENG_DELAYEDPAUSE

Pause the Filter Engine the next time it would start execution

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_FENG_DelayedPause`
Description	Pause the Filter Engine the next time it would start execution
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	The FENG is currently enabled
Clock Cycles	1
Example	`// Stop execution on the Filter Engine the next time it would restart` `Sys_FENG_DelayedPause`

14.89 SYS_FENG_EXECUTE

Execute one phase of the current Filter Engine program

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_FENG_Execute`
Description	Execute one phase of the current Filter Engine program
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	The Filter Engine clock has been enabled (Filter Engine started)
Clock Cycles	1
Example	`// Execute one phase of the current Filter Engine program` `Sys_FENG_Execute`

14.90 SYS_FENG_INTACK

Acknowledge a set of Filter Engine interrupt sources

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_FENG_IntAck( int_ack )`
Description	Acknowledge a set of Filter Engine interrupt sources
Inputs	int_ack = Filter engine interrupt sources to acknowledge
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if int_ack can be sign-extended from 8 bits; 2 if int_ack can be sign-extended from 24 bits; 3 otherwise
Example	`// Acknowledge all previous Filter Engine interrupts` `Sys_FENG_IntAck( 0x0FFFFFFF )`

14.91 SYS_FENG_PAUSE

Pause the Filter Engine immediately

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_FENG_Pause`
Description	Pause the Filter Engine immediately
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Stop execution on the Filter Engine` `Sys_FENG_Pause`

14.92 SYS_FENG_RESET

Reset the Filter Engine program decoder registers and audio path registers

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_FENG_Reset`
Description	Reset the Filter Engine program decoder registers and audio path registers
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Reset the Filter Engine` `Sys_FENG_Reset`

14.93 SYS_FENG_SINGLESTEP

Single-step the Filter Engine

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_FENG_SingleStep`
Description	Single-step the Filter Engine
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	The FENG is currently disabled
Clock Cycles	1
Example	`// Execute one instruction on the Filter Engine` `Sys_FENG_SingleStep`

14.94 SYS_FENG_START

Start (or resume) the Filter Engine

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_FENG_Start`
Description	Start (or resume) the Filter Engine
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Start execution on the Filter Engine` `Sys_FENG_Start`

14.95 SYS_FIFO_AUTOMUTE

Configure the auto-mute feature of one of the system FIFOs

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_fifo.cfi
Template	`Sys_FIFO_AutoMute( fifo_name, config )`
Description	Configure the auto-mute feature of one of the system FIFOs
Inputs	fifo_name = One of: SYS_FIFO_A0_*, SYS_FIFO_A1_*, SYS_FIFO_B0_*, or SYS_FIFO_B1_* config = The FIFO auto-mute configuration; use FIFO_**_*_AUTO_MUTE_[DISABLE | ENABLE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Mute FIFO A0_0` `Sys_FIFO_AutoMute(SYS_FIFO_A0_0, FIFO_A0_0_AUTO_MUTE_ENABLE)`

14.96 SYS_FIFO_CFXINTCONFIG

Configure source for a CFX FIFO interrupt

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_fifo.cfi
Template	`Sys_FIFO_CFXIntConfig(int_index, fifo_name)`
Description	Configure source for a CFX FIFO interrupt
Inputs	int_index = Interrupt index to configure; use a constant 0-7 fifo_name = One of: SYS_FIFO_A0_*, SYS_FIFO_A1_*, SYS_FIFO_B0_*, or SYS_FIFO_B1_*
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Configure CFX FIFO interrupt 1 to trigger on FIFO A0_0` `Sys_FIFO_CFXIntConfig( 1, SYS_FIFO_A0_0 )`

14.97 SYS_FIFO_CM3INTCONFIG

Configure source for an ARM Cortex-M3 FIFO interrupt

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_fifo.cfi
Template	`Sys_FIFO_CM3IntConfig(int_index, fifo_name)`
Description	Configure source for an ARM Cortex-M3 FIFO interrupt
Inputs	int_index = Interrupt index to configure; use a constant 0-7 fifo_name = One of: SYS_FIFO_A0_*, SYS_FIFO_A1_*, SYS_FIFO_B0_*, or SYS_FIFO_B1_*
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the ARM Cortex-M3 FIFO interrupt 0 to trigger on`
	`// FIFO A1_1`
	`Sys_FIFO_CM3IntConfig( 0, SYS_FIFO_A1_1 )`

14.98 SYS_FIFO_CONFIGURE

Configure one of the system FIFOs for use within an application

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_fifo.cfi
Template	`Sys_FIFO_Configure( fifo_name, start, fifo_size, block_size, base_ptr, ioblock_ptr )`
Description	Configure one of the system FIFOs for use within an application
Inputs	fifo_name = One of: SYS_FIFO_A0_*, SYS_FIFO_A1_*, SYS_FIFO_B0_*, or SYS_FIFO_B1_* start = Start offset for the FIFO fifo_size = Size of the FIFO block_size = Size of a block in the FIFO base_ptr = Initial value for the BASE pointer ioblock_ptr = Initial value for the IOBLOCK pointer
Outputs	None
Register Modifications	y1
Assumptions	The FIFO start offset is a multiple of 32 The FIFO size is a multiple of 32 and is not larger than 2048 The FIFO block size is a multiple of 2 and is not larger than 1024 The base and IO block pointers are multiples of 2 and point to locations between FIFO start and FIFO start + FIFO size.
Clock Cycles	13 cycles maximum, saving 1 cycle for each of start, fifo_size, start + fifo_size, block_size, base_ptr, and ioblock_ptr that can be sign-extended from 8 bits

Example	`// Configure FIFO A0_0`
	`#define BLOCK_SIZE 8`
	
	`#define FIFO_A0_0_START 0`
	`#define FIFO_A0_0_LENGTH 32`
	`#define FIFO_A0_0_END (FIFO_A0_0_START + FIFO_A0_0_LENGTH)`
	`#define FIFO_A0_0_BASE_PTR 0`
	`#define FIFO_A0_0_IOBLOCK_PTR 0`
	`#define FIFO_A0_0_BLOCK_SIZE BLOCK_SIZE`
	
	`Sys_FIFO_Configure( SYS_FIFO_A0_0, \`
	`FIFO_A0_0_START, \`
	`FIFO_A0_0_LENGTH, \`
	`FIFO_A0_0_BLOCK_SIZE, \`
	`FIFO_A0_0_BASE_PTR, \`
	`FIFO_A0_0_IOBLOCK_PTR )`

14.99 SYS_FIFO_HEARINTCONFIG

Configure source for a HEAR FIFO interrupt

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_fifo.cfi
Template	`Sys_FIFO_HEARIntConfig(int_index, fifo_name)`
Description	Configure source for a HEAR FIFO interrupt
Inputs	int_index = Interrupt index to configure; use a constant 0-7 fifo_name = One of: SYS_FIFO_A0_*, SYS_FIFO_A1_*, SYS_FIFO_B0_*, or SYS_FIFO_B1_*
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Configure the HEAR FIFO interrupt 6 to trigger on FIFO B0_3` `Sys_FIFO_HEARIntConfig( 6, SYS_FIFO_B0_3 )`

14.100 SYS_FL8N16

Convert a decimal number with up to seven digits of precision to floating-point format with an 8-bit signed exponent and a 16-bit signed mantissa

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_fl8n16( const )`
Description	Convert a decimal number with up to seven digits of precision to floating-point format with an 8-bit signed exponent and a 16-bit signed mantissa. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 7.125 becomes 71250000. The 10^7 multiplier provides 23 fractional bits of accuracy, although the 16-bit mantissa only stores 15 bits of precision, and this includes the value of any non-fractional part of the input. Although the valid range for the floating-point format is approximately -2^127 to 2^127, the whole-number input format for this macro and the 64-bit arithmetic of the preprocessor constrain the pre-multiplied inputs to have magnitude less than 2^63 or about 9 x 10^18 (representing numbers up to about 9 x 10^11). Furthermore, the 7-digit fractional part of the decimal input number constrains the inputs to have magnitude of 2^-23 or greater if they are non-zero. The input number therefore cannot exceed the limits of the floating-point format. This macro is useful for filling data registers through an immediate load and for defining floating-point data to be loaded from XMEM or YMEM.
Inputs	const = Constant to convert to floating-point format, multiplied by 10^7
Outputs	Sys_fl8n16 = Floating-point approximation (8-bit signed exponent and 16-bit signed mantissa) of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^63
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 25.125 converted to floating-point format with 8-bit`
	`// signed exponent and 16-bit signed mantissa and then store this`
	`// to a0.`
	`#set VALUE Sys_fl8n16( 251250000 )`
	`load a0, VALUE`

14.101 SYS_FRAC24

Convert a decimal number with up to seven digits of precision to <1,23> fractional format

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_frac24( const )`
Description	Convert a decimal number with up to seven digits of precision to <1,23> fractional format. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 0.125 becomes 1250000. The valid input range (after pre-multiplication) for <1,23> fractional numbers is -10000000 to +9999999. Numbers outside this range are saturated to the closest valid value, but the results are undefined for inputs with magnitude greater than about 2^46 or 70 x 10^12 (representing numbers greater than about 7000000.0). This macro is useful for filling data registers through an immediate load and for defining fractional data to be loaded from XMEM or YMEM.
Inputs	const = Constant to convert to <1,23>, multiplied by 10^7
Outputs	Sys_frac24 = <1,23> fractional approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^46
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 0.125 converted to the <1,23> fractional format`
	`// and then load this to a0.`
	`#set VALUE Sys_frac24( 1250000 )`
	`load a0, VALUE`

14.102 SYS_FRAC32

Convert a decimal number with up to seven digits of precision to <9,23> fixed-point format

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_frac32( const )`
Description	Convert a decimal number with up to seven digits of precision to <9,23> fixed-point format. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 7.125 becomes 71250000. The valid input range (after pre-multiplication) for <9,23> fixed-point numbers is -2560000000 to +2559999999. Numbers outside this range are saturated to the closest valid value, but the results are undefined for inputs with magnitude greater than about 2^46 or 70 x 10^12 (representing numbers greater than about 7000000.0). This macro is useful for filling an accumulator's high and extension parts using two immediate loads and for defining <9,23> data to be loaded from PMEM. NOTE: The frac32 data format is layout-compatible with the frac24 format.
Inputs	const = Constant to convert to <9,23>, multiplied by 10^7
Outputs	Sys_frac32 = <9,23> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^46
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 12.5 converted to the <9,23> fixed-point format`
	`// and then store this in P memory at Direct_Data_Value_frac32.`
	`Sys_Mem_Start_Segment( Frac32_Table, SYS_MEMSPACE_P )`
	`#set VALUE Sys_frac32( 125000000 )`
	`Direct_Data_Value_frac32:`
	`.data VALUE`

14.103 SYS_FRAC48

Convert a decimal number with up to seven digits of precision to <1,47> fractional format

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_frac48( const )`
Description	Convert a decimal number with up to seven digits of precision to <1,47> fractional format. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 0.125 becomes 1250000. The valid input range (after pre-multiplication) for <1,47> fractional numbers is -10000000 to +9999999. Numbers outside this range are saturated to the closest valid value, but the results are undefined for inputs with magnitude greater than about 2^24 or 16000000 (representing numbers greater than about 1.6). NOTE: This macro places the number in a <1,47> field, but provides only <1,45> accuracy. The bottom 2 bits of an unsaturated result are always zero. This macro is useful for filling accumulators through a pair of immediate loads and for defining <1,47> fractional data to be loaded from XMEM, YMEM or XYMEM.
Inputs	const = Constant to convert to <1,47>, multiplied by 10^7
Outputs	Sys_frac48 = <1,47> fractional approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^24
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 0.125 converted to the <1,47> fractional format`
	`// and then store the two halves of the 48-bit value in memory at`
	`// Direct_Data_Value_frac48.`
	`#set VALUE Sys_frac48( 1250000 )`
	`#set VAL_H VALUE >> 24`
	`#set VAL_L Sys_IMM_24( VALUE )`
	`Direct_Data_Value_frac48:`
	`.data VAL_H, VAL_L`

14.104 SYS_G722_DECODER_CLEAR_STATE

Clear the internal state of the G.722 decoder

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_acb.cfi
Template	`Sys_G722_Decoder_Clear_State`
Description	Clear the internal state of the G.722 decoder
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	The G.722 decoder is idle
Clock Cycles	2
Example	`// Clear the current state for the G.722 decoder block` `Sys_G722_Decoder_Clear_State`

14.105 SYS_G722_DECODER_CONCEAL

Set the necessary registers for concealment of a missing G.722 sample

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_acb.cfi
Template	`Sys_G722_Decoder_Conceal( PLT, RLT, SX, LSBQuant, HSBQuant, FBIN )`
Description	Set the necessary registers for concealment of a missing G.722 sample
Inputs	PLT = Replacement settings for the PLT1, PLT2 sign registers RLT = Replacement setting for the RLT1 register SX = Replacement settings for the SZL, SL pipeline registers LSBQuant = Replacement settings for the DETL, NBL quantizer registers (lower sub-band) HSBQuant = Replacement settings for the DETL, NBL quantizer registers (higher sub-band) FBIN = Replacement data for the sub-band decoders
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	Between 6 and 17

Example	`// Write all zeros as the concealment data covering for a missing`
	`// G.722 input data sample`
	`Sys_G722_Decoder_Conceal( 0, 0, 0, 0, 0, 0 )`

14.106 SYS_G722_DECODER_CONFIG

Configure the G.722 decoder

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_acb.cfi
Template	`Sys_G722_Decoder_Config( config )`
Description	Configure the G.722 decoder
Inputs	config = The G.722 decoder configuration; use G722_DEC_[DISABLE | ENABLE], G722_CONCL_[DISABLE | ENABLE], G722_DEC_FB_[BYPASS | ENABLE], and G722_DEC_OUT_[15BIT | 16BIT]
Outputs	None
Register Modifications	y1, b1
Assumptions	None
Clock Cycles	6

Example	`// Enable and configure the G.722 decoder`
	`Sys_G722_Decoder_Config( G722_DEC_ENABLE | \`
	`G722_CONCL_DISABLE | \`
	`G722_DEC_FB_BYPASS | \`
	`G722_DEC_OUT_16BIT )`

14.107 SYS_G722_ENCODER_CLEAR_STATE

Clear the internal state of the G.722 encoder

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_acb.cfi
Template	`Sys_G722_Encoder_Clear_State`
Description	Clear the internal state of the G.722 encoder
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	The G.722 encoder is idle
Clock Cycles	1
Example	`// Clear the current state for the G.722 encoder block` `Sys_G722_Encoder_Clear_State`

14.108 SYS_G722_ENCODER_CONFIG

Configure the G.722 encoder

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_acb.cfi
Template	`Sys_G722_Encoder_Config( config )`
Description	Configure the G.722 encoder
Inputs	config = The G.722 encoder configuration; use G722_ENC_[DISABLE | ENABLE], G722_ENC_FB_[BYPASS | ENABLE], and G722_ENC_IN_[15BIT | 16BIT]
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	5

Example	`// Enable and configure the G.722 encoder`
	`Sys_G722_Encoder_Config( G722_ENC_ENABLE | \`
	`G722_ENC_FB_BYPASS | \`
	`G722_ENC_IN_16BIT )`

14.109 SYS_GET_ADC_CFG

Load the ADC configuration from one of the input channels

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_Get_ADC_CFG( channel, reg )`
Description	Load the ADC configuration from one of the input channels
Inputs	channel = ADC to load this configuration from; use a constant between 0 and 3
Outputs	reg = Register to store the current ADC configuration from A_ADC_CFG_* to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current ADC0 configuration settings into y0` `Sys_Get_ADC_CFG( 0, y0 )`

14.110 SYS_GET_ADC_CTRL

Load the ADC control configuration from one of the input channels

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_Get_ADC_CTRL( channel, reg )`
Description	Load the ADC control configuration from one of the input channels
Inputs	channel = ADC to load this configuration from; use a constant between 0 and 3
Outputs	reg = Register to store the current ADC control configuration from A_ADC_CTRL_* to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current ADC0 control configuration settings into y0` `Sys_Get_ADC_CTRL( 0, y0 )`

14.111 SYS_GET_ADC_DEC_CTRL

Load the decimation filter configuration from one of the input channels

Type	Macro	
Include File	`#include <sk4_sys.cfi>`	
Source File	sk4_sys_analog_in.cfi	
Template	`Sys_Get_ADC_DEC_CTRL( channel, reg )`	
Description	Load the decimation filter configuration from one of the input channels	
Inputs	channel	= Decimation filter to load this control configuration from; use a constant between 0 and 3
Outputs	reg	= Accumulator to store the current decimation filter configuration to
Register Modifications	None	
Assumptions	None	
Clock Cycles	1	

Example	`// Load the current decimation filter configuration settings for`
	`// input channel 0 into a0`
	`Sys_Get_ADC_DEC_CTRL( 0, a0 )`	

14.112 SYS_GET_ADC_SAMPLE_FREQ_CFG

Load the sampling frequency configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_Get_ADC_SAMPLE_FREQ_CFG( reg )`
Description	Load the sampling frequency configuration
Inputs	None
Outputs	reg = Register to store the sampling frequency configuration to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current sampling frequency configuration settings into y0` `Sys_Get_ADC_SAMPLE_FREQ_CFG( y0 )`

14.113 SYS_GET_APPNUM

Return the number of the currently running application (APPNUM)

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_system.cfi
Template	`Sys_Get_AppNum( reg )`
Description	Return the number of the currently running application (APPNUM)
Inputs	None
Outputs	reg = Register to load with the current application number (APPNUM)
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load x0 with the number of the currently running application.`
	`// This value is loaded by the boot ROM when booting the device.`
	`Sys_Get_AppNum( x0 )`

14.114 SYS_GET_CHIPDEBUGVERSION

Return the CFX (primary) debug port version

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_system.cfi
Template	`Sys_Get_ChipDebugVersion( reg )`
Description	Return the CFX (primary) debug port version
Inputs	None
Outputs	reg = Register to load with the value of D_CFX_DEBUG_VERSION register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load x0 with the primary debug port protocol version number for`
	`// this device`
	`Sys_Get_ChipDebugVersion( x0 )`

14.115 SYS_GET_CHIPFAMILY

Return chip family

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_system.cfi
Template	`Sys_Get_ChipFamily( reg )`
Description	Return chip family
Inputs	None
Outputs	reg = Register to load with the value of D_CHIP_FAMILY register
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load x0 with the technology family for this chip` `Sys_Get_ChipFamily( x0 )`

14.116 SYS_GET_CHIPREVISION

Return chip revision

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_system.cfi
Template	`Sys_Get_ChipRevision( reg )`
Description	Return chip revision
Inputs	None
Outputs	reg = Register to load with the value of D_CHIP_REVISION register
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load x0 with the revision number for this chip` `Sys_Get_ChipRevision( x0 )`

14.117 SYS_GET_CHIPVERSION

Return chip version

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_system.cfi
Template	`Sys_Get_ChipVersion( reg )`
Description	Return chip version
Inputs	None
Outputs	reg = Register to load with the value of D_CHIP_VERSION register
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load x0 with the version for this chip` `Sys_Get_ChipVersion( x0 )`

14.118 SYS_GET_CM3CHIPDEBUGVERSION

Return the ARM Cortex-M3 debug port version

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_system.cfi
Template	`Sys_Get_CM3ChipDebugVersion( reg )`
Description	Return the ARM Cortex-M3 debug port version
Inputs	None
Outputs	reg = Register to load with the value of D_CM3_DEBUG_VERSION register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load x0 with the ARM Cortex-M3 debug port protocol version number`
	`// for this device`
	`Sys_Get_CM3ChipDebugVersion( x0 )`

14.119 SYS_GET_CP_VDBL_CTRL

Get the digital interface supply voltage (VDBL) charge pump configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_pwr.cfi
Template	`Sys_Get_CP_VDBL_CTRL( reg )`
Description	Get the digital interface supply voltage (VDBL) charge pump configuration
Inputs	None
Outputs	reg = Register to store the VDBL charge pump configuration to
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current digital interface supply charge pump`
	`// configuration into y1`
	`Sys_Get_CP_VDBL_CTRL( y1 )`

14.120 SYS_GET_CP_VDDA_CTRL

Get the analog supply voltage (VDDA) charge pump configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_pwr.cfi
Template	`Sys_Get_CP_VDDA_CTRL( reg )`
Description	Get the analog supply voltage (VDDA) charge pump configuration
Inputs	None
Outputs	reg = Register to store the VDDA charge pump configuration to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current analog supply charge pump configuration into y1` `Sys_Get_CP_VDDA_CTRL( y1 )`

14.121 SYS_GET_DMIC_OUTPUT_CTRL

Get the DMIC output configuration for both output channels

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_Get_DMIC_OUTPUT_CTRL( reg )`
Description	Get the DMIC output configuration for both output channels
Inputs	None
Outputs	reg = Register to store the DMIC output configurations to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current DMIC output configuration into y1` `Sys_Get_DMIC_OUTPUT_CTRL( y1 )`

14.122 SYS_GET_FENG_COEFFOFFSET

Get the current Filter Engine coefficient offset

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Get_FENG_CoeffOffset( reg )`
Description	Get the current Filter Engine coefficient offset
Inputs	None
Outputs	reg = Register to store the Filter Engine coefficient offset to; compare with an unshifted constant
Register Modifications	b1
Assumptions	None
Clock Cycles	2
Example	`// Load the current Filter Engine coefficient offset into y0` `Sys_Get_FENG_CoeffOffset( y0 )`

14.123 SYS_GET_FENG_COEFFOFFSETENABLE

Get the current Filter Engine coefficient offset enable settings

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Get_FENG_CoeffOffsetEnable( reg )`
Description	Get the current Filter Engine coefficient offset enable settings
Inputs	None
Outputs	reg = Accumulator register to store the Filter Engine coefficient offset enable settings to; compare the bit positions where the bit value is 1 to the coefficient set indexes
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current Filter Engine coefficient offset enable values`
	`// into a0`
	`Sys_Get_FENG_CoeffOffsetEnable( a0 )`

14.124 SYS_GET_FENG_INTSTATUS

Get the status of various possible sources of the Filter Engine interrupt

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Get_FENG_IntStatus( reg )`
Description	Get the status of various possible sources of the Filter Engine interrupt
Inputs	None
Outputs	reg = Accumulator register to hold the current Filter Engine interrupt status bits; compare with INT_PENDING_FENG_*
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Check the Filter Engine interrupt status`
	`Sys_Get_FENG_IntStatus( a0 )`
	`bittst a0, (FENG_INT_STATUS_SRIN2_POS + 24) ||| nop`
	`goto.0d Process_FENG_Input, carry`

14.125 SYS_GET_FENG_STATE

Get the current Filter Engine state codes

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Get_FENG_State( reg )`
Description	Get the current Filter Engine state codes
Inputs	None
Outputs	reg = Register to load with the current Filter Engine state codes; compare with a set of shifted constants where 1 indicates the condition is met and 0 indicates the condition is not met for each state code
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the Filter Engine state codes into y0` `Sys_Get_FENG_State( y0 )`

14.126 SYS_GET_FENG_STATEWRITEENABLE

Get the current Filter Engine state code write-enable settings

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Get_FENG_StateWriteEnable( reg )`
Description	Get the current Filter Engine state code write-enable settings
Inputs	None
Outputs	reg = Register to load with the current Filter Engine state code write-enable settings; compare with a set of shifted constants where 1 indicates the Filter Engine state code can be written for each state code
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the Filter Engine state code write-enable settings into y0` `Sys_Get_FENG_StateWriteEnable( y0 )`

14.127 SYS_GET_HEAR_BFP_BLOCK_EXP

Store the value of the block floating point (BFP) block exponent into a register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_hear.cfi
Template	`Sys_Get_HEAR_BFP_BLOCK_EXP( reg )`
Description	Store the value of the block floating point (BFP) block exponent into a register
Inputs	None
Outputs	reg = Register to load the HEAR BFP block exponent to
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Store the current value of the block floating point exponent used`
	`// on the HEAR Configurable Accelerator to x0`
	`Sys_Get_HEAR_BFP_BLOCK_EXP( x0 )`

14.128 SYS_GET_IDRX_CFG

Load the input dynamic range extended (IDRX) input channel configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_Get_IDRX_CFG( channel, reg )`
Description	Load the input dynamic range extended (IDRX) input channel configuration
Inputs	channel = IDRX input channel; use 0 or 1
Outputs	reg = Accumulator register to store the desired IDRX filter configuration to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current IDRX0 configuration settings into b0` `Sys_Get_IDRX_CFG( 0, b0 )`

14.129 SYS_GET_LSAD_CTRL

Get the value of control register A_LSAD_CTRL

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_lsad.cfi
Template	`Sys_Get_LSAD_CTRL( reg )`
Description	Get the value of control register A_LSAD_CTRL
Inputs	None
Outputs	reg = Register to store the value of the A_LSAD_CTRL register to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current contents of register A_LSAD_CTRL into a1` `Sys_Get_LSAD_CTRL( a1 )`

14.130 SYS_GET_O_DAC_AO_CTRL

Get the analog output configuration for both output channels

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_Get_O_DAC_AO_CTRL( reg )`
Description	Get the analog output configuration for both output channels
Inputs	None
Outputs	reg = Register to store the analog output configurations to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current analog output configuration into y1` `Sys_Get_O_DAC_AO_CTRL( y1 )`

14.131 SYS_GET_O_DAC_GF_CTRL

Get the output stage gain factor control settings

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_Get_O_DAC_GF_CTRL( reg )`
Description	Get the output stage gain factor control settings
Inputs	None
Outputs	reg = Accumulator to store output gain configuration to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current contents of register A_O_DAC_GF_CTRL into a1` `Sys_Get_O_DAC_GF_CTRL( a1 )`

14.132 SYS_GET_O_SDM_CTRL

Get the sigma-delta modulator configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_Get_O_SDM_CTRL( reg )`
Description	Get the sigma-delta modulator configuration
Inputs	None
Outputs	reg = Register to store the output modulator configuration to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Store the current sigma-delta modulator configruation to y0` `Sys_Get_O_SDM_CTRL( y0 )`

14.133 SYS_GET_O_SDMCLK

Load the output modulator prescaler

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_Get_O_SDMCLK( reg )`
Description	Load the output modulator prescaler
Inputs	None
Outputs	reg = Register to store the output modulator prescaler to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current contents of register A_O_SDMCLK into x0` `Sys_Get_O_SDMCLK( x0 )`

14.134 SYS_GET_OSC_CTRL_0

Get the oscillator frequency

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Get_OSC_CTRL_0( reg )`
Description	Get the oscillator frequency
Inputs	None
Outputs	reg = Register to store the frequency control setting from A_OSC_CTRL_0 to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current system clock setting to x0` `Sys_Get_OSC_CTRL_0( x0 )`

14.135 SYS_GET_OSC_CTRL_1

Get the oscillator multiplier and enable settings

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Get_OSC_CTRL_1( reg )`
Description	Get the oscillator multiplier and enable settings
Inputs	None
Outputs	reg = Register to store the multiplier and enable settings from A_OSC_CTRL_1 to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current clock multiplier and enable settings to x0` `Sys_Get_OSC_CTRL_1( x0 )`

14.136 SYS_GET_OSCCOMPOUT

Get the outputs from the oscillator compensation circuitry

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Get_OSCCompOut( reg )`
Description	Get the outputs from the oscillator compensation circuitry
Inputs	None
Outputs	reg = Register to load with the oscillator compensation outputs for both positive (PTAT) and negative (CTAT) temperature compensation
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current outputs of the oscillator temperature`
	`// compensation support circuitry and store this into y1`
	`Sys_Get_OSCCompOut( y1 )`

14.137 SYS_GET_OUTPUT_CTRL

Load the output driver configuration settings

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_Get_OUTPUT_CTRL( reg )`
Description	Load the output driver configuration settings
Inputs	None
Outputs	reg = Register to store the current output driver configurations to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current contents of register A_OUTPUT_CTRL into y1` `Sys_Get_OUTPUT_CTRL( y1 )`

14.138 SYS_GET_PLL_CALIBRATIONOUTPUT

Get the VCO frequency setting output from the digital calibration feature of the PLL

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Get_PLL_CalibrationOutput( reg )`
Description	Get the VCO frequency setting output from the digital calibration feature of the PLL. This macro can be used with Sys_SetReg_OSC_CTRL_0 to load this setting to the VCO frequency control bit field.
Inputs	None
Outputs	reg = Register to store the calibrated VCO frequency setting to
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Copy the output of the PLL digital calibration to the VCO trim`
	`Sys_Get_PLL_CalibrationOutput( x0 )`
	`Sys_SetReg_OSC_CTRL_0( x0 )`

14.139 SYS_GET_PLL_CLOCKSYNC

Get the PLL and SDMCLK clock synchronization status

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_clk.cfi
Template	`Sys_Get_PLL_ClockSync( reg )`
Description	Get the PLL and SDMCLK clock synchronization status
Inputs	None
Outputs	reg = Register to store the PLL and SDMCLK clock synchronization status; compare with CLK_RESYNC_[TRUE | FALSE], CLK_[SYNCHRONIZED | NOT_SYNCHRONIZED], CLK_RESYNC_INT_[ENABLE | DISABLE], CLK_SYNC_TOLERANCE_[HIGH | LOW], and CLK_SYNC_[ENABLE | DISABLE]
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the PLL/SDMCLK clock synchronization status` `Sys_Get_PLL_ClockSync( x0 )`

14.140 SYS_GET_PLL_DIGITALCAL

Get the PLL digital calibration configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Get_PLL_DigitalCal( reg )`
Description	Get the PLL digital calibration configuration
Inputs	None
Outputs	reg = Register to store the PLL digital calibration configuration to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the PLL digital calibration settings to x0` `Sys_Get_PLL_DigitalCal( x0 )`

14.141 SYS_GET_PLL_LOOPFILTERANDVCOGAIN

Get the PLL loop filter and VCO gain configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Get_PLL_LoopFilterAndVCOGain( reg )`
Description	Get the PLL loop filter and VCO gain configuration
Inputs	None
Outputs	reg = Register to store the PLL loop filter and VCO gain configuration to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the PLL loop filter and VCO gain settings to x1` `Sys_Get_PLL_LoopFilterAndVCOGain( x1 )`

14.142 SYS_GET_PLL_REFCLKDETECTCONFIG

Get the PLL reference clock detection configuration and status

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Get_PLL_RefClkDetectConfig( reg )`
Description	Get the PLL reference clock detection configuration and status
Inputs	None
Outputs	reg = Register to store the PLL reference clock detection configuration to; compare with CLK_DET_[ENABLE | DISABLE], CLK_[ACTIVE | NOT_ACTIVE], CLK_DET_SLOW_CLK_DIV*, CLK_DET_INT_[DISABLE | ACTIVATED | DEACTIVATED | ACTIVITY_CHANGE], and CLK_DET_INT_[TRUE | FALSE]
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the PLL reference clock detection status to x1 and check if`
	`// the reference clock is currently active, branching to the no clock`
	`// handler if inactive`
	`Sys_Get_PLL_RefClkDetectConfig( x1 )`
	`bittst x1, CLK_DET_STATUS   ||| nop`
	`goto.0d NoClock_Handler, no_carry`

14.143 SYS_GET_PLL_TEMPCOMPENSATION

Get the PLL temperature compensation configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Get_PLL_TempCompensation( reg )`
Description	Get the PLL temperature compensation configuration
Inputs	None
Outputs	reg = Register to store the PLL temperature compensation configuration to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the PLL temperature compensation trim to x1` `Sys_Get_PLL_TempCompensation( x1 )`

14.144 SYS_GET_SDU_CFG

Load the signal detection unit (SDU) configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_Get_SDU_CFG( acc )`
Description	Load the signal detection unit (SDU) configuration
Inputs	None
Outputs	acc = Accumulator to store the SDU configuration to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Store the current SDU configuration setting into b0` `Sys_Get_SDU_CFG( b0 )`

14.145 SYS_GET_SUP_COUNT_DATA

Get the value of control register A_SUP_COUNT_DATA

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_pwr.cfi
Template	`Sys_Get_SUP_COUNT_DATA( reg )`
Description	Get the value of control register A_SUP_COUNT_DATA
Inputs	None
Outputs	reg = Register to store the A_SUP_COUNT_DATA value to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current contents of register A_SUP_COUNT_DATA into x0` `Sys_Get_SUP_COUNT_DATA( x0 )`

14.146 SYS_GET_SUP_THRSH_DATA

Get the value of control register A_SUP_THRSH_DATA

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_pwr.cfi
Template	`Sys_Get_SUP_THRSH_DATA( reg )`
Description	Get the value of control register A_SUP_THRSH_DATA
Inputs	None
Outputs	reg = Register to store the A_SUP_THRSH_DATA value to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load the current contents of register A_SUP_THRSH_DATA into x1` `Sys_Get_SUP_THRSH_DATA( x1 )`

14.147 SYS_GPIO_CM3INTCONFIG

Configure an ARM Cortex-M3 subsystem GPIO interrupt source

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_CM3IntConfig( index, gpio_int )`
Description	Configure an ARM Cortex-M3 subsystem GPIO interrupt source
Inputs	index = GPIO interrupt source to configure; use a constant between 0 and 3 gpio_int = DIO interrupt configuration. Use CM3_DIO_SRC_* and CM3_DIO_EVENT_*
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the GPIO on DIO 5 to generate an interrupt when any`
	`// edge is detected (configured as GPIO interrupt source 1)`
	`Sys_GPIO_CM3IntConfig( 1, CM3_DIO_SRC_DIO_5 | \`
	`CM3_DIO_EVENT_TRANSITION )`

14.148 SYS_GPIO_DIRECTION

Set the GPIO directions

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Direction( gpio_io )`
Description	Set the GPIO directions
Inputs	gpio_io = Bit field for direction (0 - Input, 1 - Output). Use GPIOx_[INPUT | OUTPUT].
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if gpio_io can be sign-extended from 8 bits; 2 if gpio_io can be sign-extended from 24 bits; 3 otherwise

Example	`// Configure GPIOs 0, 2, and 3 as outputs and configure GPIOs 1, 4,`
	`// and 5 as inputs. Leaves the GPIO interrupt configuration intact`
	`// and sets the remaining GPIOs as inputs.`
	`Sys_GPIO_Direction( GPIO0_OUTPUT | GPIO1_INPUT | GPIO2_OUTPUT | \`
	`GPIO3_OUTPUT | GPIO4_INPUT | GPIO5_INPUT )`

14.149 SYS_GPIO_GET_CM3INTSTATUS

Get the current status for the ARM Cortex-M3 subsystem's GPIO (DIO) interrupt; this clears the current status

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Get_CM3IntStatus( reg )`
Description	Get the current status for the ARM Cortex-M3 subsystem's GPIO (DIO) interrupt; this clears the current status
Inputs	None
Outputs	reg = Register to store the current ARM Cortex-M3 subsystem's GPIO (DIO) interrupt status to
Register Modifications	
Assumptions	None
Clock Cycles	1
Example	`// Load x0 with the current ARM Cortex-M3 GPIO interrupt status` `Sys_GPIO_Get_CM3IntStatus( x0 )`

14.150 SYS_GPIO_GET_DATA

Get the GPIO input and store it in the specified accumulator register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Get_Data( reg )`
Description	Get the GPIO input and store it in the specified accumulator register. The returned data reflects the physical data on the GPIO pins.
Inputs	None
Outputs	reg = Accumulator to store the current GPIO input data to
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Read the value from the GPIO inputs and store it in b1` `Sys_GPIO_Get_Data( b1 )`

14.151 SYS_GPIO_GET_DATA_1

Get the GPIO input for the VDDO 1 supported DIOs and store it in the specified register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Get_Data_1( reg )`
Description	Get the GPIO input for the VDDO 1 supported DIOs and store it in the specified register. The returned data reflects the physical data on the GPIO pins.
Inputs	None
Outputs	reg = Register to store the current GPIO VDDO 1 input data to
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Read the value from the VDDO 1 supported GPIO inputs and store it`
	`// in x1`
	`Sys_GPIO_Get_Data_1( x1 )`

14.152 SYS_GPIO_GET_DATA_2

Get the GPIO input for the VDDO 2 supported DIOs and store it in the specified register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Get_Data_2( reg )`
Description	Get the GPIO input for the VDDO 2 supported DIOs and store it in the specified register. The returned data reflects the physical data on the GPIO pins.
Inputs	None
Outputs	reg = Register to store the current GPIO VDDO 2 input data to
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Read the value from the VDDO 2 supported GPIO inputs and store it`
	`// in x1`
	`Sys_GPIO_Get_Data_2( x1 )`

14.153 SYS_GPIO_GET_DATA_3

Get the GPIO input for the VDDO 3 supported DIOs and store it in the specified register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Get_Data_3( reg )`
Description	Get the GPIO input for the VDDO 3 supported DIOs and store it in the specified register. The returned data reflects the physical data on the GPIO pins.
Inputs	None
Outputs	reg = Register to store the current GPIO VDDO 3 input data to
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Read the value from the VDDO 3 supported GPIO inputs and store it`
	`// in x1`
	`Sys_GPIO_Get_Data_3( x1 )`

14.154 SYS_GPIO_GET_INTSTATUS

Get the current status for the CFX GPIO (DIO) interrupt; clears the current status

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Get_IntStatus( reg )`
Description	Get the current status for the CFX GPIO (DIO) interrupt; clears the current status
Inputs	None
Outputs	reg = Register to store the current CFX GPIO (DIO) interrupt status to
Register Modifications	
Assumptions	None
Clock Cycles	1
Example	`// Load x0 with the current CFX GPIO interrupt status` `Sys_GPIO_Get_IntStatus( x0 )`

14.155 SYS_GPIO_INTCONFIG

Configure a CFX GPIO interrupt source

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_IntConfig( index, gpio_int )`
Description	Configure a CFX GPIO interrupt source
Inputs	index = GPIO interrupt source to configure; use a constant between 0 and 3 gpio_int = DIO interrupt configuration. Use CFX_DIO_SRC_* and CFX_DIO_EVENT_*
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the GPIO on DIO 3 to generate an interrupt when a rising`
	`// edge is detected (configured as GPIO interrupt source 0)`
	`Sys_GPIO_IntConfig( 0, CFX_DIO_SRC_DIO_3 | \`
	`CFX_DIO_EVENT_RISING_EDGE )`

14.156 SYS_GPIO_SET_DATA

Set the GPIO output to the specified value

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Set_Data( gpio_data )`
Description	Set the GPIO output to the specified value
Inputs	gpio_data = GPIO pin output data
Outputs	None
Register Modifications	b1
Assumptions	Specified pins set as CFX GPIOs and GPIO direction set to output
Clock Cycles	1 if gpio_data can be sign-extended from 8 bits; 2 if gpio_data can be sign-extended from 24 bits; 3 otherwise
Example	`// Set the GPIO output data to 0x1ABCDEF0.` `Sys_GPIO_Set_Data( 0x1ABCDEF0 )`

14.157 SYS_GPIO_SET_DATA_1

Set the GPIO output for the VDDO 1 supported DIOs to the specified value

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Set_Data_1( gpio_data )`
Description	Set the GPIO output for the VDDO 1 supported DIOs to the specified value
Inputs	gpio_data = GPIO VDDO 1 output data
Outputs	None
Register Modifications	y1
Assumptions	Specified pins set as CFX GPIOs and GPIO direction set to output
Clock Cycles	1 if gpio_data can be sign-extended from 8 bits; 2 otherwise
Example	`// Set GPIO output data for the VDDO 1 DIOs to 0xAB.` `Sys_GPIO_Set_Data_1( 0xAB )`

14.158 SYS_GPIO_SET_DATA_2

Set the GPIO output for the VDDO 2 supported DIOs to the specified value

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Set_Data_2( gpio_data )`
Description	Set the GPIO output for the VDDO 2 supported DIOs to the specified value
Inputs	gpio_data = GPIO VDDO 2 output data
Outputs	None
Register Modifications	y1
Assumptions	Specified pins set as CFX GPIOs and GPIO direction set to output
Clock Cycles	1 if gpio_data can be sign-extended from 8 bits; 2 otherwise
Example	`// Set GPIO output data for the VDDO 2 DIOs to 0xAB.` `Sys_GPIO_Set_Data_2( 0xAB )`

14.159 SYS_GPIO_SET_DATA_3

Set the GPIO output for the VDDO 3 supported DIOs to the specified value

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Set_Data_3( gpio_data )`
Description	Set the GPIO output for the VDDO 3 supported DIOs to the specified value
Inputs	gpio_data = GPIO VDDO 3 output data
Outputs	None
Register Modifications	y1
Assumptions	Specified pins set as CFX GPIOs and GPIO direction set to output
Clock Cycles	1 if gpio_data can be sign-extended from 8 bits; 2 otherwise
Example	`// Set GPIO output data for the VDDO 3 DIOs to 0xAB.` `Sys_GPIO_Set_Data_3( 0xAB )`

14.160 SYS_GPIO_SET_HIGH

Set the specified CFX GPIO output value to high

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Set_High( gpio_pin )`
Description	Set the specified CFX GPIO output value to high
Inputs	gpio_pin = CFX GPIO pin to set high
Outputs	None
Register Modifications	b1
Assumptions	Specified pin set as CFX GPIO and GPIO direction set to output
Clock Cycles	3
Example	`// Set GPIO2 output to high.` `Sys_GPIO_Set_High( 2 )`

14.161 SYS_GPIO_SET_LOW

Set the specified CFX GPIO output value to low

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Set_Low( gpio_pin )`
Description	Set the specified CFX GPIO output value to low
Inputs	gpio_pin = CFX GPIO pin to set low
Outputs	None
Register Modifications	b1
Assumptions	Specified pin set as CFX GPIO and GPIO direction set to output
Clock Cycles	3
Example	`// Set GPIO2 output to low.` `Sys_GPIO_Set_Low( 2 )`

14.162 SYS_GPIO_TOGGLE

Toggle the specified CFX GPIO output pin

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Toggle( gpio_pin )`
Description	Toggle the specified CFX GPIO output pin
Inputs	gpio_pin = CFX GPIO pin to toggle
Outputs	None
Register Modifications	b1
Assumptions	Specified pin set as CFX GPIO and GPIO direction set to output
Clock Cycles	3
Example	`// Toggle GPIO2 output (low becomes high or high becomes low).` `Sys_GPIO_Toggle( 2 )`

14.163 SYS_GPIO_TOGGLE_BITS

Toggle the specified CFX GPIO output pins

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Toggle_Bits( gpio_bitfield )`
Description	Toggle the specified CFX GPIO output pins
Inputs	gpio_bitfield = CFX GPIO pins to toggle (1 - toggle, 0 - do not toggle)
Outputs	None
Register Modifications	b1
Assumptions	Specified pins set as CFX GPIOs and GPIO direction set to output
Clock Cycles	11 if gpio_bitfield can be sign-extended from 24 bits; 12 otherwise

Example	`// Toggle the odd indexed GPIO (DIO) outputs. All low bits become`
	`// high, and all high bits become low. Don't update the even indexed`
	`// GPIO outputs.`
	`Sys_GPIO_Toggle_Bits( 0x2AAAAAAA )`

14.164 SYS_GPIO_TOGGLE_BITS_1

Toggle the specified CFX GPIO output pins supported by VDDO1

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Toggle_Bits_1( gpio_bitfield )`
Description	Toggle the specified CFX GPIO output pins supported by VDDO1
Inputs	gpio_bitfield = CFX GPIO pins to toggle (1 - toggle, 0 - do not toggle)
Outputs	None
Register Modifications	y1, b1
Assumptions	Specified pins set as CFX GPIOs and GPIO direction set to output
Clock Cycles	4

Example	`// Toggle the GPIOs at positions 0, 2 from the set of VDDO 1`
	`// supported GPIO (DIO) outputs. All low bits become high, and all`
	`// high bits become low. Don't update the other GPIO outputs.`
	`Sys_GPIO_Toggle_Bits_1( 0x5 )`

14.165 SYS_GPIO_TOGGLE_BITS_2

Toggle the specified CFX GPIO output pins supported by VDDO2

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Toggle_Bits_2( gpio_bitfield )`
Description	Toggle the specified CFX GPIO output pins supported by VDDO2
Inputs	gpio_bitfield = CFX GPIO pins to toggle (1 - toggle, 0 - do not toggle)
Outputs	None
Register Modifications	y1, b1
Assumptions	Specified pins set as CFX GPIOs and GPIO direction set to output
Clock Cycles	4

Example	`// Toggle the GPIOs at positions 0, 2 from the set of VDDO 2`
	`// supported GPIO (DIO) outputs. All low bits become high, and all`
	`// high bits become low. Don't update the other GPIO outputs.`
	`Sys_GPIO_Toggle_Bits_2( 0x5 )`

14.166 SYS_GPIO_TOGGLE_BITS_3

Toggle the specified CFX GPIO output pins supported by VDDO3

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_gpio.cfi
Template	`Sys_GPIO_Toggle_Bits_3( gpio_bitfield )`
Description	Toggle the specified CFX GPIO output pins supported by VDDO3
Inputs	gpio_bitfield = CFX GPIO pins to toggle (1 - toggle, 0 - do not toggle)
Outputs	None
Register Modifications	y1, b1
Assumptions	Specified pins set as CFX GPIOs and GPIO direction set to output
Clock Cycles	4

Example	`// Toggle the GPIOs at positions 0, 2 from the set of VDDO 3`
	`// supported GPIO (DIO) outputs. All low bits become high, and all`
	`// high bits become low. Don't update the other GPIO outputs.`
	`Sys_GPIO_Toggle_Bits_3( 0x5 )`

14.167 SYS_HEAR_COMMAND

Issue a command to execute a HEAR function chain

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_hear.cfi
Template	`Sys_HEAR_Command( cmd )`
Description	Issue a command to execute a HEAR function chain.
Inputs	cmd = Command to issue to the HEAR; use a constant between 0 and 15 or HEAR_FC_CMD*_POS
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Issue a command to the HEAR to trigger function chain 1` `Sys_HEAR_Command( 1 )`

14.168 SYS_HEAR_FUNCTIONCHAIN_ENABLE

Enable specified HEAR function chains

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_hear.cfi
Template	`Sys_HEAR_FunctionChain_Enable( function_ebl )`
Description	Enable specified HEAR function chains
Inputs	function_ebl = HEAR function chains to enable
Outputs	None
Register Modifications	y1
Assumptions	Previous function chain settings are no longer needed, as they are overwritten with this macro.
Clock Cycles	1 if function_ebl can be sign-extended from 8 bits; 2 otherwise

Example	`// Enable the function chains triggered by the HEAR Configurable`
	`// Accelerator commands 0, 1 and 2 as well as FIFOs 0 and 4.`
	`Sys_HEAR_FunctionChain_Enable( HEAR_FC_EBL_CMD0 |`
	`HEAR_FC_EBL_CMD1 |`
	`HEAR_FC_EBL_CMD2 |`
	`HEAR_FC_EBL_FIFO_0 |`
	`HEAR_FC_EBL_FIFO_4 )`

14.169 SYS_HEAR_FUNCTIONCHAIN_PRIORITY

Set the priority of HEAR function chains

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_hear.cfi
Template	`Sys_HEAR_FunctionChain_Priority( function_prio )`
Description	Set the priority of HEAR function chains
Inputs	function_prio = HEAR function chains priority setting
Outputs	None
Register Modifications	y1
Assumptions	Previous HEAR function chain priority settings are no longer needed as they are overwritten with this macro
Clock Cycles	1 if function_prio can be sign-extended from 8 bits; 2 otherwise

Example	`// Assign high priority to the function chains triggered by`
	`// command 2 and FIFO 0. The remaining function chains default`
	`// to normal priority.`
	`Sys_HEAR_FunctionChain_Priority( HEAR_FC_HIGHPRIO_CMD2 | \`
	`HEAR_FC_HIGHPRIO_FIFO_0 )`

14.170 SYS_HEAR_PAUSE

Pause the HEAR Configurable Accelerator

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_hear.cfi
Template	`Sys_HEAR_Pause`
Description	Pause the HEAR Configurable Accelerator
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Pause the HEAR Configurable Accelerator` `Sys_HEAR_Pause`

14.171 SYS_HEAR_RESET

Reset the HEAR Configurable Accelerator and the FIFO controllers

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_hear.cfi
Template	`Sys_HEAR_Reset`
Description	Reset the HEAR Configurable Accelerator and the FIFO controllers
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Reset the HEAR Configurable Accelerator and FIFO controllers,`
	`// clearing the current FIFO register settings`
	`Sys_HEAR_Reset`

14.172 SYS_HEAR_RUNTIME_ADDR

Calculate the equivalent HEAR address to a CFX address

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_hear.cfi
Template	`Sys_HEAR_RunTime_Addr( memspace, addr )`
Description	Calculate the equivalent HEAR address to a CFX address
Inputs	memspace = CFX memory space; use SYS_MEMSPACE_X or SYS_MEMSPACE_Y addr = CFX address for a shared memory; use D_HEAR_*MEM_BASE, D_HEAR_*MEM_Y_BASE or a constant
Outputs	addr = HEAR address for the specified shared memory
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Calculate the equivalent HEAR address to D_HEAR_A1MEM_BASE and`
	`// store this to the application input 0 address variable`
	`#set ADDR   Sys_HEAR_RunTime_Addr(SYS_MEMSPACE_X, D_HEAR_A1MEM_BASE)`
	`load x0, ADDR`
	`load xp0, app_input0_addr`
	`nop ||| store x[xp0], x0`

14.173 SYS_HEAR_START

Start (or resume) the HEAR Configurable Accelerator

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_hear.cfi
Template	`Sys_HEAR_Start`
Description	Start (or resume) the HEAR Configurable Accelerator
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Start the HEAR Configurable Accelerator` `Sys_HEAR_Start`

14.174 SYS_HMI_DISABLE

Disable all of the Hamming and interleaving components (disables both the encoder and decoder)

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ecb.cfi
Template	`Sys_HMI_Disable`
Description	Disable all of the Hamming and interleaving components (disables both the encoder and decoder)
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	4
Example	`// Disable the Hamming and interleaving components` `Sys_HMI_Disable`

14.175 SYS_HMI_ENCODER_CONFIG

Configure the Hamming and interleaving encoder

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ecb.cfi
Template	`Sys_HMI_Encoder_Config( config )`
Description	Configure the Hamming and interleaving encoder
Inputs	config = The HMI encoder configuration; use HMI_DMA_[3REG | 4REG], HMI_INTERLEAVE_[128B | 126B], HMI_INTERLEAVE_[DISABLE | ENABLE], HMI_PARITY_[DISABLE | ENABLE], and HMI_[DISABLE | ENABLE]
Outputs	None
Register Modifications	y1, b1
Assumptions	None
Clock Cycles	6

Example	`// Enable and configure the Hamming encoder and interleaving`
	`Sys_HMI_Encoder_Config( HMI_INTERLEAVE_128B | \`
	`HMI_INTERLEAVE_ENABLE | \`
	`HMI_PARITY_ENABLE | \`
	`HMI_ENABLE )`

14.176 SYS_I2C_ACK

Manually ACK the latest I^2C byte

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_I2C_ACK`
Description	Manually ACK the latest I^2C byte.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Manually ACK the latest I2C byte` `Sys_I2C_ACK`

14.177 SYS_I2C_CONFIG

Configure the I²C interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_I2C_Config( cfg )`
Description	Configure the I²C interface
Inputs	cfg = I²C interface configuration; use I2C_MASTER_PRESCALE_x, a constant shifted to I2C_CFG_SLAVE_ID_POS, I2C_CFG_STOP_INT_[ENABLE | DISABLE], I2C_CFG_AUTO_ACK_[ENABLE | DISABLE], I2C_CFG_CONNECT_IN_STANDBY_[ENABLE | DISABLE], and I2C_CFG_SLAVE_[ENABLE | DISABLE]
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if cfg can be sign-extended from 8 bits; 2 otherwise

Example	`// Enable the I2C slave and set the address to 0x10. Also enable`
	`// auto-acknowledge mode.`
	`Sys_I2C_Config( I2C_CFG_SLAVE_ENABLE    |  \`
	`  I2C_CFG_AUTO_ACK_ENABLE |  \`
	`  (0x10 << I2C_CFG_SLAVE_ID_POS) )`

14.178 SYS_I2C_NACK

Manually NACK the latest I^2C byte

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_I2C_NACK`
Description	Manually NACK the latest I^2C byte.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Manually NACK the latest I2C byte` `Sys_I2C_NACK`

14.179 SYS_I2C_NACK_AND_STOP

Manually NACK the last I^{2}C byte and issue a stop condition (applies to I^{2}C master mode transfers only)

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_I2C_NACK_And_Stop`
Description	Manually NACK the last I^{2}C byte and issue a stop condition (applies to I^{2}C master mode transfers only)
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Manually NACK the latest I2C byte and issue a stop condition` `Sys_I2C_NACK_And_Stop`

14.180 SYS_I2C_PIN_CONFIG

Configure the I²C interface bus

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_I2C_Pin_Config( cfg )`
Description	Configure the I²C interface bus
Inputs	cfg = Combination of I2C_PIN_CFG_FILTER_[ENABLE | DISABLE], I2C_PIN_CFG_PULL_UP_[250K | 1K], and I2C_PIN_CFG_PULLUP_[ENABLE | DISABLE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the I2C bus behavior`
	`Sys_I2C_Pin_Config( I2C_PIN_CFG_FILTER_ENABLE \ \`
	`I2C_PIN_CFG_PULLUP_1K \ \`
	`I2C_PIN_CFG_PULLUP_ENABLE )`

14.181 SYS_I2C_RESET

Reset the general purpose I^2C interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_I2C_Reset`
Description	Reset the general purpose I^2C interface.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Reset the general-purpose I2C interface.` `Sys_I2C_Reset`

14.182 SYS_I2C_SLAVE_DISABLE

Disable the I²C interface by setting the I²C_SLAVE_ENABLE bit

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_I2C_Slave_Disable`
Description	Disable the I²C interface by setting the I²C_SLAVE_ENABLE bit. The I²C interface configuration is unchanged.
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	3

Example	`// Disable the I2C interface slave mode, preventing slave mode`
	`// transfers`
	`Sys_I2C_Slave_Disable`

14.183 SYS_I2C_SLAVE_ENABLE

Enable the I²C interface by setting the I²C_SLAVE_ENABLE bit

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_I2C_Slave_Enable`
Description	Enable the I²C interface by setting the I²C_SLAVE_ENABLE bit. The I²C interface configuration is unchanged.
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	3
Example	`// Enable the I2C interface for slave mode transfers` `Sys_I2C_Slave_Enable`

14.184 SYS_I2C_START_READ

Start an I^2C read transfer (applies to I^2C master mode transfers only)

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_I2C_Start_Read( addr )`
Description	Start an I^2C read transfer (applies to I^2C master mode transfers only)
Inputs	addr = Address of remote I^2C slave device
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Start an I2C master mode transfer to read from a device`
	`// connected to the bus at I2C_ADDRESS.`
	`Sys_I2C_Start_Read( I2C_ADDRESS )`

14.185 SYS_I2C_START_WRITE

Start an I^2C write transfer (applies to I^2C master mode transfers only)

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_I2C_Start_Write( addr )`
Description	Start an I^2C write transfer (applies to I^2C master mode transfers only)
Inputs	addr = Address of remote I^2C slave device
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Start an I2C master mode transfer to write to any devices`
	`// connected to the bus at I2C_ADDRESS.`
	`Sys_I2C_Start_Write( I2C_ADDRESS )`

14.186 SYS_I2C_WATCHDOG_CONFIG

Configure the I^{2}C interface bus watchdog timeout

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_i2c.cfi
Template	`Sys_I2C_Watchdog_Config( cfg )`
Description	Configure the I^{2}C interface bus watchdog timeout
Inputs	cfg = I2C_WATCHDOG_TIMEOUT_[250M | 500M | 1000M | 2000M | 4000M | DISABLED]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Configure the I2C bus watchdog timeout for 1 second` `Sys_I2C_Watchdog_Config( I2C_WATCHDOG_TIMEOUT_1000M )`

14.187 SYS_IHM_DECODER_CLEAR_STATUS

Clear the data, and hence the status, for the Hamming decoder and de-interleaver

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ecb.cfi
Template	`Sys_IHM_Decoder_Clear_Status`
Description	Clear the data, and hence the status, for the Hamming decoder and de-interleaver
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	4
Example	`// Clear the Hamming decoder and de-interleaving status` `Sys_IHM_Decoder_Clear_Status`

14.188 SYS_IHM_DECODER_CONFIG

Configure the Hamming and interleaving decoder

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ecb.cfi
Template	`Sys_IHM_Decoder_Config( config )`
Description	Configure the Hamming and interleaving decoder
Inputs	config = The IHM decoder configuration; use IHM_DMA_[3REG | 4REG]_BYTE, IHM_INTERLEAVE_[128B | 126B]_BYTE, IHM_INTERLEAVE_[DISABLE | ENABLE]_BYTE, IHM_COR_[DISABLE | ENABLE]_BYTE, and IHM_[DISABLE | ENABLE]_BYTE
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	3

Example	`// Enable and configure the Hamming decoder and de-interleaving`
	`Sys_IHM_Decoder_Config( IHM_INTERLEAVE_128B | \`
	`IHM_INTERLEAVE_ENABLE | \`
	`IHM_COR_ENABLE | \`
	`IHM_ENABLE )`

14.189 SYS_IMM_N

Sign-extend a constant expression from its n LSBs, for n = 1, 2, ..., 32

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_IMM_n( const )`
Description	Sign-extend a constant expression from its n LSBs, for n = 1, 2, ..., 32. The result is in the range of a signed n-bit number so it fits into an n-bit (or larger) signed immediate field.
Inputs	const = Constant expression to convert
Outputs	Sys_IMM_n = Constant expression with the same n LSBs as the input, but sign-extended from bit n-1 (where the LSB is bit 0) so that the result is in the signed n-bit range
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)
Example	

```
// Sign-extend a 16-bit initializer. Since all bits above bit 7
// are the same as bit 7, this value can be stored using the
// signed 8-bit immediate field of the store-immediate instruction.
store p[D_CRC_VALUE], Sys_IMM_16( 0xFFFF )

// Load 0.125 (converted to the <1,47> fractional format) into a0.
// Sys_IMM_24 extracts the low 24 bits of the 48-bit number as a
// signed 24-bit value to store into a0.l.
#set VALUE Sys_frac48( 1250000 )
#set VAL_H VALUE >> 24
#set VAL_L Sys_IMM_24( VALUE )
load a0, VAL_H
load a0.l, VAL_L

// Test whether a constant expression in a macro parameter can be
// stored into a 12-bit control register using a store-immediate
// instruction (signed 8-bit immediate field).
#if Sys_IMM_8( param ) == Sys_IMM_12( param )
    // Bits 7-11 are all the same so store-immediate can be used
    store p[CONTROL_REG], Sys_IMM_8( param )
#else
    // Value cannot be represented in 8 bits, so use a register
    load y1, (param)
    store p[CONTROL_REG], y1
#endif
```

14.190 SYS_INT_ACK

Acknowledge a set of interrupts

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_int.cfi
Template	`Sys_Int_Ack( int_ack )`
Description	Acknowledge a set of interrupts
Inputs	int_ack = Interrupts to acknowledge
Outputs	None
Register Modifications	b1
Assumptions	Previous interrupt status is no longer needed, as it is overwritten with this macro
Clock Cycles	1 if int_ack can be sign-extended from 8 bits; 2 if int_ack can be sign-extended from 24 bits; 3 otherwise
Example	`// Manually acknowledge timer 1 and PCM interrupts` `Sys_Int_Ack( INT_ACK_TIMER_1 | INT_ACK_PCM )`

14.191 SYS_INT_ENABLE

Enable a set of interrupts

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_int.cfi
Template	`Sys_Int_Enable( int_ebl )`
Description	Enable a set of interrupts
Inputs	int_ebl = Interrupts to enable
Outputs	None
Register Modifications	b1
Assumptions	Previous interrupt enable settings are no longer needed, as they are overwritten with this macro
Clock Cycles	1 if int_ebl can be sign-extended from 8 bits; 2 if int_ebl can be sign-extended from 24 bits; 3 otherwise

Example	`// Enable interrupts for watchdog, timer 1, and FIFO interrupt 0.`
	`// NOTE: Master interrupts and core interrupts must be enabled to`
	`//  receive interrupts`
	`Sys_Int_Enable( INT_EBL_MASTER  | INT_EBL_WATCHDOG | \`
	`  INT_EBL_TIMER_1 | INT_EBL_FIFO_0 )`

14.192 SYS_INT_MASTER_DISABLE

Disable all interrupts at the interrupt controller

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_int.cfi
Template	`Sys_Int_Master_Disable`
Description	Disable all interrupts at the interrupt controller
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`Critical_Section_Start:`
	`// Disables all interrupts at the interrupt controller level`
	`Sys_Int_Master_Disable`
	
	`// Do processing with interrupts disabled`
	`Critical_Section_End:`
	
	`// Re-enable previously disabled interrupts`
	`Sys_Int_Master_Enable`

14.193 SYS_INT_MASTER_ENABLE

Enable all interrupts at the interrupt controller

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_int.cfi
Template	`Sys_Int_Master_Enable`
Description	Enable all interrupts at the interrupt controller
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`Critical_Section_Start2:`
	`// Disables all interrupts at the interrupt controller level`
	`Sys_Int_Master_Disable`
	
	`// Do processing with interrupts disabled`
	`Critical_Section_End2:`
	
	`// Re-enable previously disabled interrupts`
	`Sys_Int_Master_Enable`

14.194 SYS_INT_MUXCONFIG

Configure the interrupt multiplexing

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_int.cfi
Template	`Sys_Int_MuxConfig( config )`
Description	Configure the interrupt multiplexing
Inputs	config = Interrupt multiplexing configuration; use INT_SEL_[HEAR_5 | UART], INT_SEL_[HEAR_6 | FENG], and INT_SEL_[HEAR_7 | SDU]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the multiplexed CFX interrupts to support the`
	`// Filter Engine interrupt`
	`Sys_Int_MuxConfig( INT_SEL_FENG | \`
	`INT_SEL_HEAR_5 | \`
	`INT_SEL_HEAR_7 )`

14.195 SYS_INT_PRIORITY

Configure the interrupt priority

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_int.cfi
Template	`Sys_Int_Priority( int_prio )`
Description	Configure the interrupt priority
Inputs	int_prio = Interrupt priority configuration
Outputs	None
Register Modifications	b1
Assumptions	Previous interrupt priority settings are no longer needed, as they are overwritten with this macro
Clock Cycles	1 if int_prio can be sign-extended from 8 bits; 2 if int_prio can be sign-extended from 24 bits; 3 otherwise
Example	`// Give high priority to the FIFO 0 and HEAR 0 interrupts` `Sys_Int_Priority( INT_HIGHPRIO_FIFO_0 | INT_HIGHPRIO_HEAR_0 )`

14.196 SYS_IOC_ADCINPUTCONFIG

Set up the IOC ADC input configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ioc.cfi
Template	`Sys_IOC_ADCInputConfig( arg )`
Description	Set up the IOC ADC input configuration
Inputs	arg = Combination of IOC_INPUT_CFG_ADC0_*, IOC_INPUT_CFG_ADC1_*, IOC_INPUT_CFG_ADC2_*, and IOC_INPUT_CFG_ADC3_*
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if arg can be sign-extended from 8 bits; 3 otherwise

Example	`// Connect ADC 0 to FIFO A0_0 and ADC3 FIFO A1_3`
	`#define ADC_INPUT_CFG_VAL ( IOC_INPUT_CFG_ADC0_FA0_0 | \`
	`IOC_INPUT_CFG_ADC1_NONE | \`
	`IOC_INPUT_CFG_ADC2_NONE | \`
	`IOC_INPUT_CFG_ADC3_FA1_3 )`
	
	`// Configure how the IOC handles input data for the ADCs`
	`Sys_IOC_ADCInputConfig( ADC_INPUT_CFG_VAL )`

14.197 SYS_IOC_FIFOACCESSCONFIG

Set up the IOC FIFO access configuration for each FIFO; access is either one or two data values at a time

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ioc.cfi
Template	`Sys_IOC_FIFOAccessConfig( arg )`
Description	Set up the IOC FIFO access configuration for each FIFO; access is either one or two data values at a time
Inputs	arg = A set of FIFO_**_*_DBL_ACC_[DIS | EN]
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if arg can be sign-extended from 8 bits; 3 otherwise
Example	`// Enable double access for all FIFOs` `Sys_IOC_FIFOAccessConfig( 0xFFFFFFFF )`

14.198 SYS_IOC_FIFOSHIFTCONFIG

Set up the IOC FIFO data alignment for ADCs and DACs

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ioc.cfi
Template	`Sys_IOC_FIFOShiftConfig( arg )`
Description	Set up the IOC FIFO data alignment for ADCs and DACs
Inputs	arg = Combination of IOC_FIFO_ADC0_SHIFT_[RIGHT | LEFT], IOC_FIFO_ADC1_SHIFT_[RIGHT | LEFT], IOC_FIFO_ADC2_SHIFT_[RIGHT | LEFT], IOC_FIFO_ADC3_SHIFT_[RIGHT | LEFT], IOC_FIFO_DAC0_SHIFT_[RIGHT | LEFT], and IOC_FIFO_DAC1_SHIFT_[RIGHT | LEFT]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Use a right alignment for all FIFO data`
	`#define IOC_SHIFT_CFG_VAL ( IOC_FIFO_ADC0_SHIFT_RIGHT | \`
	`IOC_FIFO_ADC1_SHIFT_RIGHT | \`
	`IOC_FIFO_ADC2_SHIFT_RIGHT | \`
	`IOC_FIFO_ADC3_SHIFT_RIGHT | \`
	`IOC_FIFO_DAC0_SHIFT_RIGHT | \`
	`IOC_FIFO_DAC1_SHIFT_RIGHT )`
	
	`Sys_IOC_FIFOShiftConfig( IOC_SHIFT_CFG_VAL )`

14.199 SYS_IOC_OUTPUTCONFIG

Set up the IOC output configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ioc.cfi
Template	`Sys_IOC_OutputConfig( arg )`
Description	Set up the IOC output configuration
Inputs	arg = Combination of IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC_[EN | DIS], IOC_OUTPUT_CFG_PCM1_*, IOC_OUTPUT_CFG_PCM0_*, IOC_OUTPUT_CFG_DAC1_*, IOC_OUTPUT_CFG_DAC0_*
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if arg can be sign-extended from 8 bits; 3 otherwise

Example	`// Connect the DACs to FIFO B0_0`
	`#define IOC_OUTPUT_CFG_VAL ( IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC_EN | \`
	`IOC_OUTPUT_CFG_DAC0_FB0_0 | \`
	`IOC_OUTPUT_CFG_DAC1_FB0_0 | \`
	`IOC_OUTPUT_CFG_PCM0_NONE | \`
	`IOC_OUTPUT_CFG_PCM1_NONE )`
	`Sys_IOC_OutputConfig( IOC_OUTPUT_CFG_VAL )`

14.200 SYS_IOC_PCMINPUTCONFIG

Set up the IOC PCM input configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ioc.cfi
Template	`Sys_IOC_PCMInputConfig( arg )`
Description	Set up the IOC PCM input configuration
Inputs	arg = Combination of IOC_INPUT_CFG_PCM0_*, IOC_INPUT_CFG_PCM1_*
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if arg can be sign-extended from 8 bits; 2 otherwise

Example	`// Connect both PCM channels to FIFO B1_0`
	`#define PCM_INPUT_CFG_VAL ( IOC_INPUT_CFG_PCM0_FB1_0 | \`
	`IOC_INPUT_CFG_PCM1_FB1_0 )`
	
	`// Configure how the IOC handles input data for the PCM`
	`Sys_IOC_PCMInputConfig( PCM_INPUT_CFG_VAL )`

14.201 SYS_LOG2

Calculate the integer log base-2 (ceiling, always positive) for a supplied constant

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Log2( const )`
Description	Calculate the integer log base-2 (ceiling, always positive) for a supplied constant. The minimum value for the constant is 1. The maximum value is 0x1FFFFFFFFF when the macro is evaluated by the preprocessor (in #set or #if) and 0x7FFFFFF when the macro is evaluated by the assembler (in an instruction or a directive).
Inputs	const = Calculate the integer log base-2 for this value
Outputs	Sys_Log2 = Approximation of log2(const), rounded up to the nearest integer value and always positive; Sys_Log2(1) is 1, not 0.
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)
Example	`// Load the value of log base-2 of 0x1000 to x0 (x0 = 0xC)` `load x0, Sys_Log2( 0x1000 )`

14.202 SYS_LOG2_CEIL

Calculate the integer log base-2 (ceiling) for a supplied constant

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Log2_Ceil( const )`
Description	Calculate the integer log base-2 (ceiling) for a supplied constant. The minimum value for the constant is 1. The maximum value is 0x1FFFFFFFFF when the macro is evaluated by the preprocessor (in #set or #if) and 0x7FFFFFF when the macro is evaluated by the assembler (in an instruction or a directive).
Inputs	const = Calculate ceiling (log base-2) for this value
Outputs	Sys_Log2_Ceil = log2(const), rounded up to the nearest integer value.
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Load the value of log base-2 (ceiling) of 0x1000 to x0 (x0 = 0xC)`
	`load x0, Sys_Log2_Ceil( 0x1000 )`
	
	`// Load the value of log base-2 (ceiling) of 0x1001 to x1 (x1 = 0xD)`
	`load x1, Sys_Log2_Ceil( 0x1001 )`

14.203 SYS_LOG2_FLOOR

Calculate the integer log base-2 (floor) for a supplied constant

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Log2_Floor( const )`
Description	Calculate the integer log base-2 (floor) for a supplied constant. The minimum value for the constant is 0. The maximum value is 0x1FFFFFFFFF when the macro is evaluated by the preprocessor (in #set or #if) and 0x7FFFFFF when the macro is evaluated by the assembler (in an instruction or a directive).
Inputs	const = Calculate floor (log base-2) for this value
Outputs	Sys_Log2_Floor = log2(const), rounded down to the nearest integer value. When the input is 0, the output is -1.
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Load the value of log base-2 (floor) of 0x1000 to x0 (x0 = 0xC)`
	`load x0, Sys_Log2_Floor( 0x1000 )`
	
	`// Load the value of log base-2 (floor) of 0x1001 to x1 (x1 = 0xC)`
	`load x1, Sys_Log2_Floor( 0x1001 )`

14.204 SYS_M24P24

Convert a decimal number with up to seven digits of precision to <24,24> fixed-point format

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_m24p24( const )`
Description	Convert a decimal number with up to seven digits of precision to <24,24> fixed-point format. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 7.125 becomes 71250000. The 10^7 multiplier provides 23 fractional bits of accuracy. The valid input range (after pre-multiplication) for <24,24> fixed-point numbers is -83886080000000 to +83886079999999. Numbers outside this range are saturated to the closest valid value, but the results are undefined for inputs with magnitude greater than about 2^62 or 4 x 10^18 (representing numbers greater than about 4 x 10^11). This macro is useful for filling accumulators through a pair of immediate loads and for defining <24,24> data to be loaded from XMEM, YMEM or XYMEM.
Inputs	const = Constant to convert to <24,24>, multiplied by 10^7
Outputs	Sys_m24p24 = <24,24> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^62
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 7.125 converted to the <24,24> fixed-point format`
	`// and then store the two halves of the 48-bit value in memory at`
	`// Direct_Data_Value_m24p24.`
	`#set VALUE Sys_m24p24( 71250000 )`
	`#set VAL_H VALUE >> 24`
	`#set VAL_L Sys_IMM_24( VALUE )`
	`Direct_Data_Value_m24p24:`
	`.data VAL_H, VAL_L`

14.205 SYS_M2P22

Convert a decimal number with up to seven digits of precision to <2,22> fixed-point format

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_m2p22( const )`
Description	Convert a decimal number with up to seven digits of precision to <2,22> fixed-point format. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 1.125 becomes 11250000. The valid input range (after pre-multiplication) for <2,22> fractional numbers is -20000000 to +19999999. Numbers outside this range are saturated to the closest valid value, but the results are undefined for inputs with magnitude greater than about 2^46 or 70 x 10^12 (representing numbers greater than about 7000000.0). This macro is useful for filling data registers through an immediate load and for defining <2,22> fixed-point data to be loaded from memory. Data in this format can be used as filter coefficients for the BIQ instruction of the Filter Engine.
Inputs	const = Constant to convert to <2,22>, multiplied by 10^7
Outputs	Sys_m2p22 = <2,22> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^46
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 1.125 converted to the <2,22> fixed-point format`
	`// and then store to x0`
	`#set VALUE Sys_m2p22( 11250000 )`
	`load x0, VALUE`

14.206 SYS_M2P46

Convert a decimal number with up to seven digits of precision to <2,46> fixed-point format

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_m2p46( const )`
Description	Convert a decimal number with up to seven digits of precision to <2,46> fixed-point format. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 1.125 becomes 11250000. The 10^7 multiplier provides 23 fractional bits of accuracy. The valid input range (after pre-multiplication) for <2,46> fixed-point numbers is -20000000 to +19999999. Numbers outside this range are saturated to the closest valid value, but the results are undefined for inputs with magnitude greater than about 2^40 or 10^12 (representing numbers greater than about 100000.0). NOTE: This macro places the number in a <2,46> field, but provides only <2,45> accuracy. The bottom bit of an unsaturated result is always zero. This macro is useful for filling accumulators through a pair of immediate loads and for defining <2,46> data to be loaded from XMEM, YMEM or XYMEM.
Inputs	const = Constant to convert to <2,46>, multiplied by 10^7
Outputs	Sys_m2p46 = <2,46> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^40
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 1.125 converted to the <2,46> fixed-point format`
	`// and then store the two halves of the 48-bit value in memory at`
	`// Direct_Data_Value_m2p46.`
	`#set VALUE Sys_m2p46( 11250000 )`
	`#set VAL_H VALUE >> 24`
	`#set VAL_L Sys_IMM_24( VALUE )`
	`Direct_Data_Value_m2p46:`
	`.data VAL_H, VAL_L`

14.207 SYS_M8P16

Convert a decimal number with up to seven digits of precision to <8,16> fixed-point format

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_m8p16( const )`
Description	Convert a decimal number with up to seven digits of precision to <8,16> fixed-point format. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 7.125 becomes 71250000. The 10^7 multiplier provides 23 fractional bits of accuracy, although the <8,16> format only stores 16 fractional bits. The valid input range (after pre-multiplication) for <8,16> fixed-point numbers is -1280000000 to +1279999999. Numbers outside this range are saturated to the closest valid value, but the results are undefined for inputs with magnitude greater than about 2^53 or 9 x 10^15 (representing numbers greater than about 900000000.0). This macro is useful for filling data registers through an immediate load and for defining <8,16> data to be loaded from XMEM or YMEM.
Inputs	const = Constant to convert to <8,16>, multiplied by 10^7
Outputs	Sys_m8p16 = <8,16> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^53
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 7.125 converted to the <8,16> fixed-point format`
	`// and then store this to a0.`
	`#set VALUE Sys_m8p16( 71250000 )`
	`load a0, VALUE`

14.208 SYS_MEM_CONFIG

Set either the CFX or the ARM Cortex-M3 as the PRAM0 memory instance controller

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_Config( arg )`
Description	Set either the CFX or the ARM Cortex-M3 as the PRAM0 memory instance controller
Inputs	arg = Either SYSTEM2_PRAM0_CFX or SYSTEM2_PRAM0_CM3
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the CFX DSP core to control and use the PRAM0 memory`
	`// instance`
	`Sys_Mem_Config( SYSTEM2_PRAM0_CFX )`

14.209 SYS_MEM_COPYPP

Copy data from program memory to program memory

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_CopyPP( src, dest, len )`
Description	Copy data from program memory to program memory
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	b1, yp2, yp3
Assumptions	The hardware loop stack is not full. The destination data block starts below the source data block or is located above the source data block.
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from program memory at address 0x8500 to`
	`// program memory at address 0x8600.`
	`Sys_Mem_CopyPP( 0x8500, 0x8600, 100 )`

14.210 SYS_MEM_COPYPX

Copy data from program memory to X memory

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_CopyPX( src, dest, len )`
Description	Copy data from program memory to X memory. Data passes through the saturation unit.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	b1, yp3, xp7
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from program memory at address 0x8500 to`
	`// X memory at address 0x1000.`
	`Sys_Mem_CopyPX( 0x8500, 0x1000, 100 )`

14.211 SYS_MEM_COPYPY

Copy data from program memory to Y memory

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_CopyPY( src, dest, len )`
Description	Copy data from program memory to Y memory. Data passes through the saturation unit.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	b1, yp2, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from program memory at address 0x8500 to`
	`// Y memory at address 0x0200.`
	`Sys_Mem_CopyPY( 0x8500, 0x0200, 100 )`

14.212 SYS_MEM_COPYXP

Copy data from X memory to program memory

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_CopyXP( src, dest, len )`
Description	Copy data from X memory to program memory. Data is sign-extended when copied.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	b1, yp3, xp7
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from X memory at address 0x1000 to`
	`// program memory at address 0x8500.`
	`Sys_Mem_CopyXP( 0x1000, 0x8500, 100 )`

14.213 SYS_MEM_COPYXX

Copy data from X memory to X memory

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_CopyXX( src, dest, len )`
Description	Copy data from X memory to X memory.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	y1, xp6, xp7
Assumptions	The hardware loop stack is not full. The destination data block starts below the source data block or is located above the source data block.
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from X memory at address 0x1000 to`
	`// X memory at address 0x1100.`
	`Sys_Mem_CopyXX( 0x1000, 0x1100, 100 )`

14.214 SYS_MEM_COPYXY

Copy data from X memory to Y memory

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_CopyXY( src, dest, len )`
Description	Copy data from X memory to Y memory.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	y1, xp7, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from X memory at address 0x1000 to Y memory at`
	`// address 0x0200. NOTE: this macro has a smaller P memory`
	`// footprint than Sys_Mem_CopyXY_Quick, but uses (3 + 200) cycles.`
	`Sys_Mem_CopyXY( 0x1000, 0x0200, 100 )`

14.215 SYS_MEM_COPYXY_QUICK

Copy data from X memory to Y memory

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_CopyXY_Quick( src, dest, len )`
Description	Copy data from X memory to Y memory. A dual-move instruction allows one word to be copied per clock cycle. This implementation has a more efficient run time for len>6, at the cost of 7 extra P-memory words.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 3, maximum 4098)
Outputs	None
Register Modifications	b1, y1, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	9 + len

Example	`// Copy 100 words from X memory at address 0x1000 to Y memory at`
	`// address 0x0200. NOTE: 7 extra P memory words are required compared`
	`// to Sys_Mem_CopyXY, but only (9 + 100) cycles are needed.`
	`Sys_Mem_CopyXY_Quick( 0x1000, 0x0200, 100 )`

14.216 SYS_MEM_COPYYP

Copy data from Y memory to program memory

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_CopyYP( src, dest, len )`
Description	Copy data from Y memory to program memory. Data is sign-extended when copied.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	b1, yp2, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from Y memory at address 0x0200 to`
	`// program memory at address 0x8500.`
	`Sys_Mem_CopyYP( 0x0200, 0x8500, 100 )`

14.217 SYS_MEM_COPYYX

Copy data from Y memory to X memory

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_CopyYX( src, dest, len )`
Description	Copy data from Y memory to X memory.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	y1, xp7, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from Y memory at address 0x0200 to X memory at`
	`// address 0x1000. NOTE: this macro has a smaller P memory`
	`// footprint than Sys_Mem_CopyYX_Quick, but uses (3 + 200) cycles.`
	`Sys_Mem_CopyYX( 0x0200, 0x1000, 100 )`

14.218 SYS_MEM_COPYYX_QUICK

Copy data from Y memory to X memory

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_CopyYX_Quick( src, dest, len )`
Description	Copy data from Y memory to X memory. A dual-move instruction allows one word to be copied per clock cycle. This implementation has a more efficient run time for len>6, at the cost of 7 extra P-memory words.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 3, maximum 4098)
Outputs	None
Register Modifications	b1, y1, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	9 + len

Example	`// Copy 100 words from Y memory at address 0x0200 to X memory at`
	`// address 0x1000. NOTE: 7 extra P memory words are required compared`
	`// to Sys_Mem_CopyYX, but only (9 + 100) cycles are needed.`
	`Sys_Mem_CopyYX_Quick( 0x0200, 0x1000, 100 )`

14.219 SYS_MEM_COPYYY

Copy data from Y memory to Y memory

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_CopyYY( src, dest, len )`
Description	Copy data from Y memory to Y memory.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	y1, yp2, yp3
Assumptions	The hardware loop stack is not full. The destination data block starts below the source data block or is located above the source data block.
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from Y memory at address 0x0200 to`
	`// Y memory at address 0x0300.`
	`Sys_Mem_CopyYY( 0x0200, 0x0300, 100 )`

14.220 SYS_MEM_FILLP

Fill a range of program memory with a value

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_FillP( value, addr, len )`
Description	Fill a range of program memory with a value.
Inputs	value = Value to be written to memory addr = Destination address len = Number of words to fill (minimum 1, maximum 4096)
Outputs	None
Register Modifications	b1, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	4 + len

Example	`// Fill program memory starting at address 0x8500 for 100 words with`
	`// the value 0x12345678`
	`Sys_Mem_FillP( 0x12345678, 0x8500, 100 )`

14.221 SYS_MEM_FILLX

Fill a range of X memory with a value

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_FillX( value, addr, len )`
Description	Fill a range of X memory with a value.
Inputs	value = Value to be written to memory addr = Destination address len = Number of words to fill (minimum 1, maximum 4096)
Outputs	None
Register Modifications	y1, xp7
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + len

Example	`// Fill X memory starting at address 0x1500 for 100 words with`
	`// the value 0x123456`
	`Sys_Mem_FillX( 0x123456, 0x1500, 100 )`

14.222 SYS_MEM_FILLY

Fill a range of Y memory with a value

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_FillY( value, addr, len )`
Description	Fill a range of Y memory with a value.
Inputs	value = Value to be written to memory addr = Destination address len = Number of words to fill (minimum 1, maximum 4096)
Outputs	None
Register Modifications	y1, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + len

Example	`// Fill Y memory starting at address 0x0200 for 100 words with`
	`// the value 0x123456`
	`Sys_Mem_FillY( 0x123456, 0x0200, 100 )`

14.223 SYS_MEM_PRIORITY

Set either the CFX or the HEAR as having higher priority when accessing shared memories

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_Priority( arg )`
Description	Set either the CFX or the HEAR as having higher priority when accessing shared memories.
Inputs	arg = Either SYSTEM_PRIORITY_CFX or SYSTEM_PRIORITY_HEAR
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Give the CFX DSP core priority access to memory in case a memory`
	`// access conflict occurs with the HEAR Configurable Accelerator`
	`Sys_Mem_Priority( SYSTEM_PRIORITY_CFX )`

14.224 SYS_MEM_PRIORITY2

Set either the CFX or the ARM Cortex-M3 as having higher priority when accessing shared memories

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_Priority2( arg )`
Description	Set either the CFX or the ARM Cortex-M3 as having higher priority when accessing shared memories.
Inputs	arg = Either SYSTEM2_PRIORITY_CFX or SYSTEM2_PRIORITY_CM3
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Give the CFX DSP core priority access to memory in case a memory`
	`// access conflict occurs with the ARM Cortex-M3 processor`
	`Sys_Mem_Priority2( SYSTEM2_PRIORITY_CFX )`

14.225 SYS_MEM_RESERVE

Reserve uninitialized memory in the current memory segment

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_Reserve( name, size )`
Description	Reserve uninitialized memory in the current memory segment
Inputs	name = Global label to give to the reserved memory size = Number of uninitialized memory words to reserve
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Reserve a block of 20 uninitialized words in X memory.`
	`Sys_Mem_Start_Segment( Table_Seg_Uninit, SYS_MEMSPACE_X )`
	`Sys_Mem_Reserve( Table_Uninitialized, 20 )`

14.226 SYS_MEM_RESERVE_INIT

Reserve memory in the current memory segment and initialize with the specified value

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_Reserve_Init( name, size, value )`
Description	Reserve memory in the current memory segment and initialize with the specified value
Inputs	name = Global label to give to the reserved memory size = Number of initialized memory words to reserve value = Data value to store into the reserved memory; to initialize XY memory use the Sys_Value_XY macro when setting this parameter
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Reserve a block of 20 words in X memory. Initialize the memory to`
	`// 0x123456.`
	`Sys_Mem_Start_Segment( Table_Seg, SYS_MEMSPACE_X )`
	`Sys_Mem_Reserve_Init( Table_Initialized, 20, 0x123456 )`

14.227 SYS_MEM_RESERVE_INIT_DEFAULT

Reserve memory in the current memory segment and initialize with 0

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_Reserve_Init_Default( name, size )`
Description	Reserve memory in the current memory segment and initialize with 0
Inputs	name = Global label to give to the reserved memory size = Number of initialized-to-zero memory words to reserve
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Reserve a block of 20 words in X memory and initialize it to 0`
	`Sys_Mem_Start_Segment( Table_Seg_Default, SYS_MEMSPACE_X )`
	`Sys_Mem_Reserve_Init_Default( Table_Default, 20 )`

14.228 SYS_MEM_START_ABSOLUTE_SEGMENT

Start an absolute memory segment

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_Start_Absolute_Segment( name, mem_space, addr )`
Description	Start an absolute memory segment
Inputs	name = Name for the memory segment mem_space = Memory space for the segment addr = Absolute starting address for this segment
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Create a segment for data at a known location in XMEM`
	`#define TABLE_START_ADDR 0x0200`
	`Sys_Mem_Start_Absolute_Segment( Table_A, SYS_MEMSPACE_X, \`
	`TABLE_START_ADDR )`
	`.global Table_A_Data`
	`Table_A_Data:`
	`.data 0x000000`
	`.data 0x000010`
	`.data 0x000012`
	`.data 0x0000FF`
	
	`// Create a segment for code at a known location in PMEM`
	`#define START_CODE_ADDR 0x1200`
	`Sys_Mem_Start_Absolute_Segment( Start_Code, SYS_MEMSPACE_P, \`
	`START_CODE_ADDR )`
	`Start_Code:`
	`nop >>> nop`

14.229 SYS_MEM_START_RELOC_SEGMENT

Start a relocatable memory segment

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_Start_Reloc_Segment( name, mem_space, addr )`
Description	Start a relocatable memory segment
Inputs	name = Name for the memory segment mem_space = Memory space for the segment addr = Base (lower-bound) starting address for this segment
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Create a relocatable segment for data at or above a specified`
	`// base address`
	`#define TABLE_START_BASE_ADDR 0x0200`
	`Sys_Mem_Start_Reloc_Segment( Table_B, SYS_MEMSPACE_X, \`
	`TABLE_START_BASE_ADDR )`
	`.global Table_B_Data`
	`Table_B_Data:`
	`.data 0x000000`
	`.data 0x000010`
	`.data 0x000012`
	`.data 0x0000FF`
	
	`// Create a relocatable code segment at or above address 0x1200`
	`#define CODE_BASE 0x1200`
	`Sys_Mem_Start_Reloc_Segment( Reloc_Code, SYS_MEMSPACE_P, \`
	`CODE_BASE )`
	`.global Reloc_Code`
	`Reloc_Code:`
	`nop >>> nop`

14.230 SYS_MEM_START_SEGMENT

Start a relocatable memory segment

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_mem.cfi
Template	`Sys_Mem_Start_Segment( name, mem_space )`
Description	Start a relocatable memory segment
Inputs	name = Name for the memory segment mem_space = Memory space for the segment
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Create a segment for the start of the application`
	`Sys_Mem_Start_Segment( main, SYS_MEMSPACE_P )`
	`.global main`
	`main:`
	`  Sys_Watchdog_Refresh`
	`  goto.0d main`
	
	`// Create a segment for a data table in X memory`
	`Sys_Mem_Start_Segment( datatable, SYS_MEMSPACE_X )`
	`.global datatable`
	`datatable:`
	`.data 0x000001`
	`.data 0x000001`
	`.data 0x000002`
	`.data 0x000003`
	`.data 0x000005`

14.231 SYS_PCM_CONFIG

Configure the PCM interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_pcm.cfi
Template	`Sys_PCM_Config( pcm_cfg )`
Description	Configure the PCM interface
Inputs	pcm_cfg = Configuration consisting of: PCM_[MASTER | SLAVE], PCM_[ENABLE | DISABLE], PCM_SUBFRAME_[ENABLE | DISABLE], PCM_BIT_ORDER_[LSB | MSB]_FIRST, PCM_FRAME_ALIGN_[LAST | FIRST]_BIT, PCM_FRAME_WIDTH_[SHORT | LONG], PCM_RX_INT_[DISABLE | ENABLE], PCM_TX_INT_[DISABLE | ENABLE], PCM_DATA_MGT_[IOC | CFX], PCM_MULTIWORD_x, PCM_WORD_SIZE_x, PCM_SHIFT_RX_DATA_[LEFT | RIGHT], PCM_FIELD_TX_DATA_[MSB | LSB], PCM_TX_DATA_[SATURATED | UNSATURATED]
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if pcm_cfg can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure the PCM interface as a slave receiver. The transfer is`
	`// configured for:`
	`// - 16 bits per word (sent MSB first), with received data placed in`
	`//   the lower 16 bits of the PCM data register.`
	`// - Pulse frame signal aligned with the first bit every 2 words`
	`#define cfg ( PCM_SLAVE | PCM_ENABLE | PCM_SUBFRAME_DISABLE | \`
	`PCM_BIT_ORDER_MSB_FIRST | \`
	`PCM_FRAME_ALIGN_FIRST_BIT | \`
	`PCM_FRAME_WIDTH_SHORT | PCM_RX_INT_ENABLE | \`
	`PCM_TX_INT_DISABLE | PCM_DATA_MGT_CFX | \`
	`PCM_MULTIWORD_2 | PCM_WORD_SIZE_16 | \`
	`PCM_SHIFT_RX_DATA_RIGHT | \`
	`PCM_FIELD_TX_DATA_LSB | \`
	`PCM_TX_DATA_UNSATURATED )`
	`Sys_PCM_Config( cfg )`

14.232 SYS_PCM_DISABLE

Disable the PCM interface by clearing the PCM_CFG_ENABLE bit

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_pcm.cfi
Template	`Sys_PCM_Disable`
Description	Disable the PCM interface by clearing the PCM_CFG_ENABLE bit. The PCM interface configuration is unchanged.
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	3
Example	`// Disable the PCM interface` `Sys_PCM_Disable`

14.233 SYS_PCM_ENABLE

Enable the PCM interface by setting the PCM_CFG_ENABLE bit

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_pcm.cfi
Template	`Sys_PCM_Enable`
Description	Enable the PCM interface by setting the PCM_CFG_ENABLE bit. The PCM interface configuration is unchanged.
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	3
Example	`// Enable the PCM interface` `Sys_PCM_Enable`

14.234 SYS_PLL_CLEARSTATUS

Clear the PLL status in A_PLL_CTRL_7

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_PLL_ClearStatus`
Description	Clear the PLL status in A_PLL_CTRL_7
Inputs	None
Outputs	None
Register Modifications	b1, y1
Assumptions	None
Clock Cycles	4
Example	`// Clear the PLL status` `Sys_PLL_ClearStatus`

14.235 SYS_PLL_CONFIGLOCK

Enable the VCO and PLL, configure the variables that are used to set the PLL frequency, and then lock the PLL

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_PLL_ConfigLock( PLL_R, PLL_M, PLL_X, PLL_P )`
Description	Enable the VCO and PLL, configure the variables that are used to set the PLL frequency, and then lock the PLL
Inputs	PLL_R = Input reference clock divider control PLL_M = VCO clock divider control PLL_X = Oscillator clock multiplier control; use OSC_MULTIPLY_BY_* PLL_P = Output clock divider
Outputs	None
Register Modifications	y1
Assumptions	The PLL reference clock source has been configured
Clock Cycles	-
Example	`// Configure the PLL frequency and lock it` `Sys_PLL_ConfigLock( R_VAL, M_VAL, X_VAL, P_VAL )`

14.236 SYS_PLL_GETSTATUS

Get the PLL status (and the PLL lock interrupt configuration settings) from A_PLL_CTRL_7

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_PLL_GetStatus( reg )`
Description	Get the PLL status (and the PLL lock interrupt configuration settings) from A_PLL_CTRL_7
Inputs	None
Outputs	reg = Register to store the PLL status to
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Check the PLL status to confirm if the PLL is unlocked`
	`Sys_PLL_GetStatus( x0 )`
	`bittst x0, PLL_CTRL_7_PLL_LOCK_INT_STATUS_POS  ||| nop`
	`goto.0d Unlock_Handler, no_carry`

14.237 SYS_PLL_WAITLOCK

Wait for the PLL to lock; exit when the PLL indicates that the PLL is locked or if a timeout occurs

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_PLL_WaitLock`
Description	Wait for the PLL to lock; exit when the PLL indicates that the PLL is locked or if a timeout occurs
Inputs	None
Outputs	SR = NO_CARRY set if PLL did not lock
Register Modifications	y1, b1
Assumptions	PLL configured pending a lock
Clock Cycles	Depends on PLL clock configuration

Example	`// Lock the PLL and wait for the lock to complete`
	`Sys_Set_PLL_ModeControl( PLL_TO_LOCK | PLL_ENABLE )`
	`Sys_PLL_WaitLock`

14.238 SYS_POP_REG_1

Restore the specified register from stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_1( reg1 )`
Description	Restore the specified register from stack.
Inputs	reg1 = Register to load from stack location [sp-1]
Outputs	None
Register Modifications	sp (sp = sp - 1)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	2
Example	`Sys_Pop_Reg_1( a0.e )`

14.239 SYS_POP_REG_10

Restore the ten specified registers from stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_10( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9, reg10 )`
Description	Restore the ten specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7] reg8 = Register to load from stack location [sp-8] reg9 = Register to load from stack location [sp-9] reg10 = Register to load from stack location [sp-10]
Outputs	None
Register Modifications	sp (sp = sp - 10)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	11
Example	`Sys_Pop_Reg_10( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h, b0.l, \` `yp0, x0 )`

14.240 SYS_POP_REG_2

Restore the two specified registers from stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_2( reg1, reg2 )`
Description	Restore the two specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2]
Outputs	None
Register Modifications	sp (sp = sp - 2)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	3
Example	`Sys_Pop_Reg_2( a0.e, a0.h )`

14.241 SYS_POP_REG_3

Restore the three specified registers from stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_3( reg1, reg2, reg3 )`
Description	Restore the three specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3]
Outputs	None
Register Modifications	sp (sp = sp - 3)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	4
Example	`Sys_Pop_Reg_3( a0.e, a0.h, a0.l )`

14.242 SYS_POP_REG_4

Restore the four specified registers from stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_4( reg1, reg2, reg3, reg4 )`
Description	Restore the four specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4]
Outputs	None
Register Modifications	sp (sp = sp - 4)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	5
Example	`Sys_Pop_Reg_4( a0.e, a0.h, a0.l, xp0 )`

14.243 SYS_POP_REG_5

Restore the five specified registers from stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_5( reg1, reg2, reg3, reg4, reg5 )`
Description	Restore the five specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5]
Outputs	None
Register Modifications	sp (sp = sp - 5)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	6
Example	`Sys_Pop_Reg_5( a0.e, a0.h, a0.l, xp0, xp1 )`

14.244 SYS_POP_REG_6

Restore the six specified registers from stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_6( reg1, reg2, reg3, reg4, reg5, reg6 )`
Description	Restore the six specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6]
Outputs	None
Register Modifications	sp (sp = sp - 6)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	7
Example	`Sys_Pop_Reg_6( a0.e, a0.h, a0.l, xp0, xp1, b0.e )`

14.245 SYS_POP_REG_7

Restore the seven specified registers from stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_7( reg1, reg2, reg3, reg4, reg5, reg6, reg7 )`
Description	Restore the seven specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7]
Outputs	None
Register Modifications	sp (sp = sp - 7)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	8
Example	`Sys_Pop_Reg_7( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h )`

14.246 SYS_POP_REG_8

Restore the eight specified registers from stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_8( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8 )`
Description	Restore the eight specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7] reg8 = Register to load from stack location [sp-8]
Outputs	None
Register Modifications	sp (sp = sp - 8)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	9
Example	`Sys_Pop_Reg_8( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h, b0.l )`

14.247 SYS_POP_REG_9

Restore the nine specified registers from stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_9( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9 )`
Description	Restore the nine specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7] reg8 = Register to load from stack location [sp-8] reg9 = Register to load from stack location [sp-9]
Outputs	None
Register Modifications	sp (sp = sp - 9)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	10
Example	`Sys_Pop_Reg_9( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h, b0.l, yp0 )`

14.248 SYS_POP_REG_LONG_1

Restore the specified register from the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_Long_1( reg1 )`
Description	Restore the specified register from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_1 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1]
Outputs	None
Register Modifications	sp (sp = sp - 1)
Assumptions	None
Clock Cycles	2
Example	`Sys_Pop_Reg_Long_1( xstep0 )`

14.249 SYS_POP_REG_LONG_10

Restore the ten specified registers from the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_Long_10( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9, reg10 )`
Description	Restore the ten specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_10 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7] reg8 = Register to load from stack location [sp-8] reg9 = Register to load from stack location [sp-9] reg10 = Register to load from stack location [sp-10]
Outputs	None
Register Modifications	sp (sp = sp - 10)
Assumptions	None
Clock Cycles	11
Example	`Sys_Pop_Reg_Long_10( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4, xstep5, xstep6, xstep7 )`

14.250 SYS_POP_REG_LONG_2

Restore the two specified registers from the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_Long_2( reg1, reg2 )`
Description	Restore the two specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_2 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2]
Outputs	None
Register Modifications	sp (sp = sp - 2)
Assumptions	None
Clock Cycles	3
Example	`Sys_Pop_Reg_Long_2( xstep0, xcirc0 )`

14.251 SYS_POP_REG_LONG_3

Restore the three specified registers from the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_Long_3( reg1, reg2, reg3 )`
Description	Restore the three specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_3 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3]
Outputs	None
Register Modifications	sp (sp = sp - 3)
Assumptions	None
Clock Cycles	4
Example	`Sys_Pop_Reg_Long_3( xstep0, xcirc0, xstep1 )`

14.252 SYS_POP_REG_LONG_4

Restore the four specified registers from the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_Long_4( reg1, reg2, reg3, reg4 )`
Description	Restore the four specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_4 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4]
Outputs	None
Register Modifications	sp (sp = sp - 4)
Assumptions	None
Clock Cycles	5
Example	`Sys_Pop_Reg_Long_4( xstep0, xcirc0, xstep1, xcirc1 )`

14.253 SYS_POP_REG_LONG_5

Restore the five specified registers from the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_Long_5( reg1, reg2, reg3, reg4, reg5 )`
Description	Restore the five specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_5 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5]
Outputs	None
Register Modifications	sp (sp = sp - 5)
Assumptions	None
Clock Cycles	6
Example	`Sys_Pop_Reg_Long_5( xstep0, xcirc0, xstep1, xcirc1, xstep2 )`

14.254 SYS_POP_REG_LONG_6

Restore the six specified registers from the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_Long_6( reg1, reg2, reg3, reg4, reg5, reg6 )`
Description	Restore the six specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_6 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6]
Outputs	None
Register Modifications	sp (sp = sp - 6)
Assumptions	None
Clock Cycles	7
Example	`Sys_Pop_Reg_Long_6( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3 )`

14.255 SYS_POP_REG_LONG_7

Restore the seven specified registers from the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_Long_7( reg1, reg2, reg3, reg4, reg5, reg6, reg7 )`
Description	Restore the seven specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_7 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7]
Outputs	None
Register Modifications	sp (sp = sp - 7)
Assumptions	None
Clock Cycles	8
Example	`Sys_Pop_Reg_Long_7( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4 )`

14.256 SYS_POP_REG_LONG_8

Restore the eight specified registers from the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_Long_8( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8 )`
Description	Restore the eight specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_8 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7] reg8 = Register to load from stack location [sp-8]
Outputs	None
Register Modifications	sp (sp = sp - 8)
Assumptions	None
Clock Cycles	9
Example	`Sys_Pop_Reg_Long_8( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4, xstep5 )`

14.257 SYS_POP_REG_LONG_9

Restore the nine specified registers from the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Pop_Reg_Long_9( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9 )`
Description	Restore the nine specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_9 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7] reg8 = Register to load from stack location [sp-8] reg9 = Register to load from stack location [sp-9]
Outputs	None
Register Modifications	sp (sp = sp - 9)
Assumptions	None
Clock Cycles	10
Example	`Sys_Pop_Reg_Long_9( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4, xstep5, xstep6 )`

14.258 SYS_POPLOOPSFROMSTACK

Pop all four sets of the hardware loop registers from the stack; uses ROMVECT_PopLoopsFromStack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_romvect.cfi
Template	`Sys_PopLoopsFromStack`
Description	Pop all four sets of the hardware loop registers from the stack; uses ROMVECT_PopLoopsFromStack
Inputs	None
Outputs	None
Register Modifications	y1, sp, all four sets of lc, lsa, lea
Assumptions	The loop registers were previously stored to the stack
Clock Cycles	18
Example	`// Pop the hardware loop registers from the stack` `Sys_PopLoopsFromStack`

14.259 SYS_POWER2

Calculate the power of 2 for a supplied constant

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Power2( const )`
Description	Calculate the power of 2 for a supplied constant.
Inputs	const = Exponent
Outputs	Sys_Power2 = 2^const
Register Modifications	None
Assumptions	Constant is a non-negative integer
Clock Cycles	0 (evaluated during compile)
Example	`// Load 2^3 to x0` `load x0, Sys_Power2( 3 )`

14.260 SYS_PUSH_REG_1

Store the specified register onto stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_1( reg1 )`
Description	Store the specified register onto stack
Inputs	reg1 = Register to store at stack location [sp-1]
Outputs	None
Register Modifications	sp (sp = sp + 1)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	2
Example	`Sys_Push_Reg_1( a0.e )`

14.261 SYS_PUSH_REG_10

Store the ten specified registers onto stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_10( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9, reg10 )`
Description	Store the ten specified registers onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7] reg8 = Register to store at stack location [sp-8] reg9 = Register to store at stack location [sp-9] reg10 = Register to store at stack location [sp-10]
Outputs	None
Register Modifications	sp (sp = sp + 10)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	11
Example	`Sys_Push_Reg_10( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h, b0.l, \` `yp0, x0 )`

14.262 SYS_PUSH_REG_2

Store the two specified registers onto stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_2( reg1, reg2 )`
Description	Store the two specified registers onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2]
Outputs	None
Register Modifications	sp (sp = sp + 2)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	3
Example	`Sys_Push_Reg_2( a0.e, a0.h )`

14.263 SYS_PUSH_REG_3

Store the three specified registers onto stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_3( reg1, reg2, reg3 )`
Description	Store the three specified registers onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3]
Outputs	None
Register Modifications	sp (sp = sp + 3)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	4
Example	`Sys_Push_Reg_3( a0.e, a0.h, a0.l )`

14.264 SYS_PUSH_REG_4

Store the four specified register onto stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_4( reg1, reg2, reg3, reg4 )`
Description	Store the four specified register onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4]
Outputs	None
Register Modifications	sp (sp = sp + 4)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	5
Example	`Sys_Push_Reg_4( a0.e, a0.h, a0.l, xp0 )`

14.265 SYS_PUSH_REG_5

Store the five specified registers onto stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_5( reg1, reg2, reg3, reg4, reg5 )`
Description	Store the five specified registers onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5]
Outputs	None
Register Modifications	sp (sp = sp + 5)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	6
Example	`Sys_Push_Reg_5( a0.e, a0.h, a0.l, xp0, xp1 )`

14.266 SYS_PUSH_REG_6

Store the six specified register onto stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_6( reg1, reg2, reg3, reg4, reg5, reg6 )`
Description	Store the six specified register onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6]
Outputs	None
Register Modifications	sp (sp = sp + 6)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	7
Example	`Sys_Push_Reg_6( a0.e, a0.h, a0.l, xp0, xp1, b0.e )`

14.267 SYS_PUSH_REG_7

Store the seven specified registers onto stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_7( reg1, reg2, reg3, reg4, reg5, reg6, reg7 )`
Description	Store the seven specified registers onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7]
Outputs	None
Register Modifications	sp (sp = sp + 7)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	8
Example	`Sys_Push_Reg_7( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h )`

14.268 SYS_PUSH_REG_8

Store the eight specified registers onto stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_8( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8 )`
Description	Store the eight specified registers onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7] reg8 = Register to store at stack location [sp-8]
Outputs	None
Register Modifications	sp (sp = sp + 8)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	9
Example	`Sys_Push_Reg_8( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h, b0.l )`

14.269 SYS_PUSH_REG_9

Store the nine specified register onto stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_9( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9 )`
Description	Store the nine specified register onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7] reg8 = Register to store at stack location [sp-8] reg9 = Register to store at stack location [sp-9]
Outputs	None
Register Modifications	sp (sp = sp + 9)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	10
Example	`Sys_Push_Reg_9( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h, b0.l, yp0 )`

14.270 SYS_PUSH_REG_LONG_1

Store the specified register onto the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_Long_1( reg1 )`
Description	Store the specified register onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_1 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1]
Outputs	None
Register Modifications	sp (sp = sp + 1)
Assumptions	None
Clock Cycles	2
Example	`Sys_Push_Reg_Long_1( xstep0 )`

14.271 SYS_PUSH_REG_LONG_10

Store the ten specified registers onto the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_Long_10( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9, reg10 )`
Description	Store the ten specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_10 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7] reg8 = Register to store at stack location [sp-8] reg9 = Register to store at stack location [sp-9] reg10 = Register to store at stack location [sp-10]
Outputs	None
Register Modifications	sp (sp = sp + 10)
Assumptions	None
Clock Cycles	11
Example	`Sys_Push_Reg_Long_10( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4, xstep5, xstep6, xstep7 )`

14.272 SYS_PUSH_REG_LONG_2

Store the two specified registers onto the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_Long_2( reg1, reg2 )`
Description	Store the two specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_2 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2]
Outputs	None
Register Modifications	sp (sp = sp + 2)
Assumptions	None
Clock Cycles	3
Example	`Sys_Push_Reg_Long_2( xstep0, xcirc0 )`

14.273 SYS_PUSH_REG_LONG_3

Store the three specified registers onto the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_Long_3( reg1, reg2, reg3 )`
Description	Store the three specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_3 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3]
Outputs	None
Register Modifications	sp (sp = sp + 3)
Assumptions	None
Clock Cycles	4
Example	`Sys_Push_Reg_Long_3( xstep0, xcirc0, xstep1 )`

14.274 SYS_PUSH_REG_LONG_4

Store the four specified registers onto the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_Long_4( reg1, reg2, reg3, reg4 )`
Description	Store the four specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_4 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4]
Outputs	None
Register Modifications	sp (sp = sp + 4)
Assumptions	None
Clock Cycles	5
Example	`Sys_Push_Reg_Long_4( xstep0, xcirc0, xstep1, xcirc1 )`

14.275 SYS_PUSH_REG_LONG_5

Store the five specified registers onto the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_Long_5( reg1, reg2, reg3, reg4, reg5 )`
Description	Store the five specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_5 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5]
Outputs	None
Register Modifications	sp (sp = sp + 5)
Assumptions	None
Clock Cycles	6
Example	`Sys_Push_Reg_Long_5( xstep0, xcirc0, xstep1, xcirc1, xstep2 )`

14.276 SYS_PUSH_REG_LONG_6

Store the six specified registers onto the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_Long_6( reg1, reg2, reg3, reg4, reg5, reg6 )`
Description	Store the six specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_6 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6]
Outputs	None
Register Modifications	sp (sp = sp + 6)
Assumptions	None
Clock Cycles	7
Example	`Sys_Push_Reg_Long_6( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3 )`

14.277 SYS_PUSH_REG_LONG_7

Store the seven specified registers onto the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_Long_7( reg1, reg2, reg3, reg4, reg5, reg6, reg7 )`
Description	Store the seven specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_7 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7]
Outputs	None
Register Modifications	sp (sp = sp + 7)
Assumptions	None
Clock Cycles	8
Example	`Sys_Push_Reg_Long_7( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4 )`

14.278 SYS_PUSH_REG_LONG_8

Store the eight specified registers onto the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_Long_8( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8 )`
Description	Store the eight specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_8 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7] reg8 = Register to store at stack location [sp-8]
Outputs	None
Register Modifications	sp (sp = sp + 8)
Assumptions	None
Clock Cycles	9
Example	`Sys_Push_Reg_Long_8( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4, xstep5 )`

14.279 SYS_PUSH_REG_LONG_9

Store the nine specified registers onto the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Push_Reg_Long_9( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9 )`
Description	Store the nine specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_9 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7] reg8 = Register to store at stack location [sp-8] reg9 = Register to store at stack location [sp-9]
Outputs	None
Register Modifications	sp (sp = sp + 9)
Assumptions	None
Clock Cycles	10
Example	`Sys_Push_Reg_Long_9( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4, xstep5, xstep6 )`

14.280 SYS_PUSHLOOPSTOSTACK

Push all four sets of the hardware loop registers onto the stack; uses ROMVECT_PushLoopsToStack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_romvect.cfi
Template	`Sys_PushLoopsToStack`
Description	Push all four sets of the hardware loop registers onto the stack; uses ROMVECT_PushLoopsToStack
Inputs	None
Outputs	None
Register Modifications	y1, sp, all four sets of lc, lsa, lea
Assumptions	There is space for 12 registers to be stored to the stack
Clock Cycles	18
Example	`// Push the hardware loop registers to the stack` `Sys_PushLoopsToStack`

14.281 SYS_REG_FL8N16

Load a data register or high part of an accumulator with a floating-point number with an 8-bit signed exponent and a 16-bit signed mantissa using the Sys_fl8n16 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Reg_fl8n16( reg, const )`
Description	Load a data register or high part of an accumulator with a floating-point number with an 8-bit signed exponent and a 16-bit signed mantissa using the Sys_fl8n16 macro.
Inputs	const = Constant to convert with Sys_fl8n16
Outputs	reg = Register to store the floating-point approximation of the supplied constant to
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^63
Clock Cycles	1
Example	`// Load 7.125 to register x1 as a <8,16> floating-point value` `Sys_Reg_fl8n16( x1, 71250000 )`

14.282 SYS_REG_FRAC24

Load a data register or high part of an accumulator with a 24-bit signed immediate using the Sys_frac24 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Reg_frac24( reg, const )`
Description	Load a data register or high part of an accumulator with a 24-bit signed immediate using the Sys_frac24 macro.
Inputs	const = Constant to convert with Sys_frac24
Outputs	reg = Register to store the fractional approximation of the supplied constant to
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^46
Clock Cycles	1
Example	`// Load 0.125 to register x1 as a <1,23> fractional value` `Sys_Reg_frac24( x1, 1250000 )`

14.283 SYS_REG_FRAC32

Load the high and extension part of an accumulator with a 32-bit signed immediate using the Sys_frac32 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Reg_frac32( accum, const )`
Description	Load the high and extension part of an accumulator with a 32-bit signed immediate using the Sys_frac32 macro. NOTE: This macro clears the low segment of the specified accumulator. NOTE: The frac32 data format is layout-compatible with the frac24 format.
Inputs	const = Constant to convert with Sys_frac32
Outputs	accum = Accumulator to store the <9,23> fixed-point approximation of the supplied constant to
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^46
Clock Cycles	2
Example	`// Load 7.125 to accumulator a1.e, a1.h as a <9,23> fractional value` `Sys_Reg_frac32( a1, 71250000 )`

14.284 SYS_REG_FRAC48

Load an accumulator with a 48-bit signed immediate using the Sys_frac48 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Reg_frac48( accum, const )`
Description	Load an accumulator with a 48-bit signed immediate using the Sys_frac48 macro.
Inputs	const = Constant to convert with Sys_frac48
Outputs	accum = Accumulator to store the fractional approximation of the supplied constant to
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^24
Clock Cycles	2
Example	`// Load 0.125 to accumulator a1 as a <1,47> fractional value` `Sys_Reg_frac48( a1, 1250000 )`

14.285 SYS_REG_M24P24

Load an accumulator with a 48-bit signed immediate using the Sys_m24p24 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Reg_m24p24( accum, const )`
Description	Load an accumulator with a 48-bit signed immediate using the Sys_m24p24 macro.
Inputs	const = Constant to convert with Sys_m24p24
Outputs	accum = Accumulator to store the <24,24> fixed-point approximation of the supplied constant to
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^62
Clock Cycles	2
Example	`// Load 7.125 to accumulator a1 as a <24,24> fixed-point value` `Sys_Reg_m24p24( a1, 71250000 )`

14.286 SYS_REG_M2P22

Load a data register or high part of an accumulator with a 24-bit signed immediate using the Sys_m2p22 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Reg_m2p22( reg, const )`
Description	Load a data register or high part of an accumulator with a 24-bit signed immediate using the Sys_m2p22 macro.
Inputs	const = Constant to convert with Sys_m2p22
Outputs	reg = Register to store the <2,22> fixed-point approximation of the supplied constant to
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^46
Clock Cycles	1
Example	`// Load 1.125 to b1.h a1 as a <2,22> fixed-point value` `Sys_Reg_m2p22( b1, 11250000 )`

14.287 SYS_REG_M2P46

Load an accumulator with a 48-bit signed immediate using the Sys_m2p46 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Reg_m2p46( accum, const )`
Description	Load an accumulator with a 48-bit signed immediate using the Sys_m2p46 macro.
Inputs	const = Constant to convert with Sys_m2p46
Outputs	accum = Accumulator to store the <2,46> fixed-point approximation of the supplied constant to
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^40
Clock Cycles	2
Example	`// Load 1.125 to accumulator a1 as a <2,46> fixed-point value` `Sys_Reg_m2p46( a1, 11250000 )`

14.288 SYS_REG_M8P16

Load a data register or high part of an accumulator with an 8-bit signed integer and a 16-bit unsigned fractional part using the Sys_m8p16 macro

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Reg_m8p16( reg, const )`
Description	Load a data register or high part of an accumulator with an 8-bit signed integer and a 16-bit unsigned fractional part using the Sys_m8p16 macro.
Inputs	const = Constant to convert with Sys_m8p16
Outputs	reg = Register to store the fixed-point approximation of the supplied constant to
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^53
Clock Cycles	1
Example	`// Load 7.125 to register x1 as a <8,16> fixed-point value` `Sys_Reg_m8p16( x1, 71250000 )`

14.289 SYS_RESTORE_HEAR_BFP

Configure the HEAR block floating point using a stored value

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_hear.cfi
Template	`Sys_Restore_HEAR_BFP( memspace, bfp_addr )`
Description	Configure the HEAR block floating point using a stored value
Inputs	mem_space = Memory space to load from; use SYS_MEMSPACE_P, SYS_MEMSPACE_X or SYS_MEMSPACE_Y bfp_addr = Memory location that contains a configuration setting for the HEAR BFP unit
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2
Example	`// Restore a previously saved HEAR block floating point configuration` `Sys_Restore_HEAR_BFP( SYS_MEMSPACE_X, X_ALGO0_BFP_CONFIG )`

14.290 SYS_RS_CLEAR_STATE

Clear the internal state and status outputs from the Reed-Solomon encoder and decoder, including the input/output pointers

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ecb.cfi
Template	`Sys_RS_Clear_State`
Description	Clear the internal state and status outputs from the Reed-Solomon encoder and decoder, including the input/output pointers. It is not possible to update the RS pointers in an atomic way unless both the encoder and decoder are idle.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	The Reed-Solomon encoder and decoder are idle
Clock Cycles	2

Example	`// Clear the current state for the Reed-Solomon encoder and decoder`
	`// blocks`
	`Sys_RS_Clear_State`

14.291 SYS_RS_DECODER_CONFIG

Configure the Reed-Solomon decoder

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ecb.cfi
Template	`Sys_RS_Decoder_Config( config )`
Description	Configure the Reed-Solomon decoder
Inputs	config = The Reed-Solomon decoder configuration; use RS_DEC_IN_*BIT, and RS_DEC_OUT_*BIT
Outputs	None
Register Modifications	y1, b1
Assumptions	None
Clock Cycles	6

Example	`// Configure the Reed-Solomon decoder`
	`Sys_RS_Decoder_Config( RS_DEC_IN_32BIT  |  \`
	`RS_DEC_OUT_32BIT )`

14.292 SYS_RS_ENCODER_CONFIG

Configure the Reed-Solomon encoder

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_ecb.cfi
Template	`Sys_RS_Encoder_Config( config )`
Description	Configure the Reed-Solomon encoder
Inputs	config = The Reed-Solomon encoder configuration; use RS_ENC_IN_*BIT, and RS_ENC_OUT_*BIT
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	5

Example	`// Configure the Reed-Solomon encoder`
	`Sys_RS_Encoder_Config( RS_ENC_IN_32BIT | \`
	`RS_ENC_OUT_32BIT )`

14.293 SYS_SAVE_HEAR_BFP

Store the current HEAR block floating point configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_hear.cfi
Template	`Sys_Save_HEAR_BFP( memspace, bfp_addr )`
Description	Store the current HEAR block floating point configuration
Inputs	mem_space = Memory space to store to; use SYS_MEMSPACE_P, SYS_MEMSPACE_X or SYS_MEMSPACE_Y bfp_addr = Memory location where a configuration setting for the HEAR BFP unit will be stored
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2

Example	`// Configure the block floating point exponent to a normalized`
	`// reference setting`
	`load x0, NORM_REFERENCE_BLOCK_EXP`
	`Sys_SetReg_HEAR_BFP_BLOCK_EXP( x0 )`
	
	`// Save the new HEAR block floating point configuration so it can`
	`// be restored later (insert a NOP to avoid reading the previous`
	`// D_HEAR_BFP value due to a buffered write of D_HEAR_BFP_BLOCK_EXP)`
	`nop`
	`Sys_Save_HEAR_BFP( SYS_MEMSPACE_X, X_ALGO0_BFP_CONFIG )`

14.294 SYS_SET_ADC_CFG

Configure the ADC for one of the input channels

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_Set_ADC_CFG( channel, config )`
Description	Configure the ADC for one of the input channels
Inputs	channel = ADC to apply this configuration to; use a constant between 0 and 3 config = ADC configuration; use ADC_INPUT_[BIASED | HIZ], P_CURRENT_*_*X, ADC_CURRENT_*_*X, ADC_OFFSET_[POS | NEG], and a constant shifted to ADC_CFG_OFFSET_MAGN_POS
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if config can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure ADC2 with the default configuration settings`
	`Sys_Set_ADC_CFG( 2, ADC_INPUT_BIASED | \`
	`P_CURRENT_1_0X | ADC_CURRENT_1_0X | \`
	`ADC_OFFSET_POS | ADC_OFFSET_16MV )`

14.295 SYS_SET_ADC_CTRL

Configure the ADC control settings for one of the input channels and calibrate the ADC DC gain offset

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_Set_ADC_CTRL( channel, config )`
Description	Configure the ADC control settings for one of the input channels and calibrate the ADC DC gain offset
Inputs	channel = ADC to apply this configuration to; use a constant between 0 and 3 config = ADC control configuration; use LINEOUT_[DISABLE | ENABLE], LPF_IN_[BYPASS | ACTIVE_20KHZ], ADC_GAIN_*DB, PGAIN_[*DB | BYPASS], ADC_SEL_* and ADC_[DISABLE | ENABLE]
Outputs	None
Register Modifications	y1, b1, xp7, yp3
Assumptions	None
Clock Cycles	Not available

Example	`// Enable input using ADC2, with 18dB of gain and low-pass`
	`// filtering at 20kHz`
	`Sys_Set_ADC_CTRL( 2, LINEOUT_DISABLE | \`
	`LPF_IN_ACTIVE_20KHZ | \`
	`ADC_GAIN_0DB | PGAIN_18DB | \`
	`ADC_SEL_AI2 | ADC_ENABLE )`

14.296 SYS_SET_ADC_CTRL_NOCALIBRATION

Configure the ADC control settings for one of the input channels without calibrating the ADC DC offset

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_Set_ADC_CTRL_NoCalibration( channel, config )`
Description	Configure the ADC control settings for one of the input channels without calibrating the ADC DC offset
Inputs	channel = ADC to apply this configuration to; use a constant between 0 and 3 config = ADC control configuration; use LINEOUT_[DISABLE | ENABLE], LPF_IN_[BYPASS | ACTIVE_20KHZ], ADC_GAIN_*DB, PGAIN_[*DB | BYPASS], ADC_SEL_* and ADC_[DISABLE | ENABLE]
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if config can be sign-extended from 8 bits; 2 otherwise

Example	`// Enable input using ADC2, with 18dB of gain and low-pass`
	`// filtering at 20kHz (do not calibrate the ADC)`
	`Sys_Set_ADC_CTRL_NoCalibration( 2, LINEOUT_DISABLE | \`
	`LPF_IN_ACTIVE_20KHZ | \`
	`ADC_GAIN_0DB | PGAIN_18DB | \`
	`ADC_SEL_AI2 | ADC_ENABLE )`

14.297 SYS_SET_ADC_DEC_CTRL

Configure the decimation filter for one of the input channels

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_Set_ADC_DEC_CTRL( channel, config )`
Description	Configure the decimation filter for one of the input channels
Inputs	channel = Decimation filter to apply this configuration to; use a constant between 0 and 3 config = Decimation filter configuration; use BAND_SELECT_ADC_*K_*K, ADC_INTEGER_DELAY_*, a constant shifted to ADC_DEC_CTRL_DELAY_FRACTIONAL_POS, ADC_[MUTE | UNMUTE], ADC_DEC_[ENABLE | DISABLE] and ADC_DC_REMOVE_CUTOFF_[*HZ | BYPASS]
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if config can be sign-extended from 8 bits; 2 if config can be sign-extended from 24 bits; 3 otherwise

Example	`// Calculate the gain factor value that will produce a unity gain`
	`// in channel 2 given a ADC_CFG_SAMPLE_FREQ setting of 0x2`
	`// NOTE: Gain factor value is #set into SYS_CALC_GF`
	`#define ADC_CFG_SAMPLE_FREQ 0x2`
	`#define UNITY_GAIN 10000000`
	
	`Sys_Calc_Gain_Factor_Val( ADC_CFG_SAMPLE_FREQ, UNITY_GAIN )`
	
	`// Configure the decimation filter for input channel 2`
	`Sys_Set_ADC_DEC_CTRL( 2, BAND_SELECT_ADC_0K_8K | \`
	`ADC_INTEGER_DELAY_0 | ADC_UNMUTE | \`
	`ADC_DEC_ENABLE | \`
	`ADC_DC_REMOVE_CUTOFF_10HZ | \`
	`SYS_CALC_GF )`

14.298 SYS_SET_ADC_SAMPLE_FREQ_CFG

Configure the sampling frequency

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_Set_ADC_SAMPLE_FREQ_CFG( config )`
Description	Configure the sampling frequency
Inputs	config = Sampling frequency configuration; use ADC_COMP_BUFFER_[BYPASS | ENABLE], ADC_HIGH_FREQ_[DISABLE | ENABLE], and a constant shifted to ADC_SAMPLE_FREQ_CFG_SAMPLE_FREQ_POS
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if config can be sign-extended from 8 bits; 2 otherwise

Example	`// Set the sampling frequency to 16 kHz with a 1.28 MHz slow clock`
	`// NOTE: Value for ADC_CFG_SAMPLE_FREQ field in ADC_CFG is #set into`
	`//  SYS_CALC_SF`
	`Sys_Calc_Sample_Freq_Val( 1280000, 16000 )`
	`Sys_Set_ADC_SAMPLE_FREQ_CFG( SYS_CALC_SF | ADC_HIGH_FREQ_DISABLE )`

14.299 SYS_SET_CLK_THROTTLECONFIGMODE

Configure the clock throttling mode

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_clk.cfi
Template	`Sys_Set_Clk_ThrottleConfigMode( config )`
Description	Configure the clock throttling mode
Inputs	config = Select the clock throttling mode; use CLK_THROTTLING_ON_OD_ZERO_[DISABLE | ENABLE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the clock throttling mode so the clock can only be`
	`// throttled when the output driver is driving a 0`
	`Sys_Set_Clk_ThrottleConfigMode( CLK_THROTTLING_ON_OD_ZERO_ENABLE )`

14.300 SYS_SET_CP_VDBL_CTRL

Set the digital interface supply voltage (VDBL) charge pump configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_pwr.cfi
Template	`Sys_Set_CP_VDBL_CTRL( config )`
Description	Set the digital interface supply voltage (VDBL) charge pump configuration
Inputs	config = VDBL charge pump configuration; use VDBL_[RESET | NO_RESET], VDBL_[DISABLE | ENABLE], VDBL_PTRIM_*MA, VDBL_TRIM_*UA and VDBL_TRIM_*_*V (or a constant shifted to CP_VDBL_CTRL_TRIM_POS)
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if config can be sign-extended from 8 bits; 2 otherwise

Example	`// Set the digital interface supply charge pump configuration`
	`// to the default settings`
	`Sys_Set_CP_VDBL_CTRL( VDBL_RESET | VDBL_ENABLE | VDBL_PTRIM_4MA | \`
	`VDBL_TRIM_36UA | VDBL_TRIM_1_8V )`

14.301 SYS_SET_CP_VDDA_CTRL

Set the analog supply voltage (VDDA) charge pump configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_pwr.cfi
Template	`Sys_Set_CP_VDDA_CTRL( config )`
Description	Set the analog supply voltage (VDDA) charge pump configuration
Inputs	config = VDDA charge pump configuration; use VDDA_[RESET | NO_RESET], VDDA_[DISABLE | ENABLE], and VDDA_TRIM_*_*V (or a constant shifted to CP_VDDA_CTRL_TRIM_POS)
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if config can be sign-extended from 8 bits; 2 otherwise

Example	`// Set the analog supply charge pump configuration to the`
	`// default settings`
	`Sys_Set_CP_VDDA_CTRL( VDDA_RESET | VDDA_ENABLE | VDDA_TRIM_2_0V )`

14.302 SYS_SET_DMIC_OUTPUT_CTRL

Configure the DMIC output channels

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_Set_DMIC_OUTPUT_CTRL( config )`
Description	Configure the DMIC output channels
Inputs	config = DMIC output configuration; use DMIC*_HIGH_SRC_[CH0 | CH1], DMIC*_LOW_SRC_[CH0 | CH1], and DMIC*_OUT_[ENABLE | DISABLE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Enable both DMIC outputs to drive the high and low portions of`
	`// the same channel.`
	`Sys_Set_DMIC_OUTPUT_CTRL( DMIC0_OUT_ENABLE | DMIC0_LOW_SRC_CH0 | \`
	`DMIC0_HIGH_SRC_CH0 | \`
	`DMIC1_OUT_ENABLE | DMIC1_LOW_SRC_CH1 | \`
	`DMIC1_HIGH_SRC_CH1 )`

14.303 SYS_SET_FENG_COEFFICIENTS

Configure one set of Filter Engine coefficients for use with the Filter Engine BIQ and FIR instructions

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Set_FENG_Coefficients( cset, c0, c1, c2, c3, c4 )`
Description	Configure one set of Filter Engine coefficients for use with the Filter Engine BIQ and FIR instructions.
Inputs	cset = Coefficient set number; use a constant between 0 and 31 c* = Coefficient * from the given coefficient set; use constants defined using Sys_m2p22 for BIQ, Sys_frac24 for FIR
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	10

Example	`// Configure a set of coefficients for an FIR filter`
	`#set c0_0  Sys_frac24( 0 )`
	`#set c0_1  Sys_frac24( 200000 )`
	`#set c0_2  Sys_frac24( 400000 )`
	`#set c0_3  Sys_frac24( 600000 )`
	`#set c0_4  Sys_frac24( 800000 )`
	
	`Sys_Set_FENG_Coefficients( 0, c0_0, c0_1, c0_2, c0_3, c0_4 )`

14.304 SYS_SET_FENG_COEFFOFFSET

Set the Filter Engine coefficient offset

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Set_FENG_CoeffOffset( offset )`
Description	Set the Filter Engine coefficient offset
Inputs	offset = Filter engine coefficient offset to use; use an unshifted constant
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	3
Example	`// Reset the Filter Engine coefficient offset to 0` `Sys_Set_FENG_CoeffOffset( 0 )`

14.305 SYS_SET_FENG_COEFFOFFSETENABLE_ACTIVE

Select which Filter Engine coefficient sets use the offset by writing the enable value to the active register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Set_FENG_CoeffOffsetEnable_Active( enable )`
Description	Select which Filter Engine coefficient sets use the offset by writing the enable value to the active register
Inputs	enable = A set of Filter Engine coefficient enable/disable bits; use 1 (enable) or 0 (disable) shifted to the coefficient set index
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if enable can be sign-extended from 8 bits; 2 if enable can be sign-extended from 24 bits, but not 8 bits; 3 otherwise

Example	`// Enable the default Filter Engine coefficient offset for this`
	`// program. This takes effect immediately.`
	`Sys_Set_FENG_CoeffOffsetEnable_Active( COEFF_OFFSET_CONFIG_DEFAULT )`

14.306 SYS_SET_FENG_COEFFOFFSETENABLE_SHADOW

Select which Filter Engine coefficient sets use the offset by writing the enable value to the shadow register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Set_FENG_CoeffOffsetEnable_Shadow( enable )`
Description	Select which Filter Engine coefficient sets use the offset by writing the enable value to the shadow register. The new setting will take affect the next time Filter Engine execution starts.
Inputs	enable = A set of Filter Engine coefficient enable/disable bits; use 1 (enable) or 0 (disable) shifted to the coefficient set index
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if enable can be sign-extended from 8 bits; 2 if enable can be sign-extended from 24 bits, but not 8 bits; 3 otherwise

Example	`// Enable the default Filter Engine coefficient offset for this`
	`// program`
	`Sys_Set_FENG_CoeffOffsetEnable_Shadow( COEFF_OFFSET_CONFIG_DEFAULT )`

14.307 SYS_SET_FENG_INTENABLE

Configure the possible sources for the Filter Engine interrupt

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Set_FENG_IntEnable( int_ebl )`
Description	Configure the possible sources for the Filter Engine interrupt
Inputs	int_ebl = Filter engine interrupt configuration; use a set of INT_DIS_FENG_* and INT_EBL_FENG_* defines
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if int_ebl can be sign-extended from 8 bits; 2 if int_ebl can be sign-extended from 24 bits; 3 otherwise

Example	`// Configure Filter Engine interrupts to trigger for errors and`
	`// input 2`
	`Sys_Set_FENG_IntEnable( INT_EBL_FENG_SRIN2 |`
	`INT_EBL_FENG_REG_ERR |`
	`INT_EBL_FENG_MEM_ERR |`
	`INT_EBL_FENG_CORE_SAT |`
	`INT_EBL_FENG_ALU_SAT |`
	`INT_EBL_FENG_OVERLOAD )`

14.308 SYS_SET_FENG_MRCOUNTERCONFIG

Configure the multi-rate counter divisors and reset the multi-rate counters

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Set_FENG_MRCounterConfig( cfg0, cfg1 )`
Description	Configure the multi-rate counter divisors and reset the multi-rate counters
Inputs	cfg0 = Configuration for Filter Engine multi-rate counter 0; use a constant indicating the number of phases desired - 1 cfg1 = Configuration for Filter Engine multi-rate counter 1; use a constant indicating the number of phases desired - 1
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the Filter Engine for four phases in the first`
	`// execution path and one phase in the other.`
	`Sys_Set_FENG_MRCounterConfig( (4 - 1), (1 - 1) )`

14.309 SYS_SET_FENG_STATE

Configure the Filter Engine state codes by writing to the active register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Set_FENG_State( state )`
Description	Configure the Filter Engine state codes by writing to the active register
Inputs	state = Filter engine state codes; use 1 (condition met) or 0 (condition not met) shifted by the state code index for each state code
Outputs	None
Register Modifications	y1
Assumptions	The Filter Engine is inactive
Clock Cycles	1 if state can be sign-extended from 8 bits; 2 otherwise
Example	`// Clear the Filter Engine state codes immediately` `Sys_Set_FENG_State( 0 )`

14.310 SYS_SET_FENG_STATE_SHADOW

Configure the Filter Engine state codes by writing to the shadow register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Set_FENG_State_Shadow( state )`
Description	Configure the Filter Engine state codes by writing to the shadow register. These state codes will take effect the next time Filter Engine execution starts.
Inputs	state = Filter Engine state codes; use 1 (condition met) or 0 (condition not met) shifted by the state code index for each state code
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if state can be sign-extended from 8 bits; 2 otherwise
Example	`// Clear the Filter Engine state codes` `Sys_Set_FENG_State_Shadow( 0 )`

14.311 SYS_SET_FENG_STATEWRITEENABLE

Configure the Filter Engine state code write-enable settings

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_Set_FENG_StateWriteEnable( cfg )`
Description	Configure the Filter Engine state code write-enable settings
Inputs	cfg = Filter engine state code write-enable settings; use a set of shifted constants where 1 indicates the Filter Engine state code can be written for each state code
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if state can be sign-extended from 8 bits; 2 otherwise
Example	`// Write-enable the Filter Engine state codes` `Sys_Set_FENG_StateWriteEnable( 0xFFFF )`

14.312 SYS_SET_HEAR_BFP

Configure the HEAR block floating point guard bits

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_hear.cfi
Template	`Sys_Set_HEAR_BFP( bfp )`
Description	Configure the HEAR block floating point guard bits
Inputs	bfp = Two-part parameter consisting of HEAR_BFP_FMD_[ENABLE | DISABLE] and HEAR_BFP_FORMAT_*BITS
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if bfp can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure the HEAR block floating point guard bits allowing 0 or 1`
	`// shifts and disabling full magnitude detection.`
	`Sys_Set_HEAR_BFP( HEAR_BFP_FORMAT_23BITS | HEAR_BFP_FMD_DISABLE )`

14.313 SYS_SET_HEAR_FUNCTIONCHAIN_VECTOR

Set a function chain vector to the specified function chain entry point

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_hear.cfi
Template	`Sys_Set_HEAR_FunctionChain_Vector( fc_pos, fc_entry )`
Description	Set a function chain vector to the specified function chain entry point
Inputs	fc_pos = HEAR function chain vector to set; use HEAR_FC_*_POS fc_entry = HEAR function chain entry point; use the _entry symbols generated by the HCT.
Outputs	None
Register Modifications	y1
Assumptions	Previous HEAR function chain vector is no longer needed
Clock Cycles	3

Example	`// Set the function chain vector for command 2 to`
	`// a previously unbound function chain.`
	`Sys_Set_HEAR_FunctionChain_Vector( HEAR_FC_CMD2_POS, \`
	`unbound_function_chain_entry )`

14.314 SYS_SET_IDRX_CFG

Configure an input dynamic range extended (IDRX) input channel

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_Set_IDRX_CFG( channel, config )`
Description	Configure an input dynamic range extended (IDRX) input channel
Inputs	channel = IDRX input channel; use 0 or 1 config = IDRX input channel configuration; use IDRX_CHANNEL_[LOW | HIGH], IDRX_INACTIVE_ADC_[ENABLED | DISABLED], IDRX_LOW_GAIN_[ENABLED | DISABLED], IDRX_[DISABLE | ENABLE], IDRX_MAX_PREAMP_GAIN_*DB, IDRX_MIN_PREAMP_GAIN_*DB, IDRX_HIGH_LIMIT_*DB, IDRX_LOW_LIMIT_*DB, IDRX_TIMER2_*, and IDRX_TIMER1_*
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if config can be sign-extended from 8 bits; 2 if config can be sign-extended from 24 bits; 3 otherwise

Example	`// Enable and configure IDRX1`
	`Sys_Set_IDRX_CFG( 1, IDRX_ENABLE | IDRX_INACTIVE_ADC_DISABLED | \`
	`IDRX_MIN_PREAMP_GAIN_0DB | \`
	`IDRX_MAX_PREAMP_GAIN_24DB | \`
	`IDRX_TIMER1_1024 | IDRX_TIMER2_1024 )`

14.315 SYS_SET_LSAD_CTRL

Set the control register A_LSAD_CTRL

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_lsad.cfi
Template	`Sys_Set_LSAD_CTRL( control_word )`
Description	Set the control register A_LSAD_CTRL
Inputs	control_word = An nine-part parameter consisting of LSAD_CTRL_GND_MODE_[TRIMMER | AUDIO_OUT], LSAD_CTRL_VBAT_MODE_[TRIMMER | AUDIO_OUT], LSAD_CTRL_CH5_MODE_[TRIMMER | AUDIO_OUT], LSAD_CTRL_CH4_MODE_[TRIMMER | AUDIO_OUT], LSAD_CTRL_CH3_MODE_[TRIMMER | AUDIO_OUT], LSAD_CTRL_CH2_MODE_[TRIMMER | AUDIO_OUT], LSAD_CTRL_CH1_MODE_[TRIMMER | AUDIO_OUT], LSAD_CTRL_CH0_MODE_[TRIMMER | AUDIO_OUT] and LSAD_CTRL_[DISABLE | PRESCALE_*]
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if control_word can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure the LSAD with the following settings:`
	`//  - Channel 0-3: Configure to audio out mode`
	`//  - Channel 4,5: Configure to trimmer mode`
	`//  - VBAT, GND: Configure to audio out mode`
	`//  - Configure the LSAD sampling frequency to`
	`//  have a prescale of 320`
	`Sys_Set_LSAD_CTRL( LSAD_CTRL_CH0_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_CH1_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_CH2_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_CH3_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_CH4_MODE_TRIMMER | \`
	`LSAD_CTRL_CH5_MODE_TRIMMER | \`
	`LSAD_CTRL_VBAT_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_GND_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_PRESCALE_320 )`

14.316 SYS_SET_MULTISYNCCONFIG

Select the clock used for multi-chip synchronization

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_clk.cfi
Template	`Sys_Set_MultiSyncConfig( config )`
Description	Select the clock used for multi-chip synchronization
Inputs	config = Clock for multi-chip synchronization; use MULT_SYNC_SLAVE_[OFF | SYSCLK | SLOWCLK | SDMCLK]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Synchronize the devices based on SLOWCLK` `Sys_Set_MultiSyncConfig( MULT_SYNC_SLAVE_SLOWCLK )`

14.317 SYS_SET_O_DAC_AO_CTRL

Configure the analog outputs for both output channels

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_Set_O_DAC_AO_CTRL( config )`
Description	Configure the analog outputs for both output channels
Inputs	config = Analog output configuration; use DIFF_OUT_[CH0 | CH1], DIFF_OUT_[DISABLE | ENABLE], AO*_LFP_OUT_ACTIVE_*KHZ, AO*_[DISABLE | ENABLE], AO*_CURRENT_*_*X, AO*_ATTENUATE_[*DB | BYPASS] and AO*_[UNMUTE | MUTE]
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if config can be sign-extended from 8 bits; 2 otherwise

Example	`// Enable both analog outputs, configuring them to apply 18dB of`
	`// attenuation and support a low-bandwidth (12kHz) output`
	`Sys_Set_O_DAC_AO_CTRL( AO0_ENABLE | AO0_CURRENT_1_0X | \`
	`AO0_ATTENUATE_18DB | \`
	`AO0_LFP_OUT_ACTIVE_12KHZ | AO0_UNMUTE | \`
	`AO1_ENABLE | AO1_CURRENT_1_0X | \`
	`AO1_ATTENUATE_18DB | \`
	`AO1_LFP_OUT_ACTIVE_12KHZ | AO1_UNMUTE )`

14.318 SYS_SET_O_DAC_GF_CTRL

Configure the output stage gain factor control settings

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_Set_O_DAC_GF_CTRL( config )`
Description	Configure the output stage gain factor control settings
Inputs	config = Output gain configuration; use O_DAC*_GF_CTRL_[DISABLE | ENABLE | BY_DIO] and constants shifted to O_DAC_GF_CTRL_DA*_GF_POS
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if config can be sign-extended from 8 bits; 2 if config can be sign-extended from 24 bits; 3 otherwise

Example	`// Enable the DAC gain factor controls, and set the gain factor`
	`// to a value of 0x350 to obtain less than unity gain across the`
	`// interpolation filter for both channels.`
	`Sys_Set_O_DAC_GF_CTRL( O_DAC0_GF_CTRL_ENABLE | \`
	`(0x350 << O_DAC_GF_CTRL_DA0_GF_POS) | \`
	`O_DAC1_GF_CTRL_ENABLE | \`
	`(0x350 << O_DAC_GF_CTRL_DA1_GF_POS) )`

14.319 SYS_SET_O_SDM_CTRL

Configure the sigma-delta modulator

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_Set_O_SDM_CTRL( config )`
Description	Configure the sigma-delta modulator
Inputs	config = Output modulator configuration; use SDM_TRUNCATE_[DISABLE | ENABLE], OD_DELAY_[DISABLE | ENABLE], L_OFFSET_*, DITHER_[DISABLE | ENABLE], DC_REMOVE_[DISABLE | ENABLE], and S_OFFSET_*
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if config can be sign-extended from 8 bits; 2 otherwise

Example	`// Reconfigure the sigma-delta modulator to its default settings`
	`Sys_Set_O_SDM_CTRL( SDM_TRUNCATE_DISABLE | \`
	`OD_DELAY_ENABLE | \`
	`L_OFFSET_NONE | \`
	`DITHER_ENABLE | \`
	`DC_REMOVE_ENABLE | \`
	`S_OFFSET_NONE )`

14.320 SYS_SET_O_SDMCLK

Configure the sigma-delta modulator prescaler

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_Set_O_SDMCLK( config )`
Description	Configure the sigma-delta modulator prescaler
Inputs	config = Output modulator prescaler; use SDMCLK_PRESCALE_*
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Configure the SDMCLK prescaler to divide SYSCLK by 2` `Sys_Set_O_SDMCLK( SDMCLK_PRESCALE_2 )`

14.321 SYS_SET_OSC_CTRL

Set the internal oscillator frequency, enable and multiplier

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Set_OSC_CTRL( ctrl0, ctrl1 )`
Description	Set the internal oscillator frequency, enable and multiplier
Inputs	ctrl0 = Frequency control setting for A_OSC_CTRL_0; use a constant ctrl1 = Multiplier and enable settings to apply to the oscillator using A_OSC_CTRL_1; use OSC_MULTIPLY_BY_*, and OSC_[DISABLE | ENABLE]
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2 if ctrl0 can be sign-extended from 8 bits; 3 otherwise
Example	`// Enable a system clock at 2.56 MHz (nominal) with a multiplier of 2` `Sys_Set_OSC_CTRL( OSC_FREQ_2M56, OSC_ENABLE | OSC_MULTIPLY_BY_2 )`

14.322 SYS_SET_OSC_CTRL_0

Set the oscillator frequency

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Set_OSC_CTRL_0( ctrl0 )`
Description	Set the oscillator frequency. This does not adjust the oscillator frequency multiplier.
Inputs	ctrl0 = Frequency control setting for A_OSC_CTRL_0; use a constant
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if ctrl0 can be sign-extended from 8 bits; 2 otherwise
Example	`// Set the system clock to a nominal 2.56 MHz` `Sys_Set_OSC_CTRL_0( OSC_FREQ_2M56 )`

14.323 SYS_SET_OSC_CTRL_1

Set the oscillator multiplier and enable

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Set_OSC_CTRL_1( ctrl1 )`
Description	Set the oscillator multiplier and enable. Does not adjust the oscillator frequency configuration.
Inputs	ctrl1 = Multiplier and enable settings to apply to the oscillator using A_OSC_CTRL_1; use OSC_MULTIPLY_BY_*, and OSC_[DISABLE | ENABLE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Enable the system clock with a multiplier of 2` `Sys_Set_OSC_CTRL_1( OSC_ENABLE | OSC_MULTIPLY_BY_2 )`

14.324 SYS_SET_OSCCOMP

Configure the internal oscillator's temperature compensation hardware

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Set_OSCComp( config )`
Description	Configure the internal oscillator's temperature compensation hardware.
Inputs	config = Temperature compensation configuration; use POS_TEMP_*_CURRENT or a constant shifted to OSC_CTRL_2_POS_TEMP_CTRL_POS for positive (PTAT) temperature compensation, NEG_TEMP_*_CURRENT or a constant shifted to OSC_CTRL_2_NEG_TEMP_CTRL_POS for negative (CTAT) temperature compensation, and OSC_TEMP_CTRL_[DISABLE | ENABLE]
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if config can be sign-extended from 8 bits; 2 otherwise
Example	`// Disable the oscillator temperature compensation support` `Sys_Set_OSCComp( OSC_TEMP_CTRL_DISABLE )`

14.325 SYS_SET_OUTPUT_CTRL

Configure the output driver configuration settings

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_Set_OUTPUT_CTRL( config )`
Description	Configure the output driver configuration settings
Inputs	config = Output driver configurations; use OD*_NEG_EDGE_[FALLING | RISING], OD*_POS_EDGE_[FALLING | RISING], OD*_HIGHPOWER_[DISABLE | ENABLE], and OD*_[DISABLE | ENABLE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the output stage to enable the output drivers, but not`
	`// the high-power outputs.`
	`Sys_Set_OUTPUT_CTRL( OD0_ENABLE | OD0_HIGHPOWER_DISABLE | \`
	`OD1_ENABLE | OD1_HIGHPOWER_DISABLE )`

14.326 SYS_SET_PLL_CLOCKSYNC

Configure the PLL and SDMCLK clock synchronization

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_clk.cfi
Template	`Sys_Set_PLL_ClockSync( config )`
Description	Configure the PLL and SDMCLK clock synchronization
Inputs	config = Clock synchronization config; use CLK_RESYNC_INT_[ENABLE | DISABLE], CLK_SYNC_TOLERANCE_[HIGH | LOW], and CLK_SYNC_[ENABLE | DISABLE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Disable the PLL and SDMCLK clock synchronization`
	`Sys_Set_PLL_ClockSync( CLK_RESYNC_INT_DISABLE | CLK_SYNC_DISABLE |\`
	`CLK_SYNC_TOLERANCE_LOW )`

14.327 SYS_SET_PLL_DIGITALCAL

Set the PLL digital calibration configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Set_PLL_DigitalCal( config )`
Description	Set the PLL digital calibration configuration
Inputs	config = PLL digital calibration configuration; use CAL_NBITS_*, CAL_MSB_[DIS | EN], and CAL_REF_DIV_*
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Configure the PLL digital calibration` `Sys_Set_PLL_DigitalCal( CAL_NBITS_12 | CAL_MSB_EN | CAL_REF_DIV_32 )`

14.328 SYS_SET_PLL_FREQUENCY

Configure the parameters that are used to set the PLL frequency; if the VCO was not previously enabled, this macro enables it

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Set_PLL_Frequency( PLL_R, PLL_M, PLL_X, PLL_P )`
Description	Configure the parameters that are used to set the PLL frequency; if the VCO was not previously enabled, this macro enables it.
Inputs	PLL_R = Input reference clock divider control PLL_M = VCO clock divider control PLL_X = Oscillator clock multiplier control; use OSC_MULTIPLY_BY_* PLL_P = Output clock divider
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	4

Example	`// Configure the PLL using the 4 multipliers and divisors that are`
	`// used to set the resulting PLL frequency`
	`Sys_Set_PLL_Frequency( R_VAL, M_VAL, X_VAL, P_VAL )`

14.329 SYS_SET_PLL_LOCKINTERRUPT

Set the PLL lock interrupt configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Set_PLL_LockInterrupt( config )`
Description	Set the PLL lock interrupt configuration
Inputs	config = PLL lock interrupt configuration; use PLL_LOCK_INT_[DISABLE | LOCKED | UNLOCKED | ACTIVITY_CHANGE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the PLL lock interrupt to detect PLL lock`
	`// status changes`
	`Sys_Set_PLL_LockInterrupt( PLL_LOCK_INT_ACTIVITY_CHANGE )`

14.330 SYS_SET_PLL_LOOPFILTERANDVCOGAIN

Set the PLL loop filter and VCO gain configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Set_PLL_LoopFilterAndVCOGain( config )`
Description	Set the PLL loop filter and VCO gain configuration
Inputs	config = PLL loop filter and VCO gain configuration; use VCO_GAIN_* or a constant shifted to PLL_CTRL_4_VCO_GAIN_POS, and constants shifted to PLL_CTRL_4_LF_RES_CFG_POS and PLL_CTRL_4_LF_CAP_CFG_POS
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if config can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure the PLL digital calibration`
	`Sys_Set_PLL_LoopFilterAndVCOGain( VCO_GAIN_25M | \`
	`(0x1A << PLL_CTRL_4_LF_CAP_CFG_POS) | \`
	`(0x01 << PLL_CTRL_4_LF_RES_CFG_POS))`

14.331 SYS_SET_PLL_MODECONTROL

Configure the PLL mode

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Set_PLL_ModeControl( config )`
Description	Configure the PLL mode
Inputs	config = PLL mode control setting; use PLL_[ENABLE | DISABLE], PLL_[TO_LOCK | FREE_RUN], and [INTERNAL | EXTERNAL]_CALIBRATION_MODE
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Enable the PLL in free run mode`
	`Sys_Set_PLL_ModeControl( PLL_ENABLE | PLL_FREE_RUN | \`
	`INTERNAL_CALIBRATION_MODE )`

14.332 SYS_SET_PLL_REFCLKDETECTCONFIG

Set the PLL reference clock detection configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Set_PLL_RefClkDetectConfig( config )`
Description	Set the PLL reference clock detection configuration
Inputs	config = PLL reference clock detection configuration; use CLK_DET_[ENABLE | DISABLE], CLK_DET_SLOW_CLK_DIV*, and CLK_DET_INT_[DISABLE | ACTIVATED | DEACTIVATED | ACTIVITY_CHANGE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Disable PLL reference clock detection` `Sys_Set_PLL_RefClkDetectConfig( CLK_DET_DISABLE )`

14.333 SYS_SET_PLL_TEMPCOMPENSATION

Set the PLL temperature compensation configuration

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_Set_PLL_TempCompensation( config )`
Description	Set the PLL temperature compensation configuration
Inputs	config = PLL temperature compensation configuration; use PLL_V2I_TEMP_COEF_* or a constant shifted to PLL_CTRL_5_V2I_TEMP_CFG_POS
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Set the default PLL temperature compensation trim` `Sys_Set_PLL_TempCompensation( PLL_V2I_TEMP_COEF_DFLT )`

14.334 SYS_SET_SDU_CFG

Configure the signal detection unit (SDU)

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_Set_SDU_CFG( config )`
Description	Configure the signal detection unit (SDU)
Inputs	config = Sampling frequency configuration; use SDU_HIGH_FREQ_[DISABLE | ENABLE], SDU_LINEOUT_[DISABLE | ENABLE], SDU_SFCR_*, SDU_P_CURRENT_*_*X, SDU_ADC_CURRENT_*_*X, SDU_PGAIN_[BYPASS | *DB], SDU_[AI* | NO_INPUT] and SDU_[DISABLE | ENABLE]
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if config can be sign-extended from 8 bits; 2 if config can be sign-extended from 24 bits; 3 otherwise

Example	`// Configure the SDU to monitor AI5 with no preamp gain`
	`Sys_Set_SDU_CFG( SDU_ENABLE | SDU_AI5 | SDU_PGAIN_BYPASS | \`
	`SDU_P_CURRENT_1_0X | SDU_ADC_CURRENT_1_0X | \`
	`SDU_SFCR_1280 | SDU_LINEOUT_DISABLE )`

14.335 SYS_SET_SUP_THRSH_DATA

Set the voltage threshold when the supply threshold counter (SUP_COUNT) will be updated

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_pwr.cfi
Template	`Sys_Set_SUP_THRSH_DATA( threshold )`
Description	Set the voltage threshold when the supply threshold counter (SUP_COUNT) will be updated
Inputs	threshold = Value to load to A_SUP_THRSH_DATA
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Set the battery monitor threshold to 0.95 volts` `Sys_Set_SUP_THRSH_DATA( 0x79 )`

14.336 SYS_SET_VDBL_CPCLK

Set the clock divisor for the VDBL charge pump

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_clk.cfi
Template	`Sys_Set_VDBL_CPClk( config )`
Description	Set the clock divisor for the VDBL charge pump
Inputs	config = VDBL charge pump clock divisor; use a constant
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Set the VDBL charge-pump clock divisor to the default setting` `Sys_Set_VDBL_CPClk( 0xF )`

14.337 SYS_SET_VDDA_CPCLK

Set the clock divisor for the VDDA charge pump

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_clk.cfi
Template	`Sys_Set_VDDA_CPClk( config )`
Description	Set the clock divisor for the VDDA charge pump
Inputs	config = VDDA charge pump clock divisor; use a constant
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Set the VDDA charge-pump clock divisor to the default setting` `Sys_Set_VDDA_CPClk( 0x0 )`

14.338 SYS_SET_VDDC_CPCLK

Set the clock divisor for the VDDC charge pump

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_clk.cfi
Template	`Sys_Set_VDDC_CPClk( config )`
Description	Set the clock divisor for the VDDC charge pump
Inputs	config = VDDC charge pump clock divisor; use a constant
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Set the VDDC charge-pump clock divisor to the default setting` `Sys_Set_VDDC_CPClk( 0x0 )`

14.339 SYS_SET_VDDM_CPCLK

Set the clock divisor for the VDDM charge pump

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_clk.cfi
Template	`Sys_Set_VDDM_CPClk( config )`
Description	Set the clock divisor for the VDDM charge pump
Inputs	config = VDDM charge pump clock divisor; use a constant
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Set the VDDM charge-pump clock divisor to the default setting` `Sys_Set_VDDM_CPClk( 0x0 )`

14.340 SYS_SET_VMIC_SW_CTRL

Configure the source for VMIC

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_pwr.cfi
Template	`Sys_Set_VMIC_SW_CTRL( config )`
Description	Configure the source for VMIC
Inputs	config = VMIC switch configuration; use VMIC_SW_[OFF | VREG | VDDA | VDBL]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Configure VREG as the source for VMIC` `Sys_Set_VMIC_SW_CTRL( VMIC_SW_VREG )`

14.341 SYS_SETREG_ADC_CFG

Configure the ADC for one of the input channels

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_SetReg_ADC_CFG( channel, reg )`
Description	Configure the ADC for one of the input channels
Inputs	channel = ADC to apply this configuration to; use a constant between 0 and 3 reg = Register containing the desired ADC configuration to be stored to A_ADC_CFG_*
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Reconfigure ADC1 using configuration settings previously loaded`
	`// to y0`
	`Sys_SetReg_ADC_CFG( 1, y0 )`

14.342 SYS_SETREG_ADC_CTRL

Configure the ADC control settings for one of the input channels and calibrate the ADC DC gain offset

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_SetReg_ADC_CTRL( channel, reg )`
Description	Configure the ADC control settings for one of the input channels and calibrate the ADC DC gain offset
Inputs	channel = ADC to apply this control configuration to; use a constant between 0 and 3 reg = Register containing the desired ADC control configuration to store to A_ADC_CTRL_*
Outputs	None
Register Modifications	y1, b1, xp7, yp3
Assumptions	None
Clock Cycles	Not available

Example	`// Reconfigure ADC1 using control configuration settings previously`
	`// stored to y0`
	`Sys_SetReg_ADC_CTRL( 1, y0 )`

14.343 SYS_SETREG_ADC_CTRL_NOCALIBRATION

Configure the ADC control settings for one of the input channels

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_SetReg_ADC_CTRL_NoCalibration( channel, reg )`
Description	Configure the ADC control settings for one of the input channels
Inputs	channel = ADC to apply this control configuration to; use a constant between 0 and 3 reg = Register containing the desired ADC control configuration to store to A_ADC_CTRL_*
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Reconfigure ADC1 using control configuration settings previously`
	`// stored to y0 (do not calibrate the ADC)`
	`Sys_SetReg_ADC_CTRL_NoCalibration( 1, y0 )`

14.344 SYS_SETREG_ADC_DEC_CTRL

Configure the decimation filter for one of the input channels

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_SetReg_ADC_DEC_CTRL( channel, reg )`
Description	Configure the decimation filter for one of the input channels
Inputs	channel = Decimation filter to apply this control configuration to; use a constant between 0 and 3 reg = Accumulator containing the desired decimation filter configuration
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the decimation filter for input channel 1 using`
	`// configuration settings stored in a0`
	`Sys_SetReg_ADC_DEC_CTRL( 1, a0 )`

14.345 SYS_SETREG_ADC_SAMPLE_FREQ_CFG

Configure the sampling frequency

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_SetReg_ADC_SAMPLE_FREQ_CFG( reg )`
Description	Configure the sampling frequency
Inputs	reg = Register containing the desired sampling frequency configuration
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Restore the previously stored sampling frequency configuration`
	`// from y0`
	`Sys_SetReg_ADC_SAMPLE_FREQ_CFG( y0 )`

14.346 SYS_SETREG_CP_VDBL_CTRL

Set the digital interface supply voltage (VDBL) charge pump configuration from a register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_pwr.cfi
Template	`Sys_SetReg_CP_VDBL_CTRL( reg )`
Description	Set the digital interface supply voltage (VDBL) charge pump configuration from a register
Inputs	reg = Register to load the VDBL charge pump configuration from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Restore a previously loaded digital interface supply charge pump`
	`// configuration from x0`
	`Sys_SetReg_CP_VDBL_CTRL( x0 )`

14.347 SYS_SETREG_CP_VDDA_CTRL

Set the analog supply voltage (VDDA) charge pump configuration from a register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_pwr.cfi
Template	`Sys_SetReg_CP_VDDA_CTRL( reg )`
Description	Set the analog supply voltage (VDDA) charge pump configuration from a register
Inputs	reg = Register to load the VDDA charge pump configuration from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Restore a previously loaded analog supply charge pump`
	`// configuration from x0`
	`Sys_SetReg_CP_VDDA_CTRL( x0 )`

14.348 SYS_SETREG_DMIC_OUTPUT_CTRL

Configure the DMIC output channels

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_SetReg_DMIC_OUTPUT_CTRL( reg )`
Description	Configure the DMIC output channels
Inputs	reg = Register to load the DMIC output configuration from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Restore a previously loaded DMIC output configuration from x0` `Sys_SetReg_DMIC_OUTPUT_CTRL( x0 )`

14.349 SYS_SETREG_FENG_COEFFOFFSET

Set the Filter Engine coefficient offset

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_SetReg_FENG_CoeffOffset( reg )`
Description	Set the Filter Engine coefficient offset
Inputs	reg = Register containing the new Filter Engine coefficient offset
Outputs	None
Register Modifications	b1
Assumptions	reg is not b1 or a component of b1
Clock Cycles	3

Example	`// Reconfigure the coefficient offset using the value previously`
	`// loaded to y0`
	`Sys_SetReg_FENG_CoeffOffset( y0 )`

14.350 SYS_SETREG_FENG_COEFFOFFSETENABLE_ACTIVE

Select which Filter Engine coefficient sets use the offset by writing the enable value to the active register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_SetReg_FENG_CoeffOffsetEnable_Active( reg )`
Description	Select which Filter Engine coefficient sets use the offset by writing the enable value to the active register
Inputs	reg = Accumulator containing the new Filter Engine coefficient offset enable value
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Update the Filter Engine coefficient offset using the value`
	`// previously stored in a0. This takes effect immediately.`
	`Sys_SetReg_FENG_CoeffOffsetEnable_Active( a0 )`

14.351 SYS_SETREG_FENG_COEFFOFFSETENABLE_SHADOW

Select which Filter Engine coefficient sets use the offset by writing the enable value to the shadow register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_SetReg_FENG_CoeffOffsetEnable_Shadow( reg )`
Description	Select which Filter Engine coefficient sets use the offset by writing the enable value to the shadow register. The new setting will take affect the next time Filter Engine execution starts.
Inputs	reg = Accumulator containing the new Filter Engine coefficient offset enable value
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Update the Filter Engine coefficient offset enable using the value`
	`// previously stored in a0`
	`Sys_SetReg_FENG_CoeffOffsetEnable_Shadow( a0 )`

14.352 SYS_SETREG_FENG_INTENABLE

Configure the possible sources for the Filter Engine interrupt

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_SetReg_FENG_IntEnable( reg )`
Description	Configure the possible sources for the Filter Engine interrupt
Inputs	reg = Accumulator register that holds the current Filter Engine interrupt configuration (a set of INT_DIS_FENG_* and INT_EBL_FENG_* defines)
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Restore the previous Filter Engine interrupt source settings now`
	`// that coefficient updates have been completed.`
	`Sys_SetReg_FENG_IntEnable( a0 )`

14.353 SYS_SETREG_FENG_STATE

Configure the Filter Engine state codes by writing to the active register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_SetReg_FENG_State( reg )`
Description	Configure the Filter Engine state codes by writing to the active register
Inputs	reg = Register containing the new Filter Engine state codes; use 1 (condition met) or 0 (condition not met) shifted by the state code index for each state code
Outputs	None
Register Modifications	None
Assumptions	The Filter Engine is inactive
Clock Cycles	1

Example	`// Restore the previously stored Filter Engine state codes`
	`// immediately`
	`Sys_SetReg_FENG_State( y0 )`

14.354 SYS_SETREG_FENG_STATE_SHADOW

Configure the Filter Engine state codes by writing to the shadow register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_SetReg_FENG_State_Shadow( reg )`
Description	Configure the Filter Engine state codes by writing to the shadow register. These new settings take effect the next time Filter Engine execution starts.
Inputs	reg = Register containing the new Filter Engine state codes; use 1 (condition met) or 0 (condition not met) shifted by the state code index for each state code
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Restore the previously stored Filter Engine state codes` `Sys_SetReg_FENG_State_Shadow( y0 )`

14.355 SYS_SETREG_FENG_STATEWRITEENABLE

Configure the Filter Engine state code write-enable settings using a register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_feng.cfi
Template	`Sys_SetReg_FENG_StateWriteEnable( reg )`
Description	Configure the Filter Engine state code write-enable settings using a register
Inputs	reg = Register containing the new Filter Engine state code write-enable settings; use a set of shifted constants where 1 indicates the Filter Engine state code can be written for each state code
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Restore the previously stored Filter Engine state code`
	`// write-enable settings`
	`Sys_SetReg_FENG_StateWriteEnable( y0 )`

14.356 SYS_SETREG_HEAR_BFP_BLOCK_EXP

Set the value of the block floating point (BFP) block exponent from a register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_hear.cfi
Template	`Sys_SetReg_HEAR_BFP_BLOCK_EXP( reg )`
Description	Set the value of the block floating point (BFP) block exponent from a register.
Inputs	reg = Register containing the value to be written to the BFP block exponent
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the new BFP exponent value into x0`
	`load x0, 5`
	
	`// Store the value loaded into x0 into the block floating point`
	`// exponent used by the HEAR`
	`Sys_SetReg_HEAR_BFP_BLOCK_EXP( x0 )`

14.357 SYS_SETREG_IDRX_CFG

Configure an input dynamic range extended (IDRX) input channel

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_SetReg_IDRX_CFG( channel, reg )`
Description	Configure an input dynamic range extended (IDRX) input channel
Inputs	channel = IDRX input channel; use 0 or 1 reg = Accumulator containing the desired IDRX filter configuration
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Reconfigure IDRX0 using configuration settings previously loaded`
	`// to b0`
	`Sys_SetReg_IDRX_CFG( 1, b0 )`

14.358 SYS_SETREG_LSAD_CTRL

Set the control register A_LSAD_CTRL from a specified register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_lsad.cfi
Template	`Sys_SetReg_LSAD_CTRL( reg )`
Description	Set the control register A_LSAD_CTRL from a specified register
Inputs	reg = Register to load A_LSAD_CTRL from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the following settings into CFX register y0:`
	`//  - Channel 0-3: Configure to audio out mode`
	`//  - Channel 4,5: Configure to trimmer mode`
	`//  - VBAT, GND: Configure to audio out mode`
	`//  - Configure the LSAD sampling frequency to`
	`//  have a prescale of 320`
	`load y0, ( LSAD_CTRL_CH0_MODE_AUDIO_OUT |`
	`  LSAD_CTRL_CH1_MODE_AUDIO_OUT |`
	`  LSAD_CTRL_CH2_MODE_AUDIO_OUT |`
	`  LSAD_CTRL_CH3_MODE_AUDIO_OUT |`
	`  LSAD_CTRL_CH4_MODE_TRIMMER |`
	`  LSAD_CTRL_CH5_MODE_TRIMMER |`
	`  LSAD_CTRL_VBAT_MODE_AUDIO_OUT |`
	`  LSAD_CTRL_GND_MODE_AUDIO_OUT |`
	`  LSAD_CTRL_PRESCALE_320 )`
	
	`// Save the settings in CFX register y0 to A_LSAD_CTRL`
	`Sys_SetReg_LSAD_CTRL( y0 )`

14.359 SYS_SETREG_O_DAC_AO_CTRL

Configure the analog outputs for both output channels

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_SetReg_O_DAC_AO_CTRL( reg )`
Description	Configure the analog outputs for both output channels
Inputs	reg = Register to load the analog output configurations from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Restore a previously loaded analog output configuration from x0` `Sys_SetReg_O_DAC_AO_CTRL( x0 )`

14.360 SYS_SETREG_O_DAC_GF_CTRL

Configure the output stage gain factor control settings

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_SetReg_O_DAC_GF_CTRL( reg )`
Description	Configure the output stage gain factor control settings
Inputs	reg = Accumulator to load output gain configuration from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load a0 with settings to enable the gain factor control, and`
	`// set the gain factor to a value of 0x400 (to obtain unity gain`
	`// across the interpolation filter). Disable the gain factor`
	`// on output channel 1.`
	`load a0, ( O_DAC0_GF_CTRL_ENABLE | 0x400 | O_DAC1_GF_CTRL_DISABLE )`
	`Sys_SetReg_O_DAC_GF_CTRL( a0 )`

14.361 SYS_SETREG_O_SDM_CTRL

Configure the sigma-delta modulator

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_SetReg_O_SDM_CTRL( reg )`
Description	Configure the sigma-delta modulator
Inputs	reg = Register to load the output modulator configuration from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Restore the sigma-delta modulator configuration temporarily stored`
	`// to y0`
	`Sys_SetReg_O_SDM_CTRL( y0 )`

14.362 SYS_SETREG_O_SDMCLK

Configure the sigma-delta modulator prescaler

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_SetReg_O_SDMCLK( reg )`
Description	Configure the sigma-delta modulator prescaler
Inputs	reg = Register to load the output modulator prescaler from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load x0 with the desired SDMCLK prescaler`
	`load x0, SDMCLK_PRESCALE_1`
	`// Configure the SDMCLK prescaler using the value in x0`
	`Sys_SetReg_O_SDMCLK( x0 )`

14.363 SYS_SETREG_OSC_CTRL_0

Set the oscillator frequency

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_SetReg_OSC_CTRL_0( reg )`
Description	Set the oscillator frequency. This does not adjust the oscillator frequency multiplier.
Inputs	reg = Register containing the frequency control setting for A_OSC_CTRL_0
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Reset the system clock to the setting previously stored in x0` `Sys_SetReg_OSC_CTRL_0( x0 )`

14.364 SYS_SETREG_OSC_CTRL_1

Set the oscillator multiplier and enable

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_SetReg_OSC_CTRL_1( reg )`
Description	Set the oscillator multiplier and enable. Does not adjust the oscillator frequency configuration.
Inputs	reg = Register containing multiplier and enable settings to apply to the oscillator using A_OSC_CTRL_1
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Enable the oscillator and restore the system clock multiplier`
	`// stored in x0`
	`or x0, x0, OSC_ENABLE ||| nop`
	`Sys_SetReg_OSC_CTRL_1( x0 )`

14.365 SYS_SETREG_OUTPUT_CTRL

Configure the output driver configuration settings

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_out.cfi
Template	`Sys_SetReg_OUTPUT_CTRL( channel, reg )`
Description	Configure the output driver configuration settings
Inputs	reg = Register containing the desired output driver configurations
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the output settings to x0. Configure the output stage`
	`// to enable both output drivers, but not the high-power outputs.`
	`load x0, ( OD0_ENABLE | OD0_HIGHPOWER_DISABLE | \`
	`OD1_ENABLE | OD1_HIGHPOWER_DISABLE )`
	`// Save the settings in CFX register x0 to A_OUTPUT_CTRL`
	`Sys_SetReg_OUTPUT_CTRL( x0 )`

14.366 SYS_SETREG_PLL_DIGITALCAL

Set the PLL digital calibration configuration from a register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_SetReg_PLL_DigitalCal( reg )`
Description	Set the PLL digital calibration configuration from a register
Inputs	reg = Register to load the PLL digital calibration configuration from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the PLL digital calibration using the settings stored`
	`// in x0`
	`Sys_SetReg_PLL_DigitalCal( x0 )`

14.367 SYS_SETREG_PLL_LOOPFILTERANDVCOGAIN

Set the PLL loop filter and VCO gain configuration from a register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_SetReg_PLL_LoopFilterAndVCOGain( reg )`
Description	Set the PLL loop filter and VCO gain configuration from a register
Inputs	reg = Register to load the loop filter and VCO gain configuration from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the PLL loop filter and VCO gain using the settings`
	`// stored in x1`
	`Sys_SetReg_PLL_LoopFilterAndVCOGain( x1 )`

14.368 SYS_SETREG_PLL_REFCLKDETECTCONFIG

Set the PLL reference clock detection configuration from a register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_SetReg_PLL_RefClkDetectConfig( reg )`
Description	Set the PLL reference clock detection configuration from a register
Inputs	reg = Register to load the PLL reference clock detection configuration from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Set the PLL reference clock detection configuration using the`
	`// setting stored in x1`
	`Sys_SetReg_PLL_RefClkDetectConfig( x1 )`

14.369 SYS_SETREG_PLL_TEMPCOMPENSATION

Set the PLL temperature compensation configuration from a register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_clk.cfi
Template	`Sys_SetReg_PLL_TempCompensation( reg )`
Description	Set the PLL temperature compensation configuration from a register
Inputs	reg = Register to load the PLL temperature compensation configuration from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Set the PLL temperature compensation trim using the`
	`// setting stored in x1`
	`Sys_SetReg_PLL_TempCompensation( x1 )`

14.370 SYS_SETREG_SDU_CFG

Configure the signal detection unit (SDU)

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_in.cfi
Template	`Sys_SetReg_SDU_CFG( acc )`
Description	Configure the signal detection unit (SDU)
Inputs	acc = Accumulator containing the desired SDU configuration
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Restore the previously stored SDU configuration from b0` `Sys_SetReg_SDU_CFG( b0 )`

14.371 SYS_SETREG_SUP_THRSH_DATA

Set the voltage threshold when the supply threshold counter (SUP_COUNT) will be updated using a register

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_analog_pwr.cfi
Template	`Sys_SetReg_SUP_THRSH_DATA( reg )`
Description	Set the voltage threshold when the supply threshold counter (SUP_COUNT) will be updated using a register
Inputs	reg = Register to load A_SUP_THRSH_DATA from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the new setting for the A_SUP_THRSH_DATA register to x0,`
	`// and set the battery monitor threshold to 1.05 volts.`
	`load x0, 0x85`
	`Sys_SetReg_SUP_THRSH_DATA( x0 )`

14.372 SYS_SPI_CS_HIGH

Set the SPI_CS line high

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_SPI_CS_High`
Description	Set the SPI_CS line high
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	SPI interface enabled
Clock Cycles	1
Example	`// Set the SPI_CS line high` `Sys_SPI_CS_High`

14.373 SYS_SPI_CS_LOW

Set the SPI_CS line low

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_SPI_CS_Low`
Description	Set the SPI_CS line low
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	SPI interface enabled
Clock Cycles	1
Example	`// Set the SPI_CS line low` `Sys_SPI_CS_Low`

14.374 SYS_SPI_DISABLE

Disable the SPI interface (clears the SPI configuration)

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_SPI_Disable`
Description	Disable the SPI interface (clears the SPI configuration)
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Disable the SPI interface` `Sys_SPI_Disable`

14.375 SYS_SPI_ENABLE

Configure the SPI interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_SPI_Enable( spi_cfg )`
Description	Configure the SPI interface
Inputs	spi_cfg = SPI configuration; use SPI_CLK_PRESCALE_*, SPI_PORT_AUTO_[ENABLE | DISABLE], SPI_CLKPOL_[NORMAL | INVERSE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Enable the SPI interface in auto mode, with a normal polarity,`
	`// and the clock prescaled by 4`
	`Sys_SPI_Enable( SPI_CLK_PRESCALE_4 | SPI_PORT_AUTO_ENABLE | \`
	`SPI_CLKPOL_NORMAL )`

14.376 SYS_SPI_READ

Read the specified number of bits over the SPI interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_SPI_Read( bits )`
Description	Read the specified number of bits over the SPI interface. NOTE: The caller must wait until SPI operation is complete before starting another SPI operation; use Sys_SPI_WaitDone.
Inputs	bits = Number of bits to receive (between 1-32) Optional: OR in SPI_CS_HIGH to force CS = 1
Outputs	None
Register Modifications	None
Assumptions	SPI interface enabled
Clock Cycles	1

Example	`// Initiate a read of 32 bits from the SPI interface. Another SPI`
	`// operation cannot be initiated until the SPI interface returns to`
	`// its idle state.`
	`Sys_SPI_Read( 32 )`
	
	`// Block and wait for the 32 bits to be read from the SPI interface`
	`Sys_SPI_WaitDone`

14.377 SYS_SPI_WAITDONE

Wait for the SPI operation to complete; exit when the SPI interface is no longer busy

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_SPI_WaitDone`
Description	Wait for the SPI operation to complete; exit when the SPI interface is no longer busy.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	SPI interface enabled Valid stack pointer
Clock Cycles	Depends on SYSCLK, SPI prescale and number of bits remaining in the current SPI transaction

Example	`// Read 16 bits from the SPI interface`
	`Sys_SPI_Read( 16 )`
	
	`// Block and wait for the 16 bits to be read from the SPI interface`
	`Sys_SPI_WaitDone`

14.378 SYS_SPI_WRITE

Write the specified number of bits over the SPI interface

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_spi.cfi
Template	`Sys_SPI_Write( bits )`
Description	Write the specified number of bits over the SPI interface. NOTE: The caller must wait until SPI operation is complete before starting another SPI operation; use Sys_SPI_WaitDone.
Inputs	bits = Number of bits to send (between 1-32) Optional: OR in SPI_CS_HIGH to force CS = 1
Outputs	None
Register Modifications	None
Assumptions	SPI interface enabled
Clock Cycles	1

Example	`// Write 24 bits to the SPI interface`
	`Sys_SPI_Write( 24 )`
	`// Block and wait for the 24 bits to be written to the SPI interface`
	`Sys_SPI_WaitDone`

14.379 SYS_STACK_ALLOC

Allocate space on the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Stack_Alloc( words )`
Description	Allocate space on the stack
Inputs	words = Number of 24-bit words to allocate on stack
Outputs	None
Register Modifications	sp (sp = sp + words)
Assumptions	None
Clock Cycles	1
Example	`// Allocate 10 words on the stack` `Sys_Stack_Alloc( 10 )`

14.380 SYS_STACK_FREE

Free up previously allocated space on the stack

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Stack_Free( words )`
Description	Free up previously allocated space on the stack
Inputs	words = Number of 24-bit words to free on stack
Outputs	None
Register Modifications	sp (sp = sp - words)
Assumptions	None
Clock Cycles	1
Example	`// Free 10 previously allocated words on the stack` `Sys_Stack_Free( 10 )`

14.381 SYS_TIMER1_CONFIG

Configure general-purpose timer 1

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_timer.cfi
Template	`Sys_Timer1_Config( config )`
Description	Configure general-purpose timer 1
Inputs	config = Four-part parameter consisting of TIMER_[START | STOP], TIMER_[ONESHOT | FREERUN], Prescaler from SLOWCLK (TIMER_PRESCALE_*) and 12-bit delay (value between 0 and 4095)
Outputs	None
Register Modifications	y1
Assumptions	Current timer run state is no longer needed
Clock Cycles	2

Example	`// Configure the general-purpose timer 1 as a free-running timer,`
	`// triggering every 100ms for a slow clock of 1.28 MHz`
	`Sys_Timer1_Config( TIMER_FREERUN | TIMER_PRESCALE_32 | 4000 )`

14.382 SYS_TIMER1_START

Start general-purpose timer 1

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_timer.cfi
Template	`Sys_Timer1_Start`
Description	Start general-purpose timer 1
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Start general-purpose timer 1` `Sys_Timer1_Start`

14.383 SYS_TIMER1_STOP

Stop general-purpose timer 1

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_timer.cfi
Template	`Sys_Timer1_Stop`
Description	Stop general-purpose timer 1
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	3
Example	`// Stop general-purpose timer 1` `Sys_Timer1_Stop`

14.384 SYS_TIMER2_CONFIG

Configure general-purpose timer 2

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_timer.cfi
Template	`Sys_Timer2_Config( config )`
Description	Configure general-purpose timer 2
Inputs	config = Four-part parameter consisting of TIMER_[START | STOP], TIMER_[ONESHOT | FREERUN], Prescaler from SLOWCLK (TIMER_PRESCALE_*) and 12-bit delay (value between 0 and 4095)
Outputs	None
Register Modifications	y1
Assumptions	Current timer run state is no longer needed
Clock Cycles	2

Example	`// Configure the general-purpose timer 2 as a free-running timer,`
	`// triggering every 10ms for a slow clock of 1.28 MHz`
	`Sys_Timer2_Config( TIMER_FREERUN | TIMER_PRESCALE_32 | 400 )`

14.385 SYS_TIMER2_START

Start general-purpose timer 2

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_timer.cfi
Template	`Sys_Timer2_Start`
Description	Start general-purpose timer 2
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Start general-purpose timer 2` `Sys_Timer2_Start`

14.386 SYS_TIMER2_STOP

Stop general-purpose timer 2

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_timer.cfi
Template	`Sys_Timer2_Stop`
Description	Stop general-purpose timer 2
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	3
Example	`// Stop general-purpose timer 2` `Sys_Timer2_Stop`

14.387 SYS_TIMERS_START

Start both general-purpose timers

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_timer.cfi
Template	`Sys_Timers_Start`
Description	Start both general-purpose timers
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Start both general-purpose timers` `Sys_Timers_Start`

14.388 SYS_TIMERS_STOP

Stop both general-purpose timers

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_timer.cfi
Template	`Sys_Timers_Stop`
Description	Stop both general-purpose timers
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	6
Example	`// Stop both general-purpose timers` `Sys_Timers_Stop`

14.389 SYS_UART_DISABLE

Disable the UART

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_uart.cfi
Template	`Sys_UART_Disable`
Description	Disable the UART
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Disable the UART interface` `Sys_UART_Disable`

14.390 SYS_UART_ENABLE

Enable the UART at the specified baud rate

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_uart.cfi
Template	`Sys_UART_Enable( clk_speed, baud )`
Description	Enable the UART at the specified baud rate.
Inputs	clk_speed = SYSCLK clock speed baud = Baud rate to which the UART should be configured
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2

Example	`// Enable the UART interface for a communications baud rate of`
	`// 115,200 and a system clock speed of 5.12MHz`
	`Sys_UART_Enable( 5120000, 115200 )`

14.391 SYS_VALUE_XY

Define an XY value list that can be used with the .data directive and similar use cases

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_utils.cfi
Template	`Sys_Value_XY( high_bits, low_bits )`
Description	Define an XY value list that can be used with the .data directive and similar use cases
Inputs	high_bits = 24-bit value to be used for the Y portion of the XY data value low_bits = 24-bit value to be used for the X portion of the XY data value
Outputs	Data_List = Correctly ordered list that can be used to set the value of an XY data element
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Reserve a block of 10 48-bit words in XY memory. Initialize the`
	`// memory to 0x123456ABCDEF.`
	`Sys_Mem_Start_Segment( Table_SegXY, SYS_MEMSPACE_XY )`
	`Sys_Mem_Reserve_Init( Table_InitializedXY, 10,`
	`Sys_Value_XY( 0x123456, 0xABCDEF ) )`

14.392 SYS_WATCHDOG_REFRESH

Refresh the watchdog timer count

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_watchdog.cfi
Template	`Sys_Watchdog_Refresh`
Description	Refresh the watchdog timer count
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`mainloop:`
	`  goto.1d mainloop`
	`  Sys_Watchdog_Refresh  // Refresh the watchdog in delay slot`

14.393 SYS_WATCHDOG_TIMEOUTCONFIG

Configure the watchdog timeout

Type	Macro
Include File	`#include <sk4_sys.cfi>`
Source File	sk4_sys_watchdog.cfi
Template	`Sys_Watchdog_TimeoutConfig( timeout )`
Description	Configure the watchdog timeout
Inputs	timeout = Timeout value for watchdog; Use WATCHDOG_TIMEOUT_*
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Set timeout to a nominal frequency of 3.276 seconds (assumes a`
	`// 1.28 MHz SYSCLK frequency)`
	`Sys_Watchdog_TimeoutConfig( WATCHDOG_TIMEOUT_3276M8 )`

CHAPTER 15
ARM Cortex-M3 Header Files and System Library Reference

This reference chapter presents a detailed description of all the macros and inline functions defined in the ARM Cortex-M3 system include files. For each macro or inline function, it describes calling parameters, modified registers and returned values.

15.1 SYS_ACB_CLEAR_STATE

Clear the internal state of all audio codec blocks

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`void Sys_ACB_Clear_State( void )`
Description	Clear the internal state of all audio codec blocks
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Clear the current state for the audio codec blocks */` `Sys_ACB_Clear_State();`

15.2 SYS_ACB_DISABLEALL

Disable all audio codec blocks

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`void Sys_ACB_DisableAll( void )`
Description	Disable all audio codec blocks
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Disable all audio codec blocks */` `Sys_ACB_DisableAll();`

15.3 SYS_ACB_INT_ACK

Acknowledge all audio codec block interrupts

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`void Sys_ACB_Int_Ack( void )`
Description	Acknowledge all audio codec block interrupts
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Acknowledge all audio codec block interrupts */` `Sys_ACB_Int_Ack();`

15.4 SYS_ACB_INT_ENABLE

Enable/disable a set of audio codec block interrupts

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`void Sys_ACB_Int_Enable( uint32_t src )`
Description	Enable/disable a set of audio codec block interrupts
Inputs	src = ACB interrupt sources to enable; use INT_EBL_* and INT_DIS_*
Outputs	None
Assumptions	None

Example	`/* Enable the CVSD audio codec block interrupts */`
	`Sys_ACB_Int_Enable( INT_EBL_CVSD_ENC_IN_ERR |`
	`INT_EBL_CVSD_DEC_IN_ERR );`

15.5 SYS_CALC_BITBAND

Calculate the bit-band location for a given address and bit position

Type	Macro
Include File	`#include <sk4.h>`
Source File	sk4_reg.h
Template	`SYS_CALC_BITBAND(addr, pos)`
Description	Calculate the bit-band location for a given address and bit position
Inputs	addr = The bit-band address for which the bit-band alias is being calculated pos = The bit for which the bit-band alias is being calculated
Outputs	return = The bit-band alias address corresponding to this address and bit position
Assumptions	None

Example	`/* Calculate the address needed to access */`
	`uint32_t AppStatus;`
	`#define X_APP_STATUS_BITBAND_BASE SYS_CALC_BITBAND(AppStatus, 0)`

15.6 SYS_CFX_COMMAND

Issue a command to trigger a set of CFX interrupts

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_cfx.h
Template	`void Sys_CFX_Command( uint32_t cmd )`
Description	Issue a command to trigger a set of CFX interrupts
Inputs	cmd = Command or commands to issue to the CFX; use CFX_CMD_*
Outputs	None
Assumptions	None
Example	`/* Issue CFX command 0 */` `Sys_CFX_Command(CFX_CMD_0);`

15.7 SYS_CM3_DELAY

Delay by at least the specified number of clock cycles

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_delay.sx
Template	`void Sys_CM3_Delay(unsigned int cycles)`
Description	Delay by at least the specified number of clock cycles
Inputs	cycles = Count of the system clock cycles to delay
Outputs	None
Assumptions	The requested delay is at least 15 cycles (15us at 1MHz) and at most 0xFFFFFFFF cycles (approx 429.5s at 10MHz) A delay between (cycles) and (cycles + 3) provides a sufficient delay resolution

Example	`/* Toggle GPIO 4 high for ~100 cycles as an indicator */`
	`Sys_GPIO_Set_High(4);`
	`Sys_CM3_Delay(100);`
	`Sys_GPIO_Set_Low(4);`

15.8 SYS_CVSD_DECODER_CLEAR_STATE

Clear the internal state of the CVSD decoder

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`void Sys_CVSD_Decoder_Clear_State( void )`
Description	Clear the internal state of the CVSD decoder
Inputs	None
Outputs	None
Assumptions	The CVSD decoder is idle
Example	`/* Clear the current state for the CVSD decoder block */` `Sys_CVSD_Decoder_Clear_State();`

15.9 SYS_CVSD_DECODER_CONFIG

Configure the CVSD decoder

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`void Sys_CVSD_Decoder_Config( uint8_t config )`
Description	Configure the CVSD decoder
Inputs	config = The CVSD decoder configuration; use CVSD_DEC_OUT_[INC | DEC]_BYTE, CVSD_DEC_[DISABLE | ENABLE]_BYTE, CVSD_DEC_J_*_BYTE, CVSD_DEC_BETA_*_BYTE, and CVSD_DEC_H_*_BYTE
Outputs	None
Assumptions	None

Example	`/* Enable and configure the CVSD decoder */`
	`Sys_CVSD_Decoder_Config( CVSD_DEC_ENABLE_BYTE |`
	`CVSD_DEC_J_4_BYTE |`
	`CVSD_DEC_BETA_8_BYTE |`
	`CVSD_DEC_H_6_BYTE );`

15.10 SYS_CVSD_ENCODER_CLEAR_STATE

Clear the internal state of the CVSD encoder

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`void Sys_CVSD_Encoder_Clear_State( void )`
Description	Clear the internal state of the CVSD encoder
Inputs	None
Outputs	None
Assumptions	The CVSD encoder is idle
Example	`/* Clear the current state for the CVSD encoder block */` `Sys_CVSD_Encoder_Clear_State();`

15.11 SYS_CVSD_ENCODER_CONFIG

Configure the CVSD encoder

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`void Sys_CVSD_Encoder_Config( uint8_t config )`
Description	Configure the CVSD encoder
Inputs	config = The CVSD encoder configuration; use CVSD_ENC_OUT_[INC | DEC]_BYTE, CVSD_ENC_[DISABLE | ENABLE]_BYTE, CVSD_ENC_J_*_BYTE, CVSD_ENC_BETA_*_BYTE, and CVSD_ENC_H_*_BYTE
Outputs	None
Assumptions	None

Example	`/* Enable and configure the CVSD encoder */`
	`Sys_CVSD_Encoder_Config( CVSD_ENC_ENABLE_BYTE |`
	`CVSD_ENC_J_4_BYTE |`
	`CVSD_ENC_BETA_8_BYTE |`
	`CVSD_ENC_H_6_BYTE );`

15.12 SYS_DIO_GET_INTSTATUS

Get the current status for the DIO interrupts; this clears the status

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dio.h
Template	`uint32_t Sys_DIO_Get_IntStatus( void )`
Description	Get the current status for the DIO interrupts; this clears the status
Inputs	None
Outputs	Return value = Current DIO interrupt status
Assumptions	None

Example	`/* Read the current DIO status to check for any new DIO interrupts */`
	`x = Sys_DIO_Get_IntStatus();`
	`if ( (x & C_DIO_INT0_TRUE) != 0)`
	`{`
	`  /* Handle DIO interrupt 0 */`
	`}`
	`else if ( (x & C_DIO_INT1_TRUE) != 0)`
	`{`
	`  /* Handle DIO interrupt 1 */`
	`}`

15.13 SYS_DIO_GET_MODE

Get the DIO mode (DIO or ARM Cortex-M3 controlled GPIO) for the DIOs

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dio.h
Template	`uint32_t Sys_DIO_Get_Mode( void )`
Description	Get the DIO mode (DIO or ARM Cortex-M3 controlled GPIO) for the DIOs
Inputs	None
Outputs	Return value = Current DIO/GPIO modes; compare with C_DIO*_IS_NOT_C_GPIO and C_DIO*_IS_C_GPIO
Assumptions	None

Example	`/* If DIO6 is currently an ARM Cortex-M3 controlled GPIO,`
	` * configure a low level on this GPIO as DIO interrupt source 0 */`
	`if ((Sys_DIO_Get_Mode() & C_DIO6_IS_C_GPIO) != 0)`
	`{`
	`  Sys_DIO_IntConfig(0, (C_DIO_SRC_DIO_6 | C_DIO_EVENT_LOW_LEVEL));`
	`}`

15.14 SYS_DIO_INTCONFIG

Configure an ARM Cortex-M3 DIO interrupt source

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dio.h
Template	`void Sys_DIO_IntConfig( uint32_t index, uint32_t cfg )`
Description	Configure an ARM Cortex-M3 DIO interrupt source
Inputs	index = DIO interrupt source to configure; use an index between 0 and 3 cfg = DIO interrupt configuration; use C_DIO_SRC_DIO_* and C_DIO_EVENT_*
Outputs	None
Assumptions	None

Example	`/* Configure DIO interrupt source 0 to generate an interrupt for`
	`* a falling edge event on DIO7. */`
	`Sys_DIO_IntConfig(0, (C_DIO_SRC_DIO_7 | C_DIO_EVENT_FALLING_EDGE));`

15.15 SYS_DIO_SET_DIRECTION

Set the input/output configuration for those DIOs configured as ARM Cortex-M3 controlled GPIOs

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dio.h
Template	`void Sys_DIO_Set_Direction( uint32_t dir )`
Description	Set the input/output configuration for those DIOs configured as ARM Cortex-M3 controlled GPIOs
Inputs	dir = Input/output configuration for those DIOs configured as Cortex-M3 controlled GPIOs; use C_DIO*_INPUT and C_DIO*_OUTPUT
Outputs	None
Assumptions	None
Example	`/* Set DIO0, DIO2 as inputs; set DIO1 as an output */` `Sys_DIO_Set_Direction(C_DIO0_INPUT | C_DIO1_OUTPUT | C_DIO2_INPUT);`

15.16 SYS_DMA_CHANNEL0_CONFIG

Configure DMA channel 0 for a data transfer

Type	Function
Include File	`#include <sk4.h>`

Source File	sk4_sys_dma.c
Template	`void Sys_DMA_Channel0_Config(uint32_t config, uint32_t transferLength, uint32_t counterInt, uint32_t srcAddr, uint32_t destAddr)`
Description	Configure DMA channel 0 for a data transfer
Inputs	config = DMA transfer behavior; use DMA_CH0_DEST_ADDR_STEP_SIZE_*, DMA_CH0_SRC_ADDR_STEP_SIZE_*, DMA_CH0_DEST_ADDR_[POS | NEG], DMA_CH0_SRC_ADDR_[POS | NEG], DMA_CH0_[LITTLE | BIG]_ENDIAN, DMA_CH0_DISABLE_INT_[DISABLE | ENABLE], DMA_CH0_ERROR_INT_[DISABLE | ENABLE], DMA_CH0_COMPLETE_INT_[DISABLE | ENABLE], DMA_CH0_COUNTER_INT_[DISABLE | ENABLE], DMA_CH0_START_INT_[DISABLE | ENABLE], DMA_CH0_DEST_WORD_SIZE_*, DMA_CH0_SRC_WORD_SIZE_*, DMA_CH0_DEST_*, DMA_CH0_SRC_*, DMA_CH0_PRIORITY_*, DMA_CH0_TRANSFER_*_TO_*, DMA_CH0_DEST_ADDR_[STATIC | INC], DMA_CH0_SRC_ADDR_[STATIC | INC], DMA_CH0_ADDR_[CIRC | LIN], DMA_CH0_[DISABLE | ENABLE] transferLength = DMA transfer length counterInt = Counter value that triggers the counter interrupt (if active) srcAddr = Base source address for the DMA transfer destAddr = Base destination address for the DMA transfer
Outputs	None
Assumptions	None
Example	

```
/* Configure DMA channel 0 for a transfer from the PCM interface to
 * a 16-word buffer in memory. Clear any previous DMA status bit
 * settings. */
#define DMA_CH0_CONFIG      (DMA_CH0_DEST_ADDR_STEP_SIZE_1  | \
                             DMA_CH0_DEST_ADDR_POS          | \
                             DMA_CH0_SRC_ADDR_STEP_SIZE_1   | \
                             DMA_CH0_SRC_ADDR_POS           | \
                             DMA_CH0_LITTLE_ENDIAN          | \
                             DMA_CH0_DISABLE_INT_DISABLE    | \
                             DMA_CH0_ERROR_INT_ENABLE       | \
                             DMA_CH0_COMPLETE_INT_ENABLE    | \
                             DMA_CH0_COUNTER_INT_ENABLE     | \
                             DMA_CH0_START_INT_DISABLE      | \
                             DMA_CH0_DEST_WORD_SIZE_32      | \
                             DMA_CH0_SRC_WORD_SIZE_32       | \
                             DMA_CH0_SRC_PCM                | \
                             DMA_CH0_PRIORITY_0             | \
                             DMA_CH0_TRANSFER_P_TO_M        | \
                             DMA_CH0_SRC_ADDR_STATIC        | \
                             DMA_CH0_DEST_ADDR_INC          | \
                             DMA_CH0_ADDR_CIRC              | \
                             DMA_CH0_DISABLE)
Sys_DMA_Channel0_Config(DMA_CH0_CONFIG, 16, 0,
                        (uint32_t)&PCM->RX_DATA,
                            (uint32_t)buffer);
Sys_DMA_Clear_Channel0_Status();
```

15.17 SYS_DMA_CHANNEL0_DISABLE

Disable DMA channel 0

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Channel0_Disable( void )`
Description	Disable DMA channel 0
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Disable DMA channel 0 */` `Sys_DMA_Channel0_Disable();`

15.18 SYS_DMA_CHANNEL0_ENABLE

Enable DMA channel 0

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Channel0_Enable( void )`
Description	Enable DMA channel 0
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Enable DMA channel 0 */` `Sys_DMA_Channel0_Enable();`

15.19 SYS_DMA_CHANNEL1_CONFIG

Configure DMA channel 1 for a data transfer

Type	Function
Include File	`#include <sk4.h>`

Source File	sk4_sys_dma.c
Template	`void Sys_DMA_Channel1_Config(uint32_t config, uint32_t transferLength, uint32_t counterInt, uint32_t srcAddr, uint32_t destAddr)`
Description	Configure DMA channel 1 for a data transfer
Inputs	config = DMA transfer behavior; use DMA_CH1_DEST_ADDR_STEP_SIZE_*, DMA_CH1_SRC_ADDR_STEP_SIZE_*, DMA_CH1_DEST_ADDR_[POS | NEG], DMA_CH1_SRC_ADDR_[POS | NEG], DMA_CH1_[LITTLE | BIG]_ENDIAN, DMA_CH1_DISABLE_INT_[DISABLE | ENABLE], DMA_CH1_ERROR_INT_[DISABLE | ENABLE], DMA_CH1_COMPLETE_INT_[DISABLE | ENABLE], DMA_CH1_COUNTER_INT_[DISABLE | ENABLE], DMA_CH1_START_INT_[DISABLE | ENABLE], DMA_CH1_DEST_WORD_SIZE_*, DMA_CH1_SRC_WORD_SIZE_*, DMA_CH1_DEST_*, DMA_CH1_SRC_*, DMA_CH1_PRIORITY_*, DMA_CH1_TRANSFER_*_TO_*, DMA_CH1_DEST_ADDR_[STATIC | INC], DMA_CH1_SRC_ADDR_[STATIC | INC], DMA_CH1_ADDR_[CIRC | LIN], DMA_CH1_[DISABLE | ENABLE] transferLength = DMA transfer length counterInt = Counter value that triggers the counter interrupt (if active) srcAddr = Base source address for the DMA transfer destAddr = Base destination address for the DMA transfer
Outputs	None
Assumptions	None
Example	

```
/* Configure DMA channel 1 for a transfer from the PCM interface to
 * a 16-word buffer in memory. Clear any previous DMA status bit
 * settings. */
#define DMA_CH1_CONFIG      (DMA_CH1_DEST_ADDR_STEP_SIZE_1  | \
                             DMA_CH1_DEST_ADDR_NEG          | \
                             DMA_CH1_SRC_ADDR_STEP_SIZE_1   | \
                             DMA_CH1_SRC_ADDR_POS           | \
                             DMA_CH1_LITTLE_ENDIAN          | \
                             DMA_CH1_DISABLE_INT_DISABLE    | \
                             DMA_CH1_ERROR_INT_ENABLE       | \
                             DMA_CH1_COMPLETE_INT_ENABLE    | \
                             DMA_CH1_COUNTER_INT_ENABLE     | \
                             DMA_CH1_START_INT_DISABLE      | \
                             DMA_CH1_DEST_WORD_SIZE_32      | \
                             DMA_CH1_SRC_WORD_SIZE_32       | \
                             DMA_CH1_SRC_PCM                | \
                             DMA_CH1_PRIORITY_0             | \
                             DMA_CH1_TRANSFER_P_TO_M        | \
                             DMA_CH1_SRC_ADDR_STATIC        | \
                             DMA_CH1_DEST_ADDR_INC          | \
                             DMA_CH1_ADDR_CIRC              | \
                             DMA_CH1_DISABLE)
Sys_DMA_Channel1_Config(DMA_CH1_CONFIG, 16, 0,
                        (uint32_t)&PCM->RX_DATA,
                            (uint32_t)buffer);
Sys_DMA_Clear_Channel1_Status();
```

15.20 SYS_DMA_CHANNEL1_DISABLE

Disable DMA channel 1

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Channel1_Disable( void )`
Description	Disable DMA channel 1
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Disable DMA channel 1 */` `Sys_DMA_Channel1_Disable();`

15.21 SYS_DMA_CHANNEL1_ENABLE

Enable DMA channel 1

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Channel1_Enable( void )`
Description	Enable DMA channel 1
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Enable DMA channel 1 */` `Sys_DMA_Channel1_Enable();`

15.22 SYS_DMA_CHANNEL2_CONFIG

Configure DMA channel 2 for a data transfer

Type	Function
Include File	`#include <sk4.h>`

Source File	sk4_sys_dma.c
Template	`void Sys_DMA_Channel2_Config(uint32_t config, uint32_t transferLength, uint32_t counterInt, uint32_t srcAddr, uint32_t destAddr)`
Description	Configure DMA channel 2 for a data transfer
Inputs	config = DMA transfer behavior; use DMA_CH2_DEST_ADDR_STEP_SIZE_*, DMA_CH2_SRC_ADDR_STEP_SIZE_*, DMA_CH2_DEST_ADDR_[POS | NEG], DMA_CH2_SRC_ADDR_[POS | NEG], DMA_CH2_[LITTLE | BIG]_ENDIAN, DMA_CH2_DISABLE_INT_[DISABLE | ENABLE], DMA_CH2_ERROR_INT_[DISABLE | ENABLE], DMA_CH2_COMPLETE_INT_[DISABLE | ENABLE], DMA_CH2_COUNTER_INT_[DISABLE | ENABLE], DMA_CH2_START_INT_[DISABLE | ENABLE], DMA_CH2_DEST_WORD_SIZE_*, DMA_CH2_SRC_WORD_SIZE_*, DMA_CH2_DEST_*, DMA_CH2_SRC_*, DMA_CH2_PRIORITY_*, DMA_CH2_TRANSFER_*_TO_*, DMA_CH2_DEST_ADDR_[STATIC | INC], DMA_CH2_SRC_ADDR_[STATIC | INC], DMA_CH2_ADDR_[CIRC | LIN], DMA_CH2_[DISABLE | ENABLE] transferLength = DMA transfer length counterInt = Counter value that triggers the counter interrupt (if active) srcAddr = Base source address for the DMA transfer destAddr = Base destination address for the DMA transfer
Outputs	None
Assumptions	None
Example	

```
/* Configure DMA channel 2 for a transfer from the PCM interface to
 * a 16-word buffer in memory. Clear any previous DMA status bit
 * settings. */
#define DMA_CH2_CONFIG      (DMA_CH2_DEST_ADDR_STEP_SIZE_1  | \
                             DMA_CH2_DEST_ADDR_POS          | \
                             DMA_CH2_SRC_ADDR_STEP_SIZE_1   | \
                             DMA_CH2_SRC_ADDR_POS           | \
                             DMA_CH2_LITTLE_ENDIAN          | \
                             DMA_CH2_DISABLE_INT_DISABLE    | \
                             DMA_CH2_ERROR_INT_ENABLE       | \
                             DMA_CH2_COMPLETE_INT_ENABLE    | \
                             DMA_CH2_COUNTER_INT_ENABLE     | \
                             DMA_CH2_START_INT_DISABLE      | \
                             DMA_CH2_DEST_WORD_SIZE_32      | \
                             DMA_CH2_SRC_WORD_SIZE_32       | \
                             DMA_CH2_SRC_PCM                | \
                             DMA_CH2_PRIORITY_0             | \
                             DMA_CH2_TRANSFER_P_TO_M        | \
                             DMA_CH2_SRC_ADDR_STATIC        | \
                             DMA_CH2_DEST_ADDR_INC          | \
                             DMA_CH2_ADDR_CIRC              | \
                             DMA_CH2_DISABLE)
Sys_DMA_Channel2_Config(DMA_CH2_CONFIG, 16, 0,
                        (uint32_t)&PCM->RX_DATA,
                            (uint32_t)buffer);
Sys_DMA_Clear_Channel2_Status();
```

15.23 SYS_DMA_CHANNEL2_DISABLE

Disable DMA channel 2

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Channel2_Disable( void )`
Description	Disable DMA channel 2
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Disable DMA channel 2 */` `Sys_DMA_Channel2_Disable();`

15.24 SYS_DMA_CHANNEL2_ENABLE

Enable DMA channel 2

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Channel2_Enable( void )`
Description	Enable DMA channel 2
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Enable DMA channel 2 */` `Sys_DMA_Channel2_Enable();`

15.25 SYS_DMA_CHANNEL3_CONFIG

Configure DMA channel 3 for a data transfer

Type	Function
Include File	`#include <sk4.h>`

Source File	sk4_sys_dma.c
Template	`void Sys_DMA_Channel3_Config(uint32_t config, uint32_t transferLength, uint32_t counterInt, uint32_t srcAddr, uint32_t destAddr)`
Description	Configure DMA channel 3 for a data transfer
Inputs	config = DMA transfer behavior; use DMA_CH3_DEST_ADDR_STEP_SIZE_*, DMA_CH3_SRC_ADDR_STEP_SIZE_*, DMA_CH3_DEST_ADDR_[POS | NEG], DMA_CH3_SRC_ADDR_[POS | NEG], DMA_CH3_[LITTLE | BIG]_ENDIAN, DMA_CH3_DISABLE_INT_[DISABLE | ENABLE], DMA_CH3_ERROR_INT_[DISABLE | ENABLE], DMA_CH3_COMPLETE_INT_[DISABLE | ENABLE], DMA_CH3_COUNTER_INT_[DISABLE | ENABLE], DMA_CH3_START_INT_[DISABLE | ENABLE], DMA_CH3_DEST_WORD_SIZE_*, DMA_CH3_SRC_WORD_SIZE_*, DMA_CH3_DEST_*, DMA_CH3_SRC_*, DMA_CH3_PRIORITY_*, DMA_CH3_TRANSFER_*_TO_*, DMA_CH3_DEST_ADDR_[STATIC | INC], DMA_CH3_SRC_ADDR_[STATIC | INC], DMA_CH3_ADDR_[CIRC | LIN], DMA_CH3_[DISABLE | ENABLE] transferLength = DMA transfer length counterInt = Counter value that triggers the counter interrupt (if active) srcAddr = Base source address for the DMA transfer destAddr = Base destination address for the DMA transfer
Outputs	None
Assumptions	None
Example	

```
/* Configure DMA channel 3 for a transfer from the PCM interface to
 * a 16-word buffer in memory. Clear any previous DMA status bit
 * settings. */
#define DMA_CH3_CONFIG      (DMA_CH3_DEST_ADDR_STEP_SIZE_1  | \
                             DMA_CH3_DEST_ADDR_POS          | \
                             DMA_CH3_SRC_ADDR_STEP_SIZE_2   | \
                             DMA_CH3_SRC_ADDR_NEG           | \
                             DMA_CH3_LITTLE_ENDIAN          | \
                             DMA_CH3_DISABLE_INT_DISABLE    | \
                             DMA_CH3_ERROR_INT_ENABLE       | \
                             DMA_CH3_COMPLETE_INT_ENABLE    | \
                             DMA_CH3_COUNTER_INT_ENABLE     | \
                             DMA_CH3_START_INT_DISABLE      | \
                             DMA_CH3_DEST_WORD_SIZE_32      | \
                             DMA_CH3_SRC_WORD_SIZE_32       | \
                             DMA_CH3_SRC_PCM                | \
                             DMA_CH3_PRIORITY_0             | \
                             DMA_CH3_TRANSFER_P_TO_M        | \
                             DMA_CH3_SRC_ADDR_STATIC        | \
                             DMA_CH3_DEST_ADDR_INC          | \
                             DMA_CH3_ADDR_CIRC              | \
                             DMA_CH3_DISABLE)
Sys_DMA_Channel3_Config(DMA_CH3_CONFIG, 16, 0,
                        (uint32_t)&PCM->RX_DATA,
                            (uint32_t)buffer);
Sys_DMA_Clear_Channel3_Status();
```

15.26 SYS_DMA_CHANNEL3_DISABLE

Disable DMA channel 3

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Channel3_Disable( void )`
Description	Disable DMA channel 3
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Disable DMA channel 3 */` `Sys_DMA_Channel3_Disable();`

15.27 SYS_DMA_CHANNEL3_ENABLE

Enable DMA channel 3

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Channel3_Enable( void )`
Description	Enable DMA channel 3
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Enable DMA channel 3 */` `Sys_DMA_Channel3_Enable();`

15.28 SYS_DMA_CLEAR_CHANNEL0_STATUS

Clear the current status for DMA channel 0

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Clear_Channel0_Status( void )`
Description	Clear the current status for DMA channel 0
Inputs	None
Outputs	None
Assumptions	None

Example	`/* Configure DMA channel 0 for a transfer from the PCM interface to`
	` * a 16-word buffer in memory. Clear any previous DMA status bit`
	` * settings. */`
	`Sys_DMA_Channel0_Config(channelConfig, 16, 0,`
	`(uint32_t)&PCM->RX_DATA,`
	`(uint32_t)buffer);`
	`Sys_DMA_Clear_Channel0_Status();`

15.29 SYS_DMA_CLEAR_CHANNEL1_STATUS

Clear the current status for DMA channel 1

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Clear_Channel1_Status( void )`
Description	Clear the current status for DMA channel 1
Inputs	None
Outputs	None
Assumptions	None

Example	`/* Configure DMA channel 1 for a transfer from the PCM interface to`
	` * a 16-word buffer in memory. Clear any previous DMA status bit`
	` * settings. */`
	`Sys_DMA_Channel1_Config(channelConfig, 16, 0,`
	`(uint32_t)&PCM->RX_DATA,`
	`(uint32_t)buffer);`
	`Sys_DMA_Clear_Channel1_Status();`

15.30 SYS_DMA_CLEAR_CHANNEL2_STATUS

Clear the current status for DMA channel 2

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Clear_Channel2_Status( void )`
Description	Clear the current status for DMA channel 2
Inputs	None
Outputs	None
Assumptions	None

Example	`/* Configure DMA channel 2 for a transfer from the PCM interface to`
	` * a 16-word buffer in memory. Clear any previous DMA status bit`
	` * settings. */`
	`Sys_DMA_Channel2_Config(channelConfig, 16, 0,`
	`(uint32_t)&PCM->RX_DATA,`
	`(uint32_t)buffer);`
	`Sys_DMA_Clear_Channel2_Status();`

15.31 SYS_DMA_CLEAR_CHANNEL3_STATUS

Clear the current status for DMA channel 3

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Clear_Channel3_Status( void )`
Description	Clear the current status for DMA channel 3
Inputs	None
Outputs	None
Assumptions	None

Example	`/* Configure DMA channel 3 for a transfer from the PCM interface to`
	` * a 16-word buffer in memory. Clear any previous DMA status bit`
	` * settings. */`
	`Sys_DMA_Channel3_Config(channelConfig, 16, 0,`
	`(uint32_t)&PCM->RX_DATA,`
	`(uint32_t)buffer);`
	`Sys_DMA_Clear_Channel3_Status();`

15.32 SYS_DMA_CLEAR_STATUS

Clear the current status for all DMA channels

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Clear_Status( void )`
Description	Clear the current status for all DMA channels
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Clear the status for all DMA channels */` `Sys_DMA_Clear_Status();`

15.33 SYS_DMA_GET_CHANNEL0_STATUS

Get the current status of DMA channel 0

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`uint8_t Sys_DMA_Get_Channel0_Status( void )`
Description	Get the current status of DMA channel 0
Inputs	None
Outputs	Return value = The current DMA_STATUS_CH0_BYTE
Assumptions	None

Example	`if ((Sys_DMA_Get_Channel0_Status() & DMA_CH0_ERROR_INT) != 0)`
	`{`
	`  /* An error has occurred on DMA channel 0. Use the error handler`
	`  * macro to handle this error. */`
	`  DMA_ErrorHandler;`
	`}`

15.34 SYS_DMA_GET_CHANNEL1_STATUS

Get the current status of DMA channel 1

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`uint8_t Sys_DMA_Get_Channel1_Status( void )`
Description	Get the current status of DMA channel 1
Inputs	None
Outputs	Return value = The current DMA_STATUS_CH1_BYTE
Assumptions	None

Example	`if ((Sys_DMA_Get_Channel1_Status() & DMA_CH0_ERROR_INT) != 0)`
	`{`
	`  /* An error has occurred on DMA channel 1. Use the error handler`
	`  * macro to handle this error. */`
	`  DMA_ErrorHandler;`
	`}`

15.35 SYS_DMA_GET_CHANNEL2_STATUS

Get the current status of DMA channel 2

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`uint8_t Sys_DMA_Get_Channel2_Status( void )`
Description	Get the current status of DMA channel 2
Inputs	None
Outputs	Return value = The current DMA_STATUS_CH2_BYTE
Assumptions	None

Example	`if ((Sys_DMA_Get_Channel2_Status() & DMA_CH0_ERROR_INT) != 0)`
	`{`
	`  /* An error has occurred on DMA channel 2. Use the error handler`
	`  * to handle this error. */`
	`  DMA_ErrorHandler;`
	`}`

15.36 SYS_DMA_GET_CHANNEL3_STATUS

Get the current status of DMA channel 3

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`uint8_t Sys_DMA_Get_Channel3_Status( void )`
Description	Get the current status of DMA channel 3
Inputs	None
Outputs	Return value = The current DMA_STATUS_CH3_BYTE
Assumptions	None

Example	`if ((Sys_DMA_Get_Channel3_Status() & DMA_CH0_ERROR_INT) != 0)`
	`{`
	`  /* An error has occurred on DMA channel 3. Use the error handler`
	`  * to handle this error. */`
	`  DMA_ErrorHandler;`
	`}`

15.37 SYS_DMA_SET_CHANNEL0_DESTADDRESS

Set the base destination address for DMA channel 0

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Set_Channel0_DestAddress( uint32_t destAddr )`
Description	Set the base destination address for DMA channel 0
Inputs	destAddr = Base destination address for the DMA transfer
Outputs	None
Assumptions	None

Example	`/* Set the DMA channel 0 destination address to Cortex-M3 data`
	` * memory at 0x20004000 */`
	`Sys_DMA_Set_Channel0_DestAddress(0x20004000);`

15.38 SYS_DMA_SET_CHANNEL0_SOURCEADDRESS

Set the base source address for DMA channel 0

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Set_Channel0_SourceAddress( uint32_t srcAddr )`
Description	Set the base source address for DMA channel 0
Inputs	srcAddr = Base source address for the DMA transfer
Outputs	None
Assumptions	None
Example	`/* Set the DMA channel 0 source address to the start of Cortex-M3 * data memory */ Sys_DMA_Set_Channel0_SourceAddress(0x20000000);`

15.39 SYS_DMA_SET_CHANNEL1_DESTADDRESS

Set the base destination address for DMA channel 1

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Set_Channel1_DestAddress( uint32_t destAddr )`
Description	Set the base destination address for DMA channel 1
Inputs	destAddr = Base destination address for the DMA transfer
Outputs	None
Assumptions	None

Example	`/* Set the DMA channel 1 destination address to Cortex-M3 data`
	` * memory at 0x20004000 */`
	`Sys_DMA_Set_Channel1_DestAddress(0x20004000);`

15.40 SYS_DMA_SET_CHANNEL1_SOURCEADDRESS

Set the base source address for DMA channel 1

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Set_Channel1_SourceAddress( uint32_t srcAddr )`
Description	Set the base source address for DMA channel 1
Inputs	srcAddr = Base source address for the DMA transfer
Outputs	None
Assumptions	None

Example	`/* Set the DMA channel 1 source address to the start of`
	`  * Cortex-M3 data memory */`
	`Sys_DMA_Set_Channel1_SourceAddress(0x20000000);`

15.41 SYS_DMA_SET_CHANNEL2_DESTADDRESS

Set the base destination address for DMA channel 2

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Set_Channel2_DestAddress( uint32_t destAddr )`
Description	Set the base destination address for DMA channel 2
Inputs	destAddr = Base destination address for the DMA transfer
Outputs	None
Assumptions	None

Example	`/* Set the DMA channel 2 destination address to Cortex-M3 data`
	`* memory at 0x20004000 */`
	`Sys_DMA_Set_Channel2_DestAddress(0x20004000);`

15.42 SYS_DMA_SET_CHANNEL2_SOURCEADDRESS

Set the base source address for DMA channel 2

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Set_Channel2_SourceAddress( uint32_t srcAddr )`
Description	Set the base source address for DMA channel 2
Inputs	srcAddr = Base source address for the DMA transfer
Outputs	None
Assumptions	None
Example	`/* Set the DMA channel 2 source address to the start of Cortex-M3 * data memory */ Sys_DMA_Set_Channel2_SourceAddress(0x20000000);`

15.43 SYS_DMA_SET_CHANNEL3_DESTADDRESS

Set the base destination address for DMA channel 3

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Set_Channel3_DestAddress( uint32_t destAddr )`
Description	Set the base destination address for DMA channel 3
Inputs	destAddr = Base destination address for the DMA transfer
Outputs	None
Assumptions	None

Example	`/* Set the DMA channel 3 destination address to Cortex-M3 data`
	` * memory at 0x20004000 */`
	`Sys_DMA_Set_Channel3_DestAddress(0x20004000);`

15.44 SYS_DMA_SET_CHANNEL3_SOURCEADDRESS

Set the base source address for DMA channel 3

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dma.h
Template	`void Sys_DMA_Set_Channel3_SourceAddress( uint32_t srcAddr )`
Description	Set the base source address for DMA channel 3
Inputs	srcAddr = Base source address for the DMA transfer
Outputs	None
Assumptions	None
Example	`/* Set the DMA channel 3 source address to the start of Cortex-M3 * data memory */ Sys_DMA_Set_Channel3_SourceAddress(0x20000000);`

15.45 SYS_ECB_INT_ACK

Acknowledge all error correction coding block interrupts

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_ecb.h
Template	`void Sys_ECB_Int_Ack( void )`
Description	Acknowledge all error correction coding block interrupts
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Acknowledge all error correction coding block interrupts */` `Sys_ECB_Int_Ack();`

15.46 SYS_ECB_INT_ENABLE

Enable/disable a set of error correction coding block interrupts

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_ecb.h
Template	`void Sys_ECB_Int_Enable( uint32_t src )`
Description	Enable/disable a set of error correction coding block interrupts
Inputs	src = ECB interrupt sources to enable; use INT_EBL_* and INT_DIS_*
Outputs	None
Assumptions	None

Example	`/* Enable the Reed-Solomon decoder interrupts */`
	`Sys_ECB_Int_Enable( INT_EBL_RS_DEC_IN_ERR |`
	`INT_EBL_RS_DEC_BLK_END |`
	`INT_EBL_RS_DEC_MISS_ERR );`

15.47 SYS_G722_DECODER_CLEAR_STATE

Clear the internal state of the G.722 decoder

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`void Sys_G722_Decoder_Clear_State( void )`
Description	Clear the internal state of the G.722 decoder
Inputs	None
Outputs	None
Assumptions	The G.722 decoder is idle
Example	`/* Clear the current state for the G.722 decoder block */` `Sys_G722_Decoder_Clear_State();`

15.48 SYS_G722_DECODER_CONCEAL

Set the necessary registers for concealment of a missing G.722 sample

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`void Sys_G722_Decoder_Conceal( uint32_t PLT, uint32_t RLT, uint32_t SX, uint32_t LSBQuant, uint32_t HSBQuant, uint32_t FBIN )`
Description	Set the necessary registers for concealment of a missing G.722 sample
Inputs	PLT = Replacement settings for the PLT1, PLT2 sign registers RLT = Replacement setting for the RLT1 register SX = Replacement settings for the SZL, SL pipeline registers LSBQuant = Replacement settings for the DETL, NBL quantizer registers (lower sub-band) HSBQuant = Replacement settings for the DETL, NBL quantizer registers (higher sub-band) FBIN = Replacement data for the sub-band decoders
Outputs	None
Assumptions	None

Example	`/* Echo the current G.722 internal register settings as a very basic`
	` * concealment of missing G.722 data */`
	`uint32_t tempPLT, tempRLT, tempSX;`
	`uint32_t tempLSBQuant, tempHSBQuant, tempFBIN;`
	
	`tempPLT = ACB->G722_DEC_PLT;`
	`tempRLT = ACB->G722_DEC_RLT;`
	`tempSX = ACB->G722_DEC_SX;`
	`tempLSBQuant = ACB->G722_DEC_LSB_QUANT;`
	`tempHSBQuant = ACB->G722_DEC_HSB_QUANT;`
	`tempFBIN = ACB->G722_DEC_FBIN;`
	
	`Sys_G722_Decoder_Conceal(tempPLT, tempRLT, tempSX,`
	`tempLSBQuant, tempHSBQuant, tempFBIN );`

15.49 SYS_G722_DECODER_CONFIG

Configure the G.722 decoder

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`void Sys_G722_Decoder_Config( uint8_t config )`
Description	Configure the G.722 decoder
Inputs	config = The G.722 decoder configuration; use G722_DEC_[DISABLE | ENABLE]_BYTE, G722_CONCL_[DISABLE | ENABLE]_BYTE, G722_DEC_FB_[BYPASS | ENABLE]_BYTE, and G722_DEC_OUT_[15BIT | 16BIT]_BYTE
Outputs	None
Assumptions	None

Example	`/* Enable and configure the G.722 decoder */`
	`Sys_G722_Decoder_Config( G722_DEC_ENABLE_BYTE |`
	`G722_CONCL_DISABLE_BYTE |`
	`G722_DEC_FB_BYPASS_BYTE |`
	`G722_DEC_OUT_16BIT_BYTE );`

15.50 SYS_G722_ENCODER_CLEAR_STATE

Clear the internal state of the G.722 encoder

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`void Sys_G722_Encoder_Clear_State( void )`
Description	Clear the internal state of the G.722 encoder
Inputs	None
Outputs	None
Assumptions	The G.722 encoder is idle
Example	`/* Clear the current state for the G.722 encoder block */` `Sys_G722_Encoder_Clear_State();`

15.51 SYS_G722_ENCODER_CONFIG

Configure the G.722 encoder

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`void Sys_G722_Encoder_Config( uint8_t config )`
Description	Configure the G.722 encoder
Inputs	config = The G.722 encoder configuration; use G722_ENC_[DISABLE | ENABLE]_BYTE, G722_ENC_FB_[BYPASS | ENABLE]_BYTE, and G722_ENC_IN_[15BIT | 16BIT]_BYTE
Outputs	None
Assumptions	None

Example	`/* Enable and configure the G.722 encoder */`
	`Sys_G722_Encoder_Config( G722_ENC_ENABLE_BYTE |`
	`G722_ENC_FB_BYPASS_BYTE |`
	`G722_ENC_IN_16BIT_BYTE );`

15.52 SYS_G722_EXTENDFROM48

Convert 6-bit samples to 8-bit samples for a 64 kbps rate

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`uint8_t Sys_G722_ExtendFrom48(uint8_t value)`
Description	Convert 6-bit samples to 8-bit samples for a 64 kbps rate
Inputs	value = 6-bits sample
Outputs	Return value = 8-bits sample
Assumptions	Encoder rate is 48 kbps

Example	`/* Extend from 48 kbps rate */`
	`x = 0x0A;`
	`y = Sys_G722_ExtendFrom48(x);`

15.53 SYS_G722_EXTENDFROM56

Convert 7-bit samples to 8-bit samples for a 64 kbps rate

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`uint8_t Sys_G722_ExtendFrom56(uint8_t value)`
Description	Convert 7-bit samples to 8-bit samples for a 64 kbps rate
Inputs	value = 7-bits sample
Outputs	Return value = 8-bits sample
Assumptions	Encoder rate is 56 kbps

Example	`/* Extend from 56 kbps rate */`
	`x = 0x0A;`
	`y = Sys_G722_ExtendFrom56(x);`

15.54 SYS_G722_TRUNCATETO48

Convert 8-bit samples to 6-bit samples for a 48 kbps rate

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`uint8_t Sys_G722_TruncateTo48(uint8_t value)`
Description	Convert 8-bit samples to 6-bit samples for a 48 kbps rate
Inputs	value = 8-bits sample
Outputs	Return value = 6-bits sample
Assumptions	None

Example	`/* Truncate to 48 kbps rate */`
	`x = 0x0A;`
	`y = Sys_G722_TruncateTo48(x);`

15.55 SYS_G722_TRUNCATETO56

Convert 8-bit samples to 7-bit samples for a 56 kbps rate

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_acb.h
Template	`uint8_t Sys_G722_TruncateTo56(uint8_t value)`
Description	Convert 8-bit samples to 7-bit samples for a 56 kbps rate
Inputs	value = 8-bits sample
Outputs	Return value = 7-bits sample
Assumptions	None

Example	`/* Truncate to 56 kbps rate */`
	`x = 0x0A;`
	`y = Sys_G722_TruncateTo56(x);`

15.56 SYS_GET_CRC_CONFIG

Get the CRC generator configuration

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_crc.h
Template	`uint32_t Sys_Get_CRC_Config( void )`
Description	Get the CRC generator configuration
Inputs	None
Outputs	config = CRC generator configuration; compare with CRC_[CCITT | 32], CRC_[BIG | LITTLE]_ENDIAN, CRC_BIT_ORDER_[STANDARD | NON_STANDARD], CRC_FINAL_REVERSE_[STANDARD | NON_STANDARD], and CRC_FINAL_XOR_[STANDARD | NON_STANDARD]
Assumptions	None

Example	`/* Temporarily store the current CRC configuration and intermediate`
	` * value while we're calculating a separate CRC */`
	`prevCRCConfig = Sys_Get_CRC_Config();`
	`prevCRCData = CRC->VALUE;`

15.57 SYS_GPIO_SET_HIGH

Set the specified GPIO output value to high

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dio.h
Template	`void Sys_GPIO_Set_High( uint32_t gpio_pin )`
Description	Set the specified GPIO output value to high
Inputs	gpio_pin = GPIO pin to set high
Outputs	None
Assumptions	None
Example	`/* Set GPIO 8 high */` `Sys_GPIO_Set_High( 8 );`

15.58 SYS_GPIO_SET_LOW

Set the specified GPIO output value to low

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dio.h
Template	`void Sys_GPIO_Set_Low( uint32_t gpio_pin )`
Description	Set the specified GPIO output value to low
Inputs	gpio_pin = GPIO pin to set low
Outputs	None
Assumptions	None
Example	`/* Set GPIO 9 low */` `Sys_GPIO_Set_Low( 9 );`

15.59 SYS_GPIO_TOGGLE

Toggle the current value of the specified GPIO output

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_dio.h
Template	`void Sys_GPIO_Toggle( uint32_t gpio_pin )`
Description	Toggle the current value of the specified GPIO output
Inputs	gpio_pin = GPIO pin to toggle
Outputs	None
Assumptions	None
Example	`/* Toggle GPIO 6 */` `Sys_GPIO_Toggle( 6 );`

15.60 SYS_HMI_DISABLE

Disable the Hamming and interleaving components (disables both the encoder and decoder)

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_ecb.h
Template	`void Sys_HMI_Disable( void )`
Description	Disable the Hamming and interleaving components (disables both the encoder and decoder)
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Disable the Hamming and interleaver components */` `Sys_HMI_Disable();`

15.61 SYS_HMI_ENCODER_CONFIG

Configure the Hamming encoder and interleaver

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_ecb.h
Template	`void Sys_HMI_Encoder_Config( uint8_t config )`
Description	Configure the Hamming encoder and interleaver
Inputs	config = The HMI encoder configuration; use HMI_DMA_[3REG | 4REG]_BYTE, HMI_INTERLEAVE_[128B | 126B]_BYTE, HMI_INTERLEAVE_[DISABLE | ENABLE]_BYTE, HMI_PARITY_[DISABLE | ENABLE]_BYTE, and HMI_[DISABLE | ENABLE]_BYTE
Outputs	None
Assumptions	None

Example	`/* Enable and configure the Hamming encoder and interleaver */`
	`Sys_HMI_Encoder_Config( HMI_INTERLEAVE_128B_BYTE |`
	`HMI_INTERLEAVE_ENABLE_BYTE |`
	`HMI_PARITY_ENABLE_BYTE |`
	`HMI_ENABLE_BYTE );`

15.62 SYS_I2C_ACK

Manually acknowledge the latest I^2C transfer

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_i2c.h
Template	`void Sys_I2C_ACK( void )`
Description	Manually acknowledge the latest I^2C transfer
Inputs	None
Outputs	None
Assumptions	None
Example	`/* ACK the latest I2C byte transfer */` `Sys_I2C_ACK();`

15.63 SYS_I2C_CONFIG

Configure the I²C interface for operation

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_i2c.h
Template	`void Sys_I2C_Config( uint32_t config )`
Description	Configure the I²C interface for operation
Inputs	config = I²C interface configuration; use C_I2C_MASTER_SPEED_*, a slave address constant shifted to I2C_CTRL0_SLAVE_ADDRESS_Pos, C_I2C_CONTROLLER_[CM3 | DMA], C_I2C_STOP_INT_[ENABLE | DISABLE], C_I2C_AUTO_ACK_[ENABLE | DISABLE], C_I2C_SAMPLE_CLK_[ENABLE | DISABLE] and C_I2C_SLAVE_[ENABLE | DISABLE]
Outputs	None
Assumptions	None

Example	`/* Configure the I2C interface to communicate as a slave at address`
	`* 0x40 in auto-acknowledgement mode */`
	`Sys_I2C_Config( (0x40 << I2C_CTRL0_SLAVE_ADDRESS_Pos) |`
	`C_I2C_CONTROLLER_CM3 | C_I2C_STOP_INT_DISABLE |`
	`C_I2C_AUTO_ACK_ENABLE | C_I2C_SLAVE_ENABLE );`

15.64 SYS_I2C_GET_STATUS

Get the current I^2C interface status

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_i2c.h
Template	`uint32_t Sys_I2C_Get_Status( void )`
Description	Get the current I^2C interface status
Inputs	None
Outputs	status = Current I^2C interface status; compare with C_I2C_[DMA | NO_DMA]_REQUEST, C_I2C_[STOP | NO_STOP]_DETECTED, C_I2C_[DATA | NON_DATA]_EVENT, C_I2C_[ERROR | NO_ERROR], C_I2C_[BUS_ERROR | NO_BUS_ERROR], C_I2C_BUFFER_[FULL | EMPTY], C_I2C_CLK_[STRETCHED | NOT_STRETCHED], C_I2C_BUS_[FREE | BUSY], C_I2C_DATA_IS_[ADDR | DATA], C_I2C_IS_[READ | WRITE], C_I2C_ADDR_[GEN_CALL | OTHER] and C_I2C_HAS_[NACK | ACK]
Assumptions	None
Example	`/* Check if a repeated start condition occurred, indicating that the * most recently received data should be treated as an address. */ if ((Sys_I2C_Get_Status() & (1 << I2C_STATUS_ADDR_DATA_Pos)) == C_I2C_DATA_IS_ADDR) { /* Initialize a new slave transfer over the I2C interface */ }`

15.65 SYS_I2C_NACK

Manually not-acknowledge the latest transfer; releases the bus to continue with a transfer

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_i2c.h
Template	`void Sys_I2C_NACK( void )`
Description	Manually not-acknowledge the latest transfer; releases the bus to continue with a transfer
Inputs	None
Outputs	None
Assumptions	None
Example	`/* NACK the latest I2C byte transfer */` `Sys_I2C_NACK();`

15.66 SYS_I2C_NACKANDSTOP

Manually not-acknowledge the latest transfer and send a stop condition (Master mode only)

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_i2c.h
Template	`void Sys_I2C_NACKAndStop( void )`
Description	Manually not-acknowledge the latest transfer and send a stop condition (Master mode only)
Inputs	None
Outputs	None
Assumptions	None

Example	`/* NACK the latest I2C byte transfer and issue a stop`
	` * condition on the I2C interface */`
	`Sys_I2C_NACKAndStop();`

15.67 SYS_I2C_RESET

Reset the I^2C interface

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_i2c.h
Template	`void Sys_I2C_Reset( void )`
Description	Reset the I^2C interface
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Reset the I2C interface */` `Sys_I2C_Reset();`

15.68 SYS_I2C_STARTREAD

Initialize an I^2C master read transfer on the I^2C interface

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_i2c.h
Template	`void Sys_I2C_StartRead( uint32_t addr )`
Description	Initialize an I^2C master read transfer on the I^2C interface
Inputs	addr = I^2C slave address to initiate a transfer with
Outputs	None
Assumptions	None
Example	`/* Initialize a read from address 0x40 over the I2C interface */` `Sys_I2C_StartRead(0x40);`

15.69 SYS_I2C_STARTWRITE

Initialize an I^{2}C master write transfer on the I^{2}C interface

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_i2c.h
Template	`void Sys_I2C_StartWrite( uint32_t addr )`
Description	Initialize an I^{2}C master write transfer on the I^{2}C interface
Inputs	addr = I^{2}C slave address to initiate a transfer with
Outputs	None
Assumptions	None

Example	`/* Initialize a write to the general call address over the I2C`
	` * interface */`
	`Sys_I2C_StartWrite(0x00);`

15.70 SYS_IHM_DECODER_CLEAR_STATUS

Clear the data, and hence the status, for the Hamming decoder and de-interleaver

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_ecb.h
Template	`void Sys_IHM_Decoder_Clear_Status( void )`
Description	Clear the data, and hence the status, for the Hamming decoder and de-interleaver
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Clear the Hamming decoder and de-interleaver status */` `Sys_IHM_Decoder_Clear_Status();`

15.71 SYS_IHM_DECODER_CONFIG

Configure the Hamming decoder and de-interleaver

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_ecb.h
Template	`void Sys_IHM_Decoder_Config( uint8_t config )`
Description	Configure the Hamming decoder and de-interleaver
Inputs	config = The IHM decoder configuration; use IHM_DMA_[3REG | 4REG]_BYTE, IHM_INTERLEAVE_[128B | 126B]_BYTE, IHM_INTERLEAVE_[DISABLE | ENABLE]_BYTE, IHM_COR_[DISABLE | ENABLE]_BYTE, and IHM_[DISABLE | ENABLE]_BYTE
Outputs	None
Assumptions	None

Example	`/* Enable and configure the Hamming decoder and de-interleaver */`
	`Sys_IHM_Decoder_Config( IHM_INTERLEAVE_128B_BYTE |`
	`IHM_INTERLEAVE_ENABLE_BYTE |`
	`IHM_COR_ENABLE_BYTE |`
	`IHM_ENABLE_BYTE );`

15.72 SYS_NVIC_CLEARALLPENDINGINT

Clear the pending status for all external interrupts

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_cm3.h
Template	`void Sys_NVIC_ClearAllPendingInt()`
Description	Clear the pending status for all external interrupts
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Clear all pending external interrupts */` `Sys_NVIC_ClearAllPendingInt();`

15.73 SYS_NVIC_DISABLEALLINT

Disable all external interrupts

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_cm3.h
Template	`void Sys_NVIC_DisableAllInt( void )`
Description	Disable all external interrupts
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Disable all external interrupts */` `Sys_NVIC_DisableAllInt();`

15.74 SYS_PCM_CONFIG

Configure the PCM interface

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_pcm.h
Template	`void Sys_PCM_Config(uint32_t config)`
Description	Configure the PCM interface
Inputs	config = Interface operation configuration; use C_PCM_BIT_ORDER_[MSB | LSB]_FIRST, C_PCM_TX_ALIGN_[MSB | LSB], C_PCM_WORD_SIZE_*, C_PCM_FRAME_ALIGN_[LAST | FIRST], C_PCM_FRAME_WIDTH_[SHORT | LONG], C_PCM_MULTIWORD_*, C_PCM_SUBFRAME_[ENABLE | DISABLE], C_PCM_CONTROLLER_[CM3 | DMA], C_PCM_[DISABLE | ENABLE], and C_PCM_SELECT_[MASTER | SLAVE]
Outputs	None
Assumptions	None

Example	`/* Configure the PCM interface as a master for use with the DMA */`
	`Sys_PCM_Config ( C_PCM_BIT_ORDER_LSB_FIRST | C_PCM_TX_ALIGN_LSB |`
	`C_PCM_WORD_SIZE_16 | C_PCM_FRAME_ALIGN_LAST |`
	`C_PCM_FRAME_WIDTH_SHORT | C_PCM_MULTIWORD_2 |`
	`C_PCM_SUBFRAME_DISABLE | C_PCM_CONTROLLER_DMA |`
	`C_PCM_ENABLE | C_PCM_SELECT_MASTER );`

15.75 SYS_PCM_DISABLE

Disable the PCM interface without changing other PCM configuration settings

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_pcm.h
Template	`void Sys_PCM_Disable( void )`
Description	Disable the PCM interface without changing other PCM configuration settings
Inputs	None
Outputs	None
Assumptions	None

Example	`/* Disable the PCM interface without overwriting its previous`
	` * configuration */`
	`Sys_PCM_Disable();`

15.76 SYS_PCM_ENABLE

Enable the PCM interface without changing other PCM configuration settings

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_pcm.h
Template	`void Sys_PCM_Enable( void )`
Description	Enable the PCM interface without changing other PCM configuration settings
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Enable the PCM interface with its previous configuration */` `Sys_PCM_Enable();`

15.77 SYS_RS_DECODER_CLEAR_STATE

Clear the internal state and status outputs from the Reed-Solomon decoder, including the input/output pointers

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_ecb.h
Template	`void Sys_RS_Decoder_Clear_State( void )`
Description	Clear the internal state and status outputs from the Reed-Solomon decoder, including the input/output pointers
Inputs	None
Outputs	None
Assumptions	The Reed-Solomon decoder is idle
Example	`/* Clear the current state for the Reed-Solomon decoder block */` `Sys_RS_Decoder_Clear_State();`

15.78 SYS_RS_DECODER_CONFIG

Configure the Reed-Solomon decoder

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_ecb.h
Template	`void Sys_RS_Decoder_Config( uint8_t config )`
Description	Configure the Reed-Solomon decoder
Inputs	config = The RS decoder configuration; use RS_DEC_OUT_[INC | DEC], RS_DEC_IN_[INC | DEC], RS_DEC_OUT_*BIT_BYTE, and RS_DEC_IN_*BIT_BYTE
Outputs	None
Assumptions	None

Example	`/* Configure the Reed-Solomon decoder */`
	`Sys_RS_Decoder_Config( RS_DEC_IN_32BIT_BYTE |`
	`RS_DEC_OUT_32BIT_BYTE );`

15.79 SYS_RS_DECODER_GETOUTPUT

Get the 11 Reed-Solomon decoder output data samples

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_ecb.c
Template	`void Sys_RS_Decoder_GetOutput( uint32_t *data )`
Description	Get the 11 Reed-Solomon decoder output data samples
Inputs	None
Outputs	data = Pointer to output data from the Reed-Solomon decoder
Assumptions	None
Example	`/* Get the output data from the Reed-Solomon decoder */` `Sys_RS_Decoder_GetOutput(data);`

15.80 SYS_RS_DECODER_SETINPUT

Set the 15 Reed-Solomon decoder data and parity input samples

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_ecb.c
Template	`void Sys_RS_Decoder_SetInput( uint32_t *data )`
Description	Set the 15 Reed-Solomon decoder data and parity input samples
Inputs	data = Pointer to input data for the Reed-Solomon decoder
Outputs	None
Assumptions	None
Example	`/* Set the input data for the Reed-Solomon decoder */` `Sys_RS_Decoder_SetInput(data);`

15.81 SYS_RS_ENCODER_CLEAR_STATE

Clear the internal state of the Reed-Solomon encoder, including the input/output pointers

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_ecb.h
Template	`void Sys_RS_Encoder_Clear_State( void )`
Description	Clear the internal state of the Reed-Solomon encoder, including the input/output pointers
Inputs	None
Outputs	None
Assumptions	The Reed-Solomon encoder is idle
Example	`/* Clear the current state for the Reed-Solomon encoder block */` `Sys_RS_Encoder_Clear_State();`

15.82 SYS_RS_ENCODER_CONFIG

Configure the Reed-Solomon encoder

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_ecb.h
Template	`void Sys_RS_Encoder_Config( uint8_t config )`
Description	Configure the Reed-Solomon encoder
Inputs	config = The RS encoder configuration; use RS_ENC_OUT_[INC | DEC], RS_ENC_IN_[INC | DEC], RS_ENC_OUT_*BIT_BYTE, and RS_ENC_IN_*BIT_BYTE
Outputs	None
Assumptions	None

Example	`/* Configure the Reed-Solomon encoder */`
	`Sys_RS_Encoder_Config( RS_ENC_IN_32BIT_BYTE |`
	`RS_ENC_OUT_32BIT_BYTE );`

15.83 SYS_RS_ENCODER_GETOUTPUT

Get the 15 Reed-Solomon encoder data and parity output samples

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_ecb.c
Template	`void Sys_RS_Encoder_GetOutput( uint32_t *data )`
Description	Get the 15 Reed-Solomon encoder data and parity output samples
Inputs	None
Outputs	data = Pointer to output data from the Reed-Solomon encoder
Assumptions	None
Example	`/* Get the output data from the Reed-Solomon encoder */` `Sys_RS_Encoder_GetOutput(data);`

15.84 SYS_RS_ENCODER_SETINPUT

Set the 11 Reed-Solomon encoder data input samples

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_ecb.c
Template	`void Sys_RS_Encoder_SetInput( uint32_t *data )`
Description	Set the 11 Reed-Solomon encoder data input samples
Inputs	data = Pointer to input data for the Reed-Solomon encoder
Outputs	None
Assumptions	None
Example	`/* Set the input data for the Reed-Solomon encoder */` `Sys_RS_Encoder_SetInput(data);`

15.85 SYS_SET_CRC_CONFIG

Configure the CRC generator type, endianness of the input data, and standard vs non-standard CRC behavior

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_crc.h
Template	`void Sys_Set_CRC_Config( uint32_t config )`
Description	Configure the CRC generator type, endianness of the input data, and standard vs non-standard CRC behavior
Inputs	config = CRC generator configuration; use CRC_[CCITT | 32], CRC_[BIG | LITTLE]_ENDIAN, CRC_BIT_ORDER_[STANDARD | NON_STANDARD], CRC_FINAL_REVERSE_[STANDARD | NON_STANDARD], and CRC_FINAL_XOR_[STANDARD | NON_STANDARD]
Outputs	None
Assumptions	None
Example	`/* Configure the CRC generator to generate CRC-CCITT compatible * values, and to expect data in big-endian format */ Sys_Set_CRC_Config(CRC_BIG_ENDIAN | CRC_CCITT);`

15.86 SYS_SPI_CONFIG

Configure the SPI operation and controller information

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_spi.h
Template	`void Sys_SPI_Config( uint32_t config )`
Description	Configure the SPI operation and controller information
Inputs	config = Interface operation configuration; use C_SPI_OVERRUN_INT_[DISABLE | ENABLE], C_SPI_UNDERRUN_INT_[DISABLE | ENABLE], C_SPI_CONTROLLER_[CM3 | DMA], C_SPI_SELECT_[MASTER | SLAVE], C_SPI_CLK_POLARITY_[NORMAL | INVERSE], C_SPI_MODE_SELECT_[MANUAL | AUTO], C_SPI_[DISABLE | ENABLE] and C_SPI_PRESCALE_*
Outputs	None
Assumptions	None

Example	`/* Configure SPI for master-mode writes of 16-bit data (controlled by * the ARM Cortex-M3 processor), running the SPI at 1/4 of the system * clock frequency. */`
	`Sys_SPI_Config( C_SPI_OVERRUN_INT_DISABLE |`
	`C_SPI_UNDERRUN_INT_DISABLE |`
	`C_SPI_CONTROLLER_CM3 |`
	`C_SPI_SELECT_MASTER |`
	`C_SPI_CLK_POLARITY_NORMAL |`
	`C_SPI_MODE_SELECT_AUTO |`
	`C_SPI_ENABLE |`
	`C_SPI_PRESCALE_4 );`
	
	`Sys_SPI_TransferConfig( C_SPI_IDLE | C_SPI_WRITE_DATA |`
	`C_SPI_CS_1 | C_SPI_WORD_SIZE_16 );`

15.87 SYS_SPI_MASTERINIT

Initialize an SPI operation on the SPI interface when running in master mode

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_spi.h
Template	`void Sys_SPI_MasterInit( void )`
Description	Initialize an SPI operation on the SPI interface when running in master mode
Inputs	None
Outputs	None
Assumptions	The SPI interface is currently idle The SPI interface is configured for master mode operation If writing over the SPI interface, the data to be written has been queued

Example	`/* Read the latest word from SPI before starting the next transfer */`
	`tempData = SPI->RX_DATA;`
	`Sys_SPI_MasterInit();`

15.88 SYS_SPI_READ

Read the specified number of bits over the SPI interface

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_spi.h
Template	`void Sys_SPI_Read( uint8_t bits )`
Description	Read the specified number of bits over the SPI interface.
Inputs	bits = Number of bits to receive; use a constant between 1 and 32
Outputs	None
Assumptions	The SPI interface is currently idle The SPI interface is configured for master mode operation

Example	`/* Save the latest word from SPI before reading the next SPI byte */`
	`tempData = SPI->RX_DATA;`
	`Sys_SPI_Read(8);`

15.89 SYS_SPI_READWRITE

Read and write the specified number of bits over the SPI interface (full-duplex)

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_spi.h
Template	`void Sys_SPI_ReadWrite( uint8_t bits )`
Description	Read and write the specified number of bits over the SPI interface (full-duplex)
Inputs	bits = Number of bits to transmit and receive; use a constant between 1 and 32
Outputs	None
Assumptions	The SPI interface is currently idle The SPI interface is configured for master mode operation The data to be written has been queued

Example	`/* Echo back to slave device the most recently received SPI word,`
	` * while reading the next word */`
	`SPI->TX_DATA = SPI->RX_DATA;`
	`Sys_SPI_ReadWrite(32);`

15.90 SYS_SPI_TRANSFERCONFIG

Set the SPI interface transfer configuration

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_spi.h
Template	`void Sys_SPI_TransferConfig( uint32_t config )`
Description	Set the SPI interface transfer configuration
Inputs	config = Interface transfer configuration; use C_SPI_[IDLE | START], C_SPI_[WRITE | READ | RW]_DATA, C_SPI_CS_* and C_SPI_WORD_SIZE_*
Outputs	None
Assumptions	None

Example	`/* Configure SPI for master-mode writes of 32-bit data (controlled by`
	` * the ARM Cortex-M3 processor), running the SPI at 1/2 of the system`
	` * clock frequency. */`
	`Sys_SPI_Config( C_SPI_OVERRUN_INT_DISABLE |`
	`C_SPI_UNDERRUN_INT_DISABLE |`
	`C_SPI_CONTROLLER_CM3 |`
	`C_SPI_SELECT_MASTER |`
	`C_SPI_CLK_POLARITY_NORMAL |`
	`C_SPI_MODE_SELECT_AUTO |`
	`C_SPI_ENABLE |`
	`C_SPI_PRESCALE_2 );`
	
	`Sys_SPI_TransferConfig( C_SPI_IDLE | C_SPI_WRITE_DATA |`
	`C_SPI_CS_1 | C_SPI_WORD_SIZE_32);`

15.91 SYS_SPI_WRITE

Write the specified number of bits over the SPI interface

Type	Function
Include File	`#include <sk4.h>`
Source File	sk4_sys_spi.h
Template	`void Sys_SPI_Write( uint8_t bits )`
Description	Write the specified number of bits over the SPI interface.
Inputs	bits = Number of bits to transmit; use a constant between 1 and 32
Outputs	None
Assumptions	The SPI interface is currently idle The SPI interface is configured for master mode operation The data to be written has been queued

Example	`/* Queue up and transmit the next SPI byte */`
	`SPI->TX_DATA = tempData;`
	`Sys_SPI_Write(8);`

15.92 SYS_WAIT_FOR_EVENT

Hold the ARM Cortex-M3 core waiting for an event, interrupt request, abort or debug entry request (ARM Thumb-2 WFE instruction)

Type	Macro
Include File	`#include <sk4.h>`
Source File	sk4_sys_cm3.h
Template	`SYS_WAIT_FOR_EVENT`
Description	Hold the ARM Cortex-M3 core waiting for an event, interrupt request, abort or debug entry request (ARM Thumb-2 WFE instruction)
Inputs	None
Outputs	None
Assumptions	None

Example	`/* Hold the processor waiting for an event (including an interrupt)`
	` * to occur */`
	`SYS_WAIT_FOR_EVENT;`

15.93 SYS_WAIT_FOR_INTERRUPT

Hold the ARM Cortex-M3 core waiting for an interrupt request, abort or debug entry request (ARM Thumb-2 WFI instruction)

Type	Macro
Include File	`#include <sk4.h>`
Source File	sk4_sys_cm3.h
Template	`SYS_WAIT_FOR_INTERRUPT`
Description	Hold the ARM Cortex-M3 core waiting for an interrupt request, abort or debug entry request (ARM Thumb-2 WFI instruction)
Inputs	None
Outputs	None
Assumptions	None
Example	`/* Hold the processor waiting for an interrupt to occur */` `SYS_WAIT_FOR_INTERRUPT;`

CHAPTER 16
CFX System Library Reference

This reference chapter presents a detailed description of all the macros and functions in the system library, including calling parameters, modified registers, and returned values.

16.1 SYS_CALIBRATEADC

Load the calibrated DC offset for the specified ADC

Type	Macro
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfi
Template	`Sys_CalibrateADC( channel )`
Description	Load the calibrated DC offset for the specified ADC. Configures the ADC based on the calibration table for the ADC and the ADC's current preamp gain setting.
Inputs	channel = Channel to load calibration data for; use a constant between 0 and 3
Outputs	sr[SR_X_ZERO] = If 0, ADC calibration value loaded. If 1, ADC calibration value not available.
Register Modifications	y1, b1, xp7, yp3
Assumptions	The ADC calibration table has been loaded
Clock Cycles	Not available

Example	`// Calibrate ADC 0`
	`// NOTE: Assumes a valid calibration table and that the preamplifier`
	`//  gain for ADC 0 has been configured`
	`Sys_CalibrateADC(0)`

16.2 SYS_CALIBRATEADCS

Load the calibrated DC offsets for all ADCs

Type	Macro
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfi
Template	`Sys_CalibrateADCs`
Description	Load the calibrated DC offsets for all ADCs. Configures the ADCs based on the calibration table and the ADCs's current preamp gain settings.
Inputs	None
Outputs	sr[SR_X_ZERO] = If 0, ADC calibration values loaded. If 1, at least one ADC calibration value was not available.
Register Modifications	y1, b1, xp7, yp3
Assumptions	Calibration table loaded
Clock Cycles	Not available

Example	`// Calibrate the ADCs`
	`// NOTE: Assumes a valid calibration table and that the preamplifier`
	`//  gains for all the ADCs have been configured`
	`Sys_CalibrateADCs`

16.3 SYS_DELAY

Delay the CFX for the specified period of time

Type	Macro
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfi
Template	`Sys_Delay( delay )`
Description	Delay the CFX for the specified period of time. Uses SYSLIB_Delay.
Inputs	delay = Delay in milliseconds -1
Outputs	None
Register Modifications	b1, y1
Assumptions	None
Clock Cycles	Not applicable (delay is in milliseconds)

Example	`// Delay for 100 ms between calling functions A and B`
	`call FunctionA`
	`Sys_Delay( 100 - 1 )`
	`call FunctionB`

16.4 SYS_DELAYREG

Delay the CFX for an amount of time specified in a register

Type	Macro
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfi
Template	`Sys_DelayReg( reg )`
Description	Delay the CFX for an amount of time specified in a register. Uses SYSLIB_Delay.
Inputs	reg = Register containing delay (in milliseconds) - 1
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	Not applicable (delay is in milliseconds)

Example	`// Delay for 100 ms between calling functions A and B`
	`load x0, (100 - 1)`
	`call FunctionA`
	`Sys_DelayReg( x0 )`
	`call FunctionB`

16.5 SYS_RESET

Reset the system through a watchdog timeout

Type	Macro
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfi
Template	`Sys_Reset`
Description	Reset the system through a watchdog timeout. Uses SYSLIB_Reset.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	Not relevant (system is being reset)
Example	`// Reset the system` `Sys_Reset`

16.6 SYS_SETOPERATINGFREQUENCY

Set the base operating frequency

Type	Macro
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfi
Template	`Sys_SetOperatingFrequency( op_freq_index )`
Description	Set the base operating frequency. Uses SYSLIB_SetOperatingFrequency.
Inputs	op_freq_index = Operating frequency index; use SYS_FREQ_*
Outputs	sr[SR_X_ZERO] = If 0, new operating frequency set. If 1, could not set frequency (no calibration value found for the requested operating frequency).
Register Modifications	x1, x0, xp0, b1, b0
Assumptions	None
Clock Cycles	Not relevant (clock frequency is being adjusted)

Example	`// Switch the operating frequency to 2.56 MHz and check result.`
	`// Execute a breakpoint if no calibration value exists for`
	`// 2.56 MHz; otherwise continue with the application.`
	`Sys_SetOperatingFrequency( SYS_FREQ_2M56 )`
	`goto.0d ContinueWithApplication, not_zero`
	
	`// No calibration data. Perform some error handling.`
	`breakpoint`

16.7 SYS_SETOPERATINGFREQUENCY_THROTTLE

Set the operating frequency throttle using the clock throttling mechanism

Type	Macro
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfi
Template	`Sys_SetOperatingFrequency_Throttle( op_freq_index, slowclk, sdmclk )`
Description	Set the operating frequency throttle using the clock throttling mechanism. Update the SLOWCLK and SDMCLK prescalers to match the updated operating frequency if the operating frequency changes. Use the clock throttling mechanism to configure all clocks in unison. Uses SYSLIB_SetOperatingFrequency_Throttle.
Inputs	op_freq_index = Operating frequency index; use SYS_FREQ_* slowclk = SLOWCLK prescale setting; use SLOWCLK_PRESCALE_* sdmclk = SDMCLK prescale setting; use SDMCLK_PRESCALE_*
Outputs	sr[SR_X_ZERO] = If 0, new operating frequency set. If 1, could not set frequency (no calibration value found for the requested operating frequency)
Register Modifications	x1, x0, xp0, b1, b0
Assumptions	None
Clock Cycles	Not relevant (clock frequency is being adjusted)

Example	`// Switch the operating frequency to 2.56 MHz, resetting the divisors`
	`// for SLOWCLK and SDMCLK to maintain a SLOWCLK frequency of`
	`// 1.28MHz and SDMCLK of 2.56 MHz. Execute a breakpoint if no`
	`// calibration value exists for 2.56 MHz; otherwise continue with`
	`// the application.`
	`Sys_SetOperatingFrequency_Throttle( SYS_FREQ_2M56, \`
	`SLOWCLK_PRESCALE_2, \`
	`SDMCLK_PRESCALE_1 )`
	`goto.0d ContinueWithApplication, not_zero`
	
	`// No calibration data. Perform some error handling.`
	`breakpoint`

16.8 SYS_SOFTPOWERDOWNDISABLE

Exit from soft power-down mode

Type	Macro
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfi
Template	`Sys_SoftPowerdownDisable`
Description	Exit from soft power-down mode. Uses SYSLIB_SoftPowerdownDisable.
Inputs	None
Outputs	None
Register Modifications	y1, x1
Assumptions	System is in soft power-down mode
Clock Cycles	Not relevant (clock source is being changed)
Example	`// Exit from soft power-down mode` `Sys_SoftPowerdownDisable`

16.9 SYS_SOFTPOWERDOWNENABLE

Put the system in soft power-down mode

Type	Macro
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfi
Template	`Sys_SoftPowerdownEnable`
Description	Put the system in soft power-down mode. In this mode analog blocks are disabled and the oscillator is slowed down to ~17 kHz. Uses SYSLIB_SoftPowerdownEnable.
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	Not relevant (clock source is being changed)
Example	`// Put the system into soft power-down mode` `Sys_SoftPowerdownEnable`

16.10 SYS_WAITFORSPIDONE

Wait for the SPI interface to become idle

Type	Macro
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfi
Template	`Sys_WaitForSPIDone`
Description	Wait for the SPI interface to become idle. Uses SYSLIB_WaitForSPIDone.
Inputs	None
Outputs	None
Register Modifications	sr[SR_X_CARRY]
Assumptions	The SPI interface has been enabled
Clock Cycles	Not applicable (arbitrary delay length)

Example	`// Transmit a word over the SPI interface in auto-mode and wait for`
	`// the SPI transmission to complete`
	`store p[D_SPI_DATA], b1`
	`Sys_WaitForSPIDone`

16.11 SYSLIB_CALIBRATEADC

Load the calibrated DC offset for the specified ADC

Type	Function
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfs
Template	`SYSLIB_CalibrateADC`
Description	Load the calibrated DC offset for the specified ADC. Configures the ADC based on the calibration table, the current sampling frequency setting, the specified ADC and the ADC's current preamp gain setting.
Inputs	yp3 = Pointer to ADC calibration table xp7 = Channel to calibrate [0-3]
Outputs	sr[SR_X_ZERO] = If 0, ADC calibration value loaded. If 1, ADC calibration value not available.
Register Modifications	y1, b1, xp7, yp3
Assumptions	The ADC calibration table has been loaded The round bit from SR is disabled
Clock Cycles	Not available

Example	`// Calibrate ADC 0`
	`// NOTE: Assumes that a calibration table has been loaded and the`
	`//  preamplifier gain has been set for ADC 0`
	`load xp7, 0`
	`call.1d SYSLIB_CalibrateADC`
	`load yp3, SYSVAR_Y_ADCCAL_START // [ds] Address of calibration table`

16.12 SYSLIB_DELAY

Delay the CFX core for the specified period of time

Type	Function
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfs
Template	`SYSLIB_Delay`
Description	Delay the CFX core for the specified period of time
Inputs	b1 = Delay in ms - 1
Outputs	None
Register Modifications	b1, y1
Assumptions	Valid SYSVAR_OPERATING_FREQ set by firmware The stack has been initialized
Clock Cycles	Not applicable (delay is in milliseconds)

Example	`// Delay for 100 ms`
	`call.1d SYSLIB_Delay`
	`load b1, (100 - 1)  // [ds] Load delay value`

16.13 SYSLIB_RESET

Reset the system with a watchdog timeout

Type	Function
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfs
Template	`SYSLIB_Reset`
Description	Reset the system with a watchdog timeout
Inputs	None
Outputs	None
Register Modifications	sr
Assumptions	None
Clock Cycles	Not relevant (system is being reset)

Example	`// Reset the system`
	`call.1d SYSLIB_Reset`
	`nop  // [ds]`

16.14 SYSLIB_SETOPERATINGFREQUENCY

Set the internal oscillator base operating frequency; the oscillator multiplication setting is not a component of the base operating frequency configuration

Type	Function	
Include File	`#include <sk4_syslib.cfi>`	
Source File	sk4_syslib.cfs	
Template	`SYSLIB_SetOperatingFrequency`	
Description	Set the internal oscillator base operating frequency; the oscillator multiplication setting is not a component of the base operating frequency configuration.	
Inputs	b1.h	= Operating frequency index; use SYS_FREQ_*
Outputs	sr[SR_X_ZERO]	= If 0, new operating frequency set If 1, could not set frequency (no calibration value found for the requested operating frequency)
Register Modifications	x1, x0, xp0, b1, b0	
Assumptions	Calibration table loaded	
Clock Cycles	Not relevant (clock frequency is being adjusted)	

Example	`// Switch the operating frequency to 2.56 MHz and check result.`
	`// Execute a breakpoint if no calibration value exists for`
	`// 2.56 MHz; otherwise continue with the application.`
	`call.1d SYSLIB_SetOperatingFrequency`
	`load b1, SYS_FREQ_2M56 // [ds] Load frequency`
	`goto.0d ContinueWithApplication, not_zero`
	
	`// No calibration data. Perform some error handling.`
	`breakpoint`	

16.15 SYSLIB_SETOPERATINGFREQUENCY_THROTTLE

Set the internal oscillator base operating frequency, using clock throttling to also configure SLOWCLK and SDMCLK

Type	Function
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfs
Template	`SYSLIB_SetOperatingFrequency_Throttle`
Description	Set the internal oscillator base operating frequency, using clock throttling to also configure SLOWCLK and SDMCLK. The SLOWCLK and SDMCLK prescalers are updated to match the updated operating frequency only if the base operating frequency changes. The oscillator multiplication setting is not a component of the base operating frequency configuration.
Inputs	b1 = Operating frequency index; use SYS_FREQ_* x0 = SLOWCLK prescale setting; use SLOWCLK_PRESCALE_* x1 = SDMCLK prescale setting; use SDMCLK_PRESCALE_*
Outputs	sr[SR_X_ZERO] = If 0, new operating frequency set If 1, could not set frequency (no calibration found for the requested operating frequency)
Register Modifications	x1, x0, xp0, b1, b0
Assumptions	Calibration table loaded
Clock Cycles	Not relevant (clock frequency is being adjusted)

Example	`// Switch the operating frequency to 2.56 MHz, resetting the divisors`
	`// for SLOWCLK and SDMCLK to maintain a SLOWCLK frequency of`
	`// 1.28MHz and SDMCLK of 2.56 MHz. Execute a breakpoint if no`
	`// calibration value exists for 2.56 MHz; otherwise continue with`
	`// the application.`
	`load b1, SYS_FREQ_2M56`
	`load x0, SLOWCLK_PRESCALE_2`
	`call.1d SYSLIB_SetOperatingFrequency_Throttle`
	`load x1, SDMCLK_PRESCALE_1 // [ds] Load SDMCLK prescaler`
	`goto.0d ContinueWithApplication, not_zero`
	
	`// No calibration data. Perform some error handling.`
	`breakpoint`

16.16 SYSLIB_SOFTPOWERDOWNDISABLE

Exit from soft power-down mode

Type	Function
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfs
Template	`SYSLIB_SoftPowerdownDisable`
Description	Exit from soft power-down mode.
Inputs	None
Outputs	None
Register Modifications	y1, x1
Assumptions	System is in soft power-down mode (through previously called SYSLIB_SoftPowerdownEnable)
Clock Cycles	Not relevant (clock source is being changed)

Example	`// Restore the system from soft power-down mode`
	`call.1d SYSLIB_SoftPowerdownDisable`
	`nop     // [ds]`

16.17 SYSLIB_SOFTPOWERDOWNENABLE

Put the system in soft power-down mode

Type	Function
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfs
Template	`SYSLIB_SoftPowerdownEnable`
Description	Put the system in soft power-down mode. In this mode, analog blocks are disabled and the oscillator is slowed down to ~17 kHz.
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	Not relevant (clock source is being changed)

Example	`// Switch to soft power-down mode`
	`call.1d SYSLIB_SoftPowerdownEnable`
	`nop     // [ds]`

16.18 SYSLIB_WAITFORSPIDONE

Wait for the SPI interface to become idle

Type	Function
Include File	`#include <sk4_syslib.cfi>`
Source File	sk4_syslib.cfs
Template	`SYSLIB_WaitForSPIDone`
Description	Wait for the SPI interface to become idle
Inputs	None
Outputs	None
Register Modifications	sr[SR_X_CARRY] (bit)
Assumptions	The SPI interface has been enabled
Clock Cycles	Not applicable (arbitrary delay length)

Example	`// Read 8 bits from the SPI port and store the data in`
	`// a0. NOTE: SPI port must be enabled (not shown here)`
	`load  x0, (SPI_START | SPI_READ | (8 - 1))      // Read 8 bits`
	`call.1d SYSLIB_WaitForSPIDone`
	`store p[D_SPI_CTRL], x0                          // [ds]`
	`load  a0, p[D_SPI_DATA]                          // Store data`

CHAPTER 17
CFX Math Library Macro Reference

This reference chapter presents a detailed description of all the macros in the CFX math library's include file, including calling parameters, modified registers, returned values and execution time.

NOTE: When using a ROM-based version of the math library, macro execution time is increased by 2 cycles per call into the ROM (as described in Section 2.4.1, “ROM-Based Libraries” on page 30).

17.1 MATHLIB_ATTACK_RELEASE

Calculate a first-order attack-release

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_ATTACK_RELEASE(XP_OUT, XP_COEFF)`
Description	Calculate a first-order attack-release. The outputs of the attack-release filter are calculated as: • y(n) = coeff * x(n) + (1-coeff) * y(n-1) = coeff * (x(n)-y(n-1)) + y(n-1) Where: • coeff = a if x(n) is larger or equal to y(n-1) • coeff = b if x(n) is smaller than y(n-1) • x(n) is the current input • y(n) is the current output • y(n-1) is the previous output • a is the coefficient for attack • b is the coefficient for release Coefficients a and b must be positive. It is assumed that (x(n)-y(n-1)) is in the range [-1, 1).
Inputs	a0.h = Input x(n) in user-defined 24-bit fixed-point format XP_OUT = Pointer to y(n-1) (24-bit fixed-point value) XP_COEFF = Pointer to coefficient a, followed by coefficient b (frac24 coefficients)
Outputs	a0.h = Output y(n) in user-defined 24-bit fixed-point format
Register Modifications	a0, x0, x1, y0
Assumptions	None
Clock Cycles	8
Example	See Section 7.6, “Examples” on page 98.

17.2 MATHLIB_ATTACK_RELEASE_VEC

Vector-wise first-order attack-release

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_ATTACK_RELEASE_VEC(XP_IN, XP_OUT, XP_COEFF, VEC_LENGTH)`
Description	Vector-wise first-order attack-release. See MATHLIB_ATTACK_RELEASE.
Inputs	XP_IN = Pointer to x(n) vector XP_OUT = Pointer to y(n-1) vector XP_COEFF = Pointer to a vector of coefficient pairs VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Also the pointer to output vector y(n)
Register Modifications	a0, x0, x1, y0
Assumptions	None
Clock Cycles	3 + 9 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.3 MATHLIB_CONV_FL8N16_FRAC48

Convert fl8n16 format to frac48 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_fl8n16_frac48`
Description	Convert fl8n16 format to frac48 format.
Inputs	a0.h = Input value for conversion (fl8n16)
Outputs	a0 = Conversion result (frac48)
Register Modifications	a0, x0
Assumptions	None
Clock Cycles	3
Example	See Section 7.6, “Examples” on page 98.

17.4 MATHLIB_CONV_FL8N16_M24P24

Convert fl8n16 format to m24p24 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_fl8n16_m24p24`
Description	Convert fl8n16 format to m24p24 format.
Inputs	a0.h = Input value for conversion (fl8n16)
Outputs	a0 = Conversion result (m24p24)
Register Modifications	a0, x0
Assumptions	None
Clock Cycles	5
Example	See Section 7.6, “Examples” on page 98.

17.5 MATHLIB_CONV_FL8N16_M8P16

Convert fl8n16 format to m8p16 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_fl8n16_m8p16`
Description	Convert fl8n16 format to m8p16 format.
Inputs	a0.h = Input value for conversion (fl8n16)
Outputs	a0.h = Conversion result (m8p16)
Register Modifications	a0, x0
Assumptions	None
Clock Cycles	4
Example	See Section 7.6, “Examples” on page 98.

17.6 MATHLIB_CONV_FRAC24_FL8N16

Convert frac24 format to fl8n16 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_frac24_fl8n16`
Description	Convert frac24 format to fl8n16 format.
Inputs	a0.h = Input value for conversion (frac24)
Outputs	a0.h = Conversion result (fl8n16)
Register Modifications	a0, y0
Assumptions	None
Clock Cycles	6
Example	See Section 7.6, “Examples” on page 98.

17.7 MATHLIB_CONV_FRAC48_FL8N16

Convert frac48 format to fl8n16 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_frac48_fl8n16`
Description	Convert frac48 format to fl8n16 format.
Inputs	a0 = Input value for conversion (frac48)
Outputs	a0.h = Conversion result (fl8n16)
Register Modifications	a0, y0
Assumptions	None
Clock Cycles	5
Example	See Section 7.6, “Examples” on page 98.

17.8 MATHLIB_CONV_INT24_FL8N16

Convert int24 format to fl8n16 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_int24_fl8n16`
Description	Convert int24 format to fl8n16 format.
Inputs	a0.h = Input value for conversion (int24)
Outputs	a0.h = Conversion result (fl8n16)
Register Modifications	a0, x0, y0
Assumptions	Flag Updates mode is enabled
Clock Cycles	8
Example	See Section 7.6, “Examples” on page 98.

17.9 MATHLIB_CONV_M24P24_FL8N16

Convert m24p24 format to fl8n16 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_m24p24_fl8n16`
Description	Convert m24p24 format to fl8n16 format.
Inputs	a0 = Input value for conversion (m24p24)
Outputs	a0.h = Conversion result (fl8n16)
Register Modifications	a0, x0, y0
Assumptions	Flag Updates mode is enabled
Clock Cycles	7
Example	See Section 7.6, “Examples” on page 98.

17.10 MATHLIB_CONV_M8P16_FL8N16

Convert m8p16 format to fl8n16 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_m8p16_fl8n16`
Description	Convert m8p16 format to fl8n16 format.
Inputs	a0.h = Input value for conversion (m8p16)
Outputs	a0.h = Conversion result (fl8n16)
Register Modifications	a0, x0, y0
Assumptions	Flag Updates mode is enabled
Clock Cycles	7
Example	See Section 7.6, “Examples” on page 98.

17.11 MATHLIB_CONV_VEC_FL8N16_FRAC48

Convert vector from fl8n16 format to frac48 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_VEC_fl8n16_frac48(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Convert vector from fl8n16 format to frac48 format. See MATHLIB_CONV_fl8n16_frac48.
Inputs	XP_IN = Pointer to an input vector of fl8n16 elements in X memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac48 elements in XY memory.
Register Modifications	a0, x0
Assumptions	XP_IN != XP_OUT
Clock Cycles	3 + 5 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.12 MATHLIB_CONV_VEC_FL8N16_M24P24

Convert vector from fl8n16 format to m24p24 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_VEC_fl8n16_m24p24(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Convert vector from fl8n16 format to m24p24 format. See MATHLIB_CONV_fl8n16_m24p24.
Inputs	XP_IN = Pointer to an input vector of fl8n16 elements in X memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m24p24 elements in XY memory.
Register Modifications	a0, x0
Assumptions	XP_IN != XP_OUT
Clock Cycles	3 + 7 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.13 MATHLIB_CONV_VEC_FL8N16_M8P16

Convert vector from fl8n16 format to m8p16 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_VEC_fl8n16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Convert vector from fl8n16 format to m8p16 format. See MATHLIB_CONV_fl8n16_m8p16.
Inputs	XP_IN = Pointer to an input vector of fl8n16 elements in X memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m8p16 elements in X memory.
Register Modifications	a0, x0
Assumptions	XP_IN != XP_OUT
Clock Cycles	3 + 6 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.14 MATHLIB_CONV_VEC_FRAC24_FL8N16

Convert vector from frac24 format to fl8n16 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_VEC_frac24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Convert vector from frac24 format to fl8n16 format. See MATHLIB_CONV_frac24_fl8n16.
Inputs	XP_IN = Pointer to an input vector of frac24 elements in X memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory.
Register Modifications	a0, y0
Assumptions	XP_IN != XP_OUT
Clock Cycles	3 + 8 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.15 MATHLIB_CONV_VEC_FRAC48_FL8N16

Convert vector from frac48 format to fl8n16 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_VEC_frac48_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Convert vector from frac48 format to fl8n16 format. See MATHLIB_CONV_frac48_fl8n16.
Inputs	XP_IN = Pointer to an input vector of frac48 elements in XY memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory.
Register Modifications	a0, y0
Assumptions	XP_IN != XP_OUT
Clock Cycles	3 + 7 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.16 MATHLIB_CONV_VEC_INT24_FL8N16

Convert vector from int24 format to fl8n16 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_VEC_int24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Convert vector from int24 format to fl8n16 format. See MATHLIB_CONV_int24_fl8n16.
Inputs	XP_IN = Pointer to an input vector of int24 elements in X memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory.
Register Modifications	a0, x0, y0
Assumptions	XP_IN != XP_OUT Flag Updates mode is enabled
Clock Cycles	3 + 10 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.17 MATHLIB_CONV_VEC_M24P24_FL8N16

Convert vector from m24p24 format to fl8n16 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_VEC_m24p24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Convert vector from m24p24 format to fl8n16 format. See MATHLIB_CONV_m24p24_fl8n16.
Inputs	XP_IN = Pointer to input vector of m24p24 elements in XY memory. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory.
Register Modifications	a0, x0, y0
Assumptions	XP_IN != XP_OUT Flag Updates mode is enabled
Clock Cycles	3 + 9 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.18 MATHLIB_CONV_VEC_M8P16_FL8N16

Convert vector from m8p16 format to fl8n16 format

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_CONV_VEC_m8p16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Convert vector from m8p16 format to fl8n16 format. See MATHLIB_CONV_m8p16_fl8n16.
Inputs	XP_IN = Pointer to input vector of m8p16 elements in X memory. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory.
Register Modifications	a0, x0, y0
Assumptions	XP_IN != XP_OUT Flag Updates mode is enabled
Clock Cycles	3 + 9 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.19 MATHLIB_COS_FRAC24_FRAC24

Approximate the cosine of a frac24 input

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_COS_frac24_frac24`	
Description	Approximate the cosine of a frac24 input.	
Inputs	a0.h	= Normalized input x (frac24). [-1, 1) range of frac24 format number maps to [-PI, PI) range for input angle x
Outputs	a0.h	= cos(x) (frac24)
Register Modifications	a0, xp0	
Assumptions	None	
Clock Cycles	5	
Example	See Section 7.6, “Examples” on page 98.	

17.20 MATHLIB_COS_VEC_FRAC24_FRAC24

Approximate the cosine of each frac24 element in a vector

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_COS_VEC_frac24_frac24(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Approximate the cosine of each frac24 element in a vector. See MATHLIB_COS_frac24_frac24.
Inputs	XP_IN = Pointer to an input vector of normalized frac24 angle values in X memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac24 elements in X memory.
Register Modifications	a0, xp0
Assumptions	None
Clock Cycles	3 + 7 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.21 MATHLIB_DB10_PWR_M8P16_FRAC48

Convert from dB10 unit to power value

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_DB10_PWR_m8p16_frac48`
Description	Convert from dB10 unit to power value. • y = 2^(x/(10*log(2)))
Inputs	a0.h = Input (m8p16), must be negative
Outputs	a0 = Conversion result (frac48)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	15
Example	See Section 7.6, “Examples” on page 98.

17.22 MATHLIB_DB10_PWR_VEC_M8P16_FRAC48

Convert vector of values from dB10 units to power values

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_DB10_PWR_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Convert vector of values from dB10 units to power values. See MATHLIB_DB10_PWR_m8p16_frac48.	
Inputs	XP_IN VEC_LENGTH	= Pointer to an input vector of m8p16 elements in X memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of frac48 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	6 + 12 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.23 MATHLIB_DB10_RMS_M8P16_FRAC48

Convert from dB10 unit to RMS value

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_DB10_RMS_m8p16_frac48`
Description	Convert from dB10 unit to RMS value. • y = 2^(x/(20*log(2)))
Inputs	a0.h = Input (m8p16), must be negative
Outputs	a0 = Conversion result (frac48)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	15
Example	See Section 7.6, “Examples” on page 98.

17.24 MATHLIB_DB10_RMS_VEC_M8P16_FRAC48

Convert vector of values from dB10 units to RMS values

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_DB10_RMS_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Convert vector of values from dB10 units to RMS values. See MATHLIB_DB10_RMS_m8p16_frac48.	
Inputs	XP_IN VEC_LENGTH	= Pointer to an input vector of m8p16 elements in X memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of frac48 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	6 + 12 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.25 MATHLIB_DB2_PWR_M8P16_FRAC48

Convert from dB2 unit to power value

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_DB2_PWR_m8p16_frac48`
Description	Convert from dB2 unit to power value. • y = 2^(x/10)
Inputs	a0.h = Input (m8p16), must be negative
Outputs	a0 = Conversion result (frac48)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	15
Example	See Section 7.6, “Examples” on page 98.

17.26 MATHLIB_DB2_PWR_VEC_M8P16_FRAC48

Convert vector of values from dB2 units to power values

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_DB2_PWR_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Convert vector of values from dB2 units to power values. See MATHLIB_DB2_PWR_m8p16_frac48.	
Inputs	XP_IN VEC_LENGTH	= Pointer to an input vector of m8p16 elements in X memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of frac48 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	6 + 12 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.27 MATHLIB_DB2_RMS_M8P16_FRAC48

Convert from dB2 unit to RMS value

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_DB2_RMS_m8p16_frac48`
Description	Convert from dB2 unit to RMS value. • y = 2^(x/20)
Inputs	a0.h = Input (m8p16), must be negative
Outputs	a0 = Conversion result (frac48)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	15
Example	See Section 7.6, “Examples” on page 98.

17.28 MATHLIB_DB2_RMS_VEC_M8P16_FRAC48

Convert vector of values from dB2 units to RMS values

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_DB2_RMS_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Convert vector of values from dB2 units to RMS values. See MATHLIB_DB2_RMS_m8p16_frac48.	
Inputs	XP_IN VEC_LENGTH	= Pointer to an input vector of m8p16 elements in X memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of frac48 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	6 + 12 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.29 MATHLIB_DIVIDE_FRAC24

Divide positive values and produce a frac24 quotient

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_DIVIDE_frac24`
Description	Divide positive values and produce a frac24 quotient. An undefined result is returned for dividend < 0 or divisor <= dividend.
Inputs	a0.h = Non-negative dividend (user-defined 24-bit fixed-point format) x0 = Strictly positive divisor (user-defined 24-bit fixed-point format)
Outputs	a0.l = Quotient (frac24)
Register Modifications	a0
Assumptions	The dividend is less than the divisor. Identical user-defined data formats are used for the dividend and divisor.
Clock Cycles	31
Example	See Section 7.6, “Examples” on page 98.

17.30 MATHLIB_DIVIDE_INT24

Divide positive values and produce an int24 quotient

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_DIVIDE_int24`
Description	Divide positive values and produce an int24 quotient. Upon completion, the remainder can be calculated in a0.h by executing the instruction "add a0, x0, lt". An undefined result is returned for dividend < 0 or divisor <= 0.
Inputs	a0.h = Non-negative dividend (user-defined 24-bit fixed-point format) x0 = Strictly positive divisor (user-defined 24-bit fixed-point format)
Outputs	a0.l = Quotient (int24)
Register Modifications	a0
Assumptions	Identical user-defined data formats are used for the dividend and divisor
Clock Cycles	32
Example	See Section 7.6, “Examples” on page 98.

17.31 MATHLIB_DIVIDE_M24P24

Divide positive values and produce an m24p24 quotient

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_DIVIDE_m24p24`
Description	Divide positive values and produce an m24p24 quotient. An undefined result is returned for dividend < 0 or divisor <= 0.
Inputs	a0.h = Non-negative dividend (user-defined 24-bit fixed-point format) x0 = Strictly positive divisor (user-defined 24-bit fixed-point format)
Outputs	a0 = Quotient (m24p24)
Register Modifications	a0, x1
Assumptions	Identical user-defined data formats are used for the dividend and divisor
Clock Cycles	61
Example	See Section 7.6, “Examples” on page 98.

17.32 MATHLIB_EXP2_M8P16_FRAC48

Approximate 2^x for m8p16 input x, with frac48 output

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_EXP2_m8p16_frac48`
Description	Approximate 2^x for m8p16 input x, with frac48 output. The input is represented as x = E + r, where E is the integer component of x and r is the fractional component. Two to the power x is approximated as: • 2^x = 2^E * 2^r = 2^(E+1) * 2^(r-1) The value of 2^(r-1) is approximated using a lookup table. Input values outside the range [-47, 0) are saturated to zero when x < -47 or to almost one (fractional full-scale value) when x >= 0.
Inputs	a0.h = Strictly negative input x (m8p16)
Outputs	a0 = Result 2^x (frac48)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	18
Example	See Section 7.6, “Examples” on page 98.

17.33 MATHLIB_EXP2_M8P16_M8P16

Approximate 2^x for m8p16 input x, with m8p16 output

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_EXP2_m8p16_m8p16`
Description	Approximate 2^x for m8p16 input x, with m8p16 output. The input is represented as x = E + r, where E is the integer component of x and r is the fractional component. Two to the power x is approximated as: • 2^x = 2^E * 2^r = 2^(E+1) * 2^(r-1) The value of 2^(r-1) is approximated using a lookup table. Input values outside the range [-40, 7) are saturated to zero when x < -40 or to almost 128 (m8p16 full-scale value) when x >= 7.
Inputs	a0.h = Input x (m8p16)
Outputs	a0 = Result 2^x (m8p16, but result also uses a0.l, as if output format were m8p40)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	19
Example	See Section 7.6, “Examples” on page 98.

17.34 MATHLIB_EXP2_VEC_M8P16_FRAC48

Approximate 2^x for each m8p16 element in a vector, with frac48 output values

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_EXP2_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Approximate 2^x for each m8p16 element in a vector, with frac48 output values. See MATHLIB_EXP2_m8p16_frac48.	
Inputs	XP_IN VEC_LENGTH	= Pointer to input vector of m8p16 elements in X memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of frac48 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	5 + 16 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.35 MATHLIB_EXP2_VEC_M8P16_M8P16

Approximate 2^x for each m8p16 element in a vector, with m8p16 output values

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_EXP2_VEC_m8p16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Approximate 2^x for each m8p16 element in a vector, with m8p16 output values. See MATHLIB_EXP2_m8p16_m8p16.
Inputs	XP_IN = Pointer to input vector of m8p16 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 16 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.36 MATHLIB_EXP_AVG

Calculate a first-order exponential average

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_EXP_AVG(XP_OUT, XP_COEFF)`
Description	Calculate a first-order exponential average. The outputs of the exponential average are calculated as: • y(n) = a * x(n) + (1-a) * y(n-1) = a * (x(n)-y(n-1)) + y(n-1) Where: • x(n) is the current input • y(n) is the current output • y(n-1) is the previous output • a is the coefficient Coefficient a must be positive. It is assumed that (x(n)-y(n-1)) is in the range [-1, 1).
Inputs	a0.h = Input x(n) in user-defined 24-bit fixed-point format XP_OUT = Pointer to y(n-1) (24-bit fixed-point value) XP_COEFF = Pointer to coefficient a (frac24)
Outputs	a0.h = Output y(n) in user-defined 24-bit fixed-point format
Register Modifications	a0, x0, y0
Assumptions	None
Clock Cycles	6
Example	See Section 7.6, “Examples” on page 98.

17.37 MATHLIB_EXP_AVG_VEC

Vector-wise first-order exponential average

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_EXP_AVG_VEC(XP_IN, XP_OUT, XP_COEFF, VEC_LENGTH)`
Description	Vector-wise first-order exponential average. See MATHLIB_EXP_AVG.
Inputs	XP_IN = Pointer to x(n) vector XP_OUT = Pointer to y(n-1) vector XP_COEFF = Pointer to a vector of coefficients VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Also the pointer to output vector y(n)
Register Modifications	a0, x0, y0
Assumptions	None
Clock Cycles	3 + 7 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.38 MATHLIB_LIN_INTERP

Linear interpolation

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_LIN_INTERP(XP_IN)`
Description	Linear interpolation. The interpolated result is calculated as: • y = t0 + x * (t1-t0) Where: • x is the index value, in range [-1, 1) • y is the interpolation result • t0 and t1 are the data points It is assumed that (t1-t0) is in the range [-1, 1).
Inputs	a0.h = Positive index value x (frac24) XP_IN = Pointer to data point t0, followed by data point t1
Outputs	a0.h = Interpolation result y (user-defined fixed-point format as defined by table data, t0 and t1)
Register Modifications	a0, x0, x1, y0
Assumptions	None
Clock Cycles	5
Example	See Section 7.6, “Examples” on page 98.

17.39 MATHLIB_LOG2_FL8N16_M8P16

Approximate the base-2 logarithm of a fl8n16 input

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_LOG2_fl8n16_m8p16`
Description	Approximate the base-2 logarithm of a fl8n16 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The base-2 logarithm of x is approximated as: • log2(x) = log2(m*2^E) = log2(m) + E The value of log2(m) is approximated using a lookup table. An undefined result is returned for input x <= 0. Overflow of the result returned in a0 occurs when x < 2^(-128).
Inputs	a0.h = Strictly positive input x (fl8n16)
Outputs	a0.h = Base-2 logarithm of x (m8p16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	16
Example	See Section 7.6, “Examples” on page 98.

17.40 MATHLIB_LOG2_FRAC24_M8P16

Approximate the base-2 logarithm of a frac24 input

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_LOG2_frac24_m8p16`
Description	Approximate the base-2 logarithm of a frac24 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The base-2 logarithm of x is approximated as: • log2(x) = log2(m*2^E) = log2(m) + E The value of log2(m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0.h = Strictly positive input x (frac24)
Outputs	a0.h = Base-2 logarithm of x (m8p16)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	15
Example	See Section 7.6, “Examples” on page 98.

17.41 MATHLIB_LOG2_FRAC48_M8P16

Approximate the base-2 logarithm of a frac48 input

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_LOG2_frac48_m8p16`
Description	Approximate the base-2 logarithm of a frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The base-2 logarithm of x is approximated as: • log2(x) = log2(m*2^E) = log2(m) + E The value of log2(m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0 = Strictly positive input x (frac48)
Outputs	a0.h = Base-2 logarithm of x (m8p16)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	15
Example	See Section 7.6, “Examples” on page 98.

17.42 MATHLIB_LOG2_M8P16_M8P16

Approximate the base-2 logarithm of an m8p16 input

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_LOG2_m8p16_m8p16`
Description	Approximate the base-2 logarithm of an m8p16 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The base-2 logarithm of x is approximated as: • log2(x) = log2(m*2^E) = log2(m) + E The value of log2(m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0.h = Strictly positive input x (m8p16)
Outputs	a0.h = Base-2 logarithm of x (m8p16)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	16
Example	See Section 7.6, “Examples” on page 98.

17.43 MATHLIB_LOG2_VEC_FL8N16_M8P16

Approximate log2 of each fl8n16 element in a vector

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_LOG2_VEC_fl8n16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Approximate log2 of each fl8n16 element in a vector. See MATHLIB_LOG2_fl8n16_m8p16.	
Inputs	XP_IN VEC_LENGTH	= Pointer to input vector of fl8n16 elements in X memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	5 + 16 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.44 MATHLIB_LOG2_VEC_FRAC24_M8P16

Approximate log2 of each frac24 element in a vector

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_LOG2_VEC_frac24_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Approximate log2 of each frac24 element in a vector. See MATHLIB_LOG2_frac24_m8p16.	
Inputs	XP_IN VEC_LENGTH	= Pointer to input vector of frac24 elements in X memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	5 + 12 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.45 MATHLIB_LOG2_VEC_FRAC48_M8P16

Approximate log2 of each frac48 element in a vector

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_LOG2_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Approximate log2 of each frac48 element in a vector. See MATHLIB_LOG2_frac48_m8p16.	
Inputs	XP_IN VEC_LENGTH	= Pointer to input vector of frac48 elements in XY memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	5 + 12 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.46 MATHLIB_LOG2_VEC_M8P16_M8P16

Approximate log2 of each m8p16 element in a vector

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_LOG2_VEC_m8p16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Approximate log2 of each m8p16 element in a vector. See MATHLIB_LOG2_m8p16_m8p16.	
Inputs	XP_IN VEC_LENGTH	= Pointer to input vector of m8p16 elements in X memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	6 + 13 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.47 MATHLIB_MAG_FRAC24_M2P46

Calculate the magnitude of a complex number

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_MAG_frac24_m2p46(XP_IN)`
Description	Calculate the magnitude of a complex number. • y = sqrt(real(x)^2 + imag(x)^2)
Inputs	XP_IN = Pointer to complex value x in X memory, with frac24 real part followed by frac24 imaginary part at next address
Outputs	a0 = Magnitude result (m2p46)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	19 (E even in SQRT) or 20 (E odd in SQRT)
Example	See Section 7.6, “Examples” on page 98.

17.48 MATHLIB_MAG_VEC_FRAC24_M2P46

Vector-wise magnitude of complex values

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_MAG_VEC_frac24_m2p46(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Vector-wise magnitude of complex values. See MATHLIB_MAG_frac24_m2p46.	
Inputs	XP_IN	= Pointer to an input vector of complex values in X memory. Real and complex components stored in interleaved manner. Cannot be xp0.
	VEC_LENGTH	= Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of m2p46 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	In range [5 + 18 * VEC_LENGTH, 5 + 19 * VEC_LENGTH]	
Example	See Section 7.6, “Examples” on page 98.	

17.49 MATHLIB_PWR_DB10_FRAC48_M8P16

Convert from power value to dB10 unit

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_PWR_DB10_frac48_m8p16`
Description	Convert from power value to dB10 unit. • y = 10 * log10(2) * log2(x) Overflow of the result returned in a0.h will occur when the input is less than 10^(-128/10).
Inputs	a0 = Input (frac48)
Outputs	a0.h = Conversion result (m8p16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	17
Example	See Section 7.6, “Examples” on page 98.

17.50 MATHLIB_PWR_DB10_VEC_FRAC48_M8P16

Convert vector of power values to dB10 units

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_PWR_DB10_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Convert vector of power values to dB10 units. See MATHLIB_PWR_DB10_frac48_m8p16.	
Inputs	XP_IN VEC_LENGTH	= Pointer to an input vector of frac48 elements in XY memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	6 + 16 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.51 MATHLIB_PWR_DB2_FRAC48_M8P16

Convert from power value to dB2 unit

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_PWR_DB2_frac48_m8p16`
Description	Convert from power value to dB2 unit. • y = 10 * log2(x) Overflow of the result returned in a0.h will occur when the input is less than 2^(-128/10).
Inputs	a0 = Input (frac48)
Outputs	a0.h = Conversion result (m8p16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	17
Example	See Section 7.6, “Examples” on page 98.

17.52 MATHLIB_PWR_DB2_VEC_FRAC48_M8P16

Convert vector of power values to dB2 units

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_PWR_DB2_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Convert vector of power values to dB2 units. See MATHLIB_PWR_DB2_frac48_m8p16.	
Inputs	XP_IN VEC_LENGTH	= Pointer to an input vector of frac48 elements in XY memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	6 + 16 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.53 MATHLIB_RECIP_FL8N16_FL8N16

Approximate the reciprocal of a signed fl8n16 input

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_RECIP_fl8n16_fl8n16`
Description	Approximate the reciprocal of a signed fl8n16 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x = 0. Overflow of the result returned in a0 occurs when the magnitude of the input x is less than 2^(-127).
Inputs	a0.h = Non-zero input x (fl8n16)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, x1, y0, y1, xp0
Assumptions	None
Clock Cycles	19
Example	See Section 7.6, “Examples” on page 98.

17.54 MATHLIB_RECIP_FRAC48_FL8N16

Approximate the reciprocal of a signed frac48 input

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_RECIP_frac48_fl8n16`
Description	Approximate the reciprocal of a signed frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x = 0.
Inputs	a0 = Non-zero input x (frac48)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	None
Clock Cycles	18
Example	See Section 7.6, “Examples” on page 98.

17.55 MATHLIB_RECIP_INT24_FL8N16

Approximate the reciprocal of a signed int24 input

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_RECIP_int24_fl8n16`
Description	Approximate the reciprocal of a signed int24 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x = 0.
Inputs	a0.h = Non-zero input x (int24)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	None
Clock Cycles	19
Example	See Section 7.6, “Examples” on page 98.

17.56 MATHLIB_RECIP_M8P16_FL8N16

Approximate the reciprocal of a signed m8p16 input

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_RECIP_m8p16_fl8n16`
Description	Approximate the reciprocal of a signed m8p16 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x = 0.
Inputs	a0.h = Non-zero input x (m8p16)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	None
Clock Cycles	19
Example	See Section 7.6, “Examples” on page 98.

17.57 MATHLIB_RECIP_POS_FL8N16_FL8N16

Approximate the reciprocal of a positive fl8n16 input

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_RECIP_POS_fl8n16_fl8n16`
Description	Approximate the reciprocal of a positive fl8n16 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x <= 0. Overflow of the result returned in a0 occurs when x < 2^(-127).
Inputs	a0.h = Strictly positive input x (fl8n16)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	None
Clock Cycles	16
Example	See Section 7.6, “Examples” on page 98.

17.58 MATHLIB_RECIP_POS_FRAC48_FL8N16

Approximate the reciprocal of a positive frac48 input

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_RECIP_POS_frac48_fl8n16`
Description	Approximate the reciprocal of a positive frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0 = Strictly positive input x (frac48)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	15
Example	See Section 7.6, “Examples” on page 98.

17.59 MATHLIB_RECIP_POS_INT24_FL8N16

Approximate the reciprocal of a positive int24 input

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_RECIP_POS_int24_fl8n16`
Description	Approximate the reciprocal of a positive int24 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0.h = Strictly positive input x (int24)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	16
Example	See Section 7.6, “Examples” on page 98.

17.60 MATHLIB_RECIP_POS_M8P16_FL8N16

Approximate the reciprocal of a positive m8p16 input

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_RECIP_POS_m8p16_fl8n16`
Description	Approximate the reciprocal of a positive m8p16 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0.h = Strictly positive input x (m8p16)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	16
Example	See Section 7.6, “Examples” on page 98.

17.61 MATHLIB_RECIP_POS_VEC_FL8N16_FL8N16

Approximate the reciprocal of each positive fl8n16 element in a vector

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_RECIP_POS_VEC_fl8n16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Approximate the reciprocal of each positive fl8n16 element in a vector. See MATHLIB_RECIP_POS_fl8n16_fl8n16.
Inputs	XP_IN = Pointer to input vector of fl8n16 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	5 + 16 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.62 MATHLIB_RECIP_POS_VEC_FRAC48_FL8N16

Approximate the reciprocal of each positive frac48 element in a vector

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_RECIP_POS_VEC_frac48_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Approximate the reciprocal of each positive frac48 element in a vector. See MATHLIB_RECIP_POS_frac48_fl8n16.
Inputs	XP_IN = Pointer to input vector of frac48 elements in XY memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	5 + 12 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.63 MATHLIB_RECIP_POS_VEC_INT24_FL8N16

Approximate the reciprocal of each positive int24 element in a vector

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_RECIP_POS_VEC_int24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Approximate the reciprocal of each positive int24 element in a vector. See MATHLIB_RECIP_POS_int24_fl8n16.	
Inputs	XP_IN VEC_LENGTH	= Pointer to input vector of int24 elements in X memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, x1, y0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	6 + 13 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.64 MATHLIB_RECIP_POS_VEC_M8P16_FL8N16

Approximate the reciprocal of each positive m8p16 element in a vector

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_RECIP_POS_VEC_m8p16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Approximate the reciprocal of each positive m8p16 element in a vector. See MATHLIB_RECIP_POS_m8p16_fl8n16.	
Inputs	XP_IN VEC_LENGTH	= Pointer to input vector of m8p16 elements in X memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, x1, y0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	6 + 13 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.65 MATHLIB_RECIP_VEC_FL8N16_FL8N16

Approximate the reciprocal of each signed fl8n16 element in a vector

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_RECIP_VEC_fl8n16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Approximate the reciprocal of each signed fl8n16 element in a vector. See MATHLIB_RECIP_fl8n16_fl8n16.	
Inputs	XP_IN VEC_LENGTH	= Pointer to input vector of fl8n16 elements in X memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, x1, y0, y1, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	5 + 19 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.66 MATHLIB_RECIP_VEC_FRAC48_FL8N16

Approximate the reciprocal of each signed frac48 element in a vector

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_RECIP_VEC_frac48_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Approximate the reciprocal of each signed frac48 element in a vector. See MATHLIB_RECIP_frac48_fl8n16.	
Inputs	XP_IN VEC_LENGTH	= Pointer to input vector of frac48 elements in XY memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, x1, y0, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	5 + 15 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.67 MATHLIB_RECIP_VEC_INT24_FL8N16

Approximate the reciprocal of each signed int24 element in a vector

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_RECIP_VEC_int24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Approximate the reciprocal of each signed int24 element in a vector. See MATHLIB_RECIP_int24_fl8n16.	
Inputs	XP_IN VEC_LENGTH	= Pointer to input vector of int24 elements in X memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, x1, y0, y1, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	6 + 16 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.68 MATHLIB_RECIP_VEC_M8P16_FL8N16

Approximate the reciprocal of each signed m8p16 element in a vector

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_RECIP_VEC_m8p16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`	
Description	Approximate the reciprocal of each signed m8p16 element in a vector. See MATHLIB_RECIP_m8p16_fl8n16.	
Inputs	XP_IN VEC_LENGTH	= Pointer to input vector of m8p16 elements in X memory. Cannot be xp0. = Number of elements in the vector
Outputs	XP_OUT	= Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, x1, y0, y1, xp0	
Assumptions	XP_IN != XP_OUT	
Clock Cycles	6 + 16 * VEC_LENGTH	
Example	See Section 7.6, “Examples” on page 98.	

17.69 MATHLIB_RMS_DB10_FRAC48_M8P16

Convert from RMS value to dB10 unit

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_RMS_DB10_frac48_m8p16`
Description	Convert from RMS value to dB10 unit. • y = 20 * log10(2) * log2(x) Overflow of the result returned in a0.h will occur when the input is less than 10^(-128/20).
Inputs	a0 = Input (frac48)
Outputs	a0.h = Conversion result (m8p16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	17
Example	See Section 7.6, “Examples” on page 98.

17.70 MATHLIB_RMS_DB10_VEC_FRAC48_M8P16

Convert vector of RMS values to dB10 units

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_RMS_DB10_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Convert vector of RMS values to dB10 units. See MATHLIB_RMS_DB10_frac48_m8p16.
Inputs	XP_IN = Pointer to an input vector of frac48 elements in XY memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 16 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.71 MATHLIB_RMS_DB2_FRAC48_M8P16

Convert from RMS value to dB2 unit

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_RMS_DB2_frac48_m8p16`
Description	Convert from RMS value to dB2 unit. • y = 20 * log2(x) Overflow of the result returned in a0.h will occur when the input is less than 2^(-128/20).
Inputs	a0 = Input (frac48)
Outputs	a0.h = Conversion result (m8p16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	17
Example	See Section 7.6, “Examples” on page 98.

17.72 MATHLIB_RMS_DB2_VEC_FRAC48_M8P16

Convert vector of RMS values to dB2 units

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_RMS_DB2_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Convert vector of RMS values to dB2 units. See MATHLIB_RMS_DB2_frac48_m8p16.
Inputs	XP_IN = Pointer to an input vector of frac48 elements in XY memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 16 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.73 MATHLIB_SIN_FRAC24_FRAC24

Approximate the sine of a frac24 input

Type	Macro	
Include File	`#include <sk4_math.cfi>`	
Source File	sk4_math.cfi	
Template	`MATHLIB_SIN_frac24_frac24`	
Description	Approximate the sine of a frac24 input.	
Inputs	a0.h	= Normalized input x (frac24). [-1, 1) range of frac24 format number maps to [-PI, PI) range for input angle x
Outputs	a0.h	= sin(x) (frac24)
Register Modifications	a0, xp0	
Assumptions	None	
Clock Cycles	5	
Example	See Section 7.6, “Examples” on page 98.	

17.74 MATHLIB_SIN_VEC_FRAC24_FRAC24

Approximate the sine of each frac24 element in a vector

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_SIN_VEC_frac24_frac24(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Approximate the sine of each frac24 element in a vector. See MATHLIB_SIN_frac24_frac24.
Inputs	XP_IN = Pointer to an input vector of normalized frac24 angle values in X memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac24 elements in X memory.
Register Modifications	a0, xp0
Assumptions	None
Clock Cycles	3 + 7 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.75 MATHLIB_SQMAG_FRAC24_M2P46

Calculate the squared magnitude of complex number x

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_SQMAG_frac24_m2p46(XP_IN)`
Description	Calculate the squared magnitude of complex number x. • y = real(x)^2 + imag(x)^2 Saturation mode is assumed to be set.
Inputs	XP_IN = Pointer to complex value x in X memory, with frac24 real part followed by frac24 imaginary part at next address
Outputs	a0 = Squared magnitude result (m2p46)
Register Modifications	a0, x0
Assumptions	None
Clock Cycles	6
Example	See Section 7.6, “Examples” on page 98.

17.76 MATHLIB_SQMAG_VEC_FRAC24_M2P46

Vector-wise squared magnitude of complex values

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_SQMAG_VEC_frac24_m2p46(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Vector-wise squared magnitude of complex values. See MATHLIB_SQMAG_frac24_m2p46.
Inputs	XP_IN = Pointer to an input vector of complex values in X memory. Real and complex components stored in interleaved manner. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m2p46 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0
Assumptions	XP_IN != XP_OUT
Clock Cycles	3 + 7 * VEC_LENGTH
Example	See Section 7.6, “Examples” on page 98.

17.77 MATHLIB_SQRT_FRAC24_FRAC48

Approximate the square root of a frac24 input

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_SQRT_frac24_frac48`
Description	Approximate the square root of a frac24 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The square root of x is approximated as either: • sqrt(x) = 2^(E/2) * sqrt(m) for even E • sqrt(x) = 2^((E+1)/2) * sqrt(m/2) for odd E The value of sqrt(m) or sqrt(m/2) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0.h = Strictly positive input x (frac24)
Outputs	a0 = Square root of x (frac48)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	17 (E even) or 18 (E odd)
Example	See Section 7.6, “Examples” on page 98.

17.78 MATHLIB_SQRT_FRAC48_FRAC48

Approximate the square root of a frac48 input

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_SQRT_frac48_frac48`
Description	Approximate the square root of a frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The square root of x is approximated as either: • sqrt(x) = 2^(E/2) * sqrt(m) for even E • sqrt(x) = 2^((E+1)/2) * sqrt(m/2) for odd E The value of sqrt(m) or sqrt(m/2) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0 = Strictly positive input x (frac48)
Outputs	a0 = Square root of x (frac48)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	17 (E even) or 18 (E odd)
Example	See Section 7.6, “Examples” on page 98.

17.79 MATHLIB_SQRT_VEC_FRAC24_FRAC48

Approximate the square root of each frac24 element in a vector

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_SQRT_VEC_frac24_frac48(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Approximate the square root of each frac24 element in a vector. See MATHLIB_SQRT_frac24_frac48.
Inputs	XP_IN = Pointer to input vector of frac24 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac48 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	In range [5 + 14 * VEC_LENGTH, 5 + 15 * VEC_LENGTH]
Example	See Section 7.6, “Examples” on page 98.

17.80 MATHLIB_SQRT_VEC_FRAC48_FRAC48

Approximate the square root of each frac48 element in a vector

Type	Macro
Include File	`#include <sk4_math.cfi>`
Source File	sk4_math.cfi
Template	`MATHLIB_SQRT_VEC_frac48_frac48(XP_IN, XP_OUT, VEC_LENGTH)`
Description	Approximate the square root of each frac48 element in a vector. See MATHLIB_SQRT_frac48_frac48.
Inputs	XP_IN = Pointer to input vector of frac48 elements in XY memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac48 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	In range [5 + 14 * VEC_LENGTH, 5 + 15 * VEC_LENGTH]
Example	See Section 7.6, “Examples” on page 98.

CHAPTER 18
CFX Math Library Function Reference

This reference chapter presents a detailed description of all the functions in the CFX math library, including calling parameters, modified registers, returned values and execution time.

NOTE: When using a ROM-based version of the math library, function execution time is increased by 2 cycles (as described in Section 2.4.1, “ROM-Based Libraries” on page 30).

18.1 _MATHLIB_DIVIDE

Divide a non-negative dividend by a positive divisor

Type	Function
Include File	`#include <sk4_math.cfi>`
Source File	sk4_mathlib.cfs
Template	`_MATHLIB_DIVIDE`
Description	Divide a non-negative dividend by a positive divisor. The data format of the dividend and divisor is user-defined but must be the same 24-bit fixed-point format. This function is used as a component of larger division routines implemented by the Math Library macros. An undefined result is returned for dividend < 0 or divisor <= 0.
Inputs	a0.h = Non-negative dividend (user-defined 24-bit fixed-point format) x0 = Strictly positive divisor (user-defined 24-bit fixed-point format)
Outputs	a0.l = Quotient (int24)
Register Modifications	a0
Assumptions	None
Clock Cycles	26
Example	See Section 7.6, “Examples” on page 98.

18.2 _MATHLIB_EXP2_M8P16_FRAC48

Approximate 2^x for m8p16 input x, with frac48 output

Type	Function
Include File	`#include <sk4_math.cfi>`
Source File	sk4_mathlib.cfs
Template	`_MATHLIB_EXP2_m8p16_frac48`
Description	Approximate 2^x for m8p16 input x, with frac48 output. The input is represented as x = E + r, where E is the integer component of x and r is the fractional component. Two to the power x is approximated as: • 2^x = 2^E * 2^r = 2^(E+1) * 2^(r-1) The value of 2^(r-1) is approximated using a lookup table. An undefined result is returned for input x >= 0.
Inputs	a0.h = Strictly negative input x (m8p16)
Outputs	a0 = Result 2^x (frac48)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	7
Example	See Section 7.6, “Examples” on page 98.

18.3 _MATHLIB_LOG2_FRAC48_M8P16

Approximate the base-2 logarithm of a frac48 input

Type	Function
Include File	`#include <sk4_math.cfi>`
Source File	sk4_mathlib.cfs
Template	`_MATHLIB_LOG2_frac48_m8p16`
Description	Approximate the base-2 logarithm of a frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The base-2 logarithm of x is approximated as: • log2(x) = log2(m*2^E) = log2(m) + E The value of log2(m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0 = Strictly positive input x (frac48)
Outputs	a0.h = Base-2 logarithm of x (m8p16)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	9
Example	See Section 7.6, “Examples” on page 98.

18.4 _MATHLIB_RECIP_GEN_FRAC48_FL8N16

Approximate the reciprocal of a signed frac48 input

Type	Function
Include File	`#include <sk4_math.cfi>`
Source File	sk4_mathlib.cfs
Template	`_MATHLIB_RECIP_GEN_frac48_fl8n16`
Description	Approximate the reciprocal of a signed frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x = 0.
Inputs	a0 = Non-zero input x (frac48)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	None
Clock Cycles	12
Example	See Section 7.6, “Examples” on page 98.

18.5 _MATHLIB_RECIP_POS_FRAC48_FL8N16

Approximate the reciprocal of a positive frac48 input

Type	Function
Include File	`#include <sk4_math.cfi>`
Source File	sk4_mathlib.cfs
Template	`_MATHLIB_RECIP_POS_frac48_fl8n16`
Description	Approximate the reciprocal of a positive frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0 = Strictly positive input x (frac48)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	9
Example	See Section 7.6, “Examples” on page 98.

18.6 _MATHLIB_SQRT_FRAC48_FRAC48

Approximate the square root of a frac48 input

Type	Function
Include File	`#include <sk4_math.cfi>`
Source File	sk4_mathlib.cfs
Template	`_MATHLIB_SQRT_frac48_frac48`
Description	Approximate the square root of a frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The square root of x is approximated as either: • sqrt(x) = 2^(E/2) * sqrt(m) for even E • sqrt(x) = 2^((E+1)/2) * sqrt(m/2) for odd E The value of sqrt(m) or sqrt(m/2) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0 = Strictly positive input x (frac48)
Outputs	a0 = Square root of x (frac48)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	11 (E even) or 12 (E odd)
Example	See Section 7.6, “Examples” on page 98.

CHAPTER 19
ARM Cortex-M3 Math Library Reference

This reference chapter presents a detailed description of all the functions in the ARM Cortex-M3 math library, including calling parameters, returned values and assumptions.

19.1 MATH_ADD_FRAC32

Add two 32-bit signed fractional numbers, then saturate

Type	Function
Include File	`#include <sk4_math.h>`
Source File	sk4_math_frac32.c
Template	`int Math_Add_frac32(int x, int y)`
Description	Add two 32-bit signed fractional numbers, then saturate. The result is one of the following: • (x + y), if MIN_FRAC32 <= (x + y) <= MAX_FRAC32 • MAX_FRAC32, if (x + y) > MAX_FRAC32 • MIN_FRAC32, if (x + y) < MIN_FRAC32
Inputs	x = Fractional number represented as int y = Fractional number represented as int
Outputs	z = (x + y) with saturation
Assumptions	None

Example	`/* Addition with saturation: add two 32-bit signed fractional`
	` * numbers, then saturate to 0x7FFFFFFF if positive overflow`
	` * occurs, or 0x80000000 if negative overflow occurs. */`
	`z  = Math_Add_frac32(x, y);`

19.2 MATH_ATTACKRELEASE

Calculate a first-order attack-release filter

Type	Function
Include File	`#include <sk4_math.h>`
Source File	sk4_math_float.c
Template	`float Math_AttackRelease(float a, float b, float x, float y1)`
Description	Calculate a first-order attack-release filter. The outputs of the attack-release filter are calculated as: • y[n] = beta * x[n] + (1 - beta) * y[n-1] Where: • beta = a if x[n] >= y[n-1] • beta = b if x[n] < y[n-1] • a is the coefficient for attack, 0 < a < 1 • b is the coefficient for release, 0 < b < 1
Inputs	a = Filter coefficient b = Filter coefficient x = Current input y1 = Previous output
Outputs	y = New output
Assumptions	None

Example	`/* Dual-time constant attack-release averaging filter: generate new`
	` * output data based on the filter coefficients a and b, new input`
	` * data data_in, and previous output data data_out. */`
	`data_out = Math_AttackRelease(a, b, data_in, data_out);`

19.3 MATH_ATTACKRELEASE_FRAC32

Calculate a fixed-point first-order attack-release filter

Type	Function
Include File	`#include <sk4_math.h>`
Source File	sk4_math_frac32.c
Template	`int Math_AttackRelease_frac32(int a, int b, int x, int y1)`
Description	Calculate a fixed-point first-order attack-release filter. The outputs of the attack-release filter are calculated as: • y[n] = beta * x[n] + (1 - beta) * y[n-1] Where: • beta = a if x[n] >= y[n-1] • beta = b if x[n] < y[n-1] • a is the coefficient for attack, 0 < a < 1 • b is the coefficient for release, 0 < b < 1
Inputs	a = Filter coefficient b = Filter coefficient x = Current input y1 = Previous output
Outputs	y = New output
Assumptions	None

Example	`/* Dual-time constant attack-release averaging filter: generate new`
	` * output data based on the filter coefficients a and b, new input`
	` * data data_in, and previous output data data_out. All input and`
	` * output types are 32-bit signed fractional. */`
	`data_out = Math_AttackRelease_frac32(a, b, data_in, data_out);`

19.4 MATH_EXPAVG

Calculate a first-order exponential average

Type	Function
Include File	`#include <sk4_math.h>`
Source File	sk4_math_float.c
Template	`float Math_ExpAvg(float alpha, float x, float y1)`
Description	Calculate a first-order exponential average. The outputs of the exponential average are calculated as: • y[n] = alpha * x[n] + (1 - alpha) * y[n-1] Where: • 0 < alpha < 1
Inputs	alpha = Filter coefficient x = Current input y1 = Previous output
Outputs	y = New output
Assumptions	None

Example	`/* Exponential moving average filter: generate new output data based`
	` * on the filter coefficient alpha, new input data data_in, and`
	` * previous output data data_out. */`
	`data_out = Math_ExpAvg(alpha, data_in, data_out);`

19.5 MATH_EXPAVG_FRAC32

Calculate a fixed-point first-order exponential average

Type	Function
Include File	`#include <sk4_math.h>`
Source File	sk4_math_frac32.c
Template	`int Math_ExpAvg_frac32(int alpha, int x, int y1)`
Description	Calculate a fixed-point first-order exponential average. The outputs of the exponential average are calculated as: • y[n] = alpha * x[n] + (1 - alpha) * y[n-1] Where: • 0 < alpha < 1
Inputs	alpha = Filter coefficient x = Current input y1 = Previous output
Outputs	y = New output
Assumptions	None

Example	`/* Exponential moving average filter: generate new output data based`
	` * on the filter coefficient alpha, new input data data_in, and`
	` * previous output data data_out. All input and output types are`
	` * 32-bit signed fractional. */`
	`data_out = Math_ExpAvg_frac32(alpha, data_in, data_out);`

19.6 MATH_LINEARINTERP

Calculate linear interpolation on the interval [x0, x1)

Type	Function
Include File	`#include <sk4_math.h>`
Source File	sk4_math_float.c
Template	`float Math_LinearInterp(float x0, float x1, float y0, float y1, float x)`
Description	Calculate linear interpolation on the interval [x0, x1). The interpolation is calculated as: • y = y0 + (y1 - y0) / (x1 - x0) * (x - x0)
Inputs	x0 = First boundary point x-axis value x1 = Second boundary point x-axis value y0 = First boundary point y-axis value y1 = Second boundary point y-axis value x = Interpolation point
Outputs	y = Interpolated value
Assumptions	x0 != x1

Example	`/* Linear interpolation on the interval [x0, x1) with boundary points`
	` * (x0, y0) and (x1, y1). */`
	`y = Math_LinearInterp(x0, x1, y0, y1, x);`

19.7 MATH_LINEARINTERP_FRAC32

Calculate fixed-point linear interpolation on the interval [0, 1)

Type	Function
Include File	`#include <sk4_math.h>`
Source File	sk4_math_frac32.c
Template	`int Math_LinearInterp_frac32(int y0, int y1, int x)`
Description	Calculate fixed-point linear interpolation on the interval [0, 1). The interpolation is calculated as: • y = y0 + x * (y1 - y0)
Inputs	y0 = Left boundary point y1 = Right boundary point x = Interpolation point
Outputs	y = Interpolated value
Assumptions	0 <= x < 1

Example	`/* Linear interpolation on the interval [0, 1) with boundary points`
	` * y0 and y1. */`
	`y = Math_LinearInterp_frac32(y0, y1, x);`

19.8 MATH_MULT_FRAC32

Multiply two 32-bit signed fractional numbers, then saturate

Type	Function
Include File	`#include <sk4_math.h>`
Source File	sk4_math_frac32.c
Template	`int Math_Mult_frac32(int x, int y)`
Description	Multiply two 32-bit signed fractional numbers, then saturate. The result is either: • x * y, if x > MIN_FRAC32 or y > MIN_FRAC32 • MAX_FRAC32, if x = MIN_FRAC32 and y = MIN_FRAC32
Inputs	x = Fractional number represented as int y = Fractional number represented as int
Outputs	z = (x * y) with saturation
Assumptions	None

Example	`/* Multiplication with saturation: multiply two 32-bit signed`
	` * fractional numbers, then saturate to 0x7FFFFFFF if overflow`
	` * occurs. */`
	`z = Math_Mult_frac32(x, y);`

19.9 MATH_SINGLEVAR_REG

Find the least-squares solution for a single variable linear regression model

Type	Function
Include File	`#include <sk4_math.h>`
Source File	sk4_math_float.c
Template	`void Math_SingleVar_Reg(float* x, float* y, unsigned int N, float* a)`
Description	Find the least-squares solution for a single variable linear regression model. A linear regression model with a single predictor variable can be represented as: • y[i] = a0 + a1 * x[i] + e[i], i = 0, 1, 2, ..., N-1
Inputs	x = Pointer to the input variable vector x[] y = Pointer to the dependent variable vector y[] N = Length of vector x[] and y[]
Outputs	a = Pointer to the coefficient vector {a0, a1}
Assumptions	x[] and y[] are of the same length x[] is not a constant vector (constant vector here means x[0] = x[1] = ... = x[N-1]) a is a pointer to a coefficient vector of length 2

Example	`/* For an input variable vector x[] and a dependent variable vector`
	` * y[], find the least-squares solution to the linear regression`
	` * model. */`
	`Math_SingleVar_Reg(x, y, DATA_LENGTH, coeff_vector);`

19.10 MATH_SUB_FRAC32

Subtract one 32-bit signed fractional number from another, then saturate

Type	Function
Include File	`#include <sk4_math.h>`
Source File	sk4_math_frac32.c
Template	`int Math_Sub_frac32(int x, int y)`
Description	Subtract one 32-bit signed fractional number from another, then saturate. The result is one of the following: • (x - y), if MIN_FRAC32 <= (x - y) <= MAX_FRAC32 • MAX_FRAC32, if (x - y) > MAX_FRAC32 • MIN_FRAC32, if (x - y) < MIN_FRAC32
Inputs	x = Fractional number represented as int y = Fractional number represented as int
Outputs	z = (x - y) with saturation
Assumptions	None

Example	`/* Subtract with saturation: subtract one 32-bit signed fractional`
	` * number from another, then saturate to 0x7FFFFFFF if positive`
	` * overflow occurs, or 0x80000000 if negative overflow occurs. */`
	`z  = Math_Sub_frac32(x, y);`

CHAPTER 20
EEPROM Library Reference

This reference chapter presents a detailed description of all the macros and functions in the EEPROM library, including calling parameters, modified registers and returned values.

20.1 EELIB_AUTOSET_ADDRESSMODE

Determine the address mode of the EEPROM attached to the SPI interface, and set the address mode accordingly

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_AutoSet_AddressMode`
Description	Determine the address mode of the EEPROM attached to the SPI interface, and set the address mode accordingly
Inputs	None
Outputs	SYSVAR_EE_ADDR_MODE= Set to the correct address mode; compare against EE_ADDRESS_[16BIT | 24BIT]
Register Modifications	b0, b1
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode EEPROM block write protect is not configured to protect the bottom section of the EEPROM
Clock Cycles	Not applicable

Example	`// Automatically configure the EEPROM for the correct addressing mode`
	`call.1d EELIB_AutoSet_AddressMode`
	`nop // [ds]`

20.2 EELIB_CLOSE

Close the EEPROM library

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_Close`
Description	Close the EEPROM library.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	6

Example	`// Close the EEPROM Library`
	`call.1d EELIB_Close`
	`nop // [ds]`

20.3 EELIB_COMMIT

Commit EEPROM buffer to EEPROM non-volatile memory

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_Commit`
Description	Commit EEPROM buffer to EEPROM non-volatile memory
Inputs	None
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1
Assumptions	None
Clock Cycles	Not applicable

Example	`// Commit the current writes to the EEPROM library. Branch to`
	`// function A if the writes failed to commit.`
	`call.1d EELIB_Commit`
	`nop // [ds]`
	`goto.0d FunctionA, zero`

20.4 EELIB_INITIALIZE

Initialize the EEPROM library

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_Initialize`
Description	Initialize the EEPROM library. Sets the default commit timeout, write-disables the library, and sets the default EEPROM type to use 16-bit addresses and 64-byte pages.
Inputs	b1 = SPI speed; use SPI_CLK_PRESCALE_* define
Outputs	None
Register Modifications	b1, x1
Assumptions	The sources for the SPI interface pads have been configured
Clock Cycles	Not applicable

Example	`// Initialize the EEPROM Library`
	`call.1d EELIB_Initialize`
	`load b1, SPI_CLK_PRESCALE_2 // [ds] Load SPI prescaler`

20.5 EELIB_INITWRITEWORD

Initialize the EEPROM for writing at the specified address

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_InitWriteWord`
Description	Initialize the EEPROM for writing at the specified address.
Inputs	b0.h = EEPROM address to write to
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode EEPROM address mode has been set (invalid settings default to 16-bit addressing)
Clock Cycles	Not applicable

Example	`// Set up to write a block of data at EEPROM_WRITE_ADDRESS`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_InitWriteWord`
	`load b0.h, EEPROM_WRITE_ADDRESS // [ds] Load the address to write to`
	
	`// Write the 16-bit data from memory buffer to EEPROM`
	`Write_Loop:`
	`  call.1d EELIB_WriteNextWord16`
	`  nop ||| load b1.h, x[xp0++] // [ds] Load the word to write`
	`  nop ||| move b1, xp0`
	`  cmp b1, MEMORY_BUFFER_SIZE ||| nop`
	`  goto.0d Write_Loop, ne`

20.6 EELIB_LIBWRITEDISABLE

Write-disable the EEPROM library

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_LibWriteDisable`
Description	Write-disable the EEPROM library.
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	3

Example	`// Write-disable the EEPROM Library`
	`call.1d EELIB_LibWriteDisable`
	`nop // [ds]`

20.7 EELIB_LIBWRITEENABLE

Write-enable the EEPROM library

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_LibWriteEnable`
Description	Write-enable the EEPROM library.
Inputs	b1 = AT_WREN opcode
Outputs	None
Register Modifications	None
Assumptions	EEPROM not busy
Clock Cycles	3

Example	`// Write-enable the EEPROM Library`
	`call.1d EELIB_LibWriteEnable`
	`load b1, AT_WREN // [ds] Load the write-enable opcode`

20.8 EELIB_POWERDOWN

Attempt to place the EEPROM in power-down mode

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_PowerDown`
Description	Attempt to place the EEPROM in power-down mode
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	EEPROM not busy
Clock Cycles	Not applicable

Example	`// Power down the EEPROM to reduce the current consumed by this`
	`// device while it is not in use.`
	`call.1d EELIB_PowerDown`
	`nop  // [ds]`

20.9 EELIB_POWERUP

Place the EEPROM into power-up mode

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_PowerUp`
Description	Place the EEPROM into power-up mode
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	EEPROM not busy
Clock Cycles	Not applicable

Example	`// Ensure the EEPROM is powered up before trying to read from it`
	`call.1d EELIB_PowerUp`
	`nop // [ds]`

20.10 EELIB_READBLOCK_P

Read a block of data from EEPROM to PMEM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadBlock_P`
Description	Read a block of data from EEPROM to PMEM. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM address xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM not busy
Clock Cycles	Not applicable

Example	`// Read a block of MEMORY_BUFFER_SIZE words, from EEPROM at`
	`// EEPROM_READ_ADDRESS, to P-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_P`
	`load xp0, MEMORY_BUFFER_START  // [ds] Load the buffer pointer`

20.11 EELIB_READBLOCK_P16

Read a block of data from EEPROM to PMEM in 16-bit words

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadBlock_P16`
Description	Read a block of data from EEPROM to PMEM in 16-bit words. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM address xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM not busy
Clock Cycles	Not applicable

Example	`// Read a block of MEMORY_BUFFER_SIZE 16-bit words, from EEPROM at`
	`// EEPROM_READ_ADDRESS, to P-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_P16`
	`load xp0, MEMORY_BUFFER_START  // [ds] Load the buffer pointer`

20.12 EELIB_READBLOCK_X

Read a block of data from EEPROM to XMEM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadBlock_X`
Description	Read a block of data from EEPROM to XMEM. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM address xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM not busy
Clock Cycles	Not applicable

Example	`// Read a block of MEMORY_BUFFER_SIZE words, from EEPROM at`
	`// EEPROM_READ_ADDRESS, to X-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_X`
	`load xp0, MEMORY_BUFFER_START  // [ds] Load the buffer pointer`

20.13 EELIB_READBLOCK_X16

Read a block of data from EEPROM to XMEM in 16-bit words

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadBlock_X16`
Description	Read a block of data from EEPROM to XMEM in 16-bit words. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM address xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM not busy
Clock Cycles	Not applicable

Example	`// Read a block of MEMORY_BUFFER_SIZE 16-bit words, from EEPROM at`
	`// EEPROM_READ_ADDRESS, to X-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_X16`
	`load xp0, MEMORY_BUFFER_START  // [ds] Load the buffer pointer`

20.14 EELIB_READBLOCK_Y

Read a block of data from EEPROM to YMEM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadBlock_Y`
Description	Read a block of data from EEPROM to YMEM. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM address yp0 = Destination memory address
Outputs	None
Register Modifications	x0, yp0, b1
Assumptions	EEPROM not busy
Clock Cycles	Not applicable

Example	`// Read a block of MEMORY_BUFFER_SIZE words, from EEPROM at`
	`// EEPROM_READ_ADDRESS, to Y-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_Y`
	`load yp0, MEMORY_BUFFER_START  // [ds] Load the buffer pointer`

20.15 EELIB_READNEXTBLOCK_P

Read the next block of data from a queued EEPROM to PMEM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadNextBlock_P`
Description	Read the next block of data from a queued EEPROM to PMEM.
Inputs	x0 = Size of block (in words) xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	Not applicable

Example	`// Read the next block of MEMORY_BUFFER_SIZE words from EEPROM to`
	`// P-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_P`
	`load xp0, MEMORY_BUFFER_START  // [ds] Load the buffer pointer`

20.16 EELIB_READNEXTBLOCK_P16

Read the next block of data from a queued EEPROM to PMEM in 16-bit words

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadNextBlock_P16`
Description	Read the next block of data from a queued EEPROM to PMEM in 16-bit words
Inputs	x0 = Size of block (in words) xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	Not applicable

Example	`// Read the next block of MEMORY_BUFFER_SIZE 16-bit words from`
	`// EEPROM to P-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_P16`
	`load xp0, MEMORY_BUFFER_START  // [ds] Load the buffer pointer`

20.17 EELIB_READNEXTBLOCK_X

Read the next block of data from a queued EEPROM to XMEM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadNextBlock_X`
Description	Read the next block of data from a queued EEPROM to XMEM
Inputs	x0 = Size of block (in words) xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	Not applicable

Example	`// Read the next block of MEMORY_BUFFER_SIZE words from EEPROM to`
	`// X-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_X`
	`load xp0, MEMORY_BUFFER_START  // [ds] Load the buffer pointer`

20.18 EELIB_READNEXTBLOCK_X16

Read the next block of data from a queued EEPROM to XMEM in 16-bit words

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadNextBlock_X16`
Description	Read the next block of data from a queued EEPROM to XMEM in 16-bit words
Inputs	x0 = Size of block (in words) xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	Not applicable

Example	`// Read the next block of MEMORY_BUFFER_SIZE 16-bit words from`
	`// EEPROM to X-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_X16`
	`load xp0, MEMORY_BUFFER_START  // [ds] Load the buffer pointer`

20.19 EELIB_READNEXTBLOCK_Y

Read the next block of data from a queued EEPROM to YMEM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadNextBlock_Y`
Description	Read the next block of data from a queued EEPROM to YMEM
Inputs	x0 = Size of block (in words) yp0 = Destination memory address
Outputs	None
Register Modifications	x0, yp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	Not applicable

Example	`// Read the next block of MEMORY_BUFFER_SIZE words from EEPROM to`
	`// Y-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_Y`
	`load yp0, MEMORY_BUFFER_START  // [ds] Load the buffer pointer`

20.20 EELIB_READNEXTWORD16

Read a 16-bit word from the queued EEPROM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadNextWord16`
Description	Read a 16-bit word from the queued EEPROM. Update the CRC value with the 16-bit word.
Inputs	None
Outputs	b1.h = Next word
Register Modifications	b1
Assumptions	EEPROM queued for read
Clock Cycles	Not applicable

Example	`// Read the next word from EEPROM into the memory buffer`
	`call.1d EELIB_ReadNextWord16`
	`nop // [ds]`
	`nop ||| store x[xp0++], b1.h // Store the word read to the buffer`

20.21 EELIB_READNEXTWORD24

Read a 24-bit word from the queued EEPROM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadNextWord24`
Description	Read a 24-bit word from the queued EEPROM. Update the CRC value with the 24-bit word.
Inputs	None
Outputs	b1.h = Next word
Register Modifications	b1
Assumptions	EEPROM queued for read
Clock Cycles	Not applicable

Example	`// Read the next word from EEPROM into the memory buffer`
	`call.1d EELIB_ReadNextWord24`
	`nop // [ds]`
	`nop ||| store x[xp0++], b1.h // Store the word read to the buffer`

20.22 EELIB_READNEXTWORD32

Read a 32-bit word from the queued EEPROM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadNextWord32`
Description	Read a 32-bit word from the queued EEPROM. Update the CRC value with the 32-bit word.
Inputs	None
Outputs	b1 = Next word
Register Modifications	b1
Assumptions	EEPROM queued for read
Clock Cycles	Not applicable

Example	`// Read the next word from EEPROM into the memory buffer`
	`call.1d EELIB_ReadNextWord32`
	`nop // [ds]`
	`store.ex p[xp0++], b1 // Store the word read to the buffer`

20.23 EELIB_READNEXTWORD8

Read an 8-bit word from the queued EEPROM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadNextWord8`
Description	Read an 8-bit word from the queued EEPROM. Update the CRC value with the 8-bit word.
Inputs	None
Outputs	b1.h = Next word
Register Modifications	b1
Assumptions	EEPROM queued for read
Clock Cycles	Not applicable

Example	`// Read the next byte from EEPROM into the memory buffer`
	`call.1d EELIB_ReadNextWord8`
	`nop // [ds]`
	`nop ||| store x[xp0++], b1.h // Store the byte read to the buffer`

20.24 EELIB_READSTATUSREGISTER

Read the EEPROM status register

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadStatusRegister`
Description	Read the EEPROM status register.
Inputs	None
Outputs	b1.h = EEPROM status byte
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	Not applicable

Example	`// Check the AT_WEN bit from the EEPROM status register and branch`
	`// to function A if not set`
	`call.1d EELIB_ReadStatusRegister`
	`nop // [ds]`
	`bittst b1, (AT_WEN_POS + 24) ||| nop`
	`goto.0d FunctionA, no_carry`

20.25 EELIB_READWORD16

Initialize the EEPROM and read a 16-bit word

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadWord16`
Description	Initialize the EEPROM and read a 16-bit word. Update the CRC value with the 16-bit word.
Inputs	b0.h = EEPROM address
Outputs	b1.h = Read word
Register Modifications	b1
Assumptions	EEPROM not busy EEPROM address mode has been set (invalid settings default to 16-bit addressing)
Clock Cycles	Not applicable

Example	`// Read a word from EEPROM into the memory buffer`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_ReadWord16`
	`load b0.h, EEPROM_READ_ADDRESS  // [ds] Load the address to read from`
	`nop ||| store x[xp0++], b1.h    // Store the word read to the buffer`

20.26 EELIB_READWORD24

Initialize the EEPROM and read a 24-bit word

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadWord24`
Description	Initialize the EEPROM and read a 24-bit word. Update the CRC value with the 24-bit word.
Inputs	b0.h = EEPROM address
Outputs	b1.h = Read word
Register Modifications	b1
Assumptions	EEPROM not busy EEPROM address mode has been set (invalid settings default to 16-bit addressing)
Clock Cycles	Not applicable

Example	`// Read a word from EEPROM into the memory buffer`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_ReadWord24`
	`load b0.h, EEPROM_READ_ADDRESS  // [ds] Load the address to read from`
	`nop ||| store x[xp0++], b1.h    // Store the word read to the buffer`

20.27 EELIB_READWORD32

Initialize the EEPROM and read a 32-bit word

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadWord32`
Description	Initialize the EEPROM and read a 32-bit word. Update the CRC value with the 32-bit word.
Inputs	b0.h = EEPROM address
Outputs	b1 = Read word
Register Modifications	b1
Assumptions	EEPROM not busy EEPROM address mode has been set (invalid settings default to 16-bit addressing)
Clock Cycles	Not applicable

Example	`// Read a word from EEPROM into the memory buffer`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_ReadWord32`
	`load b0.h, EEPROM_READ_ADDRESS  // [ds] Load the address to read from`
	`store.ex p[xp0++], b1  // Store the word read to the buffer`

20.28 EELIB_READWORD8

Initialize the EEPROM and read an 8-bit word

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_ReadWord8`
Description	Initialize the EEPROM and read an 8-bit word. Updates the CRC value with the 8-bit word.
Inputs	b0.h = EEPROM address
Outputs	b1.h = Read word
Register Modifications	b1
Assumptions	EEPROM not busy EEPROM address mode has been set (invalid settings default to 16-bit addressing)
Clock Cycles	Not applicable

Example	`// Read a byte from EEPROM into the memory buffer`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_ReadWord8`
	`load b0.h, EEPROM_READ_ADDRESS  // [ds] Load the address to read from`
	`nop ||| store x[xp0++], b1.h    // Store the byte read to the buffer`

20.29 EELIB_SELECTSEGMENT

Selects EEPROM segment 0 or 1 (address bit 17)

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_SelectSegment`
Description	Selects EEPROM segment 0 or 1 (address bit 17)
Inputs	b0.h = Segment number [0|1]
Outputs	None
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	Not applicable

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS in the high`
	`// segment of the custom E2M2 EEPROM memory.`
	`call.1d EELIB_SelectSegment`
	`load b0.h, 1 // [ds]`
	
	`load b1.h, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord16`
	`load b0.h, EEPROM_WRITE_ADDRESS // [ds] Load the address to write to`

20.30 EELIB_SET_ADDRESSMODE

Set the address mode for the EEPROM library

Type	Macro
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfi
Template	`EELIB_Set_AddressMode( mode )`
Description	Set the address mode for the EEPROM library
Inputs	mode = Address mode to use; use EE_ADDRESS_[16BIT | 24BIT]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Change the EEPROM addressing mode to support a 24-bit`
	`// addressed EEPROM`
	`EELIB_Set_AddressMode(EE_ADDRESS_24BIT)`

20.31 EELIB_SET_PAGESIZE

Set the page size for the EEPROM library

Type	Macro
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfi
Template	`EELIB_Set_PageSize( size )`
Description	Set the page size for the EEPROM library
Inputs	size = Page size to use; use a constant
Outputs	None
Register Modifications	y1
Assumptions	The specified page size is a power of two
Clock Cycles	2 if size is greater than or equal to 256; 1 otherwise
Example	`// Change the EEPROM page size to 64 bytes per page` `EELIB_Set_PageSize(64)`

20.32 EELIB_VERIFYEEPROMCONNECTION

Verify that an EEPROM is connected to the SPI port

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_VerifyEEPROMConnection`
Description	Verify that an EEPROM is connected to the SPI port
Inputs	None
Outputs	b1.h = Connected status (0 - Connected; 1 - Not Connected)
Register Modifications	b0, b1
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	Not applicable

Example	`// Check if an EEPROM is connected, branching to function A if one`
	`// is detected`
	`call.1d EELIB_VerifyEEPROMConnection`
	`nop // [ds]`
	`cmp b1, 0 ||| nop`
	`goto.0d FunctionA, eq`

20.33 EELIB_WRITEBLOCK_CONST32

Writes a block of data from one constant PMEM word to EEPROM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteBlock_Const32`
Description	Writes a block of data from one constant PMEM word to EEPROM. Use to fill an EEPROM block with a single word pattern.
Inputs	x0 = Size of buffer (in words) b0.h = EEPROM address xp0 = Pointer to source word (32-bit, in PMEM)
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, b0
Assumptions	The EEPROM is not busy EELIB_LibWriteEnable was called with the AT_WREN opcode The EEPROM page size has been set
Clock Cycles	Not applicable

Example	`// Write a block of EEPROM at EEPROM_WRITE_ADDRESS, from a 32-bit`
	`// constant stored in P-memory at MEMORY_CONSTANT`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_WRITE_ADDRESS`
	`call.1d EELIB_WriteBlock_Const32`
	`load xp0, MEMORY_CONSTANT  // [ds] Load the buffer pointer`

20.34 EELIB_WRITEBLOCK_P

Writes a block of data from PMEM to EEPROM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteBlock_P`
Description	Writes a block of data from PMEM to EEPROM
Inputs	x0 = Size of buffer (in words) b0.h = EEPROM address xp0 = Source buffer address
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, b0, xp0
Assumptions	The EEPROM is not busy EELIB_LibWriteEnable was called with the AT_WREN opcode The EEPROM page size has been set
Clock Cycles	Not applicable

Example	`// Write a block of EEPROM at EEPROM_WRITE_ADDRESS, from P-memory at`
	`// MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_WRITE_ADDRESS`
	`call.1d EELIB_WriteBlock_P`
	`load xp0, MEMORY_BUFFER_START  // [ds] Load the buffer pointer`

20.35 EELIB_WRITEBLOCK_X

Writes a block of data from XMEM to EEPROM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteBlock_X`
Description	Writes a block of data from XMEM to EEPROM
Inputs	x0 = Size of buffer (in words) b0.h = EEPROM address xp0 = Source buffer address
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, b0, xp0
Assumptions	The EEPROM is not busy EELIB_LibWriteEnable was called with the AT_WREN opcode The EEPROM page size has been set
Clock Cycles	Not applicable

Example	`// Write a block of EEPROM at EEPROM_WRITE_ADDRESS, from X-memory at`
	`// MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_WRITE_ADDRESS`
	`call.1d EELIB_WriteBlock_X`
	`load xp0, MEMORY_BUFFER_START  // [ds] Load the buffer pointer`

20.36 EELIB_WRITEBLOCK_Y

Writes a block of data from YMEM to EEPROM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteBlock_Y`
Description	Writes a block of data from YMEM to EEPROM
Inputs	x0 = Size of buffer (in words) b0.h = EEPROM address yp0 = Source buffer address
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, b0, yp0
Assumptions	The EEPROM is not busy EELIB_LibWriteEnable was called with the AT_WREN opcode The EEPROM page size has been set
Clock Cycles	Not applicable

Example	`// Write a block of EEPROM at EEPROM_WRITE_ADDRESS, from Y-memory at`
	`// MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_WRITE_ADDRESS`
	`call.1d EELIB_WriteBlock_Y`
	`load yp0, MEMORY_BUFFER_START  // [ds] Load the buffer pointer`

20.37 EELIB_WRITEDISABLE

Write-disable the EEPROM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteDisable`
Description	Write-disable the EEPROM.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	EEPROM not busy
Clock Cycles	Not applicable

Example	`// Write-disable the EEPROM`
	`call.1d EELIB_WriteDisable`
	`nop  // [ds]`

20.38 EELIB_WRITEENABLE

Write-enable the EEPROM

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteEnable`
Description	Write-enable the EEPROM
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	Not applicable

Example	`// Write-enable the EEPROM`
	`call.1d EELIB_WriteEnable`
	`nop  // [ds]`

20.39 EELIB_WRITENEXTWORD16

Writes a 16-bit word to the EEPROM buffer

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteNextWord16`
Description	Writes a 16-bit word to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 16-bit word.
Inputs	b1.h = Word to write
Outputs	None
Register Modifications	None
Assumptions	EEPROM queued for write
Clock Cycles	Not applicable

Example	`// Continue writing to EEPROM`
	`call.1d EELIB_WriteNextWord16`
	`nop ||| load b1.h, x[xp0++]  // [ds] Load the next word to write`

20.40 EELIB_WRITENEXTWORD24

Writes a 24-bit word to the EEPROM buffer

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteNextWord24`
Description	Writes a 24-bit word to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 24-bit word.
Inputs	b1.h = Word to write
Outputs	None
Register Modifications	None
Assumptions	EEPROM queued for write
Clock Cycles	Not applicable

Example	`// Continue writing to EEPROM`
	`call.1d EELIB_WriteNextWord24`
	`nop ||| load b1.h, x[xp0++]  // [ds] Load the next word to write`

20.41 EELIB_WRITENEXTWORD32

Writes a 32-bit word to the EEPROM buffer

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteNextWord32`
Description	Writes a 32-bit word to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 32-bit word.
Inputs	b1 = Word to write (Bits 23:0 from b1.h. Bits 31:24 from b1.e)
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM queued for write
Clock Cycles	Not applicable

Example	`// Continue writing to EEPROM`
	`call.1d EELIB_WriteNextWord32`
	`load.ex b1, p[xp0++]  // [ds] Load the next word to write`

20.42 EELIB_WRITENEXTWORD8

Writes a 8-bit word to the EEPROM buffer

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteNextWord8`
Description	Writes a 8-bit word to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 8-bit word.
Inputs	b1.h = Word to write
Outputs	None
Register Modifications	None
Assumptions	EEPROM queued for write
Clock Cycles	Not applicable

Example	`// Continue writing to EEPROM`
	`call.1d EELIB_WriteNextWord8`
	`nop ||| load b1.h, x[xp0++]  // [ds] Load the next byte to write`

20.43 EELIB_WRITESTATUSREGISTER

Write the EEPROM status register

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteStatusRegister`
Description	Write the EEPROM status register.
Inputs	b0.h = New EEPROM status byte
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	Not applicable

Example	`// Write the EEPROM status register, setting the block write protect`
	`// bits`
	`call.1d EELIB_WriteStatusRegister`
	`load b0.h, (3 << AT_BP0_POS)    // [ds] Load new value`

20.44 EELIB_WRITEWORD16

Initialize the EEPROM for writing and send a 16-bit word to be written to the EEPROM buffer

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteWord16`
Description	Initialize the EEPROM for writing and send a 16-bit word to be written to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 16-bit word.
Inputs	b0.h = EEPROM address b1.h = Word to write
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode EEPROM address mode has been set (invalid settings default to 16-bit addressing)
Clock Cycles	Not applicable

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS`
	`load b1.h, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord16`
	`load b0.h, EEPROM_WRITE_ADDRESS // [ds] Load the address to write to`

20.45 EELIB_WRITEWORD24

Initialize the EEPROM for writing and send a 24-bit word to be written to the EEPROM buffer

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteWord24`
Description	Initialize the EEPROM for writing and send a 24-bit word to be written to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 24-bit word.
Inputs	b0.h = EEPROM address b1.h = Word to write
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode EEPROM address mode has been set (invalid settings default to 16-bit addressing)
Clock Cycles	Not applicable

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS`
	`load b1.h, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord24`
	`load b0.h, EEPROM_WRITE_ADDRESS // [ds] Load the address to write to`

20.46 EELIB_WRITEWORD32

Initialize the EEPROM for writing and send a 32-bit word to be written to the EEPROM buffer

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteWord32`
Description	Initialize the EEPROM for writing and send a 32-bit word to be written to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 32-bit word.
Inputs	b0.h = EEPROM address b1 = Word to write (Bits 23:0 from b1.h. Bits 31:24 from b1.e)
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode EEPROM address mode has been set (invalid settings default to 16-bit addressing)
Clock Cycles	Not applicable

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS`
	`load b1, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord32`
	`load b0.h, EEPROM_WRITE_ADDRESS // [ds] Load the address to write`

20.47 EELIB_WRITEWORD8

Initialize the EEPROM for writing and send a 8-bit word to be written to the EEPROM buffer

Type	Function
Include File	`#include <sk4_eeprom.cfi>`
Source File	sk4_eeprom.cfs
Template	`EELIB_WriteWord8`
Description	Initialize the EEPROM for writing and send a 8-bit word to be written to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 8-bit word.
Inputs	b0.h = EEPROM address b1.h = Word to write
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode EEPROM address mode has been set (invalid settings default to 16-bit addressing)
Clock Cycles	Not applicable

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS`
	`load b1.h, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord8`
	`load b0.h, EEPROM_WRITE_ADDRESS // [ds] Load the address to write to`

CHAPTER 21

Calibration Library Reference

This reference chapter presents a detailed description of all the macros and functions in the calibration library, including calling parameters, modified registers and returned values.

21.1 CALIBRATE_ADC_CALIBRATECOMPLETESYSTEM

Builds a table of all ADC DC offset values (starting at xp0)

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_adc.cfs
Template	`CALIBRATE_ADC_CalibrateCompleteSystem`
Description	Builds a table of all ADC DC offset values (starting at xp0). Includes a setting for each PGA setting for each ADC. Uses the CalibrateCurrentConfiguration subroutine to calibrate each setting to the target DC offset.
Inputs	xp0 = The starting address in XMEM where the calibration values will be stored
Outputs	SYSVAR_CAL_ERRNO = Set to indicate if the ADC calibration did not converge for one or more values
Register Modifications	b1, y1, yp3, xp7
Assumptions	VREG has been calibrated The desired sampling frequency has been set CALIBRATE_ADC_Initialize has been called
Clock Cycles	N/A

Example	`// Calibrate all ADC and PGA combinations and save results starting`
	`// at X_UNPACKED_TABLE. Note: Results must be consolidated into the`
	`// ADC calibration table stored in the Y memory system variables`
	`// before using the system library function SYSLIB_CalibrateADC.`
	`call.1d CALIBRATE_ADC_CalibrateCompleteSystem`
	`load xp0, X_UNPACKED_TABLE // [ds]`

21.2 CALIBRATE_ADC_CALIBRATECURRENTCONFIGURATION

Given the current PGA gain setting and the target DC offset, an 8-bit calibration value is calculated for each ADC channel

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_adc.cfs
Template	`CALIBRATE_ADC_CalibrateCurrentConfiguration`
Description	Given the current PGA gain setting and the target DC offset, an 8-bit calibration value is calculated for each ADC channel.
Inputs	xp0 = Location where the calibration values will be saved
Outputs	SYSVAR_CAL_ERRNO = Set to indicate if the ADC calibration did not converge for one or more values
Register Modifications	b1, y1, yp3, xp7
Assumptions	The input stage has been configured Input selection for input stage is set to MUTE The desired sampling frequency has been set VREG has been calibrated CALIBRATE_ADC_Initialize has been called
Clock Cycles	N/A

Example	`// Calibrate all ADCs for the currently configured PGA gain and save`
	`// the results to X_SINGLE_GAIN_RESULTS. Note: Results must be copied`
	`// to the ADC calibration table stored in the Y memory system`
	`// variables before using the system library function`
	`// SYSLIB_CalibrateADC.`
	`call.1d CALIBRATE_ADC_CalibrateCurrentConfiguration`
	`load xp0, X_SINGLE_GAIN_RESULTS // [ds]`

21.3 CALIBRATE_ADC_INITIALIZE

Initialize the system to support the ADC calibration

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_adc.cfs
Template	`CALIBRATE_ADC_Initialize`
Description	Initialize the system to support the ADC calibration
Inputs	y0 = Target accuracy in the 18-bit LSBs. Convert to mV by dividing by 2^17. y1 = Tone placement as a multiple of Fs, encoded in a <4,20> fixed-point format (m4p20)
Outputs	None
Register Modifications	sr[IE] = 0, sr[FLAG_UPDATE] = 1, b1, y1
Assumptions	The desired sampling frequency has been set
Clock Cycles	N/A

Example	`// Initialize the system for ADC calibration, using the default`
	`// tone placement and accuracy`
	`load y0, CALIBRATION_ACCURACY_1P51_MV`
	`call.1d CALIBRATE_ADC_Initialize`
	`load y1, DEFAULT_PLACEMENT_FS_MULTIPLE // [ds]`

21.4 CALIBRATE_ADCCOPYTORAM

Copy the ADC calibration data from the local manufacturing area to the system variables in low Y RAM

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_ADCCopyToRAM`
Description	Copy the ADC calibration data from the local manufacturing area to the system variables in low Y RAM.
Inputs	None
Outputs	None
Register Modifications	b1, y1, xp7, yp3
Assumptions	One hardware loop is available
Clock Cycles	90

Example	`// Store the previously calibrated ADC data from the local`
	`// manufacturing area to the system variables`
	`Calibrate_ADCCopyToRAM`

21.5 CALIBRATE_ADDADCCALIBRATION

Update the ADC calibration entry specified in the local manufacturing area using the current SAMP_FREQ and the data pointed to by xpreg

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_AddADCCalibration( adc, xpreg )`
Description	Update the ADC calibration entry specified in the local manufacturing area using the current SAMP_FREQ and the data pointed to by xpreg
Inputs	adc = ADC to update the calibration entry for xpreg = Pointer to unpacked ADC calibration data
Outputs	None
Register Modifications	b1, y1
Assumptions	xpreg points to a valid ADC calibration table The current SAMP_FREQ setting matches the ADC calibration data
Clock Cycles	125

Example	`// ADC0 and ADC1 are calibrated. Store their ADC calibration settings`
	`// to the local manufacturing area (xp0 points to the temporary`
	`// calibration table)`
	`Calibrate_AddADCCalibration( 0, xp0 )`
	`Calibrate_AddADCCalibration( 1, xp0 )`

21.6 CALIBRATE_ADDCLOCKCALIBRATION

Update the specified clock calibration entry in the local manufacturing area using the current clock configuration

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_AddClockCalibration( entry, op_freq )`
Description	Update the specified clock calibration entry in the local manufacturing area using the current clock configuration
Inputs	entry = The clock calibration entry to update; the default clock frequency is loaded from clock calibration entry 0 op_freq = The operating frequency index to be used for the added clock calibration entry; use SYS_FREQ_*M**
Outputs	None
Register Modifications	y1, b1
Assumptions	The current clock configuration provides the specified operating frequency
Clock Cycles	14 to 21, depending on the byte alignment of the clock calibration entry being updated

Example	`// Add the current clock configuration to the manufacturing area as`
	`// the default entry, indicating the clock is currently calibrated`
	`// to 5.12MHz`
	`Calibrate_AddClockCalibration(0, SYS_FREQ_5M12)`

21.7 CALIBRATE_ADDVDBLCALIBRATION

Update the VDBL calibration entry in the local manufacturing area using the current VDBL configuration

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_AddVDBLCalibration`
Description	Update the VDBL calibration entry in the local manufacturing area using the current VDBL configuration
Inputs	None
Outputs	None
Register Modifications	y1, b1
Assumptions	The current VDBL setting is a valid calibration
Clock Cycles	5

Example	`// Store the current VDBL calibration setting to the local`
	`// manufacturing area`
	`Calibrate_AddVDBLCalibration`

21.8 CALIBRATE_ADDVDDACALIBRATION

Update the VDDA calibration entry in the local manufacturing area using the current VDDA configuration

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_AddVDDACalibration`
Description	Update the VDDA calibration entry in the local manufacturing area using the current VDDA configuration
Inputs	None
Outputs	None
Register Modifications	y1, b1
Assumptions	The current VDDA setting is a valid calibration
Clock Cycles	13

Example	`// Store the current VDDA calibration setting to the local`
	`// manufacturing area`
	`Calibrate_AddVDDACalibration`

21.9 CALIBRATE_ADDVDDCALTCALIBRATION

Update the VDDC alternate calibration entry in the local manufacturing area using the current VDDC configuration

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_AddVDDCAltCalibration`
Description	Update the VDDC alternate calibration entry in the local manufacturing area using the current VDDC configuration
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	The current VDDC setting is a valid calibration
Clock Cycles	4

Example	`// Store the current VDDC calibration setting to the alternate`
	`// location in the local manufacturing area`
	`Calibrate_AddVDDCAltCalibration`

21.10 CALIBRATE_ADDVDDCCALIBRATION

Update the VDDC calibration entry in the local manufacturing area using the current VDDC configuration

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_AddVDDCCalibration`
Description	Update the VDDC calibration entry in the local manufacturing area using the current VDDC configuration
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	The current VDDC setting is a valid calibration
Clock Cycles	4

Example	`// Store the current VDDC calibration setting to the local`
	`// manufacturing area`
	`Calibrate_AddVDDCCalibration`

21.11 CALIBRATE_ADDVDDMCALIBRATION

Update the VDDM calibration entry in the local manufacturing area using the current VDDM configuration

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_AddVDDMCalibration`
Description	Update the VDDM calibration entry in the local manufacturing area using the current VDDM configuration
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	The current VDDM setting is a valid calibration
Clock Cycles	4

Example	`// Store the current VDDM calibration setting to the local`
	`// manufacturing area`
	`Calibrate_AddVDDMCalibration`

21.12 CALIBRATE_ADDVREGCALIBRATION

Update the VREG calibration entry in the local manufacturing area using the current VREG configuration

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_AddVREGCalibration`
Description	Update the VREG calibration entry in the local manufacturing area using the current VREG configuration
Inputs	None
Outputs	None
Register Modifications	y1, b1
Assumptions	The current VREG setting is a valid calibration
Clock Cycles	8

Example	`// Store the current VREG calibration setting to the local`
	`// manufacturing area`
	`Calibrate_AddVREGCalibration`

21.13 CALIBRATE_CLOCK

Calibrate the system clock

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_clock.cfs
Template	`CALIBRATE_Clock`
Description	Calibrate the system clock
Inputs	y0 = Number of cycles required to achieve the desired calibrated frequency; use y0 = freq / (REF_FREQ * TIMING_LOOP_CYCLE_COUNT) where freq is the desired clock frequency in hertz
Outputs	SR[X_ZERO] = Set if a timeout has occurred, cleared otherwise SYSVAR_CAL_ERRNO = Set to indicate if a failure occurred
Register Modifications	a1, b0, b1, y1
Assumptions	CALIBRATE_Clock_Initialize has been called VREG has been calibrated The verified input reference signal is still present
Clock Cycles	N/A

Example	`// Calibrate the clock to 5.12MHz`
	`call.1d CALIBRATE_Clock`
	`load y0, (5120000 / (REF_FREQ * TIMING_LOOP_CYCLE_COUNT)) // [ds]`

21.14 CALIBRATE_CLOCK

Calculate the calibration routine's reference clock cycle target for the given clock frequency

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_Clock( freq )`
Description	Calculate the calibration routine's reference clock cycle target for the given clock frequency
Inputs	freq = The clock frequency in hertz to calibrate to
Outputs	SR[X_ZERO] = Set if an error has occurred; cleared otherwise
Register Modifications	a1, b1, y0, y1
Assumptions	CALIBRATE_Clock_Initialize has been called VREG has been calibrated The verified input reference signal is still present
Clock Cycles	N/A
Example	`// Calibrate the current system clock to 1.28MHz` `Calibrate_Clock(1280000)`

21.15 CALIBRATE_CLOCK_INITIALIZE

Initialize the system to support the clock calibration

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_clock.cfs
Template	`CALIBRATE_Clock_Initialize`
Description	Initialize the system to support the clock calibration
Inputs	y1 = DIO to read the input reference from
Outputs	SR[X_ZERO] = Set if the reference signal cannot be verified; cleared otherwise SYSVAR_CAL_ERRNO = Set to indicate if a failure occurred
Register Modifications	SR[FLAG_UPDATE], b0, b1, y1, yp3
Assumptions	None
Clock Cycles	N/A

Example	`// Initialize the system for clock calibration, using a reference signal`
	`// provided on DIO 4`
	`call.1d CALIBRATE_Clock_Initialize`
	`load y1, 4 // [ds]`

21.16 CALIBRATE_INITIALIZEMA

Reset the contents of the local manufacturing area

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_InitializeMA`
Description	Reset the contents of the local manufacturing area.
Inputs	None
Outputs	None
Register Modifications	b1, yp3
Assumptions	None
Clock Cycles	57
Example	`// Initialize the local manufacturing area` `Calibrate_InitializeMA`

21.17 CALIBRATE_LOADMA

Load the local manufacturing area instance from an attached EEPROM

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_LoadMA`
Description	Load the local manufacturing area instance from an attached EEPROM. Replaces the content of the local manufacturing area with the content from the EEPROM.
Inputs	None
Outputs	None
Register Modifications	x0, b0, xp0
Assumptions	The EEPROM library has been initialized The sources for the SPI interface pads have been configured
Clock Cycles	N/A

Example	`// Initialize the EEPROM library to support loading the`
	`// manufacturing area`
	`call.1d EELIB_Initialize`
	`load b1, SPI_CLK_PRESCALE_2 // [ds] Load SPI prescaler`
	
	`// Load the manufacturing area from the attached EEPROM`
	`Calibrate_LoadMA`

21.18 CALIBRATE_POWER_INITIALIZE

Initialize the system for power supply (VREG, VDDC, VDDM, VDDA and VDBL) calibration

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_pwr.cfs
Template	`CALIBRATE_Power_Initialize`
Description	Initialize the system for power supply (VREG, VDDC, VDDM, VDDA and VDBL) calibration. Resets the VREG, VDDA, VDBL, VDDC, VDDM and oscillator trimming values to their default settings.
Inputs	None
Outputs	None
Register Modifications	b1, y1
Assumptions	None
Clock Cycles	N/A

Example	`// Initialize the system for power supply calibration`
	`call.1d CALIBRATE_Power_Initialize`
	`nop // [ds]`

21.19 CALIBRATE_POWER_VDBLALTCALIBRATE

Calibrate the interface charge pump (VDBL) to a specified voltage

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_pwr.cfs
Template	`CALIBRATE_Power_VDBLAltCalibrate`
Description	Calibrate the interface charge pump (VDBL) to a specified voltage
Inputs	y1 = LSAD measurement of the desired target voltage for VDBL
Outputs	SR[X_ZERO] = Set if the calibration did not result in a valid setting; cleared otherwise SYSVAR_CAL_ERRNO = Set to indicate if a failure occurred
Register Modifications	b1, y1, xp7, yp3
Assumptions	VREG has been calibrated CALIBRATE_Power_Initialize has been called
Clock Cycles	N/A

Example	`// Calibrate VDBL to 1.85 V`
	`call.1d CALIBRATE_Power_VDBLAltCalibrate`
	`load y1, 0xE8 // [ds]`

21.20 CALIBRATE_POWER_VDBLCALIBRATE

Calibrate the interface charge pump (VDBL)

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_pwr.cfs
Template	`CALIBRATE_Power_VDBLCalibrate`
Description	Calibrate the interface charge pump (VDBL)
Inputs	None
Outputs	SR[X_ZERO] = Set if the calibration did not result in a valid setting; cleared otherwise SYSVAR_CAL_ERRNO = Set to indicate if a failure occurred
Register Modifications	b1, y1, xp7, yp3
Assumptions	VREG has been calibrated CALIBRATE_Power_Initialize has been called
Clock Cycles	N/A

Example	`// Calibrate VDBL`
	`call.1d CALIBRATE_Power_VDBLCalibrate`
	`nop // [ds]`

21.21 CALIBRATE_POWER_VDDACALIBRATE

Calibrate the analog charge pump (VDDA)

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_pwr.cfs
Template	`CALIBRATE_Power_VDDACalibrate`
Description	Calibrate the analog charge pump (VDDA)
Inputs	None
Outputs	SR[X_ZERO] = Set if the calibration did not result in a valid setting; cleared otherwise SYSVAR_CAL_ERRNO = Set to indicate if a failure occurred
Register Modifications	b1, y1, xp7, yp3
Assumptions	VREG has been calibrated CALIBRATE_Power_Initialize has been called
Clock Cycles	N/A

Example	`// Calibrate VDDA`
	`call.1d CALIBRATE_Power_VDDACalibrate`
	`nop // [ds]`

21.22 CALIBRATE_POWER_VDDCALTCALIBRATE

Calibrate the digital core voltage (VDDC) to a specified voltage

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_pwr.cfs
Template	`CALIBRATE_Power_VDDCAltCalibrate`
Description	Calibrate the digital core voltage (VDDC) to a specified voltage
Inputs	y1 = LSAD measurement of the desired target voltage for VDDC
Outputs	SR[X_ZERO] = Set if the calibration did not result in a valid setting; cleared otherwise SYSVAR_CAL_ERRNO = Set to indicate if a failure occurred
Register Modifications	b1, y1, xp7, yp3
Assumptions	VREG has been calibrated CALIBRATE_Power_Initialize has been called
Clock Cycles	N/A

Example	`// Calibrate VDDC to 0.73 V (useful for f_SYSCLK < 5.12 MHz)`
	`call.1d CALIBRATE_Power_VDDCAltCalibrate`
	`load y1, 0xC2 // [ds]`

21.23 CALIBRATE_POWER_VDDCCALIBRATE

Calibrate the digital core voltage (VDDC)

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_pwr.cfs
Template	`CALIBRATE_Power_VDDCCalibrate`
Description	Calibrate the digital core voltage (VDDC)
Inputs	None
Outputs	SR[X_ZERO] = Set if the calibration did not result in a valid setting; cleared otherwise SYSVAR_CAL_ERRNO = Set to indicate if a failure occurred
Register Modifications	b1, y1, xp7, yp3
Assumptions	VREG has been calibrated CALIBRATE_Power_Initialize has been called
Clock Cycles	N/A

Example	`// Calibrate VDDC`
	`call.1d CALIBRATE_Power_VDDCCalibrate`
	`nop // [ds]`

21.24 CALIBRATE_POWER_VDDMALTCALIBRATE

Calibrate the digital memory voltage (VDDM) to a specified voltage

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_pwr.cfs
Template	`CALIBRATE_Power_VDDMAltCalibrate`
Description	Calibrate the digital memory voltage (VDDM) to a specified voltage
Inputs	y1 = LSAD measurement of the desired target voltage for VDDM
Outputs	SR[X_ZERO] = Set if the calibration did not result in a valid setting; cleared otherwise SYSVAR_CAL_ERRNO = Set to indicate if a failure occurred
Register Modifications	b1, y1, xp7, yp3
Assumptions	VREG has been calibrated CALIBRATE_Power_Initialize has been called
Clock Cycles	N/A

Example	`// Calibrate VDDM to 0.88 V (useful for f_SYSCLK = 15.36 MHz)`
	`call.1d CALIBRATE_Power_VDDMAltCalibrate`
	`load y1, 0xEA // [ds]`

21.25 CALIBRATE_POWER_VDDMCALIBRATE

Calibrate the digital memory voltage (VDDM)

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_pwr.cfs
Template	`CALIBRATE_Power_VDDMCalibrate`
Description	Calibrate the digital memory voltage (VDDM)
Inputs	None
Outputs	SR[X_ZERO] = Set if the calibration did not result in a valid setting; cleared otherwise SYSVAR_CAL_ERRNO = Set to indicate if a failure occurred
Register Modifications	b1, y1, xp7, yp3
Assumptions	VREG has been calibrated CALIBRATE_Power_Initialize has been called
Clock Cycles	N/A

Example	`// Calibrate VDDM`
	`call.1d CALIBRATE_Power_VDDMCalibrate`
	`nop // [ds]`

21.26 CALIBRATE_POWER_VREGALTCALIBRATE

Calibrate the bandgap voltage (VREG) against a specified VBAT supply voltage

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_pwr.cfs
Template	`CALIBRATE_Power_VREGAltCalibrate`
Description	Calibrate the bandgap voltage (VREG) against a specified VBAT supply voltage.
Inputs	y1 = LSAD measurement of the expected VBAT voltage (without correction factored in for the differences between VREG and VREF)
Outputs	SR[X_ZERO] = Set if the calibration did not result in a valid setting; cleared otherwise SYSVAR_CAL_ERRNO = Set to indicate if a failure occurred
Register Modifications	b1, y1, xp7, yp3
Assumptions	CALIBRATE_Power_Initialize has been called
Clock Cycles	N/A

Example	`// Calibrate VREG relative to a 1.5 V VBAT reference`
	`call.1d CALIBRATE_Power_VREGAltCalibrate`
	`load y1, 0x180 // [ds]`

21.27 CALIBRATE_POWER_VREGCALIBRATE

Calibrate the bandgap voltage (VREG)

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_pwr.cfs
Template	`CALIBRATE_Power_VREGCalibrate`
Description	Calibrate the bandgap voltage (VREG)
Inputs	None
Outputs	SR[X_ZERO] = Set if the calibration did not result in a valid setting; cleared otherwise SYSVAR_CAL_ERRNO = Set to indicate if a failure occurred
Register Modifications	b1, y1, yp3, xp7
Assumptions	VBAT is supplied at 1.25V CALIBRATE_Power_Initialize has been called
Clock Cycles	N/A

Example	`// Calibrate VREG`
	`call.1d CALIBRATE_Power_VREGCalibrate`
	`nop // [ds]`

21.28 CALIBRATE_StoreMA

Store the contents of the local manufacturing area to an attached EEPROM

Type	Function
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate_init.cfs
Template	`CALIBRATE_StoreMA`
Description	Store the contents of the local manufacturing area to an attached EEPROM.
Inputs	None
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, b0, xp0
Assumptions	The EEPROM library has been initialized The sources for the SPI interface pads have been configured The EEPROM is not busy EELIB_LibWriteEnable was called with the AT_WREN opcode The EEPROM page size and addressing type have been set
Clock Cycles	N/A

Example	`// Initialize the EEPROM library, setting the SPI prescaler to 8`
	`//(transfers at 1.28MHz), write enable the library, and make sure that`
	`// the BWP bits are cleared.`
	`call.1d EELIB_Initialize`
	`load b1, SPI_CLK_PRESCALE_8 // [ds]`
	
	`call.1d EELIB_LibWriteEnable`
	`load b1, AT_WREN // [ds]`
	
	`call.1d EELIB_WriteStatusRegister`
	`load b0, 0 // [ds]`
	
	`call.1d EELIB_AutoSet_AddressMode`
	`nop // [ds]`
	
	`// Store the manufacturing data area created to the EEPROM`
	`call.1d CALIBRATE_StoreMA`
	`nop // [ds]`

21.29 CALIBRATE_STOREMA

Store the contents of the local manufacturing area to an attached EEPROM

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_StoreMA`
Description	Store the contents of the local manufacturing area to an attached EEPROM.
Inputs	None
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, b0, xp0
Assumptions	The EEPROM library has been initialized The sources for the SPI interface pads have been configured The EEPROM is not busy The EEPROM page size and addressing type have been set
Clock Cycles	support macro declarations

Example	`// Store the manufacturing area containing calibration information`
	`// to the attached EEPROM`
	`Calibrate_StoreMA`

21.30 CALIBRATE_UPDATEMA24

Update the local manufacturing area by writing at the table offset specified

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_UpdateMA24( offset, dreg )`
Description	Update the local manufacturing area by writing at the table offset specified
Inputs	offset = Offset into the MA table to update (in bytes) dreg = Data register holding the 24-bit value to update the MA with
Outputs	None
Register Modifications	b1
Assumptions	The MA has been initialized through a call to Calibrate_InitializeMA or Calibrate_LoadMA
Clock Cycles	5 if the offset is divisible by 4 with no remainder; 3 if the offset is divisible by 4 with a remainder of 1; 8 if the offset is divisible by 4 with a remainder of 2; 10 if the offset is divisible by 4 with a remainder of 3

Example	`// Clear the default clock calibration setting`
	`load x0, EEFS_ADC_CAL_BLOCK_INVALID`
	`Calibrate_UpdateMA24( EEFS_CLKCAL_BLOCK_BASE - EEFS_MANUAREA_BASE, x0 )`

21.31 CALIBRATE_UPDATEMA32

Update the local manufacturing area by writing at the table offset specified

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_UpdateMA32( offset, accum )`
Description	Update the local manufacturing area by writing at the table offset specified
Inputs	offset = Offset into the MA table to update (in bytes) accum = Accumulator holding the 32-bit value to update the MA with; the value is contained within the extension and high parts of this accumulator (accum cannot be b1)
Outputs	None
Register Modifications	b1
Assumptions	The MA has been initialized through a call to Calibrate_InitializeMA or Calibrate_LoadMA offset is 32-bit aligned
Clock Cycles	1 if the offset is divisible by 4 with no remainder; 8 if the offset is divisible by 4 with a remainder of 1; 12 otherwise

Example	`// Update the manufacturing area to store the passed-in`
	`// timestamp for the production test (this value was`
	`// previously stored in b0)`
	`Calibrate_UpdateMA32( TIMESTAMP_OFFSET, b0 )`

21.32 CALIBRATE_UPDATEMA8

Update the local manufacturing area by writing at the table offset specified

Type	Macro
Include File	`#include <sk4_calibrate.cfi>`
Source File	sk4_calibrate.cfi
Template	`Calibrate_UpdateMA8( offset, dreg )`
Description	Update the local manufacturing area by writing at the table offset specified
Inputs	offset = Offset into the MA table to update (in bytes) dreg = Data register holding the 8-bit value to update the MA with; the value is contained in the 8 LSBs of this register
Outputs	None
Register Modifications	y1, b1
Assumptions	The MA has been initialized through a call to Calibrate_InitializeMA or Calibrate_LoadMA
Clock Cycles	3 if the offset divisible by 4 with no remainder; 9 otherwise

Example	`// Update the manufacturing area to mark the calibration settings`
	`// for ADC0 as invalid`
	`load x0, EEFS_ADC_CAL_BLOCK_INVALID`
	`Calibrate_UpdateMA8( EEFS_ADC0_CAL_ADDR - EEFS_MANUAREA_BASE, x0 )`

APPENDIX A
Glossary

The following abbreviations and terms are used in this manual:

breakpoints, external
: These breakpoints allow the configuration of a breakpoint event based on accesses to data and the value of the written data.

breakpoints, hardware
: These breakpoints are configured as pointers to P-Memory. They are triggered when the instruction pointed to by the hardware breakpoint is issued for CFX processor execution.

breakpoints, software
: These breakpoints are included in code for the CFX processor as `breakpoint` instructions.

CFX Digital Signal Processor
: the Ezairo 7100 processing core that is used to configure the system and coordinate the flow of signal data progressing through the system

CID
: chip identifier

CMSIS
: Cortex Microcontroller Software Interface Standard

CRC
: cyclic redundancy code

FENG
: Filter Engine

FIFO
: A section of memory that is treated as a circular buffer where components that use this memory use pointers to handle data in a first-in, first-out manner. A *hardware FIFO* is a FIFO instance where its registers are controlled by the IOC. A *software FIFO* is a FIFO instance where its registers are controlled by an application on the CFX or microcode configuration on the HEAR.

GPIO
: general purpose input/output

HEAR Configurable Accelerator
: the Ezairo 7100 core that is a microcode configurable signal processing engine used to execute a variety of signal processing functions; designed to work with the CFX to reduce the signal processing load on the CFX

I^2C
: Industry standardized Inter-IC Communication protocol.

I^2S
: Industry standardized Inter-IC Sound protocol

IOC
: input/output controller

LDP
: low-delay path

LSAD
: low-speed analog-to-digital

LSB
: least significant bit

LUT
: look up table

MSB
: most significant bit

NVM
: non-volatile memory

PCM
: pulse code modulation

POR
: power-on reset

RAM
: random access memory

ROM
: read-only memory

SCL
: serial clock signal (part of I^2C bus)

SDA
: serial data signal (part of I^2C bus)

SiP
: system-in-package

SoC
: system-on-chip

SPI
: serial peripheral interface

UART

universal asynchronous receiver-transmitter

ON Semiconductor and ON are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is either a trademark or registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative